

Contents

guntner.dasm	6
nesdefs.dasm	14
zp_vars.asm	18
ppu_lib_and_tables.asm	23
state__handler.asm	38
state_menu_screens.asm	40
state_cutscenes.asm	51
state_attract.asm	57
state_game.asm	59
arctang.asm	64
collision_detection.dasm	69
dashboard.asm	71
palette.asm	77
powerups.asm	83
sine.dasm	88
starfield_simple.asm	90
timer_global.dasm	95
phase_handler.asm	97
phase_table.asm	109
enemy_handler.dasm	113
enemy_tables.dasm	123
enemy_death.asm	125
enemy_boss_moufs.asm	128
enemy_boss_vamp.asm	134
enemy_boss_scarab.asm	143
enemy_boss_swordtner.asm	147
enemy_boss_eyes.asm	156
enemy_ant.asm	157
enemy_birb.asm	160

enemy_chomps.asm	162
enemy_dart.asm	164
enemy_dumbface.asm	167
enemy_galger.asm	168
enemy_ikes_mom.asm	175
enemy_lasso.asm	178
enemy_maggs.asm	180
enemy_muya.asm	182
enemy_skeet.asm	184
enemy_skully.asm	186
enemy_spark.asm	188
enemy_starglasses.asm	190
enemy_throber.asm	192
enemy_uzi.asm	194
enemy_zigzag.asm	197
player_main.dasm	199
player_bullets.dasm	203
player_controls.dasm	209
player_movement.dasm	211
player_demo.asm	215
apu_engine.asm	218
apu_sfx.asm	225
apu_song.asm	235
tables_at_top_of_rom.asm	245

guntner.dasm

```

include "nesdefs.dasm"
include "zp_vars.asm"

;;;;; RAM NOTES
; $000 zero page
; $100 6502 stack
; $200 PPU OAM sprite data
; $300 entity data
; $400-7ff sine scale tables

01 50      phase_spawn_table      = $0150
01 70      spare_stack            = $170
01 e0      top_of_stack           = $1e0
08 00      end_of_ram             = $800

        sprite_0_y      EQM      $b6 ; 182
        sprite_0_y_diff EQM      $4a ; 74

;;;;; NES CARTRIDGE HEADER

; mapper 0
; 1 16k PRG bank
; 1 8k CHR bank
; vertical mirroring

seg Header
org $Bff0
.NES_MAPPER      SET 0
.NES_PRG_BANKS   SET 1
.NES_CHR_BANKS   SET 1
.NES_MIRRORING   SET NES_MIRR_VERT

bff0 byte      $4e,$45,$53,$1a
bff4 byte      .NES_PRG_BANKS
bff5 byte      .NES_CHR_BANKS
bff6 byte      .NES_MIRRORING|(.NES_MAPPER<<4)
bff7 byte      .NES_MAPPER&$f0
bff8 byte      0,0,0,0,0,0,0,0

```

```

seg Code
org $C000

;;;;; START OF CODE

cart_start: subroutine
        NES_INIT ; set up stack pointer, turn off PPU
c000 78      sei
c001 d8      cld
c002 a2 ff   ldx #$ff
c004 9a      txs
c005 e8      inx
c006 8e 01 20 stx PPU_MASK
c009 8e 10 40 stx DMC_FREQ
c00c 8e 00 20 stx PPU_CTRL
c00f 2c 02 20 bit PPU_STATUS
c012 2c 15 40 bit APU_CHAN_CTRL
c015 a9 40    lda #$40
c017 8d 17 40 sta APU_FRAME
c01a a9 0f    lda #$0F
c01c 8d 15 40 sta APU_CHAN_CTRL

        ; CLEAR CPU RAM
c01f 20 15 c1 jsr ClearRAM ; clear RAM
c022 20 f5 fd jsr WHOOPS

        ; MY SETUP
        ; setup rng
c025 a9 01    lda #$01
c027 85 26    sta rng0
c029 85 27    sta rng1
c02b a9 ff    lda #$ff
c02d 85 28    sta rng2
c02f 85 43    sta dashboard_message

c031 a9 e8    lda #232
c033 85 03    sta scroll_y ; just seems right

c035 20 84 d4 jsr sine_init

c038 a9 ff    lda #>arctang_velocity_tables
c03a 85 85    sta arctang_velocity_hi

c03c a9 c0    lda #$c0
c03e 85 30    sta scroll_to_counter

c040 a9 01    lda #1
c042 85 23    sta game_difficulty
c044 a9 01    lda #$01
c046 85 1a    sta player_speed

```

```

; set default player colors
c048 a9 14 lda #$14
c04a 85 a9 sta player_color0
c04c a9 21 lda #$21
c04e 85 aa sta player_color1
c050 a9 37 lda #$37
c052 85 ab sta player_color2
c054 20 48 f2 jsr player_update_colors

```

```

; bookbinder colors
c057 a2 51 ldx #81
c059 a0 06 ldy #6
c05b 20 cf d1 jsr palette_load

```

```

;=====
; M O D E   I N I T   H E R E
c05e a9 0f lda #$0f
c060 85 90 sta pal_uni_bg

```

```

;jsr game_init
;jsr menu_screens_init
;jsr attract_init
;jsr cut_scene_intro_init
;jsr cut_scene_outro_init

;jsr options_screen_init
;jsr sandbox_init
;jsr sandbox2_init

```

```

.endless
c065 4c 65 c0 jmp .endless ; endless loop

```

```

;;;;; INTERRUPT HANDLERS

```

```

nmi_handler: subroutine

```

```

; ~2250 cycles for PPU access (PAL is 7450 cycles)
; "On NTSC, count on being able to copy 160 bytes
; to nametables or the palette using a moderately
; unrolled loop"
; write 64 tiles?
; write all palettes
; oam dma

```

```

SAVE_REGS ; 13 cycles

```

```

c068 48 pha
c069 8a txa
c06a 48 pha
c06b 98 tya
c06c 48 pha

```

```

; enable NMI lockout
c06d a5 00 lda nmi_lockout
c06f c9 00 cmp #$00
c071 f0 03 beq .no_lock
c073 4c 0f c1 jmp .nmi_end
.no_lock
c076 e6 00 inc nmi_lockout

```

```

; PPU VBLANK STUFF
; RENDER va cycles
c078 a5 55 lda state_render_addr
c07a 4c f1 c4 jmp jump_to_subroutine
state_render_done:
; OAM DMA 513 cycles
c07d a9 02 lda #$02
c07f 8d 14 40 sta PPU_OAM_DMA
; PALETTE 236 cycles
c082 20 df d2 jsr palette_render
; SCROLL POS 17 cycles
c085 2c 02 20 bit PPU_STATUS
c088 a5 01 lda scroll_x_hi
c08a 8d 05 20 sta PPU_SCROLL
c08d a5 03 lda scroll_y
c08f 8d 05 20 sta PPU_SCROLL
; NAMETABLE++ 9 cycles
c092 a5 04 lda scroll_page
c094 29 01 and #$01
c096 09 90 ora #CTRL_NMI|CTRL_BG_1000
c098 8d 00 20 sta PPU_CTRL
c09b a5 09 lda ppu_mask_emph
c09d 49 18 eor #MASK_BG|MASK_SPR
c09f 8d 01 20 sta PPU_MASK

```

```

; hope everything above was under
; ~2250 cycles!

```

```

; main state logic
c0a2 a5 56 lda state_update_addr
c0a4 4c f1 c4 jmp jump_to_subroutine
state_update_done:

sprite0_wait:
c0a7 a5 52 lda state_sprite0
c0a9 f0 30 beq sprite0_off

; wait for Sprite 0; SPRITE 0 WAIT TIME!!!
c0ab 2c 02 20 .wait0 bit PPU_STATUS
c0ae 70 fb bvs .wait0
c0b0 a9 c0 lda #$c0
c0b2 2c 02 20 .wait1 bit PPU_STATUS
c0b5 f0 fb beq .wait1

; HUD POSITIONING
; Horizontal Scroll Solution
c0b7 2c 02 20 bit PPU_STATUS
c0ba a9 00 lda #$00
c0bc 8d 05 20 sta PPU_SCROLL
c0bf 8d 05 20 sta PPU_SCROLL
; Vertical Scroll Solution
;lda #$26
;sta PPU_ADDR
;lda #$c0
;sta PPU_ADDR
;lda #$00
;sta PPU_SCROLL
;sta PPU_SCROLL

; set bg pos to page 2 for dashboard
c0c2 a9 91 lda #$01|CTRL_NMI|CTRL_BG_1000
c0c4 8d 00 20 sta PPU_CTRL
c0c7 a5 09 lda ppu_mask_emph
c0c9 49 08 eor #MASK_BG
c0cb 8d 01 20 sta PPU_MASK

; must be game mode
c0ce 20 fc f3 jsr player_bullets_update
c0d1 20 96 d5 jsr starfield_update

; HUD prepare for next draw
; stop updating HUD if game over
c0d4 a5 65 lda player_death_flag
c0d6 d0 03 bne skip_dash_update
c0d8 20 c2 d0 jsr dashboard_update
skip_dash_update:
sprite0_off:

; debug in lifebar space

;lda phase_spawn_counter
;jsr get_char_hi
;sta $109
;lda phase_spawn_counter

```

```

;jsr get_char_lo
;sta $10a

;lda phase_kill_counter
;jsr get_char_hi
;sta $10c
;lda phase_kill_counter
;jsr get_char_lo
;sta $10d

c0db 20 3e fb jsr song_update
c0de 20 9d f7 jsr apu_update

c0e1 20 a6 f4 jsr player_controls_read
c0e4 20 3d d4 jsr powerup_bomb_update
c0e7 20 0e d4 jsr powerup_mushroom_update
c0ea 20 00 d2 jsr palette_update

c0ed e6 24 inc wtf ; frame counter lo
c0ef d0 02 bne .skip_ftw
c0f1 e6 25 inc ftw ; frame counter hi
.skip_ftw
c0f3 20 44 d6 jsr timer_update ; update global time
c0f6 a5 26 lda rng0 ; rng updates
c0f8 20 5a c1 jsr NextRandom
c0fb 85 26 sta rng0
c0fd a5 27 lda rng1
c0ff 20 60 c1 jsr PrevRandom
c102 85 27 sta rng1
c104 a5 28 lda rng2
c106 20 5a c1 jsr NextRandom
c109 85 28 sta rng2

; disable NMI lockout
c10b a9 00 lda #$00
c10d 85 00 sta nmi_lockout

.nmi_end
RESTORE_REGS ; 16 cycles
c10f 68 pla
c110 a8 tay
c111 68 pla
c112 aa tax
c113 68 pla
c114 40 rti

; all includes

c115 include "ppu_lib_and_tables.asm"

c621 include "state_handler.asm"
c654 include "state_menu_screens.asm"
ca6e include "state_cutscenes.asm"
cc4c include "state_attract.asm"
ccc4 include "state_game.asm"

```

```

sandbox_init:
sandbox_update:
    ; include "state_sandbox.asm"
    ;include "state_sandbox2.asm"

ce12      include "arctang.asm"
cf35      include "collision_detection.dasm"

cf65      include "dashboard.asm"
d173      include "palette.asm"
d308      include "powerups.asm"
d484      include "sine.dasm"
          ;include "starfield.dasm"
d4ea      include "starfield_simple.asm"

d644      include "timer_global.dasm"

d6a4      include "phase_handler.asm"
d9d1      include "phase_table.asm"
da77      include "enemy_handler.dasm"
dcc5      include "enemy_tables.dasm"
dcfb      include "enemy_death.asm"

ddc3      include "enemy_boss_moufs.asm"
df6f      include "enemy_boss_vamp.asm"
e204      include "enemy_boss_scarab.asm"
e302      include "enemy_boss_swordtner.asm"

e5b6      include "enemy_boss_eyes.asm"

e5f9      include "enemy_ant.asm"
          ;include "enemy_bat.asm"
e6b6      include "enemy_birb.asm"
e725      include "enemy_chomps.asm"
e7a8      include "enemy_dart.asm"
e87c      include "enemy_dumbface.asm"
e8d9      include "enemy_galger.asm"
ec34      include "enemy_ikes_mom.asm"
ed04      include "enemy_lasso.asm"
ed89      include "enemy_maggs.asm"
edfb      include "enemy_muya.asm"
ee4d      include "enemy_skeet.asm"
eede      include "enemy_skully.asm"
ef6e      include "enemy_spark.asm"
efe6      include "enemy_starglasses.asm"
f067      include "enemy_throber.asm"
f0c8      include "enemy_uzi.asm"
f191      include "enemy_zigzag.asm"

f217      include "player_main.dasm"
f2f9      include "player_bullets.dasm"
f488      include "player_controls.dasm"
f4be      include "player_movement.dasm"
f5a0      include "player_demo.asm"

f65b      include "apu_engine.asm"
f846      include "apu_sfx.asm"
fb3e      include "apu_song.asm"

```

```

; space remaining history notes:

;fd13 - bosses have 2 spawn assistants
;fd0c - boss spawn fade from white
;fcd9 - removed score system
;fd4e - conglomerated all jump tables
;fd82 - refactored scarab sprite handlings
;fdc6 - nop's used for size gauges commented out
;fdf5 - all 4 bosses and game play able
;f953 - moar optimized 1197 bytes left! \:U/
;f9ea - optimized enemy code / using arctang more
;fbb8 - all audio done
;fa5d - phase engine, all text, and distress alien
;f535 - super secret code / a new song
;f3c0 - gutted superfluous enemy handling code
;f4d9 - gutted fancy starfield/msg feature
;fa82 - 1st level playable
;f7e3 - scarab boss
;f6e3 - ant/lasso/throber/uzi
;f37d - new audio engine and powerup sfx
;f162 - rewrote player movement
;f258 - powerups done except sfx
;f267 - vampire done?
;f14c - with arctang now
;e7be
;e8b6 newnewnew
;ea9f bookmark new
;e666 bookmark old

```

```

fdfe      ; LOOKUP TABLES AT TOP OF ROM
          org $fe00
fe00      include "tables_at_top_of_rom.asm"
fff8 a9 0d lda #13 ; super secret operation

```

```

          ;;;; CPU VECTORS
          seg Vectors
          org $fffa
fffa word nmi_handler ; vblank nmi
fffc word cart_start ; reset
fffe word nmi_handler ; irq / brk

```

```

          ;;;; GRAPHX
          org $010000
          incbin "guntner.chr"

```

nesdefs.dasm

processor 6502

;;;;; CONSTANTS

```
20 00      PPU_CTRL      = $2000
20 01      PPU_MASK      = $2001
20 02      PPU_STATUS    = $2002
20 03      OAM_ADDR      = $2003
20 04      OAM_DATA      = $2004
20 05      PPU_SCROLL    = $2005
20 06      PPU_ADDR      = $2006
20 07      PPU_DATA      = $2007

40 14      PPU_OAM_DMA    = $4014

40 16      JOYPAD1       = $4016
40 17      JOYPAD2       = $4017
00 80      BUTTON_A      = 1 << 7
00 40      BUTTON_B      = 1 << 6
00 20      BUTTON_SELECT = 1 << 5
00 10      BUTTON_START  = 1 << 4
00 08      BUTTON_UP     = 1 << 3
00 04      BUTTON_DOWN   = 1 << 2
00 02      BUTTON_LEFT   = 1 << 1
00 01      BUTTON_RIGHT  = 1 << 0
```

; NOTE: I've put this outside of the PPU & APU,
; because it is a feature of the CPU that is
; primarily of use to the PPU.

```
40 14      OAM_DMA        = $4014
```

```
02 00      ; OAM local RAM copy goes from $0200-$02FF:
OAM_RAM      = $0200
```

```
; PPU_CTRL flags
; Execute Non-Maskable Interrupt on VBlank
CTRL_NMI      EQM %10000000
; Use 8x8 Sprites
CTRL_8x8      EQM %00000000
; Use 8x16 Sprites
CTRL_8x16     EQM %00100000
; Background Pattern Table at $0000 in VRAM
CTRL_BG_0000  EQM %00000000
; Background Pattern Table at $1000 in VRAM
CTRL_BG_1000  EQM %00010000
; Sprite Pattern Table at $0000 in VRAM
CTRL_SPR_0000 EQM %00000000
; Sprite Pattern Table at $1000 in VRAM
CTRL_SPR_1000 EQM %00001000
; Increment PPU Address by 1 (Horizontal rendering)
CTRL_INC_1    EQM %00000000
; Increment PPU Address by 32 (Vertical rendering)
CTRL_INC_32   EQM %00000100
; Name Table Address at $2000
CTRL_NT_2000  EQM %00000000
; Name Table Address at $2400
CTRL_NT_2400  EQM %00000001
; Name Table Address at $2800
CTRL_NT_2800  EQM %00000010
; Name Table Address at $2C00
CTRL_NT_2C00  EQM %00000011
```



```

; PPU_MASK flags
MASK_TINT_RED      EQM %00100000 ; Red
MASK_TINT_BLUE     EQM %01000000 ; Blue
MASK_TINT_GREEN    EQM %10000000 ; Green
; Sprites Visible
MASK_SPR           EQM %00010000
; Backgrounds Visible
MASK_BG            EQM %00001000
; Sprites clipped on left column
MASK_SPR_CLIP      EQM %00000100
; Background clipped on left column
MASK_BG_CLIP       EQM %00000010
; Display in Color
MASK_COLOR         EQM %00000000
; Display in Monochrome
MASK_MONO          EQM %00000001

; read flags
; VBlank Active
F_BLANK            EQM %10000000
; VBlank hit Sprite 0
F_SPRITE0          EQM %01000000
; More than 8 sprites on current scanline
F_SCAN8           EQM %00100000
; VRAM writes currently ignored.
F_WIGNORE          EQM %00010000

```

```

;;;;; CARTRIDGE FILE HEADER

```

```

NES_MIRR_HORIZ     EQM 0
NES_MIRR_VERT      EQM 1
NES_MIRR_QUAD      EQM 8

```

```

;;;;; NES_INIT SETUP MACRO (place at start)

```

```

MAC nes_init
sei                ; disable IRQs
cld                ; decimal mode not supported
ldx #$ff
txs                ; set up stack pointer
inx                ; increment X to 0
stx PPU_MASK       ; disable rendering
stx DMC_FREQ       ; disable DMC interrupts
stx PPU_CTRL       ; disable NMI interrupts
bit PPU_STATUS     ; clear VBL flag
bit APU_CHAN_CTRL; ack DMC IRQ bit 7
lda #$40
sta APU_FRAME      ; disable APU Frame IRQ
lda #$0F
sta APU_CHAN_CTRL; initialize channels
ENDM

```

```

;;;;; PPU_SETADDR <address> - set PPU address

```

```

MAC ppu_setaddr
lda #{1}           ; upper byte
sta PPU_ADDR
lda #{1}           ; lower byte
sta PPU_ADDR
ENDM

```

```

;;;;; SAVE_REGS - save A/X/Y registers

```

```

MAC save_regs
pha
txa
pha
tya
pha
ENDM

```

```

;;;;; RESTORE_REGS - restore Y/X/A registers

```

```

MAC restore_regs
pla
tay
pla
tax
pla
ENDM

```

zp_vars.asm

```

        ;;;; VARIABLES

        seg.u ZEROPAGE
        org $0

0000      nmi_lockout      byte
0001      scroll_x_hi      byte
0002      scroll_x_lo      byte
0003      scroll_y         byte
0004      scroll_page      byte
0005      scroll_speed_lo  byte
0006      scroll_speed_hi  byte
0007      scroll_cache_lo  byte
0008      scroll_cache_hi  byte
0009      ppu_mask_emph   byte
000a      bg_tile_default  byte
000b      player_health   byte
000c      player_lives    byte
000d      player_heal_c   byte
        ; if set game in demo mode
000e      attract_true     byte
        ; dest x for demo computer controls
000f      player_demo_x    byte
        ; dest y for same
0010      player_demo_y    byte
0011      player_demo_lr   byte
0012      player_demo_ud   byte

0013      player_x_hi      byte
0014      player_x_lo      byte
0015      player_y_hi      byte
0016      player_y_lo      byte
        ; player_x plus 2 because collision box
0017      player_coll_x    byte
        ; player_y plus 3 because collision box
0018      player_coll_y    byte
0019      player_gun_str    byte
        ; 2 bit value 0..3 translates to 1..4
001a      player_speed     byte
001b      player_paused    byte

001c      player_controls  byte
001d      player_controls_debounced  byte

001e      player_boundless byte
001f      player_autofire_c byte
0020      player_autofire_s byte
0021      player_damage    byte
0022      player_damage_flash byte

0023      game_difficulty  byte

```

```

0024      wtf              byte ; frame counter lo
0025      ftw              byte ; frame counter hi
0026      rng0             byte
0027      rng1             byte
0028      rng2             byte

0029      title_rudy_pos   byte
002a      options_rudy_pos byte
002b      options_music_on byte
002c      options_song_id  byte
002d      options_sound_id byte
002e      options_rudy_color1 byte
002f      options_rudy_color2 byte
0030      scroll_to_counter byte

0031      starfield_state  byte
0032      starfield_column byte
0033      starfield_page    byte
0034      starfield_color0  byte
0035      starfield_color1  byte
0036      starfield_trans_frame byte
0037      starfield_msg_pos_lo byte
0038      starfield_msg_pos_hi byte
0039      starfield_msg_return_lo byte
003a      starfield_msg_return_hi byte

003b      collision_0_x     byte
003c      collision_0_y     byte
003d      collision_0_w     byte
003e      collision_0_h     byte
003f      collision_1_x     byte
0040      collision_1_y     byte
0041      collision_1_w     byte
0042      collision_1_h     byte

0043      dashboard_message byte

0044      timer_frames_1s    byte
0045      timer_frames_10s   byte
0046      timer_seconds_1s   byte
0047      timer_seconds_10s  byte
0048      timer_minutes_1s   byte
0049      timer_minutes_10s  byte

```

004a	state_v0	byte		0076	00	enemy_ram_offset	byte
004b	state_v1	byte		0077	00	enemy_oam_offset	byte
004c	state_v2	byte		0078		enemy_slot_id	byte
004d	state_v3	byte		0079		enemy_dmg_accumulator	byte
004e	state_v4	byte		007a		boss_dmg_handle_true	byte
004f	state_v5	byte		007b		boss_death_happening	byte
0050	state_v6	byte		007c		boss_heart_stars	byte
0051	state_v7	byte		007d		boss_x	byte
0052	state_sprite0	byte		007e		boss_y	byte
0053	state_iframes	byte		007f		boss_eyes_pal	byte
0054	state_iframe_length	byte					
0055	state_render_addr	byte		0080		arc_sequence_id	byte
0056	state_update_addr	byte		0081		arc_sequence_lo	byte
				0082		arc_sequence_hi	byte
				0083		arc_sequence_length	byte
0057	nametable_tile_planter_addr_lo	byte		0084		arctang_velocity_lo	byte
0058	nametable_tile_planter_addr_hi	byte		0085		arctang_velocity_hi	byte
0059	phase_state	byte		0086		temp00	byte
005a	phase_current	byte		0087		temp01	byte
005b	phase_level	byte		0088		temp02	byte
005c	phase_kill_counter	byte		0089		temp03	byte
005d	phase_table_ptr	byte					
005e	phase_spawn_type	byte		008a		pal_fade_c	byte
005f	phase_spawn_long_c	byte		008b	00	pal_fade_offset	byte
0060	phase_arctang_counter	byte		008c		pal_fade_target	byte
0061	phase_spawn_counter	byte		008d		state_fade_in	byte
0062	phase_interval_counter	byte		008e		state_fade_out	byte
0063	phase_end_game	byte		008f		state_from_white	byte
0064	player_frag_counter	byte		0090		pal_uni_bg	byte
0065	player_death_flag	byte		0091		pal_bg_0_1	byte
0066	player_death_counter	byte		0092		pal_bg_0_2	byte
0067	you_dead_counter	byte		0093		pal_bg_0_3	byte
				0094		pal_bg_1_1	byte
0068	powerup_counter	byte		0095		pal_bg_1_2	byte
0069	00 powerup_offset	byte		0096		pal_bg_1_3	byte
				0097		pal_bg_2_1	byte
				0098		pal_bg_2_2	byte
006a	bomb_counter	byte		0099		pal_bg_2_3	byte
				009a		pal_bg_3_1	byte
006b	r_bag_counter	byte		009b		pal_bg_3_2	byte
				009c		pal_bg_3_3	byte
006c	shroom_counter	byte		009d		pal_spr_0_1	byte
006d	shroom_mod	byte		009e		pal_spr_0_2	byte
006e	orbit_shield_speed	byte		009f		pal_spr_0_3	byte
006f	orbit_shield_pos	byte		00a0		pal_spr_1_1	byte
				00a1		pal_spr_1_2	byte
				00a2		pal_spr_1_3	byte
0070	dart_frame_max	byte		00a3		pal_spr_2_1	byte
0071	dart_x_origin	byte		00a4		pal_spr_2_2	byte
0072	dart_y_origin	byte		00a5		pal_spr_2_3	byte
0073	dart_velocity	byte		00a6		pal_spr_3_1	byte
0074	dart_sprite	byte		00a7		pal_spr_3_2	byte
0075	dart_dir_adjust	byte		00a8		pal_spr_3_3	byte

```

00a9      player_color0 byte
00aa      player_color1 byte
00ab      player_color2 byte

00ac      apu_pu1_counter      byte
00ad      apu_pu1_envelope     byte
00ae      apu_pu1_last_hi      byte
           ; !!! this order for x offset
00af      apu_tri_counter      byte
00b0      apu_pu2_counter      byte
00b1      apu_pu2_envelope     byte
00b2      apu_pu2_last_hi      byte
           ; !!! must update apu_env_run if moved
00b3      apu_noi_counter      byte
00b4      apu_noi_envelope     byte

           ; apu variables
00b5      apu_rng0             byte
00b6      apu_rng1             byte
00b7      apu_temp             byte
00b8      sfx_temp00           byte
00b9      sfx_temp01           byte

00ba      sfx_phase_next_counter byte
           ; counters to mask other channel audio
00bb      sfx_pu1_counter      byte
00bc      sfx_pu2_counter      byte
00bd      sfx_noi_counter      byte
           ; table offsets for update subroutines
00be      sfx_pu2_update_type  byte
00bf      sfx_noi_update_type  byte

00c0      audio_song_id        byte
00c1      audio_frame_counter  byte
00c2      audio_root_tone      byte
00c3      audio_pattern_pos    byte
00c4      audio_pattern_num     byte

00 c6      bullet_x_vel = $c6
00c5 00 c7      bullet_y_vel = $c7

```

ppu_lib_and_tables.asm

```

;;;;; SUBROUTINES

ClearRAM: subroutine
c115 a9 00      lda #0 ; A = 0
c117 aa         tax ; X = 0
           .clearRAM
c118 95 00      sta $0,x ; clear $0-$ff
c11a e0 fe      cpx #$fe ; last 2 bytes of stack?
c11c b0 03      bcs .skipStack ; don't clear it
c11e 9d 00 01   sta $100,x ; clear $100-$1fd
           .skipStack
c121 9d 00 02   sta $200,x ; clear $200-$2ff
c124 9d 00 03   sta $300,x ; clear $300-$3ff
c127 9d 00 04   sta $400,x ; clear $400-$4ff
c12a 9d 00 05   sta $500,x ; clear $500-$5ff
c12d 9d 00 06   sta $600,x ; clear $600-$6ff
c130 9d 00 07   sta $700,x ; clear $700-$7ff
c133 e8         inx ; X = X + 1
c134 d0 e2      bne .clearRAM ; loop 256 times
c136 60         rts

do_nothing: subroutine
c137 60         rts

           ; disable PPU drawing and NMI
render_enable:
c138 a9 18      lda #MASK_BG|MASK_SPR
c13a 8d 01 20   sta PPU_MASK
c13d a9 90      lda #CTRL_NMI|CTRL_BG_1000
c13f 8d 00 20   sta PPU_CTRL
c142 60         rts

render_disable:
c143 a9 00      lda #$00
c145 8d 01 20   sta PPU_MASK
c148 8d 00 20   sta PPU_CTRL
c14b 60         rts

           ; wait for VSYNC to start
WaitSync:
c14c 2c 02 20   bit PPU_STATUS
c14f 10 fb      bpl WaitSync
c151 60         rts

;;;;; RANDOM NUMBERS

get_next_random: subroutine
c152 a5 26      lda rng0
c154 20 5a c1    jsr NextRandom
c157 85 26      sta rng0
c159 60         rts

```

```

NextRandom: subroutine
c15a 4a      lsr
c15b 90 02   bcc .NoEor
c15d 49 d4   eor #$d4
.NoEor:
c15f 60      rts
; Get previous random value
PrevRandom subroutine
c160 0a      asl
c161 90 02   bcc .NoEor
c163 49 a9   eor #$a9
.NoEor:
c165 60      rts

shake_8: subroutine
; returns rng in -4..3 range ready for addition
c166 20 52 c1 jsr get_next_random
c169 4a      lsr
c16a 29 07   and #$07
c16c 38      sec
c16d e9 04   sbc #$04
c16f 18      clc
c170 60      rts

```

tile_empty EQM \$03

```

nametables_clear:
; tiles
c171 a2 07   ldx #$07
c173 a0 00   ldy #$00
PPU_SETADDR $2000
c175 a9 20   lda #>$2000
c177 8d 06 20 sta PPU_ADDR
c17a a9 00   lda #<$2000
c17c 8d 06 20 sta PPU_ADDR
c17f a9 03   lda #tile_empty
.page_loop
.byte_loop
c181 8d 07 20 sta PPU_DATA
c184 c8      iny
c185 d0 fa   bne .byte_loop
c187 ca      dex
c188 10 f7   bpl .page_loop
; attributes
c18a a0 00   ldy #0
PPU_SETADDR $23c0
c18c a9 23   lda #>$23c0
c18e 8d 06 20 sta PPU_ADDR
c191 a9 c0   lda #<$23c0
c193 8d 06 20 sta PPU_ADDR
c196 a2 c0   ldx #$c0
.nm1_attr_loop
c198 8c 07 20 sty PPU_DATA
c19b e8      inx

```

```

c19c d0 fa   bne .nm1_attr_loop
PPU_SETADDR $27c0
c19e a9 27   lda #>$27c0
c1a0 8d 06 20 sta PPU_ADDR
c1a3 a9 c0   lda #<$27c0
c1a5 8d 06 20 sta PPU_ADDR
c1a8 a2 c0   ldx #$c0
.nm2_attr_loop
c1aa 8c 07 20 sty PPU_DATA
c1ad e8      inx
c1ae d0 fa   bne .nm2_attr_loop
c1b0 60      rts

sprite_clear:
c1b1 a9 ff   lda #$ff
c1b3 a2 00   ldx #$00
.clear_sprite_ram
c1b5 9d 00 02 sta oam_ram_y,x ; PPU OAM sprite data
c1b8 e8      inx
c1b9 d0 fa   bne .clear_sprite_ram
c1bb 60      rts

ppu_mess_emph: subroutine
c1bc a5 28   lda rng2
c1be 29 e0   and #%11100000
c1c0 85 09   sta ppu_mask_emph
c1c2 60      rts

```

```

; HEX views
get_char_hi: subroutine
; a = value in / hex char out
c1c3 4a      lsr
c1c4 4a      lsr
c1c5 4a      lsr
c1c6 4a      lsr
c1c7 18      clc
c1c8 69 60   adc #char_set_offset
c1ca 60      rts
get_char_lo: subroutine
; a = value in / hex char out
c1cb 29 0f   and #$0f
c1cd 18      clc
c1ce 69 60   adc #char_set_offset
c1d0 60      rts

;;;;; nametable_tile_planter
;;;;; data set address <address> - set 16-bit PPU
address

```

```

MAC nmtpl_setaddr
lda #{1} ; upper byte
sta nametable_tile_planter_addr_hi
lda #{1} ; lower byte
sta nametable_tile_planter_addr_lo
ENDM

```

```

nametable_tile_planter:
c1d1 a0 00 ldy #$00
c1d3 b1 57 lda (nametable_tile_planter_addr_lo),y ; upper byte
        .tileset_loop
c1d5 8d 06 20 sta PPU_ADDR
c1d8 c8 iny
c1d9 b1 57 lda (nametable_tile_planter_addr_lo),y ; lower byte
c1db 8d 06 20 sta PPU_ADDR
c1de c8 iny
        .string_loop
c1df b1 57 lda (nametable_tile_planter_addr_lo),y ; read
string
c1e1 c8 iny
c1e2 c9 00 cmp #$00
c1e4 f0 05 beq .terminate_string
c1e6 8d 07 20 sta PPU_DATA
c1e9 d0 f4 bne .string_loop
        .terminate_string
c1eb b1 57 lda (nametable_tile_planter_addr_lo),y ; look for
ff
c1ed c9 ff cmp #$ff
c1ef d0 e4 bne .tileset_loop
c1f1 60 rts

```

```

dashboard_messages:
; " Y O u D 3 A D "
c1f2 hex 03970360037d03038503630382038503
; " G A M E O V E r "
c202 hex 03870382038d0386038f039403867a03
; "gg ConGraTion gg"
c212 hex 707003847776877a6a93728f8e037070
; " please unpause "
c222 hex 0378746e6a7b6e037d76786a7d7b6e03
; "GoodLuck Hav3fuN"
c232 hex 87777785747d6c8b03886a7e636f7d8e

```

```

difficulty_messages:
; " Weiner Burgh "
c242 hex 03956e72766e7a03837d7a707103
; " Kids' Gloves "
c250 hex 038b726d7b9e038774777e6e7b03
; "Mega Gooderest"
c25e hex 8d6e706a038777776d6e7a6e7b7c
; "A-OK #1 Expert"
c26c hex 82ab8f8b039f61038680786e7a7c

```

```

menu_screen_tile_data:
; "Lange1 Bookbinder's"
c27a hex 2027
c27c hex 8c6a76706e740383777736b72766d6e7a9e7b
c28f byte #$00
; "Please START"
c290 hex 21ea
c292 hex 90746e6a7b6e03039293829193
c29f byte #$00

```

```

; "Much Options"
c2a0 hex 222a
c2a2 hex 8d7d6c7103038f787c7277767b
c2af byte #$00
; numbered version
c2b0 hex 22c3
; " #00 "
;hex 039f606003
; dashes for fun
;hex 03ababab03
; normal pin line
c2b2 hex b1b1b1b1b1
c2b7 byte #$00
; " ax1147 "
c2b8 hex 22d5
c2ba hex 036a806161646703
c2c2 byte #$00

; PUBLISHER BYLINE
c2c3 hex 2301
; MEGA CAT byline
; " Published by MEGA CAT Studios "
;hex 03907d6b74727b716e6d036b81038d8
;hex 687820384829303927c7d6d72777b03
; " Licensed by MEGA CAT STVDIOS "
;hex 03907d6b74727b716e6d036b81038d8
;hex 68782038482930392939485898f9203
; " Published by Mega Cat Studios "
;hex 03907d6b74727b716e6d036b81038d6
;hex e706a03846a7c03927c7d6d72777b03
; " Midwest Gaming Classic #MCS "
;hex 038d726d7f6e7b7c03876a757276700
;hex 384746a7b7b726c03039f8d84920303
; RETROSTAGE byline
; " PCB Mfr. by RetroStage "
;hex 0303030303908483038d6f7a99036b8
;hex 103916e7c7a77927c6a706e03030303
; "Hand Soldered Proto Board Edtn."
;hex 886a766d039277746d6e7a6e6d03907
;hex a777c770383776a7a6d03866d7c7699
; " Manually Soldered Edition "
;hex 0303038d6a767d6a747481039277746
;hex d6e7a6e6d03866d727c727776030303
; " Scrimp Error Box Edition "
;hex 030303926c7a72757803867a7a777a0
;hex 383778003866d727c72777603030303

; default byline
; " Publisher's Line Here "
c2c5 hex 0303030303907d6b74727b716e7a9e7
c2e4 byte b038c72766e03886e7a6e0303030303
#$00

; "(C)opyright MMXXII LoBlast"
c2e5 hex 2323
c2e7 hex 9c849d7778817a7270717c038d8d969
68989038c7783746a7b7c
c301 byte #$00

```

```

; " Options Screen "
c302 hex 2423
c304 hex 038f787c7277767b03926c7a6e6e7603
c314 byte #$00
; "Menu Return"
c315 hex 24ca
c317 hex 8d6e767d0303916e7c7d7a76
c323 byte #$00
; "Song"
c324 hex 250a
c326 hex 92777670
c32a byte #$00
; "Sound"
c32b hex 254a
c32d hex 92777d766d
c332 byte #$00
; "Color 1 " + cfcf
c333 hex 258a
c335 hex 847774777a0361030303cfcf
c341 byte #$00
; "Color 2 " + c0c0
c342 hex 25ca
c344 hex 847774777a0362030303c0c0
c350 byte #$00
; "Difficulties"
c351 hex 260a
c353 hex 85726f6f726c7d747c726e7b
c35f byte #$00
c360 byte #$ff

```

cut_scene_intro_tile_data:

```

; "MY DINGLE"
c361 hex 2171
c363 hex 8d970385898e878c86
c36c byte #$00
; "is very sick and there are"
c36d hex 21a3
c36f hex 727b037e6e7a81037b726c73036a766d037c716e7a6e036a7a6e
c389 byte #$00
; "life saving drugs very far"
c38a hex 21e3
c38c hex 74726f6e037b6a7e727670036d7a7d707b037e6e7a81036f6a7a
c3a6 byte #$00
; "away. Please drive through"
c3a7 hex 2223
c3a9 hex 6a7f6a81990390746e6a7b6e036d7a727e6e037c717a777d7071
c3c3 byte #$00
; "the 13th dimension quickly"
c3c4 hex 2263
c3c6 hex 7c716e0361637c71036d72756e767b72777603797d726c737481
c3e0 byte #$00

```

```

; "so my dingle is saved!"
c3e1 hex 22a3
c3e3 hex 7b77037581036d727670746e03727b037b6a7e6e6d9b
c3f9 byte #$00
c3fa byte #$ff

```

cut_scene_ending_bad_tile_data:

```

; "why did you bring me"
c3fb hex 21c2
c3fd hex 957181036d726d0381777d036b7a72767003756e
c411 byte #$00
; "a dead dingle?"
c412 hex 2210
c414 hex 6a036d6e6a6d036d727670746e9a
c422 byte #$00
; "You took too long!"
c423 hex 2287
c425 hex 97777d037c777773037c777703747776709b
c437 byte #$00
c438 byte #$ff

```

cut_scene_ending_ok_tile_data:

```

; "The dingle is now in a coma."
c439 hex 21c2
c43b hex 93716e036d727670746e03727b0376777f037276036a036c77756a99
c457 byte #$00
; "It may recover."
c458 hex 2229
c45a hex 897c03756a81037a6e6c777e6e7a99
c469 byte #$00
; "You could have been faster."
c46a hex 2283
c46c hex 97777d036c777d746d03716a7e6e036b6e6e76036f6a7b7c6e7a99
c487 byte #$00
c488 byte #$ff

```

cut_scene_ending_good_tile_data:

```

; "what a happy, healthy DINGLE!"
c489 hex 21c2
c48b hex 95716a7c036a03716a78788198716e6a747c71810385898e878c869b
c4a7 byte #$00
; "J O O D J O R B!!"
c4a8 hex 2226
c4aa hex 8a038f038f0385030303038a038f039103839b9b
c4be byte #$00
; "You seem to be so expedient!"
c4bf hex 2282
c4c1 hex 97777d037b6e6e75037c77036b6e037b77036e80786e6d726e767c9b
c4dd byte #$00
c4de byte #$ff

```

```

cut_scene_ending_time_tile_data:
; "Your Time was "
c4df hex      2305
c4e1 hex      97777d7a039372756e037f6a7b03
c4ef byte     #$00
c4f0 byte     #$ff

char_set_offset      EQM $60
char_set_0           EQM $60
char_set_1           EQM $61
char_set_2           EQM $62
char_set_3           EQM $63
char_set_4           EQM $64
char_set_5           EQM $65
char_set_6           EQM $66
char_set_7           EQM $67
char_set_8           EQM $68
char_set_9           EQM $69
char_set_a           EQM $6a
char_set_b           EQM $6b
char_set_c           EQM $6c
char_set_d           EQM $6d
char_set_e           EQM $6e
char_set_f           EQM $6f
char_set_g           EQM $70
char_set_h           EQM $71
char_set_i           EQM $72
char_set_k           EQM $73
char_set_l           EQM $74
char_set_m           EQM $75
char_set_n           EQM $76
char_set_o           EQM $77
char_set_p           EQM $78
char_set_q           EQM $79
char_set_r           EQM $7a
char_set_s           EQM $7b
char_set_t           EQM $7c
char_set_u           EQM $7d
char_set_v           EQM $7e
char_set_w           EQM $7f
char_set_x           EQM $80
char_set_y           EQM $81
char_set_a           EQM $6a
char_set_b           EQM $6b
char_set_c           EQM $6c
char_set_d           EQM $6d
char_set_e           EQM $6e
char_set_g           EQM $70
char_set_h           EQM $71
char_set_i           EQM $72
char_set_j           EQM $8a
char_set_k           EQM $73
char_set_l           EQM $74
char_set_m           EQM $75
char_set_n           EQM $76
char_set_o           EQM $77
char_set_p           EQM $78

```

```

char_set_r           EQM $7a
char_set_s           EQM $7b
char_set_t           EQM $7c
char_set_v           EQM $7e
char_set_w           EQM $7f
char_set_x           EQM $80
char_set_y           EQM $81
char_set_comma       EQM $98
char_set_period      EQM $99
char_set_question    EQM $9a
char_set_bang        EQM $9b
char_set_paren_open  EQM $9c
char_set_paren_close EQM $9d
char_set_apostrophe  EQM $9e
char_set_hash        EQM $9f
char_set_dash        EQM $ab
char_set_colon       EQM $ac
char_set_space       EQM $3

```

```

jump_to_subroutine: subroutine
; a = jump table offset
; caches x and leaves y alone
c4f1 86 89      stx temp03
c4f3 aa         tax
c4f4 bd 03 c5   lda mega_jump_table_to_end_all_jump_tables_lo,x
c4f7 85 86      sta temp00
c4f9 bd 92 c5   lda mega_jump_table_to_end_all_jump_tables_hi,x
c4fc 85 87      sta temp01
c4fe a6 89      ldx temp03
c500 6c 86 00   jmp (temp00)

```

```

; jump tables defined

enemy_update_jump_table_offset      EQM 0
enemy_spawn_jump_table_offset       EQM 27
boss_vamp_state_jump_table_offset   EQM 54
boss_swordtner_state_jump_table_offset EQM 60
powerup_pickup_jump_table_offset    EQM 63
options_screen_state_jump_table_offset EQM 71
phase_handlers_jump_table_offset     EQM 77
sfx_test_jump_table_offset          EQM 82
apu_env_jump_table_offset           EQM 102
sfx_update_jump_table_offset        EQM 108
song_update_jump_table_offset       EQM 115
state_init_jump_table_offset        EQM 124
state_render_jump_table_offset      EQM 129
state_update_jump_table_offset      EQM 134

```



```

mega_jump_table_to_end_all_jump_tables_lo:
enemy_update_jump_table_lo:
c503 byte <enemy_do_nothing
c504 byte <crossbones_cycle
c505 byte <powerups_cycle
c506 byte <enemy_do_nothing
c507 byte <birb_cycle
c508 byte <spark_cycle
c509 byte <zigzag_cycle
c50a byte <skeet_cycle
c50b byte <galger_cycle
c50c byte <bullet_cycle
c50d byte <dart_cycle
c50e byte <boss_vamp_bat_cycle
c50f byte <chomps_cycle
c510 byte <maggs_cycle
c511 byte <muya_cycle
c512 byte <starglasses_cycle
c513 byte <skully_cycle
c514 byte <dumbface_cycle
c515 byte <throber_cycle
c516 byte <ant_cycle
c517 byte <lasso_cycle
c518 byte <ikes_mom_cycle
c519 byte <uzi_cycle
c51a byte <boss_moufs_cycle
c51b byte <boss_vamp_cycle
c51c byte <boss_scarab_cycle
c51d byte <boss_swordtner_cycle
enemy_spawn_jump_table_lo:
c51e byte <do_nothing
c51f byte <do_nothing
c520 byte <do_nothing
c521 byte <do_nothing
c522 byte <birb_spawn
c523 byte <spark_spawn
c524 byte <zigzag_spawn
c525 byte <skeet_spawn
c526 byte <galger_spawn
c527 byte <bullet_spawn
c528 byte <dart_spawn
c529 byte <boss_vamp_bat_spawn
c52a byte <chomps_spawn
c52b byte <maggs_spawn
c52c byte <muya_spawn
c52d byte <starglasses_spawn
c52e byte <skully_spawn
c52f byte <dumbface_spawn
c530 byte <throber_spawn
c531 byte <ant_spawn
c532 byte <lasso_spawn
c533 byte <ikes_mom_spawn
c534 byte <uzi_spawn
c535 byte <boss_moufs_spawn
c536 byte <boss_vamp_spawn
c537 byte <boss_scarab_spawn
c538 byte <boss_swordtner_spawn

```

```

boss_vamp_state_jump_table_lo:
c539 byte <boss_vamp_state_idle_update
c53a byte <boss_vamp_state_suck_bats
c53b byte <boss_vamp_state_shake
c53c byte <boss_vamp_state_lunge
c53d byte <boss_vamp_state_retreat
c53e byte <boss_vamp_state_blow_bats
boss_swordtner_state_jump_table_lo:
c53f byte <boss_swordtner_mode_0
c540 byte <boss_swordtner_mode_shake
c541 byte <boss_swordtner_mode_2
powerup_pickup_jump_table_lo:
c542 byte <powerup_pickup_mask
c543 byte <powerup_pickup_mushroom
c544 byte <powerup_pickup_plus_one
c545 byte <powerup_pickup_bomb
c546 byte <powerup_pickup_r_bag
c547 byte <powerup_pickup_health_25
c548 byte <powerup_pickup_health_50
c549 byte <powerup_pickup_health_100
options_screen_state_jump_table_lo:
c54a byte <options_menu_return
c54b byte <options_screen_song_handler
c54c byte <options_screen_sfx_handler
c54d byte <options_screen_color1_handler
c54e byte <options_screen_color2_handler
c54f byte <options_screen_difficulty_handler
phase_handlers_jump_table_lo:
c550 byte <phase_zero
c551 byte <phase_galger
c552 byte <phase_spawns
c553 byte <phase_spawn_long
c554 byte <phase_boss_fight
sfx_test_jump_table_lo:
c555 byte <sfx_pewpew
c556 byte <sfx_player_damage
c557 byte <sfx_player_death
c558 byte <sfx_enemy_damage
c559 byte <sfx_enemy_death
c55a byte <sfx_powerup_hit
c55b byte <sfx_powerup_bomb
c55c byte <sfx_powerup_mushroom
c55d byte <sfx_powerup_mask
c55e byte <sfx_powerup_lup
c55f byte <sfx_powerup_battery_25
c560 byte <sfx_powerup_battery_50
c561 byte <sfx_powerup_battery_100
c562 byte <sfx_shoot_dart
c563 byte <sfx_shoot_bullet
c564 byte <sfx_rng_chord
c565 byte <sfx_phase_next
c566 byte <sfx_snare
c567 byte <sfx_hat
c568 byte <sfx_ghost_snare

```

```

apu_env_jump_table_lo:
c569 byte    <apu_env_lin_long      ; 0
c56a byte    <apu_env_lin_tiny     ; 2
c56b byte    <apu_env_exp_long     ; 3
c56c byte    <apu_env_exp_short    ; 4
c56d byte    <apu_env_exp_tiny     ; 5
c56e byte    <apu_env_exp_pico     ; 6

sfx_update_jump_table_lo:
c56f byte    <do_nothing           ; 0
c570 byte    <sfx_player_death_update ; 1
c571 byte    <sfx_enemy_death_update ; 2
c572 byte    <sfx_powerup_battery_update ; 3
c573 byte    <sfx_powerup_bomb_update ; 4
c574 byte    <sfx_powerup_lup_update ; 5
c575 byte    <sfx_phase_next_update ; 6

song_update_jump_table_lo:
c576 byte    <do_nothing           ; rng chord
c577 byte    <song_01              ; sick dingle
c578 byte    <song_02              ; in game
c579 byte    <song_03              ; boss intro
c57a byte    <song_04              ; boss fight
c57b byte    <song_05              ; game over
c57c byte    <song_06              ; end bad
c57d byte    <song_07              ; end ok
c57e byte    <song_08              ; end good

state_init_jump_table_lo:
c57f byte    <menu_screens_init    ; 0
c580 byte    <attract_init         ; 1
c581 byte    <game_init            ; 2
c582 byte    <cut_scene_intro_init ; 3
c583 byte    <cut_scene_outro_init ; 4

state_render_jump_table_lo:
c584 byte    <state_render_do_nothing ; 0
c585 byte    <menu_screens_render    ; 1
c586 byte    <starfield_render       ; 3
c587 byte    <state_render_do_nothing ; placeholder
c588 byte    <dashboard_render       ; 5

state_update_jump_table_lo:
c589 byte    <state_update_do_nothing ; 0
c58a byte    <title_screen_update    ; 1
c58b byte    <scrollto_options_update ; 2
c58c byte    <options_screen_update  ; 3
c58d byte    <scrollto_titles_update ; 4
c58e byte    <attract_update         ; 5
c58f byte    <game_update            ; 6
c590 byte    <sandbox_update         ; 7
c591 byte    <cut_scene_update_generic ; 8

```

```

mega_jump_table_to_end_all_jump_tables_hi:
enemy_update_jump_table_hi:
c592 byte    >enemy_do_nothing
c593 byte    >crossbones_cycle
c594 byte    >powerups_cycle
c595 byte    >enemy_do_nothing
c596 byte    >birb_cycle
c597 byte    >spark_cycle
c598 byte    >zigzag_cycle
c599 byte    >skeet_cycle
c59a byte    >galger_cycle
c59b byte    >bullet_cycle
c59c byte    >dart_cycle
c59d byte    >boss_vamp_bat_cycle
c59e byte    >chomps_cycle
c59f byte    >maggs_cycle
c5a0 byte    >muya_cycle
c5a1 byte    >starglasses_cycle
c5a2 byte    >skully_cycle
c5a3 byte    >dumbface_cycle
c5a4 byte    >throber_cycle
c5a5 byte    >ant_cycle
c5a6 byte    >lasso_cycle
c5a7 byte    >ikes_mom_cycle
c5a8 byte    >uzi_cycle
c5a9 byte    >boss_moufs_cycle
c5aa byte    >boss_vamp_cycle
c5ab byte    >boss_scarab_cycle
c5ac byte    >boss_swordtner_cycle

enemy_spawn_jump_table_hi:
c5ad byte    >do_nothing
c5ae byte    >do_nothing
c5af byte    >do_nothing
c5b0 byte    >do_nothing
c5b1 byte    >birb_spawn
c5b2 byte    >spark_spawn
c5b3 byte    >zigzag_spawn
c5b4 byte    >skeet_spawn
c5b5 byte    >galger_spawn
c5b6 byte    >bullet_spawn
c5b7 byte    >dart_spawn
c5b8 byte    >boss_vamp_bat_spawn
c5b9 byte    >chomps_spawn
c5ba byte    >maggs_spawn
c5bb byte    >muya_spawn
c5bc byte    >starglasses_spawn
c5bd byte    >skully_spawn
c5be byte    >dumbface_spawn
c5bf byte    >throber_spawn
c5c0 byte    >ant_spawn
c5c1 byte    >lasso_spawn
c5c2 byte    >ikes_mom_spawn
c5c3 byte    >uzi_spawn
c5c4 byte    >boss_moufs_spawn
c5c5 byte    >boss_vamp_spawn
c5c6 byte    >boss_scarab_spawn
c5c7 byte    >boss_swordtner_spawn

```

```

boss_vamp_state_jump_table_hi:
c5c8 byte    >boss_vamp_state_idle_update
c5c9 byte    >boss_vamp_state_suck_bats
c5ca byte    >boss_vamp_state_shake
c5cb byte    >boss_vamp_state_lunge
c5cc byte    >boss_vamp_state_retreat
c5cd byte    >boss_vamp_state_blow_bats
boss_swordtner_state_jump_table_hi:
c5ce byte    >boss_swordtner_mode_0
c5cf byte    >boss_swordtner_mode_shake
c5d0 byte    >boss_swordtner_mode_2

powerup_pickup_jump_table_hi:
c5d1 byte    >powerup_pickup_mask
c5d2 byte    >powerup_pickup_mushroom
c5d3 byte    >powerup_pickup_plus_one
c5d4 byte    >powerup_pickup_bomb
c5d5 byte    >powerup_pickup_r_bag
c5d6 byte    >powerup_pickup_health_25
c5d7 byte    >powerup_pickup_health_50
c5d8 byte    >powerup_pickup_health_100
options_screen_state_jump_table_hi:
c5d9 byte    >options_menu_return
c5da byte    >options_screen_song_handler
c5db byte    >options_screen_sfx_handler
c5dc byte    >options_screen_color1_handler
c5dd byte    >options_screen_color2_handler
c5de byte    >options_screen_difficulty_handler
phase_handlers_jump_table_hi:
c5df byte    >phase_zero
c5e0 byte    >phase_galger
c5e1 byte    >phase_spawns
c5e2 byte    >phase_spawn_long
c5e3 byte    >phase_boss_fight
sfx_test_jump_table_hi:
c5e4 byte    >sfx_pewpew
c5e5 byte    >sfx_player_damage
c5e6 byte    >sfx_player_death
c5e7 byte    >sfx_enemy_damage
c5e8 byte    >sfx_enemy_death
c5e9 byte    >sfx_powerup_hit
c5ea byte    >sfx_powerup_bomb
c5eb byte    >sfx_powerup_mushroom
c5ec byte    >sfx_powerup_mask
c5ed byte    >sfx_powerup_lup
c5ee byte    >sfx_powerup_battery_25
c5ef byte    >sfx_powerup_battery_50
c5f0 byte    >sfx_powerup_battery_100
c5f1 byte    >sfx_shoot_dart
c5f2 byte    >sfx_shoot_bullet
c5f3 byte    >sfx_rng_chord
c5f4 byte    >sfx_phase_next
c5f5 byte    >sfx_snare
c5f6 byte    >sfx_hat
c5f7 byte    >sfx_ghost_snare

```

```

apu_env_jump_table_hi:
c5f8 byte    >apu_env_lin_long          ; 0
c5f9 byte    >apu_env_lin_tiny         ; 2
c5fa byte    >apu_env_exp_long         ; 3
c5fb byte    >apu_env_exp_short        ; 4
c5fc byte    >apu_env_exp_tiny         ; 5
c5fd byte    >apu_env_exp_pico         ; 6
sfx_update_jump_table_hi:
c5fe byte    >do_nothing                ; 0
c5ff byte    >sfx_player_death_update  ; 1
c600 byte    >sfx_enemy_death_update   ; 2
c601 byte    >sfx_powerup_battery_update ; 3
c602 byte    >sfx_powerup_bomb_update   ; 4
c603 byte    >sfx_powerup_lup_update    ; 5
c604 byte    >sfx_phase_next_update     ; 6
song_update_jump_table_hi:
c605 byte    >do_nothing                ; rng chord
c606 byte    >song_01                   ; sick dingle
c607 byte    >song_02                   ; in game
c608 byte    >song_03                   ; boss intro
c609 byte    >song_04                   ; boss fight
c60a byte    >song_05                   ; game over
c60b byte    >song_06                   ; end bad
c60c byte    >song_07                   ; end ok
c60d byte    >song_08                   ; end good
state_init_jump_table_hi:
c60e byte    >menu_screens_init         ; 0
c60f byte    >attract_init              ; 1
c610 byte    >game_init                 ; 2
c611 byte    >cut_scene_intro_init      ; 3
c612 byte    >cut_scene_outro_init      ; 4
state_render_jump_table_hi:
c613 byte    >state_render_do_nothing   ; 0
c614 byte    >menu_screens_render       ; 1
c615 byte    >starfield_render          ; 3
c616 byte    >state_render_do_nothing   ; placeholder
c617 byte    >dashboard_render          ; 5
state_update_jump_table_hi:
c618 byte    >state_update_do_nothing   ; 0
c619 byte    >title_screen_update       ; 1
c61a byte    >scrollto_options_update   ; 2
c61b byte    >options_screen_update     ; 3
c61c byte    >scrollto_titles_update    ; 4
c61d byte    >attract_update            ; 5
c61e byte    >game_update               ; 6
c61f byte    >sandbox_update            ; 7
c620 byte    >cut_scene_update_generic  ; 8

```

state__handler.asm

```
; === STATE MACHINE ===

; nmi order of operations :
; jmp() to state renderer
; most likely starfield mode
; jmp to dashboard
; jmp back
; OAM DMA
; palette transfer
; set scroll pos

; jmp() to state logic
; jmp back

; check flag sprite0 and if set :
; wait for sprite 0
; set scroll pos
; (palette hackery?)

; apu manage
; starfield manage jmp()?
; palette manage

; init functions order of operations :
; disable rendering and nmi
; (halt audio?)
; update vram
; wait for vsync/vblank
; reenale rendering and nmi

state_clear:
c621 a2 07    ldx #$07
c623 a9 00    lda #$00
        .loop
c625 95 4a    sta state_v0,x
c627 ca      dex
c628 10 fb    bpl .loop
c62a 60      rts

state_render_do_nothing:
c62b 4c 7d c0    jmp state_render_done

state_update_do_nothing:
c62e 4c a7 c0    jmp state_update_done

state_sprite0_enable:
c631 a9 01    lda #$01
c633 8d 03 02    sta oam_ram_x
c636 a9 b6    lda #sprite_0_y
c638 8d 00 02    sta oam_ram_y
c63b a9 ff    lda #$ff
c63d 8d 01 02    sta oam_ram_spr
c640 a9 20    lda #$20
c642 8d 02 02    sta oam_ram_att
c645 a9 ff    lda #$ff
c647 85 52    sta state_sprite0
c649 60      rts

state_sprite0_disable:
c64a a9 ff    lda #$ff
c64c 8d 00 02    sta oam_ram_y
c64f a9 00    lda #$00
c651 85 52    sta state_sprite0
c653 60      rts
```

state_menu_screens.asm

```

; zp vars

;title_rudy_pos EQU $181

;options_rudy_pos EQU $182
;options_music_on EQU $183
;options_song_id EQU $184
;options_sound_id EQU $185
;options_rudy_color1 EQU $186
;options_rudy_color2 EQU $187

;scroll_to_counter EQU $188

menu_screens_init: subroutine

c654 20 43 c1    jsr render_disable
c657 20 71 c1    jsr nametables_clear
c65a 20 b1 c1    jsr sprite_clear
c65d 20 4a c6    jsr state_sprite0_disable

c660 a9 82      lda #state_render_jump_table_offset+1
c662 85 55      sta state_render_addr
c664 a9 87      lda #state_update_jump_table_offset+1
c666 85 56      sta state_update_addr
c668 20 21 c6    jsr state_clear ; a = 0
c66b 85 01      sta scroll_x_hi
c66d 85 04      sta scroll_page
c66f 85 22      sta player_damage_flash ; no flash on fade-in
c671 85 09      sta ppu_mask_emph ; reset mask
c673 20 95 d6    jsr timer_reset
c676 20 d5 c7    jsr title_screen_set_rudy_y
; setup colors
c679 a2 12      ldx #18
c67b a0 00      ldy #0
c67d 20 cf d1    jsr palette_load
c680 a9 0f      lda #$0f
c682 85 90      sta pal_uni_bg
; setup everything else
c684 20 17 f2    jsr player_game_reset
c687 20 7c fb    jsr song_stop
c68a 20 65 cf    jsr dashboard_init
c68d 20 c2 d0    jsr dashboard_update
c690 20 75 d0    jsr dashboard_render

; copylines pal att
PPU_SETADDR $23c0
c693 a9 23      lda #>$23c0
c695 8d 06 20    sta PPU_ADDR
c698 a9 c0      lda #<$23c0
c69a 8d 06 20    sta PPU_ADDR
c69d a2 37      ldx #$37
; copylines_pal_loop
c69f 8a          txa
c6a0 c9 30      cmp #$30

```

```

c6a2 b0 05      bcs .bookbinder_gray
c6a4 8a          txa
c6a5 29 30      and #$30
c6a7 d0 04      bne .on8
;bookbinder_gray
c6a9 a9 0a      lda #00001010
c6ab d0 02      bne .plot_pal
.on8
c6ad a9 00      lda #$00000000
;plot_pal
c6af 8d 07 20    sta PPU_DATA
c6b2 ca          dex
c6b3 10 ea      bpl .copylines_pal_loop

; G u n T n e R

; BIG TITLE
PPU_SETADDR $2060
c6b5 a9 20      lda #>$2060
c6b7 8d 06 20    sta PPU_ADDR
c6ba a9 60      lda #<$2060
c6bc 8d 06 20    sta PPU_ADDR
c6bf a0 00      ldy #$00
big_title_loop:
c6c1 b9 f1 c7    lda guntner_title_name_table,y
c6c4 8d 07 20    sta PPU_DATA
c6c7 c8          iny
c6c8 d0 f7      bne big_title_loop

; pinline on title screen
PPU_SETADDR $22c0
c6ca a9 22      lda #>$22c0
c6cc 8d 06 20    sta PPU_ADDR
c6cf a9 c0      lda #<$22c0
c6d1 8d 06 20    sta PPU_ADDR
c6d4 a9 b1      lda #$b1
c6d6 a0 20      ldy #$20
.pinline_footer
c6d8 8d 07 20    sta PPU_DATA
c6db 88          dey
c6dc d0 fa      bne .pinline_footer

; pinline on options screen
PPU_SETADDR $2420
c6de a9 24      lda #>$2420
c6e0 8d 06 20    sta PPU_ADDR
c6e3 a9 20      lda #<$2420
c6e5 8d 06 20    sta PPU_ADDR
c6e8 a9 b1      lda #$b1
c6ea a0 20      ldy #$20
.pinline_header
c6ec 8d 07 20    sta PPU_DATA
c6ef 88          dey
c6f0 d0 fa      bne .pinline_header

```

```

; various stuff on screen
NMTP_SETADDR menu_screen_tile_data
c6f2 a9 c2    lda #>menu_screen_tile_data
c6f4 85 58    sta nametable_tile_planter_addr_hi
c6f6 a9 7a    lda #<menu_screen_tile_data
c6f8 85 57    sta nametable_tile_planter_addr_lo
c6fa 20 d1 c1 jsr nametable_tile_planter

; set rudy stuff
c6fd 20 af c7 jsr menus_position_rudy
c700 20 98 f2 jsr set_player_sprite

c703 20 4e d2 jsr palette_fade_in_init
c706 20 76 fa jsr sfx_rng_chord

; turn ppu back on
c709 20 38 c1 jsr render_enable
c70c 60      rts

; VBLANK RENDER HANDLER

menu_screens_render:
; pretty much just for options screen
; show song id
PPU_SETADDR $2514
c70d a9 25    lda #>$2514
c70f 8d 06 20 sta PPU_ADDR
c712 a9 14    lda #<$2514
c714 8d 06 20 sta PPU_ADDR
c717 a9 60    lda #char_set_offset
c719 8d 07 20 sta PPU_DATA
c71c a5 2c    lda options_song_id
c71e 20 cb c1 jsr get_char_lo
c721 8d 07 20 sta PPU_DATA
; show sound id
PPU_SETADDR $2554
c724 a9 25    lda #>$2554
c726 8d 06 20 sta PPU_ADDR
c729 a9 54    lda #<$2554
c72b 8d 06 20 sta PPU_ADDR
c72e a5 2d    lda options_sound_id
c730 20 c3 c1 jsr get_char_hi
c733 8d 07 20 sta PPU_DATA
c736 a5 2d    lda options_sound_id
c738 20 cb c1 jsr get_char_lo
c73b 8d 07 20 sta PPU_DATA
; show difficulty
PPU_SETADDR $2649
c73e a9 26    lda #>$2649
c740 8d 06 20 sta PPU_ADDR
c743 a9 49    lda #<$2649
c745 8d 06 20 sta PPU_ADDR
c748 a9 00    lda #0
c74a 18      clc
c74b a4 23    ldy game_difficulty
c74d f0 05    beq .diff_offset_found

```

```

.diff_offset_finder
c74f 69 0e    adc #14
c751 88      dey
c752 d0 fb    bne .diff_offset_finder
.diff_offset_found
c754 a8      tay
c755 a2 00    ldx #0
.diff_place_loop
c757 b9 42 c2 lda difficulty_messages,y
c75a 8d 07 20 sta PPU_DATA
c75d e8      inx
c75e c8      iny
c75f e0 0e    cpx #14
c761 d0 f4    bne .diff_place_loop
c763 4c 7d c0 jmp state_render_done

; SCROLL HANDLERS

scrollto_options_update: subroutine
c766 a5 30    lda scroll_to_counter
c768 18      clc
c769 69 04    adc #$04
c76b 85 30    sta scroll_to_counter
c76d aa      tax
c76e bd 00 fe lda sine_table,x
c771 85 01    sta scroll_x_hi
c773 e0 40    cpx #$40
c775 d0 0c    bne .done
; setup options
c777 a9 89    lda #state_update_jump_table_offset+3
c779 85 56    sta state_update_addr
c77b a9 01    lda #1
c77d 85 04    sta scroll_page
c77f a9 00    lda #$00
c781 85 01    sta scroll_x_hi
; jmp options_screen_init
.done
c783 20 af c7 jsr menus_position_rudy
c786 4c a7 c0 jmp state_update_done

scrollto_titles_update: subroutine
c789 a9 00    lda #$00
c78b 85 04    sta scroll_page
c78d a5 30    lda scroll_to_counter
c78f 38      sec
c790 e9 04    sbc #$04
c792 85 30    sta scroll_to_counter
c794 aa      tax
c795 bd 00 fe lda sine_table,x
c798 85 01    sta scroll_x_hi

```

```

c79a e0 c0      cpx #$c0
c79c d0 0b      bne .done
                ; setup options
c79e a9 87      lda #state_update_jump_table_offset+1
c7a0 85 56      sta state_update_addr
c7a2 a9 00      lda #$00
c7a4 85 04      sta scroll_page
c7a6 20 95 d6   jsr timer_reset
                .done
c7a9 20 af c7   jsr menus_position_rudy
c7ac 4c a7 c0   jmp state_update_done

```

```

                menus_position_rudy: subroutine
c7af a5 01      lda scroll_x_hi
c7b1 d0 01      bne .do_eeet
c7b3 60         rts
                .do_eeet
c7b4 c9 48      cmp #$48
c7b6 b0 0b      bcs .coming_from_right
                .going_to_the_left
c7b8 20 d5 c7   jsr title_screen_set_rudy_y
c7bb a9 3a      lda #$3a
c7bd 38         sec
c7be e5 01      sbc scroll_x_hi
c7c0 4c cb c7   jmp .plot_tiles
                .coming_from_right
c7c3 20 e3 c7   jsr options_screen_set_rudy_y
c7c6 a9 38      lda #$38
c7c8 38         sec
c7c9 e5 01      sbc scroll_x_hi
                .plot_tiles
c7cb 8d 07 02   sta oam_ram_rudy+3
c7ce 18         clc
c7cf 69 08      adc #$08
c7d1 8d 0b 02   sta oam_ram_rudy+7
c7d4 60         rts

```

```

                title_screen_set_rudy_y: subroutine
c7d5 a5 29      lda title_rudy_pos
c7d7 0a         asl
c7d8 0a         asl
c7d9 0a         asl
c7da 0a         asl
c7db 18         clc
c7dc 69 7f      adc #$7f
c7de 85 15      sta player_y_hi
c7e0 4c 98 f2   jmp set_player_sprite

```

```

                options_screen_set_rudy_y: subroutine
c7e3 a5 2a      lda options_rudy_pos
c7e5 0a         asl
c7e6 0a         asl
c7e7 0a         asl
c7e8 0a         asl
c7e9 18         clc
c7ea 69 37      adc #$37
c7ec 85 15      sta player_y_hi
c7ee 4c 98 f2   jmp set_player_sprite

```

; STRINGS AND NAMETABLES

```

                guntner_title_name_table: ; 256 bytes
c7f1 byte      $03,$03,$03,$c1,$c2,$c3,$c4,$c2,
                $c2,$c2,$c2,$c2,$c2,$c2,$c2,$c2
c801 byte      $c2,$c2,$c2,$c2,$c2,$c2,$c2,$c2,
                $c2,$c2,$c2,$c2,$c5,$03,$03,$03
c811 byte      $03,$03,$03,$c6,$c7,$c8,$c9,$ca,
                $ca,$ca,$ca,$ca,$ca,$ca,$ca,$cb
c821 byte      $cc,$ca,$ca,$ca,$ca,$ca,$ca,$ca,
                $ca,$cb,$cc,$cd,$ce,$03,$03,$03
c831 byte      $03,$03,$03,$cf,$d0,$03,$d1,$d2,
                $d3,$d4,$d5,$d6,$d7,$d8,$d9,$d2
c841 byte      $d3,$d6,$d7,$d8,$d9,$da,$db,$dc,
                $dd,$d2,$d3,$de,$cf,$03,$03,$03
c851 byte      $03,$03,$03,$cf,$df,$e0,$e1,$d2,
                $d3,$e2,$d5,$e3,$e4,$e5,$d5,$d2
c861 byte      $d3,$e3,$e4,$e5,$d5,$d2,$e6,$e7,
                $d5,$d2,$e8,$e9,$ea,$03,$03,$03
c871 byte      $03,$03,$03,$cf,$d0,$e3,$d5,$d2,
                $d3,$e2,$d5,$e3,$d5,$e2,$d5,$d2
c881 byte      $d3,$e3,$d5,$e2,$d5,$d2,$eb,$ec,
                $ed,$d2,$d3,$e2,$ee,$03,$03,$03
c891 byte      $03,$03,$03,$cf,$d0,$e3,$d5,$ef,
                $f0,$f1,$f2,$f3,$ce,$f4,$d5,$d2
c8a1 byte      $d3,$f3,$ce,$f4,$d5,$ef,$f0,$f1,
                $f2,$d2,$f5,$f6,$cf,$03,$03,$03
c8b1 byte      $03,$03,$03,$cf,$f7,$f8,$d5,$03,
                $03,$03,$03,$03,$03,$03,$03,$d2
c8c1 byte      $d3,$03,$03,$03,$03,$03,$03,$03,
                $03,$03,$03,$e2,$cf,$03,$03,$03
c8d1 byte      $03,$03,$03,$f9,$fa,$fa,$fb,$03,
                $03,$03,$03,$03,$03,$03,$fc,$fa
c8e1 byte      $fa,$fd,$03,$03,$03,$03,$03,$03,
                $03,$03,$03,$fe,$fa,$fd,$03,$03

```

```

                title_screen_super_secret_code:
c8f1 byte      $08,$08,$04,$04,$02,$01,$02,$01,$40,$80

```



```

; TITLE SCREEN UPDATES

title_screen_update: subroutine
; set rudy x
c8fb a9 38    lda #$38
c8fd 85 13    sta player_x_hi

; number of seconds before demo
c8ff a5 46    lda timer_seconds_1s
; add #$30 because timer based on numerical tiles
c901 c9 63    cmp #char_set_0+3
c903 d0 05    bne .sit_and_wait

.start_demo
c905 a9 7d    lda #state_init_jump_table_offset+1
c907 20 82 d2 jsr palette_fade_out_init

.sit_and_wait
; check for super secret code
c90a a4 4a    ldy state_v0
c90c 30 24    bmi .super_secret_code_done
c90e a5 1c    lda player_controls
c910 29 cf    and #$cf
c912 c5 4c    cmp state_v2
c914 f0 1c    beq .super_secret_code_done
c916 85 4c    sta state_v2
c918 c9 00    cmp #$00
c91a f0 16    beq .super_secret_code_done
c91c d9 f1 c8 cmp title_screen_super_secret_code,y
c91f f0 06    beq .super_secret_code_next
c921 a9 ff    lda #$ff
c923 85 4a    sta state_v0
c925 30 0b    bmi .super_secret_code_done

.super_secret_code_next
c927 c8      iny
c928 84 4a    sty state_v0
c92a c0 0a    cpy #$0a
c92c 90 04    bcc .super_secret_code_done
c92e a9 01    lda #$01
c930 85 4b    sta state_v1

.super_secret_code_done
c932 a9 0f    lda #BUTTON_UP|BUTTON_DOWN|BUTTON_LEFT|BUTTON_RIGHT
c934 25 1d    and player_controls_debounced
c936 f0 0d    beq .dont_change_pos
c938 e6 29    inc title_rudy_pos
c93a a9 01    lda #%00000001
c93c 25 29    and title_rudy_pos
c93e 85 29    sta title_rudy_pos
c940 20 95 d6 jsr timer_reset
c943 d0 1e    bne .do_nothing

.dont_change_pos
c945 a9 10    lda #BUTTON_START
c947 a4 4a    ldy state_v0
c949 c0 ff    cpy #$ff
c94b d0 02    bne .button_check
c94d 09 c0    ora #BUTTON_B|BUTTON_A

.button_check
c94f 25 1d    and player_controls_debounced
c951 f0 10    beq .do_nothing
c953 a5 29    lda title_rudy_pos

```

```

c955 c9 01    cmp #$01
c957 f0 06    beq .goto_options

.start_game:
c959 20 76 c9 jsr menu_start_game
c95c 4c 63 c9 jmp .do_nothing

.goto_options
c95f a9 88    lda #state_update_jump_table_offset+2
c961 85 56    sta state_update_addr

.do_nothing
c963 20 d5 c7 jsr title_screen_set_rudy_y
; except animate that color tho
; increase color
c966 a5 24    lda wtf
c968 29 0f    and #$f
c96a d0 07    bne .dont_next_title_color
c96c a5 91    lda pal_bg_0_1
c96e 20 e0 d1 jsr palette_next_rainbow_color
c971 85 91    sta pal_bg_0_1

.dont_next_title_color
c973 4c a7 c0 jmp state_update_done

menu_start_game: subroutine
c976 a5 1c    lda player_controls
c978 29 20    and #BUTTON_SELECT
c97a f0 02    beq .not_cheater_boundless
c97c e6 1e    inc player_boundless

.not_cheater_boundless
;; disable start_d so game doesn't instantly pause
c97e a9 00    lda #$00
c980 85 1c    sta player_controls
c982 85 1d    sta player_controls_debounced
; init cut scene 00
c984 a9 7f    lda #state_init_jump_table_offset+3
c986 20 82 d2 jsr palette_fade_out_init
c989 60      rts

; OPTION SCREEN UPDATES

options_screen_update: subroutine
c98a a5 6c    lda shroom_counter
c98c d0 03    bne .skip_player_colors
c98e 20 48 f2 jsr player_update_colors

.skip_player_colors
c991 a5 1c    lda player_controls
c993 29 10    and #BUTTON_START
c995 f0 0a    beq .dont_start_game
c997 a9 c0    lda #$c0 ; set scroll to title screen
c999 85 30    sta scroll_to_counter

.start_game:
c99b 20 76 c9 jsr menu_start_game
c99e 4c a7 c0 jmp state_update_done

```



```

.dont_start_game
c9a1 20 e3 c7 jsr options_screen_set_rudy_y
; check if option changes
c9a4 a5 1d lda player_controls_debounced
c9a6 29 04 and #BUTTON_DOWN
c9a8 f0 02 beq .dont_change_option_down
c9aa e6 2a inc options_rudy_pos
.dont_change_option_down
c9ac a5 1d lda player_controls_debounced
c9ae 29 08 and #BUTTON_UP
c9b0 f0 02 beq .dont_change_option_up
c9b2 c6 2a dec options_rudy_pos
.dont_change_option_up
; check option pos is in range
c9b4 a5 2a lda options_rudy_pos
c9b6 c9 ff cmp #$ff ; min value - 1
c9b8 d0 02 bne .dont_wrap_up
c9ba a9 05 lda #$05
.dont_wrap_up
c9bc c9 06 cmp #$06 ; max value + 1
c9be d0 02 bne .dont_wrap_down
c9c0 a9 00 lda #$00
.dont_wrap_down
c9c2 85 2a sta options_rudy_pos
; which option handler?
options_screen_handler
c9c4 18 clc
c9c5 69 47 adc #options_screen_state_jump_table_offset
c9c7 4c f1 c4 jmp jump_to_subroutine

```

```

options_menu_return: subroutine
c9ca a5 1d lda player_controls_debounced
c9cc 29 c0 and #BUTTON_B|BUTTON_A
c9ce f0 04 beq .do_nothing
c9d0 a9 8a lda #state_update_jump_table_offset+4
c9d2 85 56 sta state_update_addr
.do_nothing
c9d4 4c a7 c0 jmp state_update_done

```

```

options_min_max:
c9d7 byte $00, $08 ; song
c9d9 byte $00, $13 ; sfx
c9db byte $10, $1c ; color1
c9dd byte $20, $2c ; color2
c9df byte $00, $03 ; difficulty

```

```

options_handle_change: subroutine
; a = current value
; y = table offset
c9e1 85 86 sta temp00
c9e3 a5 1d lda player_controls_debounced
c9e5 29 01 and #BUTTON_RIGHT
c9e7 f0 0e beq .dont_increase
c9e9 e6 86 inc temp00
c9eb b9 d8 c9 lda options_min_max+1,y
c9ee c5 86 cmp temp00
c9f0 b0 1b bcs .done
c9f2 b9 d7 c9 lda options_min_max,y
c9f5 85 86 sta temp00
.dont_increase
c9f7 a5 1d lda player_controls_debounced
c9f9 29 02 and #BUTTON_LEFT
c9fb f0 10 beq .dont_decrease
c9fd c6 86 dec temp00
c9ff 30 07 bmi .go_max
ca01 a5 86 lda temp00
ca03 d9 d7 c9 cmp options_min_max,y
ca06 b0 05 bcs .done
.go_max
ca08 b9 d8 c9 lda options_min_max+1,y
ca0b 85 86 sta temp00
.dont_decrease
.done
ca0d a5 86 lda temp00
ca0f 60 rts

```

```

options_screen_song_handler: subroutine
ca10 a5 2c lda options_song_id
ca12 a0 00 ldy #0
ca14 20 e1 c9 jsr options_handle_change
ca17 85 2c sta options_song_id
; trigger song with button
ca19 a5 1d lda player_controls_debounced
ca1b 29 40 and #BUTTON_B
ca1d f0 03 beq .no_b
ca1f 20 7c fb jsr song_stop
.no_b
ca22 a5 1d lda player_controls_debounced
ca24 29 80 and #BUTTON_A
ca26 f0 05 beq .no_a
ca28 a5 2c lda options_song_id
ca2a 20 4b fb jsr song_start
.no_a
ca2d 4c a7 c0 jmp state_update_done

```

```

options_screen_sfx_handler: subroutine
ca30  a5 2d      lda options_sound_id
ca32  a0 02      ldy #2
ca34  20 e1 c9   jsr options_handle_change
ca37  85 2d      sta options_sound_id
           ; trigger sound with button
ca39  a5 1d      lda player_controls_debounced
ca3b  29 c0      and #BUTTON_B|BUTTON_A
ca3d  d0 03      bne .trigger_sound
ca3f  4c a7 c0   jmp state_update_done
.trigger_sound
ca42  a5 2d      lda options_sound_id
ca44  20 46 f8   jsr sfx_test_delegator
ca47  4c a7 c0   jmp state_update_done

options_screen_color1_handler: subroutine
ca4a  a5 a9      lda player_color0
ca4c  a0 04      ldy #4
ca4e  20 e1 c9   jsr options_handle_change
ca51  85 a9      sta player_color0
ca53  4c a7 c0   jmp state_update_done

options_screen_color2_handler: subroutine
ca56  a5 aa      lda player_color1
ca58  a0 06      ldy #6
ca5a  20 e1 c9   jsr options_handle_change
ca5d  85 aa      sta player_color1
ca5f  4c a7 c0   jmp state_update_done

options_screen_difficulty_handler: subroutine
ca62  a5 23      lda game_difficulty
ca64  a0 08      ldy #8
ca66  20 e1 c9   jsr options_handle_change
ca69  85 23      sta game_difficulty
ca6b  4c a7 c0   jmp state_update_done

```

state_cutsscenes.asm

```

cut_scene_update_generic: subroutine
ca6e  a5 63      lda phase_end_game
ca70  f0 0a      beq .start_game
ca72  a5 1d      lda player_controls_debounced
ca74  29 30      and #BUTTON_START|BUTTON_SELECT
ca76  f0 12      beq .do_nothing
           ; game complete go back to title screen
ca78  a9 7c      lda #state_init_jump_table_offset+0
ca7a  d0 08      bne .trigger_fadeout
.start_game
ca7c  a5 1d      lda player_controls_debounced
ca7e  29 d0      and #BUTTON_START|BUTTON_B|BUTTON_A
ca80  f0 08      beq .do_nothing
ca82  a9 7e      lda #state_init_jump_table_offset+2
.trigger_fadeout
ca84  20 82 d2   jsr palette_fade_out_init
ca87  20 7c fb   jsr song_stop
.do_nothing
ca8a  4c a7 c0   jmp state_update_done

cut_scene_intro_init: subroutine
           ; zero everything out
ca8d  20 43 c1   jsr render_disable
ca90  20 b1 c1   jsr sprite_clear
ca93  20 71 c1   jsr nametables_clear
           ; colors
ca96  a2 0c      ldx #12
ca98  a0 00      ldy #0
ca9a  20 cf d1   jsr palette_load
ca9d  20 cf d1   jsr palette_load
           ; Distressed Alien
          PPU_SETADDR #$2086
caa0  a9 20      lda #>#$2086
caa2  8d 06 20   sta PPU_ADDR
caa5  a9 86      lda #<#$2086
caa7  8d 06 20   sta PPU_ADDR
caa9  20 fa ca   jsr cut_scene_alien_main_draw
           ; alien attributes
caad  a2 55      ldx #%01010101
          PPU_SETADDR #$23c9
caaf  a9 23      lda #>#$23c9
cab1  8d 06 20   sta PPU_ADDR
cab4  a9 c9      lda #<#$23c9
cab6  8d 06 20   sta PPU_ADDR
cab9  8e 07 20   stx PPU_DATA
cab9  8e 07 20   stx PPU_DATA
cabf  8e 07 20   stx PPU_DATA
          PPU_SETADDR #$23d1
cac2  a9 23      lda #>#$23d1
cac4  8d 06 20   sta PPU_ADDR
cac7  a9 d1      lda #<#$23d1
cac9  8d 06 20   sta PPU_ADDR
cacc  8e 07 20   stx PPU_DATA

```

```

cacf 8e 07 20    stx PPU_DATA
cad2 8e 07 20    stx PPU_DATA
                ; put text on screen
                NMTP_SETADDR cut_scene_intro_tile_data
cad5 a9 c3       lda #>cut_scene_intro_tile_data
cad7 85 58       sta nametable_tile_planter_addr_hi
cad9 a9 61       lda #<cut_scene_intro_tile_data
cadd 85 57       sta nametable_tile_planter_addr_lo
cadd 20 d1 c1    jsr nametable_tile_planter
                ; setup everything else
cae0 a9 00       lda #$00
cae2 85 04       sta scroll_page
cae4 85 01       sta scroll_x_hi
cae6 a9 81       lda #state_render_jump_table_offset
cae8 85 55       sta state_render_addr
caea a9 8e       lda #state_update_jump_table_offset+8
caec 85 56       sta state_update_addr
caee 20 38 c1    jsr render_enable
caf1 20 4e d2    jsr palette_fade_in_init
caf4 a9 01       lda #1
caf6 20 4b fb    jsr song_start
caf9 60          rts

cut_scene_alien_main_draw: subroutine
cafa a9 00       lda #0
cafc 85 86       sta temp00 ; pattern tile counter
cafe a2 07       ldx #7
cb00 a0 07       .alien_tile_loop
                ldy #7
                .alien_tile_row_loop
cb02 a5 86       lda temp00
cb04 8d 07 20    sta PPU_DATA
cb07 e6 86       inc temp00
cb09 88         dey
cb0a 10 f6       bpl .alien_tile_row_loop
                ; setup empty tile fill
cb0c a0 17       ldy #23
cb0e a9 03       lda #tile_empty
                .alien_row_filler_loop
cb10 8d 07 20    sta PPU_DATA
cb13 88         dey
cb14 10 fa       bpl .alien_row_filler_loop
cb16 ca         dex
cb17 10 e7       bpl .alien_tile_loop
cb19 60          rts

cut_scene_alien_end_draw_row: subroutine
                ; x = start tile id
cb1a a0 07       ldy #$07
                .row_loop
cb1c 8e 07 20    stx PPU_DATA
cb1f e8         inx
cb20 88         dey
cb21 10 f9       bpl .row_loop
cb23 60          rts

cut_scene_outro_init: subroutine
cb24 20 43 c1    jsr render_disable
cb27 20 b1 c1    jsr sprite_clear
cb2a 20 71 c1    jsr nametables_clear

cb2d a9 00       lda #$00
cb2f 85 52       sta state_sprite0
cb31 85 04       sta scroll_page
cb33 85 01       sta scroll_x_hi

                ; draw original alien
                PPU_SETADDR #$208c
cb35 a9 20       lda #>#$208c
cb37 8d 06 20    sta PPU_ADDR
cb3a a9 8c       lda #<#$208c
cb3c 8d 06 20    sta PPU_ADDR
cb3f 20 fa ca    jsr cut_scene_alien_main_draw
                ; update alien image
cb42 a2 40       ldx #$40
                PPU_SETADDR #$208c
cb44 a9 20       lda #>#$208c
cb46 8d 06 20    sta PPU_ADDR
cb49 a9 8c       lda #<#$208c
cb4b 8d 06 20    sta PPU_ADDR
cb4e 20 1a cb    jsr cut_scene_alien_end_draw_row
                PPU_SETADDR #$20ac
cb51 a9 20       lda #>#$20ac
cb53 8d 06 20    sta PPU_ADDR
cb56 a9 ac       lda #<#$20ac
cb58 8d 06 20    sta PPU_ADDR
cb5b 20 1a cb    jsr cut_scene_alien_end_draw_row
                PPU_SETADDR #$20cc
cb5e a9 20       lda #>#$20cc
cb60 8d 06 20    sta PPU_ADDR
cb63 a9 cc       lda #<#$20cc
cb65 8d 06 20    sta PPU_ADDR
cb68 20 1a cb    jsr cut_scene_alien_end_draw_row
                PPU_SETADDR #$212c
cb6b a9 21       lda #>#$212c
cb6d 8d 06 20    sta PPU_ADDR
cb70 a9 2c       lda #<#$212c
cb72 8d 06 20    sta PPU_ADDR
cb75 20 1a cb    jsr cut_scene_alien_end_draw_row

                ; alien attributes
cb78 a2 55       ldx #%01010101
                PPU_SETADDR #$23ca
cb7a a9 23       lda #>#$23ca
cb7c 8d 06 20    sta PPU_ADDR
cb7f a9 ca       lda #<#$23ca
cb81 8d 06 20    sta PPU_ADDR
cb84 8e 07 20    stx PPU_DATA
cb87 8e 07 20    stx PPU_DATA
cb8a 8e 07 20    stx PPU_DATA
                PPU_SETADDR #$23d2

```

```

cb8d a9 23      lda #>#$23d2
cb8f 8d 06 20   sta PPU_ADDR
cb92 a9 d2      lda #<#$23d2
cb94 8d 06 20   sta PPU_ADDR
cb97 8e 07 20   stx PPU_DATA
cb9a 8e 07 20   stx PPU_DATA
cb9d 8e 07 20   stx PPU_DATA
               ; alien colors
cba0 a2 09      ldx #9
cba2 a0 03      ldy #3
cba4 20 cf d1   jsr palette_load

cba7 e6 63      inc phase_end_game

               ; allow future plays to be boundless
cba9 e6 1e      inc player_boundless

               ; use timer to decide end screen
cbab a5 49      lda timer_minutes_10s
cbad c9 60      cmp ##$0+char_set_offset
cbaf f0 3c      beq .good
cbb1 a5 48      lda timer_minutes_1s
cbb3 c9 65      cmp #$5+char_set_offset
cbb5 90 1b      bcc .ok

               .bad
               ; 15+ minutes
               NMTP_SETADDR cut_scene_ending_bad_tile_data
cbb7 a9 c3      lda #>cut_scene_ending_bad_tile_data
cbb9 85 58      sta nametable_tile_planter_addr_hi
cbbb a9 fb      lda #<cut_scene_ending_bad_tile_data
cbbd 85 57      sta nametable_tile_planter_addr_lo
               ; palette
cbbf a9 06      lda #$06
cbc1 85 90      sta pal_uni_bg
cbc3 a2 00      ldx #0
cbc5 a0 00      ldy #0
cbc7 20 cf d1   jsr palette_load
cbca a9 06      lda #song_end_bad
cbcc 20 4b fb   jsr song_start
cbcf 4c 05 cc   jmp .plot_screen

               .ok
               ; 10+ minutes
               NMTP_SETADDR cut_scene_ending_ok_tile_data
cbd2 a9 c4      lda #>cut_scene_ending_ok_tile_data
cbd4 85 58      sta nametable_tile_planter_addr_hi
cbd6 a9 39      lda #<cut_scene_ending_ok_tile_data
cbd8 85 57      sta nametable_tile_planter_addr_lo
               ; palette
cbda a9 07      lda #$07
cbdc 85 90      sta pal_uni_bg
cbde a2 03      ldx #3
cbe0 a0 00      ldy #0
cbe2 20 cf d1   jsr palette_load
cbe5 a9 07      lda #song_end_ok
cbe7 20 4b fb   jsr song_start
cbea 4c 05 cc   jmp .plot_screen

               .good
               ; under 10 minutes
               NMTP_SETADDR cut_scene_ending_good_tile_data
cbed a9 c4      lda #>cut_scene_ending_good_tile_data
cbef 85 58      sta nametable_tile_planter_addr_hi
cbf1 a9 89      lda #<cut_scene_ending_good_tile_data
cbf3 85 57      sta nametable_tile_planter_addr_lo
               ; palette
cbf5 a9 01      lda #$01
cbf7 85 90      sta pal_uni_bg
cbf9 a2 06      ldx #6
cbfb a0 00      ldy #0
cbfd 20 cf d1   jsr palette_load
cc00 a9 08      lda #song_end_good
cc02 20 4b fb   jsr song_start

               .plot_screen
cc05 20 d1 c1   jsr nametable_tile_planter
               ; DEATHLESS!! display
               ; XXX needs some work
cc08 a5 66      lda player_death_counter
               ; final time display
               NMTP_SETADDR cut_scene_ending_time_tile_data
cc0a a9 c4      lda #>cut_scene_ending_time_tile_data
cc0c 85 58      sta nametable_tile_planter_addr_hi
cc0e a9 df      lda #<cut_scene_ending_time_tile_data
cc10 85 57      sta nametable_tile_planter_addr_lo
cc12 20 d1 c1   jsr nametable_tile_planter
cc15 a5 49      lda timer_minutes_10s
cc17 8d 07 20   sta PPU_DATA
cc1a a5 48      lda timer_minutes_1s
cc1c 8d 07 20   sta PPU_DATA
cc1f a9 ac      lda #char_set_colon
cc21 8d 07 20   sta PPU_DATA
cc24 a5 47      lda timer_seconds_10s
cc26 8d 07 20   sta PPU_DATA
cc29 a5 46      lda timer_seconds_1s
cc2b 8d 07 20   sta PPU_DATA
cc2e a9 99      lda #char_set_period
cc30 8d 07 20   sta PPU_DATA
cc33 a5 45      lda timer_frames_10s
cc35 8d 07 20   sta PPU_DATA
cc38 a5 44      lda timer_frames_1s
cc3a 8d 07 20   sta PPU_DATA

cc3d a9 81      lda #state_render_jump_table_offset
cc3f 85 55      sta state_render_addr
cc41 a9 8e      lda #state_update_jump_table_offset+8
cc43 85 56      sta state_update_addr
cc45 20 38 c1   jsr render_enable
cc48 20 4e d2   jsr palette_fade_in_init
cc4b 60         rts

```

state_attract.asm

```
    attract_init:
        ; SCROLL SPEED
cc4c 20 c4 cc    jsr game_init_generic
cc4f a9 03      lda #03
cc51 85 06      sta scroll_speed_hi
cc53 a9 ad      lda #173
cc55 85 05      sta scroll_speed_lo
cc57 a9 8b      lda #state_update_jump_table_offset+5
cc59 85 56      sta state_update_addr
cc5b a5 0e      lda attract_true
cc5d d0 0b      bne .attract_mode_set
cc5f 20 32 dd    jsr clear_all_enemies
cc62 a9 ff      lda #$ff
cc64 85 0e      sta attract_true
cc66 a9 08      lda #iframe_attract_length
cc68 85 54      sta state_iframe_length
    .attract_mode_set
cc6a 20 38 c1    jsr render_enable
cc6d 20 4e d2    jsr palette_fade_in_init
cc70 60         rts
```

```
    attract_update: subroutine
        ; some buttons return to menu
cc71 a5 1c      lda player_controls
cc73 29 d0      and #BUTTON_START|BUTTON_B|BUTTON_A
cc75 f0 08      beq .buttons_for_menu_not_pressed
cc77 a9 7c      lda #state_init_jump_table_offset+0
cc79 20 82 d2    jsr palette_fade_out_init
cc7c 4c a7 c0    jmp state_update_done
    .buttons_for_menu_not_pressed
cc7f a5 24      lda wtf
cc81 29 07      and #$07
cc83 d0 03      bne .no_enemy_spawn
cc85 20 9a cc    jsr attract_spawn_enemy
    .no_enemy_spawn
cc88 a5 0b      lda player_health
cc8a c9 00      cmp #$00
cc8c f0 06      beq .done
cc8e 20 a0 f5    jsr player_demo_controls
cc91 20 b8 f3    jsr player_bullets_check_controls
    .done
cc94 20 18 cd    jsr game_update_generic
cc97 4c a7 c0    jmp state_update_done
```

```
    attract_spawn_enemy: subroutine
        ; starglasses
cc9a ad a0 03    lda $03a0
cc9d d0 06      bne .no_starglasses
cc9f a2 a0      ldx #$a0
cca1 a9 0f      lda #starglasses_id
cca3 d0 1c      bne .delegate
    .no_starglasses
        ; birb
cca5 20 86 dc    jsr get_enemy_slot_1_sprite
cca8 b0 04      bcs .no_birb
ccaa a9 04      lda #birb_id
ccac d0 13      bne .delegate
    .no_birb
        ; maggs
ccae 20 9b dc    jsr get_enemy_slot_2_sprite
ccb1 b0 04      bcs .no_maggs
ccb3 a9 0d      lda #maggs_id
ccb5 d0 0a      bne .delegate
    .no_maggs
        ; skully
ccb7 20 b0 dc    jsr get_enemy_slot_4_sprite
ccba a9 10      lda #skully_id
ccbc e0 c0      cpx #$c0
ccbe 90 01      bcc .delegate
ccc0 60         rts
    .delegate
ccc1 4c 77 da    jmp enemy_spawn_delegator
```

state_game.asm

```
game_init_generic: subroutine
ccc4 20 43 c1    jsr render_disable
ccc7 20 71 c1    jsr nametables_clear

ccca a9 b3      lda #scroll_speed_lo_default
cccc 85 05      sta scroll_speed_lo
ccce a9 02      lda #scroll_speed_hi_default
ccd0 85 06      sta scroll_speed_hi

ccd2 a9 83      lda #state_render_jump_table_offset+2
ccd4 85 55      sta state_render_addr

ccd6 20 95 d6    jsr timer_reset
ccd9 20 17 f2    jsr player_game_reset
ccdc 20 65 cf    jsr dashboard_init
ccdf 20 ff d4    jsr starfield_init
cce2 20 ea d4    jsr starfield_twinkle_reset
cce5 20 dd d5    jsr starfield_draw_dash_top_bar_nametable0
cce8 20 c2 d0    jsr dashboard_update
cceb 20 75 d0    jsr dashboard_render

ccee 20 31 c6    jsr state_sprite0_enable
ccf1 60         rts
```

```
game_init:
ccf2 20 c4 cc    jsr game_init_generic
ccf5 a9 8c      lda #state_update_jump_table_offset+6
ccf7 85 56      sta state_update_addr
ccf9 a9 00      lda #$00
ccfb 85 0e      sta attract_true
ccfd 20 32 dd    jsr clear_all_enemies
cd00 20 38 c1    jsr render_enable
cd03 20 4e d2    jsr palette_fade_in_init
cd06 a9 04      lda #iframe_game_length
cd08 85 54      sta state_iframe_length
cd0a a9 02      lda #2
cd0c 20 4b fb    jsr song_start
cd0f a5 4b      lda state_v1
cd11 f0 04      beq .no_super_secret_code
cd13 a9 20      lda #$20
cd15 85 0c      sta player_lives
cd17 60         .no_super_secret_code
                rts
```

```
game_update_generic: subroutine
cd18 a5 0b      lda player_health
cd1a d0 7d      bne .player_not_dead
                .player_dead_anim
cd1c a5 65      lda player_death_flag
cd1e d0 39      bne .death_already_set
cd20 e6 65      inc player_death_flag
cd22 20 7c fb    jsr song_stop
cd25 a5 06      lda scroll_speed_hi
cd27 85 08      sta scroll_cache_hi
cd29 a5 05      lda scroll_speed_lo
cd2b 85 07      sta scroll_cache_lo
cd2d c6 0c      dec player_lives
cd2f d0 1d      bne .youdead
                ; attract mode has different behaviour
cd31 a5 0e      lda attract_true
cd33 c9 ff      cmp #$ff
cd35 f0 17      beq .youdead
                ; set to grayscale
cd37 a9 01      lda #%00000001
cd39 85 09      sta ppu_mask_emph
                ; cinematics
cd3b 85 06      sta scroll_speed_hi
cd3d a9 05      lda #song_game_over
cd3f 20 4b fb    jsr song_start
cd42 a9 79      lda #121
cd44 85 67      sta you_dead_counter
                ; "GAME OVER"
cd46 a0 10      ldy #$10
cd48 20 66 d0    jsr dashboard_message_set
cd4b 4c b8 cd    jmp .done
                .youdead
cd4e 20 85 f8    jsr sfx_player_death
                ; "YOU DEAD"
cd51 a0 00      ldy #$00
cd53 20 66 d0    jsr dashboard_message_set
cd56 4c b8 cd    jmp .done
                .death_already_set
cd59 20 c5 d5    jsr death_scroll_speed
cd5c e6 67      inc you_dead_counter
cd5e a5 67      lda you_dead_counter
cd60 c9 50      cmp #80
cd62 f0 0a      beq .trigger_fadeout
cd64 c9 78      cmp #120
cd66 d0 50      bne .done
                ; next life
cd68 a5 0c      lda player_lives
cd6a d0 14      bne .next_life
                ; no lives left game over
                ; GO BACK TO TITLE SCREEN AFTER DEATH SEQUENCE
cd6c f0 4a      beq .done
                .trigger_fadeout
                ; demo mode has different behaviour
cd6e a5 0e      lda attract_true
cd70 c9 ff      cmp #$ff
cd72 f0 04      beq .return_to_title_screen
                ; do not return to title if game is not over
cd74 a5 0c      lda player_lives
```

```

cd76 d0 40      bne .done
               .return_to_title_screen
               ; GO BACK TO TITLE SCREEN
               ; AFTER DEATH SEQUENCE
cd78 a9 7c      lda #state_init_jump_table_offset+0
cd7a 20 82 d2    jsr palette_fade_out_init
cd7d 4c b8 cd    jmp .done
               .next_life
cd80 20 81 fb    jsr song_unstop
               ; reset health
cd83 a9 ff      lda #$ff
cd85 85 0b      sta player_health
               ; turn on iframes
cd87 a9 78      lda #120
cd89 85 53      sta state_iframes
               ; reset death sequence timers
cd8b a9 00      lda #$00
cd8d 85 65      sta player_death_flag
cd8f 85 67      sta you_dead_counter
               ; revert scroll speed
cd91 a5 08      lda scroll_cache_hi
cd93 85 06      sta scroll_speed_hi
cd95 a5 07      lda scroll_cache_lo
cd97 85 05      sta scroll_speed_lo

               .player_not_dead

               .healing_with_time
cd99 a5 0b      lda player_health
cd9b c9 ff      cmp #$ff
cd9d f0 0b      beq .no_heal
cd9f a5 0d      lda player_heal_c
cda1 18         clc
cda2 69 23      adc #$23 ; speed of heal counter increment
cda4 85 0d      sta player_heal_c
cda6 90 02      bcc .no_heal
               .reset_heal_counter
cda8 e6 0b      inc player_health
               .no_heal

               player_change_speed:
cdaa a5 1d      lda player_controls_debounced
cdac 29 20      and #BUTTON_SELECT
cdae f0 08      beq .read_select_done
               ; update speed
cdb0 e6 1a      inc player_speed
cdb2 a5 1a      lda player_speed
cdb4 29 03      and #$03
cdb6 85 1a      sta player_speed
               .read_select_done

               .done
               ; player collision update
cdb8 a6 13      ldx player_x_hi
cdba e8         inx
cbbb e8         inx
cdbc 86 17      stx player_coll_x

```

```

cdbe a6 15      ldx player_y_hi
cdc0 e8         inx
cdc1 e8         inx
cdc2 86 18      stx player_coll_y

cdc4 20 56 db    jsr enemies_update_all
cdc7 20 98 f2    jsr set_player_sprite
cdca 60          rts

               game_update: subroutine
               ; check for (un)pausings
cdcb a5 63      lda phase_end_game
cdcd c9 01      cmp #$01
cdcf f0 28      beq .read_start_done
cdd1 a5 1d      lda player_controls_debounced
cdd3 29 10      and #BUTTON_START
cdd5 f0 22      beq .read_start_done
cdd7 a5 1b      lda player_paused
cdd9 c9 00      cmp #$00
cddb f0 0e      beq .pause
               .unpause
cddd a9 00      lda #$00
cdde 85 1b      sta player_paused
cde1 a9 ff      lda #$ff
cde3 85 43      sta dashboard_message
cde5 20 81 fb    jsr song_unstop
cde8 4c f9 cd    jmp .read_start_done
               .pause
cdeb a9 ff      lda #$ff
cded 85 1b      sta player_paused
cdef a9 30      lda #$30
cdf1 85 43      sta dashboard_message
cdf3 20 7c fb    jsr song_stop
cdf6 20 76 fa    jsr sfx_rng_chord
               .read_start_done
               ; test for paused
cdf9 a5 1b      lda player_paused
cdfb d0 12      bne .done_and_paused

cdfd 20 a4 d6    jsr phase_handler
ce00 20 18 cd    jsr game_update_generic

ce03 a5 0b      lda player_health
ce05 c9 00      cmp #$00
ce07 f0 06      beq .done_and_paused

ce09 20 be f4    jsr player_move_position
ce0c 20 b8 f3    jsr player_bullets_check_controls
               .done_and_paused
ce0f 4c a7 c0    jmp state_update_done

```


arctang.asm

```

; quadrants
;
; 0 | 1
; --|--
; 2 | 3

; 24 angular regions total (0-23)
; 6 regions per quadrant
; direction
; 12 o'clock 6
; 3 o'clock 0
; 6 o'clock 18
; 9 o'clock 12

enemy_get_direction_of_player:
; references these zp vars
; collision_0_x
; collision_0_y
; player_x_hi
; player_y_hi
; uses these zp vars
; collision_0_w = x delta
; collision_0_h = y delta
; collision_1_x = "small"
; collision_1_y = "large"
; collision_1_h = "half"
; temp00 = quadrant
; temp01 = region
; temp02 = x register stash
; temp03 = y register stash
; returns a = direction (or region)
ce12 86 88 stx temp02
ce14 84 89 sty temp03
; find quadrant
ce16 a9 00 lda #0
ce18 85 86 sta temp00
; y quadrants
ce1a a5 15 lda player_y_hi
ce1c 38 sec
ce1d e5 3c sbc collision_0_y
ce1f 90 07 bcc .top_quadrant
.bottom_quadrant
ce21 e6 86 inc temp00
ce23 e6 86 inc temp00
ce25 4c 2d ce jmp .y_quadrant_done
.top_quadrant
ce28 a5 3c lda collision_0_y
ce2a 38 sec
ce2b e5 15 sbc player_y_hi
.y_quadrant_done
ce2d 85 3e sta collision_0_h
ce2f a8 tay
; x quadrants
ce30 a5 13 lda player_x_hi
ce32 38 sec
ce33 e5 3b sbc collision_0_x

```

```

ce35 90 05 bcc .left_quadrant
.right_quadrant
ce37 e6 86 inc temp00
ce39 4c 41 ce jmp .x_quadrant_done
.left_quadrant
ce3c a5 3b lda collision_0_x
ce3e 38 sec
ce3f e5 13 sbc player_x_hi
.x_quadrant_done
ce41 85 3d sta collision_0_w
ce43 aa tax
; quadrant is now in temp00
; start finding that region
ce44 e4 3e cpx collision_0_h
ce46 b0 0a bcs .x_greater_or_equal_y
.x_less_than_y
ce48 a9 10 lda #16
ce4a 85 87 sta temp01
ce4c 86 3f stx collision_1_x
ce4e 84 40 sty collision_1_y
ce50 d0 08 bne .determine_region
.x_greater_or_equal_y
ce52 a9 00 lda #0
ce54 85 87 sta temp01
ce56 86 40 stx collision_1_y
ce58 84 3f sty collision_1_x
.determine_region
ce5a a5 3f lda collision_1_x
ce5c 4a lsr
ce5d 85 42 sta collision_1_h
ce5f a5 3f lda collision_1_x
ce61 0a asl
ce62 b0 09 bcs .q_smaller
ce64 18 clc
ce65 65 42 adc collision_1_h
ce67 b0 04 bcs .q_smaller
ce69 c5 40 cmp collision_1_y
ce6b 90 0d bcc .q_larger
.q_smaller
; S * 2.5 > L
ce6d 46 42 lsr collision_1_h
ce6f a5 3f lda collision_1_x
ce71 18 clc
ce72 65 42 adc collision_1_h
ce74 c5 40 cmp collision_1_y
ce76 90 18 bcc .region1
ce78 b0 11 bcs .region0
.q_larger
; S * 2.5 < L
ce7a a5 3f lda collision_1_x
ce7c 0a asl
ce7d 0a asl
ce7e 0a asl
ce7f b0 16 bcs .region2
ce81 38 sec
ce82 e5 42 sbc collision_1_h
ce84 c5 40 cmp collision_1_y
ce86 90 16 bcc .region3

```



```

ce88 4c 97 ce      jmp .region2
               .region0
               ; L / S < 1.25
               ; d = 3,9,15,21
ce8b a5 87      lda temp01
ce8d 18          clc
ce8e 30 13      bmi .result_lookup
               .region1
               ; 1.25 < L / S < 2.5
               ; d = 2,4,8,10,14,16,20,22
ce90 a5 87      lda temp01
ce92 18          clc
ce93 69 04      adc #4
ce95 10 0c      bpl .result_lookup
               .region2
               ; 2.5 < L / S < 7.5
               ; d = 1,5,7,11,13,17,19,23
ce97 a5 87      lda temp01
ce99 18          clc
ce9a 69 08      adc #8
ce9c 10 05      bpl .result_lookup
               .region3
               ; 7.5 < L / S
               ; d = 0,6,12,18
ce9e a5 87      lda temp01
cea0 18          clc
cea1 69 0c      adc #12
               .result_lookup
               ; temp01 should be ready in the accumulator
cea3 65 86      adc temp00
cea5 aa         tax
cea6 bd 00 ff   lda ARCTANG_TRANSLATION_LOOKUP_TABLE,x
               ; restore enemy_ram_offset
cea9 a6 88      ldx temp02
ceab a4 89      ldy temp03
cead 60         rts

               arctang_bound_dir: subroutine
               ; a = value to be bounded
               ; returns value 0..23 in a
ceae 10 04      bpl .check_above_24
               .below_0
ceb0 18          clc
ceb1 69 18      adc #24
ceb3 60         rts
               .check_above_24
ceb4 c9 18      cmp #24
ceb6 90 03      bcc .bounded
ceb8 38          sec
ceb9 e9 18      sbc #24
               .bounded
cebb 60         rts

               ; arctang movement is 16-bit
               ; oam ram x,y = high byte
               ; enemy ram x,y = low byte

               arctang_update_x: subroutine
               ; temp00 = hi
               ; temp01 = lo
               ; temp02 = region
cebc a6 88      ldx temp02
cebe bd 20 ff   lda ARCTANG_REGION_TO_X_VELOCITY_TABLE,x
cec1 0a         asl
cec2 a8         tay
cec3 bd 50 ff   lda ARCTANG_REGION_X_PLUS_OR_MINUS_TABLE,x
cec6 10 0c      bpl arctang_16bit_maths

               arctang_update_y: subroutine
               ; temp00 = hi
               ; temp01 = lo
               ; temp02 = region
cec8 a6 88      ldx temp02
ceca bd 38 ff   lda ARCTANG_REGION_TO_Y_VELOCITY_TABLE,x
cecd 0a         asl
cece a8         tay
cecf bd 68 ff   lda ARCTANG_REGION_Y_PLUS_OR_MINUS_TABLE,x
ced2 10 00      bpl arctang_16bit_maths

               arctang_16bit_maths: subroutine
               ; a = plus or minus
               ; y = velocity table offset
ced4 d0 11      bne .velocity_add
               .velocity_sub
               ; lo byte
ced6 a5 87      lda temp01
ced8 38          sec
ced9 f1 84      sbc (arctang_velocity_lo),y
cedb 85 87      sta temp01
               ; hi byte
cedd a5 86      lda temp00
cedf c8          iny
cee0 f1 84      sbc (arctang_velocity_lo),y
cee2 85 86      sta temp00
cee4 4c f5 ce   jmp .done_x
               .velocity_add
               ; lo byte
cee7 a5 87      lda temp01
cee9 18          clc
ceea 71 84      adc (arctang_velocity_lo),y
ceec 85 87      sta temp01
               ; hi byte
eeee a5 86      lda temp00
cef0 c8          iny
cef1 71 84      adc (arctang_velocity_lo),y
cef3 85 86      sta temp00
               .done_x
cef5 60         rts

```

collision_detection.dasm

```

arctang_enemy_update:
    ; updates both x and y position given:
    ; - enemy_ram_ex holds region/direction
    ; - enemy_ram_x/y holds lo bytes
    ; - oam_ram_x/y holds hi bytes

    ; set region for arctang
cef6  bd 07 03  lda enemy_ram_ex,x
cef9  85 88      sta temp02 ; region

    ; set x byte for arctang
cefb  b9 03 02  lda oam_ram_x,y
cefe  85 86      sta temp00 ; hi byte
cf00  bd 02 03  lda enemy_ram_x,x
cf03  85 87      sta temp01 ; lo byte
cf05  20 bc ce  jsr arctang_update_x
cf08  a4 77      ldy enemy_oam_offset
cf0a  a5 86      lda temp00
cf0c  99 03 02  sta oam_ram_x,y
cf0f  a6 76      ldx enemy_ram_offset
cf11  a5 87      lda temp01
cf13  9d 02 03  sta enemy_ram_x,x

    ; set y byte for arctang
cf16  b9 00 02  lda oam_ram_y,y
cf19  85 86      sta temp00 ; hi byte
cf1b  bd 03 03  lda enemy_ram_y,x
cf1e  85 87      sta temp01 ; lo byte
cf20  20 c8 ce  jsr arctang_update_y
cf23  a4 77      ldy enemy_oam_offset
cf25  a5 86      lda temp00
cf27  20 ff da  jsr enemy_fix_y_visible
cf2a  99 00 02  sta oam_ram_y,y
cf2d  a6 76      ldx enemy_ram_offset
cf2f  a5 87      lda temp01
cf31  9d 03 03  sta enemy_ram_y,x

cf34  60          rts

```

```

;;; COLLISION DETECTOR
;
; works by putting object data into the following
; zero page variables
;
; collision_0_x byte
; collision_0_y byte
; collision_0_w byte
; collision_0_h byte
; collision_1_x byte
; collision_1_y byte
; collision_1_w byte
; collision_1_h byte
;
; returns ff (true) or 00 (false) in accumulator

; the detect_collision subroutine
; was moved to player_bullets.asm
; saving over 12 cycles per usage

```

```

player_collision_detect: subroutine
    ; returns true/false in a
    lda state_iframes
    bne .no_collision
    clc
    lda collision_0_x
    cf3a  a5 3b
    cf3c  65 3d  adc collision_0_w
    cf3e  b0 22  bcs .no_collision ; make sure x+w is not less than
    x
    cf40  c5 13  cmp player_x_hi
    cf42  90 1e  bcc .no_collision
    cf44  18      clc
    cf45  a5 17  lda player_coll_x
    cf47  69 0b  adc #$0b ; player width
    cf49  c5 3b  cmp collision_0_x
    cf4b  90 15  bcc .no_collision
    cf4d  18      clc
    cf4e  a5 3c  lda collision_0_y
    cf50  65 3e  adc collision_0_h
    cf52  c5 18  cmp player_coll_y
    cf54  90 0c  bcc .no_collision
    cf56  18      clc
    cf57  a5 18  lda player_coll_y
    cf59  69 03  adc #$03 ; player height
    cf5b  c5 3c  cmp collision_0_y
    cf5d  90 03  bcc .no_collision
    .collision
    cf5f  a9 ff  lda #$ff
    cf61  60      rts
    .no_collision
    cf62  a9 00  lda #$00
    cf64  60      rts

```

```
;;; constants
```

68

```
cf98      8d 06 20          sta PPU_ADDR  
cf9b      a9 ad           lda #dash_top_bar_tile  
cf9d      a0 20           ldy #$20  
  
                .set_top_bar  
cf9f      8d 07 20        sta PPU_DATA  
cfa2      88             dey  
cfa3      d0 fa           bne .set_top_bar  
  
                ; fill page 2 dashboard tiles  
cfa5      a2 00           ldx #$00  
                .dashboard_fill_tiles  
cfa7      bd c6 cf        lda dashboard_bg_tiles,x  
cfaa      8d 07 20        sta PPU_DATA  
cfad      e8             inx  
cfae      e0 a0           cpx #$a0  
cfb0      d0 f5           bne .dashboard_fill_tiles  
cfb2      a2 00           ldx #$00  
                .dash_cache_fill  
cfb4      bd e6 cf        lda dashboard_bg_tiles+$20,x  
cfb7      9d 00 01        sta dash_cache,x  
cfba      bd 26 d0        lda dashboard_bg_tiles+$60,x  
cfbd      9d 20 01        sta dash_cache+$20,x  
cfc0      e8             inx  
cfc1      e0 20           cpx #$20  
cfc3      d0 ef           bne .dash_cache_fill  
cfc5      60             rts  
  
                dashboard_bg_tiles:  
                ; row 0 is top bar  
                ; row 1 : top hud frames  
cfc6      hex            b4b4bcbbcbbcbcbce  
cfce      hex            bcbbcbbcbcbcbcbc  
dfd6      hex            bcbbcbbcbcbcbcbc  
cfde      hex            bebcbcbcbcbcbdb4  
                ; row 2 : health, meter, lives  
cfe6      hex            b4b0  
cfe8      byte           #char_set_S  
cfe9      hex            a8a9aa  
cfec      hex            ffb3  
cfee      hex            ffffffffffffffff  
cff6      hex            ffffffffffffffff  
cffe      hex            b3aeaf  
d001      byte           #char_set_x  
d002      hex            30b1b2b6  
                ; row 3 : middle hud frames  
d006      hex            b4b8b9b9b9b9b9b7  
d00e      hex            b9b9bfbbb9b9b9b9  
d016      hex            b9b9b9b9bfbbb9b9  
d01e      hex            b7b9b9b9b9b9bab4  
                ; row 4 : wave, score, time  
d026      hex            b4b0  
d028      byte           #char_set_P,#char_set_H,#char_set_A  
d02b      byte           #char_set_S,#char_set_E,#tile_empty  
d02e      hex            fffffb2b6fffffffff  
d036      hex            ffffffffffb2b0fffff  
d03e      hex            ffffffffffffffb2b4
```

69

```

; row 5 : bottom hud frames
d046 hex      b4b4b5b5b5b5b5b5
d04e hex      b5b5b6b4b5b5b5b5
d056 hex      b5b5b5b5b6b4b5b5
d05e hex      b5b5b5b5b5b5b6b6

```

```

dashboard_message_set: subroutine
; set y for message start offset
d066 a2 00      ldx #$00
; .loop
d068 b9 f2 c1    lda dashboard_messages,y
d06b 9d 08 01    sta dash_cache+$08,x
d06e c8          iny
d06f e8          inx
d070 e0 10       cpx #$10
d072 d0 f4       bne .loop
d074 60          rts

```

```

; RENDER DASH STATS TO SCREEN
dashboard_render: subroutine
; top hud info row
PPU_SETADDR $2706 ; PPU_SETADDR is 12 cycles
d075 a9 27      lda #>$2706
d077 8d 06 20    sta PPU_ADDR
d07a a9 06      lda #<$2706
d07c 8d 06 20    sta PPU_ADDR
d07f a2 06      ldx #$06
; .top_row_loop
d081 bd 00 01    lda dash_cache,x
d084 8d 07 20    sta PPU_DATA
d087 e8          inx
d088 e0 1e       cpx #$1e
d08a d0 f5       bne .top_row_loop

```

```

; .lives
PPU_SETADDR lives_addr
d08c a9 27      lda #>lives_addr
d08e 8d 06 20    sta PPU_ADDR
d091 a9 1c      lda #<lives_addr
d093 8d 06 20    sta PPU_ADDR
d096 a5 0c      lda player_lives
d098 20 c3 c1    jsr get_char_hi
d09b 8d 07 20    sta PPU_DATA
d09e a5 0c      lda player_lives
d0a0 20 cb c1    jsr get_char_lo
d0a3 8d 07 20    sta PPU_DATA

```

```

; bottom hud info row
PPU_SETADDR $2748
d0a6 a9 27      lda #>$2748
d0a8 8d 06 20    sta PPU_ADDR
d0ab a9 48      lda #<$2748

```

```

d0ad 8d 06 20    sta PPU_ADDR
d0b0 a2 08      ldx #$08
; .bottom_row_loop
d0b2 bd 20 01    lda dash_cache+$20,x
d0b5 8d 07 20    sta PPU_DATA
d0b8 e8          inx
d0b9 e0 1e       cpx #$1e
d0bb d0 f5       bne .bottom_row_loop
d0bd 60          rts

```

```

dashboard_gauges_table:
d0be byte      phase_kill_counter
d0bf byte      player_frag_counter
d0c0 byte      player_x_hi
d0c1 byte      player_y_hi

```

```

; GET DASH UPDATES READY TO RENDER
dashboard_update: subroutine

```

```

; player speed
d0c2 a5 1a      lda player_speed
d0c4 18          clc
d0c5 69 61      adc #char_set_offset+1
d0c7 8d 06 01    sta $0100+6

```

```

; LIFE BARF
d0ca a4 43      ldy dashboard_message
d0cc 30 06      bmi .lifebarf_is_live
d0ce 20 66 d0    jsr dashboard_message_set
d0d1 4c 0c d1    jmp .no_lifebar

```

```

; .lifebarf_is_live
; 64 health lines on screen
; 16 tiles across
; 256 possible values
; displaying 1/4th the range

```

```

d0d4 a0 00      ldy #$00
d0d6 a9 3c      lda #$3c ; #60 (4 less than max)
d0d8 85 86      sta temp00 ; life value floor for tile

```

```

d0da a5 0b      lda player_health
d0dc 4a          lsr
d0dd 4a          lsr
d0de 85 87      sta temp01 ; life / 4 aka 0..63

```

```

; .find_top_tile
d0e0 a5 87      lda temp01
d0e2 c5 86      cmp temp00
d0e4 b0 0f      bcs .top_tile_found ; life > tile floor value
d0e6 a5 86      lda temp00
d0e8 38          sec
d0e9 e9 04      sbc #$04
d0eb 85 86      sta temp00
d0ed a9 03      lda #tile_empty

```

```

d0ef 99 08 01    sta dash_cache+$08,y
d0f2 c8         iny
d0f3 d0 eb      bne .find_top_tile
                .top_tile_found
d0f5 a5 87      lda temp01
d0f7 38         sec
d0f8 e5 86      sbc temp00
d0fa 18         clc
d0fb 69 a4      adc #$a4
d0fd 99 08 01    sta dash_cache+$08,y
                .fill_remaining_tiles
d100 c8         iny
d101 c0 10      cpy #$10
d103 f0 07      beq .no_lifebar
d105 a9 a7      lda #$a7 ; full tile
d107 99 08 01    sta dash_cache+$08,y
d10a d0 f4      bne .fill_remaining_tiles
                .no_lifebar

                ; PHASE
d10c a5 5a      lda phase_current
d10e 20 c3 c1    jsr get_char_hi
d111 8d 28 01    sta dash_cache+$28
d114 a5 5a      lda phase_current
d116 20 cb c1    jsr get_char_lo
d119 8d 29 01    sta dash_cache+$29

                ; GAUGES
                ; used to be scoreboard

                ; phase_kill_counter
                ; player_frag_counter
                ; player_x_hi
                ; player_y_hi

d11c a2 07      ldx #$07
d11e a0 03      ldy #$03
                .gauges_display_loop
d120 86 86      stx temp00
d122 be be d0    ldx dashboard_gauges_table,y
d125 b5 00      lda $0000,x
d127 85 87      sta temp01
d129 a6 86      ldx temp00
d12b 20 cb c1    jsr get_char_lo
d12e 9d 2c 01    sta dash_cache+$2c,x
d131 a5 87      lda temp01
d133 20 c3 c1    jsr get_char_hi
d136 ca         dex
d137 9d 2c 01    sta dash_cache+$2c,x
d13a ca         dex
d13b 88         dey
d13c 10 e2      bpl .gauges_display_loop

                ; TIMER
                .timer
                ; make sure player has lives / not game over
d13e a5 0c      lda player_lives
d140 c9 00      cmp #$00
d142 f0 2e      beq .timer_done
                ; make sure player hasn't finished the game
d144 a5 63      lda phase_end_game
d146 c9 01      cmp #$01
d148 f0 28      beq .timer_done
                ; we good to update the visual timer
d14a a5 49      lda timer_minutes_10s
d14c 8d 36 01    sta dash_cache+$36
d14f a5 48      lda timer_minutes_1s
d151 8d 37 01    sta dash_cache+$37
                ; colon
d154 a9 ac      lda #char_set_colon
d156 8d 38 01    sta dash_cache+$38
d159 a5 47      lda timer_seconds_10s
d15b 8d 39 01    sta dash_cache+$39
d15e a5 46      lda timer_seconds_1s
d160 8d 3a 01    sta dash_cache+$3a
                ; period
d163 a9 99      lda #char_set_period
d165 8d 3b 01    sta dash_cache+$3b
d168 a5 45      lda timer_frames_10s
d16a 8d 3c 01    sta dash_cache+$3c
d16d a5 44      lda timer_frames_1s
d16f 8d 3d 01    sta dash_cache+$3d
                .timer_done

d172 60         rts

```

palette.asm

```

00 e7      palette_cache EQU $e7

;Palette00:  ; 25 bytes

; hex 0f ;screen color
; hex 022830 ;background 0
; hex 071624 ;background 1
; hex 02113c ;background 2
; hex 0b1a3b ;background 3
; hex 192939 ;sprite 0 ; rudy
; hex 132130 ;sprite 1; old birb palete
; ;hex 051637 ; chomps
; hex 122130 ; vamp eyes & bats
; hex 071727 ;sprite 2
; hex 150037 ; vamp head/face
; hex 013530 ;sprite 3 ; starglasses ; crossbones

; bg 0 - stars bg_tiles 1 / title screen
; bg 1 - stars bg_tiles 2
; bg 2 - n/a (bg enemies haha)
; bg 3 - dashboard / options screen
; spr 0 - player
; spr 1 - enemy / darts
; spr 2 - enemy / stars sprites 1
; spr 3 - enemy / powerups / stars sprite 2

; level palettes
; spr 1, 2 and 3
; boss palettes
; spr 1 and 2

palette_table:
; #00 end bad
d173 byte  $0f, $30, $27
; #03 end ok
d176 byte  $0f, $10, $30
; #06 end good
d179 byte  $0a, $37, $27
; #09 end alien
d17c byte  $3c, $1c, $24
; #12 intro bg
d17f byte  $0c, $30, $3c
; #15 intro alien
d182 byte  $04, $1b, $37
; #18 title screen
d185 byte  $02, $27, $34

```

```

; #21 level 0 palettes (and attract mode)
d188 hex    13 21 30
d18b hex    07 17 27
d18e hex    01 35 30
; #30 level 1 palettes
d191 hex    06 16 38
d194 hex    0b 18 39
d197 hex    08 32 20
; #39 level 2 palettes
d19a hex    0c 2d 32
d19d hex    13 25 37
d1a0 hex    09 27 10
; #48 level 3 palettes
d1a3 hex    2d 10 30
d1a6 hex    12 23 37
d1a9 hex    07 17 27
; #57 boss
; moufs palettes
d1ac hex    04 16 20
d1af hex    02 07 38
; #63 boss
; vamp+bats palettes
d1b2 hex    15 2d 37
d1b5 hex    02 11 30
; #69 boss
; scarab palettes
d1b8 hex    0c 27 37
d1bb hex    03 1a 39
; #75 boss
; swordtner palettes
d1be hex    07 17 3d
d1c1 hex    11 2d 31
; #81 bookbinder
d1c4 hex    0f 00 37

palette_level_offset_table:
d1c7 byte  #21, #30, #39, #48
palette_boss_offset_table:
d1cb byte  #57, #63, #69, #75

palette_load:
; x = palette table offset
; y = pal ram offset
; loads 1 set of 3 colors
; not used to write to main bg color
d1cf a9 02    lda #$02
d1d1 85 86    sta temp00
.loader_loop
d1d3 bd 73 d1  lda palette_table,x
d1d6 99 91 00  sta pal_uni_bg+1,y
d1d9 e8        inx
d1da c8        iny
d1db c6 86     dec temp00
d1dd 10 f4     bpl .loader_loop
d1df 60        rts

```

```

palette_next_rainbow_color: subroutine
; a = current and return color
d1e0 85 86    sta temp00
d1e2 e6 86    inc temp00
d1e4 a5 86    lda temp00
d1e6 29 0f    and #$0f
d1e8 c9 0d    cmp #$0d
d1ea 10 03    bpl .wrap
d1ec a5 86    lda temp00
d1ee 60       rts
; .wrap
d1ef a5 86    lda temp00
d1f1 38       sec
d1f2 e9 0c    sbc #$0c
d1f4 60       rts

bomb_bg_animation_table:
d1f5 byte     $0f,$07,$06,$05,$16,$26,$27,$28,$37,$38,$30

palette_update: subroutine
; shroom
d200 a5 6c    lda shroom_counter
d202 f0 14    beq .no_shroom_effect
d204 a5 24    lda wtf
d206 29 03    and #$03
d208 d0 0e    bne .no_shroom_effect
d20a a0 0e    ldy #$0e
; .shroom_loop
d20c b9 9a 00 lda pal_bg_3_1,y
d20f 20 e0 d1 jsr palette_next_rainbow_color
d212 99 9a 00 sta pal_bg_3_1,y
d215 88       dey
d216 10 f4    bpl .shroom_loop
; .no_shroom_effect
; fade in
d218 a5 8d    lda state_fade_in
d21a f0 03    beq .dont_fade_in
d21c 4c 5b d2 jmp palette_fade_in_update
; .dont_fade_in
; fade out
d21f a5 8e    lda state_fade_out
d221 f0 03    beq .dont_fade_out
d223 4c 91 d2 jmp palette_fade_out_update
; .dont_fade_out
; fade from white
d226 a5 8f    lda state_from_white
d228 f0 03    beq .dont_fade_from_white
d22a 4c bb d2 jmp palette_fade_from_white_update
; .dont_fade_from_white
; bomb
d22d a5 6a    lda bomb_counter
d22f f0 08    beq .normal_bg
d231 4a       lsr
d232 4a       lsr
d233 aa       tax
d234 bd f5 d1 lda bomb_bg_animation_table,x

```

```

d237 85 90    sta pal_uni_bg
; .normal_bg
; do normal
d239 a0 19    ldy #25
; .loop
d23b b9 90 00 lda pal_uni_bg,y
d23e 99 e7 00 sta palette_cache,y
d241 88       dey
d242 10 f7    bpl .loop
d244 60       rts

palette_state_reset:
d245 a9 00    lda #$00
d247 85 8d    sta state_fade_in
d249 85 8e    sta state_fade_out
d24b 85 8f    sta state_from_white
d24d 60       rts

palette_fade_in_init: subroutine
; make sure we're not already fading out
d24e a5 8e    lda state_fade_out
d250 d0 08    bne .init_skip
; .init_fade
d252 a9 00    lda #$00
d254 85 8a    sta pal_fade_c ; frame counter
d256 a9 ff    lda #$ff
d258 85 8d    sta state_fade_in
; .init_skip
d25a 60       rts

palette_fade_in_update: subroutine
d25b a5 8a    lda pal_fade_c
d25d 29 30    and #%00110000
d25f 49 30    eor #%00110000
d261 85 8b    sta pal_fade_offset
d263 a2 17    ldx #23
; .loop
d265 b5 90    lda pal_uni_bg,x
d267 38       sec
d268 e5 8b    sbc pal_fade_offset
d26a 10 02    bpl .dont_set_black
d26c a9 0f    lda #$0f
; .dont_set_black
d26e 95 e7    sta palette_cache,x
d270 ca       dex
d271 10 f2    bpl .loop
d273 a5 8a    lda pal_fade_c
d275 18       clc
d276 69 03    adc #$03
d278 c9 40    cmp #$40
d27a 90 03    bcc .fade_mode_not_done
d27c 20 45 d2 jsr palette_state_reset
; .fade_mode_not_done
d27f 85 8a    sta pal_fade_c
d281 60       rts

```

```

palette_fade_out_init: subroutine
    ; fade out leads to forced tansition and new state
    ; a = init callback index
d282 85 8c    sta pal_fade_target
    ; make sure we're not already fading out
d284 a5 8e    lda state_fade_out
d286 d0 08    bne .init_skip
    .init_fade
d288 a9 10    lda #$10
d28a 85 8a    sta pal_fade_c ; frame counter
d28c a9 ff    lda #$ff
d28e 85 8e    sta state_fade_out
    .init_skip
d290 60      rts

palette_fade_out_update: subroutine
d291 a5 8a    lda pal_fade_c
d293 29 f0    and #%11110000
d295 85 8b    sta pal_fade_offset
d297 a2 19    ldx #25
    .loop
d299 b5 90    lda pal_uni_bg,x
d29b 38      sec
d29c e5 8b    sbc pal_fade_offset
d29e b0 02    bcs .no_reset
d2a0 a9 0f    lda #$0f
    .no_reset
d2a2 95 e7    sta palette_cache,x
d2a4 ca      dex
d2a5 10 f2    bpl .loop
d2a7 a5 8a    lda pal_fade_c
d2a9 18      clc
d2aa 69 03    adc #$03
d2ac c9 60    cmp #$60
d2ae 90 08    bcc .fade_mode_not_done
d2b0 20 45 d2 jsr palette_state_reset

d2b3 a5 8c    lda pal_fade_target
d2b5 4c f1 c4 jmp jump_to_subroutine
    .fade_mode_not_done
d2b8 85 8a    sta pal_fade_c
d2ba 60      rts

```

```

palette_fade_from_white_update: subroutine
d2bb a5 8a    lda pal_fade_c
d2bd 29 f0    and #%11110000
d2bf 85 8b    sta pal_fade_offset
d2c1 a2 17    ldx #23
    .loop
d2c3 b5 90    lda pal_uni_bg,x
d2c5 18      clc
d2c6 65 8b    adc pal_fade_offset
d2c8 c9 3f    cmp #$3f
d2ca 90 02    bcc .dont_set_white
d2cc a9 30    lda #$30
    .dont_set_white
d2ce 95 e7    sta palette_cache,x
d2d0 ca      dex
d2d1 10 f0    bpl .loop
d2d3 c6 8a    dec pal_fade_c
d2d5 c6 8a    dec pal_fade_c
d2d7 c6 8a    dec pal_fade_c
d2d9 10 03    bpl .fade_mode_not_done
d2db 20 45 d2 jsr palette_state_reset
    .fade_mode_not_done
d2de 60      rts

    ; 12 + 32 x 7 = 236 cycles
palette_render: subroutine
d2df a9 3f    lda #>$3f00
d2e1 8d 06 20 sta PPU_ADDR
d2e4 a9 00    lda #<$3f00
d2e6 8d 06 20 sta PPU_ADDR
d2e9 a2 00    ldx #0
d2eb a0 08    ldy #8
    .loop
d2ed a5 e7    lda palette_cache
d2ef 8d 07 20 sta PPU_DATA
d2f2 e8      inx
d2f3 b5 e7    lda palette_cache,x
d2f5 8d 07 20 sta PPU_DATA
d2f8 e8      inx
d2f9 b5 e7    lda palette_cache,x
d2fb 8d 07 20 sta PPU_DATA
d2fe e8      inx
d2ff b5 e7    lda palette_cache,x
d301 8d 07 20 sta PPU_DATA
d304 88      dey
d305 d0 e6    bne .loop
d307 60      rts

```


powerups.asm

```

; powerups always spawn from starglasses

; powerup juggling brought to you by sine waves
; x pos is point of origin
; only need to use rise and fall arc
; must capture max fall velocity (4px/frame)
; rise and fall needs full 8bit range

; palette sprites start at $24
; 0 - skull shield
; 1 - mushroom
; 2 - plus one
; 3 - bomb
; 4 - r bag
; 5 - health 25%
; 6 - health 50%
; 7 - health full

```

```

powerup_drop_table:
d308 hex      04 05 00 03 06 02 01 07
d310 hex      00 03 06 04 01 00 07 02

```

```

powerup_from_starglasses:
d318 a9 02      lda #powerups_id
d31a 9d 00 03    sta enemy_ram_type,x
d31d b9 03 02    lda oam_ram_x,y
d320 18          clc
d321 69 04      adc #$04
d323 9d 02 03    sta enemy_ram_x,x
d326 b9 00 02    lda oam_ram_y,y
d329 9d 03 03    sta enemy_ram_y,x
d32c a9 40      lda #$40
d32e 9d 04 03    sta enemy_ram_pc,x
d331 a9 01      lda #$01
d333 9d 01 03    sta enemy_ram_hp,x
; rng powerup type
d336 a5 68      lda powerup_counter
d338 29 07      and #$07
d33a d0 07      bne .dont_reset_powerup_offset
d33c 20 52 c1    jsr get_next_random
d33f 29 0f      and #$0f
d341 85 69      sta powerup_offset
.dont_reset_powerup_offset
d343 a5 69      lda powerup_offset
d345 29 0f      and #$0f
d347 aa         tax
d348 bd 08 d3    lda powerup_drop_table,x
d34b a6 76      ldx enemy_ram_offset
d34d 9d 07 03    sta enemy_ram_ex,x
d350 e6 69      inc powerup_offset
d352 e6 68      inc powerup_counter
d354 60         rts

```

```

powerups_cycle: subroutine
.process_this_frame
d355 b9 03 02    lda oam_ram_x,y
d358 e9 03      sbc #3
d35a 85 3b      sta collision_0_x
d35c bd 03 03    lda enemy_ram_y,x
d35f 99 00 02    sta oam_ram_y,y
d362 e9 03      sbc #3
d364 85 3c      sta collision_0_y
d366 a9 0d      lda #$0d
d368 85 3d      sta collision_0_w
d36a 85 3e      sta collision_0_h
d36c a9 00      lda #$00
d36e 85 79      sta enemy_dmg_accumulator
d370 20 04 f3    jsr player_bullet_collision_handler
d373 a5 79      lda enemy_dmg_accumulator
d375 d0 14      bne .reset_velocity
d377 20 35 cf    jsr player_collision_detect
d37a f0 25      beq .frame
.despawn_powerup
.player_picksup_powerup
d37c bd 07 03    lda enemy_ram_ex,x
d37f 18          clc
d380 69 3f      adc #powerup_pickup_jump_table_offset
d382 20 f1 c4    jsr jump_to_subroutine
d385 20 47 dd    jsr enemy_clear
d388 4c 92 db    jmp update_enemies_handler_next
.reset_velocity
d38b a9 01      lda #$01
d38d 9d 04 03    sta enemy_ram_pc,x
d390 b9 03 02    lda oam_ram_x,y
d393 9d 02 03    sta enemy_ram_x,x
; will wrap to the left so add hp
d396 fe 01 03    inc enemy_ram_hp,x
d399 10 03      bpl .dont_max_hp
d39b de 01 03    dec enemy_ram_hp,x
.dont_max_hp
; give it sfx
d39e 20 32 f9    jsr sfx_powerup_hit
.frame
d3a1 bd 04 03    lda enemy_ram_pc,x
d3a4 c9 80      cmp #$80
d3a6 f0 2f      beq .max_velocity
.sine_arc
d3a8 bc 04 03    ldy enemy_ram_pc,x
d3ab c8         iny
d3ac b9 00 fe    lda sine_table,y
d3af 85 86      sta temp00
d3b1 88         dey
d3b2 b9 00 fe    lda sine_table,y
d3b5 38         sec
d3b6 e5 86      sbc temp00
d3b8 85 86      sta temp00
d3ba 9d 05 03    sta enemy_ram_ac,x
d3bd a4 77      ldy enemy_oam_offset
d3bf bd 02 03    lda enemy_ram_x,x
d3c2 18          clc

```

```

d3c3 65 86      adc temp00
d3c5 9d 02 03   sta enemy_ram_x,x
d3c8 99 03 02   sta oam_ram_x,y
d3cb a5 24      lda wtf
d3cd 29 01      and #$01
d3cf f0 14      beq .move_done
d3d1 fe 04 03   inc enemy_ram_pc,x
d3d4 4c e5 d3   jmp .move_done
               .max_velocity
d3d7 b9 03 02   lda oam_ram_x,y
d3da 18         clc
d3db 69 04      adc #$04
d3dd 99 03 02   sta oam_ram_x,y
d3e0 90 03      bcc .move_done
               ; will fall off the right so minus hp
d3e2 de 01 03   dec enemy_ram_hp,x
               .move_done
               ; sprite
d3e5 bd 07 03   lda enemy_ram_ex,x
d3e8 18         clc
d3e9 69 24      adc #$24
d3eb 99 01 02   sta oam_ram_spr,y
               ; palette
d3ee a9 03      lda #$03
d3f0 99 02 02   sta oam_ram_att,y
               .despawn_check
d3f3 bd 01 03   lda enemy_ram_hp,x
d3f6 c9 00      cmp #$00
d3f8 d0 03      bne .done
               ;; despawns to the right
d3fa 20 47 dd   jsr enemy_clear
               .done
d3fd 4c 92 db   jmp update_enemies_handler_next

               powerup_pickup_mask: subroutine
d400 e6 6e      inc orbit_shield_speed
d402 e6 6e      inc orbit_shield_speed
d404 e6 6e      inc orbit_shield_speed
d406 20 86 f9   jsr sfx_powerup_mask
d409 60         rts

               powerup_pickup_mushroom: subroutine
               ; shroom_counter set inside sfx call
d40a 20 62 f9   jsr sfx_powerup_mushroom
d40d 60         rts
               powerup_mushroom_update: subroutine
d40e a5 6c      lda shroom_counter
d410 f0 20      beq .shroom_done
d412 a5 24      lda wtf
d414 29 03      and #$03
d416 d0 1a      bne .shroom_done
d418 c6 6c      dec shroom_counter

```

```

d41a d0 0b      bne .audio_bend_up_or_down
               .end_of_shroom
d41c 20 39 d7   jsr phase_palette_load
d41f 20 48 f2   jsr player_update_colors
d422 a9 00      lda #$00
d424 85 6d      sta shroom_mod
d426 60         rts
               .audio_bend_up_or_down
d427 a5 6c      lda shroom_counter
d429 c9 3f      cmp #$3f
d42b 90 03      bcc .down
               .up
d42d e6 6d      inc shroom_mod
d42f 60         rts
               .down
d430 c6 6d      dec shroom_mod
               .shroom_done
d432 60         rts

               powerup_pickup_plus_one: subroutine
d433 e6 0c      inc player_lives
d435 20 a3 f9   jsr sfx_powerup_lup
d438 60         rts

               powerup_pickup_bomb: subroutine
               ; bomb_counter set inside sfx call
               ; 1hp enemy damage every other frame
d439 20 44 f9   jsr sfx_powerup_bomb
d43c 60         rts
               powerup_bomb_update: subroutine
d43d a5 6a      lda bomb_counter
d43f f0 0a      beq .bomb_done
d441 c6 6a      dec bomb_counter
d443 a5 bf      lda sfx_noi_update_type
d445 d0 04      bne .bomb_done
d447 a9 04      lda #$04
d449 85 bf      sta sfx_noi_update_type
               .bomb_done
d44b 60         rts

               powerup_pickup_r_bag: subroutine
d44c a9 ff      lda #$ff
d44e 85 6b      sta r_bag_counter
d450 18         clc
d451 a9 05      lda #$05
d453 65 20      adc player_autofire_s
d455 85 20      sta player_autofire_s
d457 a9 27      lda #$27
d459 c5 20      cmp player_autofire_s
d45b 90 02      bcc .not_maxed_out
d45d a5 20      lda player_autofire_s
               .not_maxed_out
d45f 85 20      sta player_autofire_s
d461 60         rts

```

```

powerup_pickup_health_25: subroutine
d462 20 ee f9    jsr sfx_powerup_battery_25
d465 a5 0b      lda player_health
d467 18         clc
d468 69 40      adc #$40
d46a b0 13      bcs powerup_player_max_energy
d46c 85 0b      sta player_health
d46e 60         rts

```

```

powerup_pickup_health_50: subroutine
d46f 20 f4 f9   jsr sfx_powerup_battery_50
d472 a5 0b      lda player_health
d474 18         clc
d475 69 80      adc #$80
d477 b0 06      bcs powerup_player_max_energy
d479 85 0b      sta player_health
d47b 60         rts

```

```

powerup_pickup_health_100: subroutine
d47c 20 fa f9   jsr sfx_powerup_battery_100
powerup_player_max_energy:
d47f a9 ff      lda #$ff
d481 85 0b      sta player_health
d483 60         rts

```

sine.dasm

```

04 80          sine_7bits equ $0480
05 00          sine_6bits equ $0500
05 80          sine_5bits equ $0580
06 00          sine_4bits equ $0600
06 80          sine_3bits equ $0680
07 00          sine_2bits equ $0700
07 80          sine_1bits equ $0780

```

```

; 40 bytes
sine_init: subroutine
d484 a2 00      ldx #$00
d486 a0 00      ldy #$00
.loop
d488 b9 00 fe   lda sine_table,y
d48b 4a         lsr
d48c 9d 80 04   sta sine_7bits,x
d48f 4a         lsr
d490 9d 00 05   sta sine_6bits,x
d493 4a         lsr
d494 9d 80 05   sta sine_5bits,x
d497 4a         lsr
d498 9d 00 06   sta sine_4bits,x
d49b 4a         lsr
d49c 9d 80 06   sta sine_3bits,x
d49f 4a         lsr
d4a0 9d 00 07   sta sine_2bits,x
d4a3 4a         lsr
d4a4 9d 80 07   sta sine_1bits,x
d4a7 e8         inx
d4a8 c8         iny
d4a9 c8         iny
d4aa d0 dc      bne .loop
d4ac 60         rts

```

starfield_simple.asm

```

; 61 bytes
sine_of_scale: subroutine
; returns scaled value of sine table
; a = sine max
; x = sine pos
d4ad 49 ff    eor #$ff
d4af 85 86    sta temp00
; half x for lookup table
d4b1 8a      txa
d4b2 4a      lsr
d4b3 aa      tax
d4b4 a9 00    lda #$00
d4b6 46 86    lsr temp00
d4b8 46 86    lsr temp00
d4ba b0 03    bcs .not_1bits
d4bc 7d 80 07 adc sine_1bits,x
; .not_1bits
d4bf 46 86    lsr temp00
d4c1 b0 03    bcs .not_2bits
d4c3 7d 00 07 adc sine_2bits,x
; .not_2bits
d4c6 46 86    lsr temp00
d4c8 b0 03    bcs .not_3bits
d4ca 7d 80 06 adc sine_3bits,x
; .not_3bits
d4cd 46 86    lsr temp00
d4cf b0 03    bcs .not_4bits
d4d1 7d 00 06 adc sine_4bits,x
; .not_4bits
d4d4 46 86    lsr temp00
d4d6 b0 03    bcs .not_5bits
d4d8 7d 80 05 adc sine_5bits,x
; .not_5bits
d4db 46 86    lsr temp00
d4dd b0 03    bcs .not_6bits
d4df 7d 00 05 adc sine_6bits,x
; .not_6bits
d4e2 46 86    lsr temp00
d4e4 b0 03    bcs .not_7bits
d4e6 7d 80 04 adc sine_7bits,x
; .not_7bits
d4e9 60      rts

```

```

; star probability
STARFIELD_MASK      EQM %00111111
STARFIELD_COLO_START EQM $00
STARFIELD_COL1_START EQM $07

```

```

starfield_sprite EQM $6b
star_pattern      EQM $a0

```

```

; 32 bytes for string data
starfield_msg_cache = $03e0
star_cache          = $d0
star_oam_start      EQM $40
star_oam_end        EQM <(star_oam_start+$80)

```

```

scroll_speed_lo_default EQM #179
scroll_speed_hi_default EQM #2

```

```

starfield_twinkle_reset: subroutine
d4ea a9 00    lda #STARFIELD_COLO_START
d4ec 85 34    sta starfield_color0
d4ee a9 07    lda #STARFIELD_COL1_START
d4f0 85 35    sta starfield_color1
d4f2 60      rts

```

```

starfield_speed_increase
; increase star speed
d4f3 a9 39    lda #57
d4f5 18      clc
d4f6 65 05    adc scroll_speed_lo
d4f8 85 05    sta scroll_speed_lo
d4fa 90 02    bcc .scroll_hi_done
d4fc e6 06    inc scroll_speed_hi
; .scroll_hi_done
d4fe 60      rts

```

```

starfield_init: subroutine
; set starfield state
d4ff a9 00    lda #$00
d501 85 31    sta starfield_state
; set bg tile palette attributes / colors
; $23c0 and $27c0
; page 1 attributes
PPU_SETADDR $23c0
d503 a9 23    lda #>$23c0
d505 8d 06 20 sta PPU_ADDR
d508 a9 c0    lda #<$23c0
d50a 8d 06 20 sta PPU_ADDR
d50d a9 14    lda #%00010100
d50f a2 d8    ldx #$d8
; .23c0_loop
d511 8d 07 20 sta PPU_DATA

```

```

d514 e8      inx
d515 d0 fa    bne .23c0_loop
                ; COLOR SPLIT
                ; set split color attr for top row
                PPU_SETADDR $23e8
d517 a9 23    lda #>$23e8
d519 8d 06 20 sta PPU_ADDR
d51c a9 e8    lda #<$23e8
d51e 8d 06 20 sta PPU_ADDR
d521 a9 f1    lda #%11110001
d523 a2 f8    ldx #$f8
                .27e8_loop
d525 8d 07 20 sta PPU_DATA
d528 e8      inx
d529 d0 fa    bne .27e8_loop
                ; page 2 attributes
                PPU_SETADDR $27c0
d52b a9 27    lda #>$27c0
d52d 8d 06 20 sta PPU_ADDR
d530 a9 c0    lda #<$27c0
d532 8d 06 20 sta PPU_ADDR
d535 a9 14    lda #%00010100
d537 a2 d8    ldx #$d8
                .27c0_loop
d539 8d 07 20 sta PPU_DATA
d53c e8      inx
d53d d0 fa    bne .27c0_loop
                ; make sure sprite0 has something to collide with
d53f 20 dd d5 jsr starfield_draw_dash_top_bar_nametable0
                ; draw stars on nametable 1
d542 a9 04    lda #CTRL_INC_32
d544 8d 00 20 sta PPU_CTRL
                ;stx starfield_page
d547 a2 00    ldx #$00
                .draw_stars_loop
d549 86 32    stx starfield_column
d54b 20 52 c1 jsr get_next_random
d54e 20 9c d5 jsr starfield_cache_next_col
d551 a9 20    lda #$20
d553 8d 06 20 sta PPU_ADDR
d556 a5 32    lda starfield_column
d558 8d 06 20 sta PPU_ADDR
d55b a0 15    ldy #$15
                .transfer_loop
d55d b9 d0 00 lda star_cache,y
d560 8d 07 20 sta PPU_DATA
d563 88      dey
d564 d0 f7    bne .transfer_loop
d566 e8      inx
d567 e0 20    cpx #$20
d569 d0 de    bne .draw_stars_loop
d56b a9 00    lda #0
d56d 8d 00 20 sta PPU_CTRL
d570 60      rts

```

```

starfield_render: subroutine
                ; copy tiles to PPU VRAM during vblank
                ;update_starfield_column
d571 a9 04    lda #CTRL_INC_32
d573 8d 00 20 sta PPU_CTRL
                ; set base PPU address
d576 a5 33    lda starfield_page
d578 8d 06 20 sta PPU_ADDR
d57b a5 32    lda starfield_column
d57d 8d 06 20 sta PPU_ADDR
                ; rip from cache to PPU
d580 a0 15    ldy #$15
                .col_loop
d582 b9 d0 00 lda star_cache,y
d585 8d 07 20 sta PPU_DATA
d588 88      dey
d589 d0 f7    bne .col_loop
                ; put that shit back to sequential order
d58b a9 00    lda #0
d58d 8d 00 20 sta PPU_CTRL
d590 20 75 d0 jsr dashboard_render
d593 4c 7d c0 jmp state_render_done

starfield_update: subroutine
                ; will not work with scroll speeds above 8
                ; precalculate all starfield things post sprite
d596 20 f2 d5 jsr starfield_scroll
                ; twinkle them stars with palette stuff
d599 20 1a d6 jsr starfield_twinkle_colors
                starfield_cache_next_col:
                ; setup rng for star probability
d59c a5 26    lda rng0
d59e 4a      lsr
d59f 29 3f    and #STARFIELD_MASK
d5a1 85 86    sta temp00
d5a3 a0 00    ldy #$00
                .col_assign_tile_loop
d5a5 c4 86    cpy temp00
d5a7 d0 11    bne .empty_tile
                .star_tile
d5a9 a5 7c    lda boss_heart_stars
d5ab f0 04    beq .normal_stars
                .heart_stars
d5ad a9 ff    lda #$ff
d5af d0 0b    bne .cache_tile
                .normal_stars
d5b1 a5 32    lda starfield_column
d5b3 29 03    and #$03
d5b5 18      clc
d5b6 69 a0    adc #star_pattern
d5b8 d0 02    bne .cache_tile
                .empty_tile
d5ba a9 03    lda #tile_empty
                .cache_tile
d5bc 99 d0 00 sta star_cache,y

```

```

d5bf c8      iny
d5c0 c0 16    cpy #$16
d5c2 d0 e1    bne .col_assign_tile_loop
d5c4 60      rts

```

```

                                death_scroll_speed: subroutine
d5c5 a5 06      lda scroll_speed_hi
d5c7 f0 13      beq .scroll_slow_done
d5c9 a5 05      lda scroll_speed_lo
d5cb 38         sec
d5cc e9 0d      sbc #13
d5ce 85 05      sta scroll_speed_lo
d5d0 b0 0a      bcs .scroll_slow_done
d5d2 c6 06      dec scroll_speed_hi
d5d4 a5 06      lda scroll_speed_hi
d5d6 d0 04      bne .scroll_slow_done
d5d8 a9 00      lda #0
d5da 85 05      sta scroll_speed_lo
                                .scroll_slow_done
d5dc 60      rts

```

```

                                starfield_draw_dash_top_bar_nametable0: subroutine
                                ; fill page 1 bar for sprite 0 collisions
                                PPU_SETADDR dash_page1_top_bar
d5dd a9 22      lda #>dash_page1_top_bar
d5df 8d 06 20    sta PPU_ADDR
d5e2 a9 c0      lda #<dash_page1_top_bar
d5e4 8d 06 20    sta PPU_ADDR
d5e7 a9 ad      lda #dash_top_bar_tile
d5e9 a0 1f      ldy #$1f
                                .tile_dash_set_page1
d5eb 8d 07 20    sta PPU_DATA
d5ee 88         dey
d5ef 10 fa      bpl .tile_dash_set_page1
d5f1 60      rts

```

```

                                starfield_scroll: subroutine
                                ; update scroll pos
d5f2 a5 02      lda scroll_x_lo
d5f4 38         sec
d5f5 e5 05      sbc scroll_speed_lo
d5f7 85 02      sta scroll_x_lo
d5f9 a5 01      lda scroll_x_hi
d5fb e5 06      sbc scroll_speed_hi
d5fd 85 01      sta scroll_x_hi
d5ff b0 02      bcs .samepage
d601 e6 04      inc scroll_page
                                .samepage
                                ; find starfield column for updating
d603 4a         lsr
d604 4a         lsr

```

```

d605 4a         lsr
d606 85 32      sta starfield_column
                                ; set nametable for column updating
d608 a5 04      lda scroll_page
d60a 29 01      and #$01
d60c d0 07      bne .scroll_page
d60e a9 20      lda #$20
d610 85 33      sta starfield_page
d612 4c 19 d6   jmp .scroll_page_done
                                .scroll_page
d615 a9 24      lda #$24
d617 85 33      sta starfield_page
                                .scroll_page_done
d619 60      rts

```

```

                                starfield_twinkle_colors: subroutine
                                ; update palettes every 8 frames
d61a a5 24      lda wtf
d61c 29 07      and #$07
d61e d0 23      bne .done
                                ; color 0
d620 a5 34      lda starfield_color0
d622 20 e0 d1    jsr palette_next_rainbow_color
d625 85 34      sta starfield_color0
d627 85 91      sta pal_bg_0_1
d629 18         clc
d62a 69 10      adc #$10
d62c 85 92      sta pal_bg_0_2
d62e 69 10      adc #$10
d630 85 93      sta pal_bg_0_3
                                ; color 1
d632 a5 35      lda starfield_color1
d634 20 e0 d1    jsr palette_next_rainbow_color
d637 85 35      sta starfield_color1
d639 85 94      sta pal_bg_1_1
d63b 69 10      adc #$10
d63d 85 95      sta pal_bg_1_2
d63f 69 10      adc #$10
d641 85 96      sta pal_bg_1_3
                                .done
d643 60      rts

```

timer_global.dasm

; counts in ranges that match numerical tiles

```
timer_update: subroutine
; UPDATE TIMER DIGITS
; but not if game is over
d644 a5 63 lda phase_end_game
d646 d0 4c bne .timer_done
; frames 1s
d648 e6 44 inc timer_frames_1s
d64a a9 6a lda #char_set_offset+$a
d64c c5 44 cmp timer_frames_1s
d64e d0 44 bne .timer_done
d650 a9 60 lda #char_set_offset
d652 85 44 sta timer_frames_1s
; frames 10s
d654 e6 45 inc timer_frames_10s
d656 a9 66 lda #char_set_offset+$6
d658 c5 45 cmp timer_frames_10s
d65a d0 38 bne .timer_done
d65c a9 60 lda #char_set_offset
d65e 85 45 sta timer_frames_10s
; seconds 1s
d660 e6 46 inc timer_seconds_1s
d662 a9 6a lda #char_set_offset+$a
d664 c5 46 cmp timer_seconds_1s
d666 d0 2c bne .timer_done
d668 a9 60 lda #char_set_offset
d66a 85 46 sta timer_seconds_1s
; seconds 10s
d66c e6 47 inc timer_seconds_10s
d66e a9 66 lda #char_set_offset+$6
d670 c5 47 cmp timer_seconds_10s
d672 d0 20 bne .timer_done
d674 a9 60 lda #char_set_offset
d676 85 47 sta timer_seconds_10s
; minutes 1s
d678 e6 48 inc timer_minutes_1s
d67a a9 6a lda #char_set_offset+$a
d67c c5 48 cmp timer_minutes_1s
d67e d0 14 bne .timer_done
d680 a9 60 lda #char_set_offset
d682 85 48 sta timer_minutes_1s
; minutes 10s
; we really checking this?
; let it go into tile limbo
; GAME OVER at 99:59.59 !!
d684 e6 49 inc timer_minutes_10s
d686 a5 47 lda timer_seconds_10s
d688 c9 6a cmp #char_set_offset+$a
d68a d0 08 bne .timer_done
d68c a9 00 lda #$00
d68e 85 0b sta player_health
d690 a9 01 lda #$01
d692 85 0c sta player_lives
```

```
.timer_done
d694 60 rts

timer_reset: subroutine
d695 a9 60 lda #char_set_offset
d697 85 44 sta timer_frames_1s
d699 85 45 sta timer_frames_10s
d69b 85 46 sta timer_seconds_1s
d69d 85 47 sta timer_seconds_10s
d69f 85 48 sta timer_minutes_1s
d6a1 85 49 sta timer_minutes_10s
d6a3 60 rts
```

phase_handler.asm

```

;phase_spawn_table EQM $0150

; zp vars

;phase_kill_count byte
;phase_current byte
;phase_state byte
;phase_table_ptr byte
;phase_spawn_type byte
;phase_arctang_counter byte
;phase_spawn_counter byte
;phase_interval_counter byte
;phase_end_game byte

; phase_state : 0 = still spawning

phase_handling_stuff

phase_handler: subroutine
; only advance phase activity if player is alive
d6a4 a5 65 lda player_death_flag
d6a6 f0 01 beq .player_not_dead
d6a8 60 rts
.player_not_dead
; always check if its time for an interval spawn
d6a9 20 6c d9 jsr phase_interval_spawn
; call the appropriate phase handler
d6ac a5 5a lda phase_current
d6ae 29 0f and #$0f
d6b0 aa tax
d6b1 bd d1 d9 lda phase_type_table,x
d6b4 18 clc
d6b5 69 4d adc #phase_handlers_jump_table_offset
d6b7 4c f1 c4 jmp jump_to_subroutine

; PHASE UTILITIES

phase_check_next_phase: subroutine
; if phase_state == 0 then
; we don't know the kill count yet
d6ba a5 59 lda phase_state
d6bc f0 06 beq .not_next_phase
; kill_counter > 0 means we need more killing
d6be a5 5c lda phase_kill_counter
d6c0 30 03 bmi .end_of_current_phase
d6c2 f0 01 beq .end_of_current_phase
.not_next_phase

```

```

d6c4 60 rts
.end_of_current_phase
; use phase_state as a timer
d6c5 e6 59 inc phase_state
; transition time
d6c7 a5 59 lda phase_state
d6c9 c9 20 cmp #$20
d6cb d0 06 bne .dont_crossbones
d6cd 20 10 d7 jsr phase_clear_1_sprite_spawns
d6d0 20 04 f9 jsr sfx_enemy_death
.dont_crossbones
d6d3 a5 59 lda phase_state
; wait ~60 frames before trigger next phase sfx
d6d5 c9 40 cmp #$40
d6d7 d0 03 bne .dont_trigger_sfx
d6d9 20 ae fa jsr sfx_phase_next
.dont_trigger_sfx
d6dc a5 59 lda phase_state
; wait ~64 frames before starting next phase
; this should be synced with sfx 2nd chime
d6de c9 44 cmp #$44
d6e0 d0 e2 bne .not_next_phase
; next phase process

phase_next: subroutine
; reset phase vars
d6e2 a9 00 lda #$00
d6e4 85 5c sta phase_kill_counter
d6e6 85 61 sta phase_spawn_counter
d6e8 85 59 sta phase_state
d6ea e6 5a inc phase_current
; check for starfield lumen reset
d6ec a5 5a lda phase_current
d6ee 29 01 and #$01
d6f0 f0 03 beq .no_starfield_lumen_reset
d6f2 20 ea d4 jsr starfield_twinkle_reset
.no_starfield_lumen_reset
; set phase level
d6f5 a5 5a lda phase_current
d6f7 29 0f and #$0f
d6f9 d0 02 bne .not_next_level
d6fb e6 5b inc phase_level
.not_next_level
d6fd 20 f3 d4 jsr starfield_speed_increase
d700 20 39 d7 jsr phase_palette_load
d703 68 pla
d704 68 pla
d705 60 rts

phase_check_spawn_frame: subroutine
d706 a5 24 lda wtf
d708 29 07 and #$07
d70a f0 03 beq .spawn_frame_go
d70c 68 pla
d70d 68 pla
d70e 60 rts
.spawn_frame_go
d70f 60 rts

```



```

phase_clear_1_sprite_spawns: subroutine
; small enemies spawned by phase become crossbones
d710 38      sec
d711 a0 0f    ldy #$0f
d713 a2 78    ldx #$78
        .clear_spawn_loop
d715 b9 50 01  lda phase_spawn_table,y
d718 f0 0a    beq .prep_next_loop
d71a a9 00    lda #0
d71c 99 50 01  sta phase_spawn_table,y
d71f a9 01    lda #crossbones_id
d721 9d 00 03  sta enemy_ram_type,x
        .prep_next_loop
d724 8a      txa
d725 e9 08    sbc #$08
d727 aa      tax
d728 88      dey
d729 10 ea    bpl .clear_spawn_loop
d72b 60      rts

phase_spawn_track: subroutine
; x = enemy ram pos
d72c 86 86    stx temp00
d72e 8a      txa
d72f 4a      lsr
d730 4a      lsr
d731 4a      lsr
d732 aa      tax
d733 fe 50 01  inc phase_spawn_table,x
d736 a6 86    ldx temp00
d738 60      rts

phase_palette_load: subroutine
; each level has its own palette
d739 a4 5b    ldy phase_level
d73b be c7 d1  ldx palette_level_offset_table,y
d73e a0 0f    ldy #15
d740 20 cf d1  jsr palette_load
d743 20 cf d1  jsr palette_load
d746 20 cf d1  jsr palette_load
; boss phases have palettes too!
d749 a5 5a    lda phase_current
d74b 29 0f    and #$0f
d74d c9 0f    cmp #$0f
d74f d0 0e    bne .done
d751 a6 5b    ldx phase_level
d753 bd cb d1  lda palette_boss_offset_table,x
d756 aa      tax
d757 a0 0f    ldy #15
d759 20 cf d1  jsr palette_load
d75c 20 cf d1  jsr palette_load
        .done
d75f 60      rts

```

```

; PHASE TYPES

phase_zero: subroutine
d760 20 ba d6  jsr phase_check_next_phase
d763 a5 59    lda phase_state
d765 d0 20    bne .done
d767 a5 5a    lda phase_current
d769 d0 04    bne .congration
d76b a9 40    lda #$40
d76d d0 02    bne .continue
        .congration
d76f a9 20    lda #$20
        .continue
d771 85 43    sta dashboard_message
d773 e6 61    inc phase_spawn_counter
d775 a9 64    lda #100
d777 c5 61    cmp phase_spawn_counter
d779 d0 0c    bne .stay_zero
d77b a9 ff    lda #$ff
d77d 85 43    sta dashboard_message
d77f a5 5a    lda phase_current
; phase #$40 == game done/end
d781 c9 40    cmp #$40
d783 f0 03    beq .end_game
d785 e6 59    inc phase_state
        .stay_zero
        .done
d787 60      rts
        .end_game
d788 a9 00    lda #$00
d78a 85 01    sta scroll_x_hi
d78c 85 04    sta scroll_page
d78e a9 80    lda #state_init_jump_table_offset+4
d790 20 82 d2  jsr palette_fade_out_init
d793 60      rts

phase_galger: subroutine
d794 20 ba d6  jsr phase_check_next_phase
d797 20 06 d7  jsr phase_check_spawn_frame
; phase state > 0 == done initializing phase
d79a a5 59    lda phase_state
d79c d0 16    bne .phase_init_done
; set kill counter
d79e a5 5b    lda phase_level
d7a0 0a      asl
d7a1 18      clc
d7a2 69 06    adc #$06
d7a4 85 5c    sta phase_kill_counter
d7a6 85 61    sta phase_spawn_counter
; set arctang sequence and advance
d7a8 a5 60    lda phase_arctang_counter
d7aa 29 0f    and #$0f
d7ac aa      tax
d7ad 20 89 ea  jsr arc_sequence_set
d7b0 e6 60    inc phase_arctang_counter

```

```

; set phase_state to init done
d7b2 e6 59 inc phase_state
; phase_init_done
d7b4 a5 61 lda phase_spawn_counter
d7b6 f0 11 beq .dont_spawn
; do_a_spawn
d7b8 20 86 dc jsr get_enemy_slot_1_sprite
d7bb b0 0c bcs .dont_spawn
d7bd 20 2c d7 jsr phase_spawn_track
d7c0 a9 08 lda #galger_id
d7c2 85 5e sta phase_spawn_type
d7c4 20 77 da jsr enemy_spawn_delegator
d7c7 c6 61 dec phase_spawn_counter
; dont_spawn
d7c9 60 rts

phase_spawns: subroutine
d7ca 20 ba d6 jsr phase_check_next_phase
d7cd 20 06 d7 jsr phase_check_spawn_frame
; phase state > 0 == done spawning
d7d0 a5 59 lda phase_state
d7d2 d0 2f bne .dont_spawn
; if downcounter > 0 then spawn
d7d4 a5 61 lda phase_spawn_counter
d7d6 d0 18 bne .do_spawn
; check for next spawn type
d7d8 a4 5d ldy phase_table_ptr
d7da e6 5d inc phase_table_ptr
d7dc b9 19 da lda phase_enemy_table,y
; if type == 0 then stop spawning
d7df d0 04 bne .next_spawn_type
d7e1 e6 59 inc phase_state
; phase_state should be 1 now
d7e3 d0 1e bne .dont_spawn
; next_spawn_type
d7e5 85 5e sta phase_spawn_type
; load number to spawn
d7e7 a4 5d ldy phase_table_ptr
d7e9 e6 5d inc phase_table_ptr
d7eb b9 19 da lda phase_enemy_table,y
d7ee 85 61 sta phase_spawn_counter
; do_spawn
d7f0 a4 5e ldy phase_spawn_type
d7f2 20 72 dc jsr enemy_slot_from_type
d7f5 b0 0c bcs .dont_spawn
d7f7 20 2c d7 jsr phase_spawn_track
d7fa c6 61 dec phase_spawn_counter
d7fc e6 5c inc phase_kill_counter
d7fe a5 5e lda phase_spawn_type
d800 20 77 da jsr enemy_spawn_delegator
; dont_spawn
d803 60 rts

```

```

phase_spawn_long_table:
d804 byte 8, 16, 12, 24, 16, 32, 20, 40

phase_spawn_long: subroutine
d80c 20 ba d6 jsr phase_check_next_phase
d80f 20 06 d7 jsr phase_check_spawn_frame
; phase state > 0 == done spawning
d812 a5 59 lda phase_state
d814 d0 0b bne .phase_init_done
; init
; oiriginal kill count calculation
; 1 + level + difficulty * (4 or 8)
; -----;
; difficulty 0 1 2 3
; -----|----|----|----|----;
; phase 06 4 8 12 16
; phase 0d 8 16 24 32
; phase 16 8 12 16 20
; phase 1d 16 24 32 40
; phase 26 12 16 20 24
; phase 2d 24 32 40 48
; phase 36 16 20 24 28
; phase 3d 32 40 48 56
d816 a6 5f ldx phase_spawn_long_c
d818 bd 04 d8 lda phase_spawn_long_table,x
; store kill count target and advance state
d81b 85 5c sta phase_kill_counter
d81d e6 5f inc phase_spawn_long_c
d81f e6 59 inc phase_state
; phase_init_done
; make sure state is 1 or dont spawn
d821 c9 01 cmp #$01
d823 d0 21 bne .dont_spawn
d825 a5 5b lda phase_level
d827 c9 03 cmp #$03
d829 d0 0b bne .not_final_level
; final_level_spawn_rng
d82b 20 52 c1 jsr get_next_random
d82e 4a lsr
d82f 29 03 and #$03
d831 f0 f8 beq .final_level_spawn_rng
d833 38 sec
d834 e9 01 sbc #$01
; not_final_level
d836 18 clc
d837 69 05 adc #spark_id
d839 a8 tay
d83a 20 86 dc jsr get_enemy_slot_1_sprite
d83d b0 07 bcs .dont_spawn
; set_and_jump
d83f 20 2c d7 jsr phase_spawn_track
d842 98 tya
d843 4c 77 da jmp enemy_spawn_delegator
; dont_spawn
d846 60 rts

```

```

phase_boss_fight: subroutine
; handle boss intro/outro cinematics here
; phase_spawn_counter = scene timer
; phase_kill_counter = which scene
d847 a5 5c lda phase_kill_counter
d849 d0 03 bne .not_intro
d84b 4c 61 d8 jmp phase_boss_fight_intro
; .not_intro
d84e c9 02 cmp #$02
d850 d0 03 bne .not_cooldown
d852 4c 4d d9 jmp phase_boss_fight_cooldown
; .not_cooldown
d855 a5 7b lda boss_death_happening
d857 f0 03 beq .not_death
d859 4c db d8 jmp phase_boss_dying
; .not_death
d85c 60 rts

phase_boss_assistants_table:
d85d byte chomps_id, skully_id, maggs_id, muya_id

phase_boss_fight_intro:
d861 a5 61 lda phase_spawn_counter
d863 d0 0f bne .dont_init_cinematics
; .init_cinematics
; play boss fight encounter jingle
d865 a9 03 lda #song_boss_intro
d867 20 4b fb jsr song_start
; increase scroll speed
d86a e6 06 inc scroll_speed_hi
d86c e6 06 inc scroll_speed_hi
d86e e6 06 inc scroll_speed_hi
; set spawn_counter timing
d870 a9 a0 lda #$a0
d872 85 61 sta phase_spawn_counter
; .dont_init_cinematics
; WHOOOOOSH!!~
d874 e6 13 inc player_x_hi
d876 e6 13 inc player_x_hi
d878 a5 13 lda player_x_hi
d87a c9 e8 cmp #232
d87c 90 04 bcc .right_max
d87e a9 e6 lda #230
d880 85 13 sta player_x_hi
; .right_max
d882 20 bc c1 jsr ppu_mess_emph
; .done_messing_emph
d885 e6 61 inc phase_spawn_counter
d887 d0 4d bne .done

; brighten stars
d889 a5 34 lda starfield_color0
d88b 69 10 adc #$10
d88d 85 34 sta starfield_color0
d88f a5 35 lda starfield_color1
d891 69 10 adc #$10
d893 85 35 sta starfield_color1

```

```

; boss spawning this frame
d895 20 21 c6 jsr state_clear
d898 e6 5c inc phase_kill_counter
d89a a9 00 lda #0
d89c 85 09 sta ppu_mask_emph
d89e 85 61 sta phase_spawn_counter

d8a0 c6 06 dec scroll_speed_hi
d8a2 c6 06 dec scroll_speed_hi
; play boss fight song
d8a4 a9 04 lda #song_boss_fight
d8a6 20 4b fb jsr song_start
; do a fade in FROM HWITE!! :U
d8a9 a9 37 lda #$37
d8ab 85 8a sta pal_fade_c
d8ad 85 8f sta state_from_white
; sfx!
d8af 20 04 f9 jsr sfx_enemy_death
; spawn assistant enemies
d8b2 a6 5b ldx phase_level
d8b4 bd 5d d8 lda phase_boss_assistants_table,x
d8b7 85 88 sta temp02
d8b9 a8 tay
d8ba 20 72 dc jsr enemy_slot_from_type
d8bd a5 88 lda temp02
d8bf 20 77 da jsr enemy_spawn_delegator
d8c2 a4 88 ldy temp02
d8c4 20 72 dc jsr enemy_slot_from_type
d8c7 a5 88 lda temp02
d8c9 20 77 da jsr enemy_spawn_delegator
; spawn boss
d8cc a6 5b ldx phase_level
d8ce bd 15 da lda level_boss_table,x
d8d1 a2 b8 ldx #$b8
d8d3 4c 77 da jmp enemy_spawn_delegator
; .done
d8d6 60 rts

phase_boss_dying_sfx_table:
d8d7 hex 03 06 0d 11

phase_boss_dying: subroutine
d8db a5 61 lda phase_spawn_counter
d8dd d0 0f bne .dont_init
; stop music
d8df 20 7c fb jsr song_stop
; calculate length of boss death
; 8 frame increments
d8e2 a5 5b lda phase_level
d8e4 18 clc
d8e5 69 02 adc #$02
d8e7 0a asl ; ( phase level + 2 ) * 2 = 4, 6, 8, 10
d8e8 69 08 adc #$08 ; + 8 = 12, 14, 16, 18
d8ea 85 4a sta state_v0
d8ec e6 61 inc phase_spawn_counter
; .dont_init
d8ee a0 08 ldy #8

```

```

.rng_pal_loop
d8f0 20 52 c1 jsr get_next_random
d8f3 4a      lsr
d8f4 29 3c    and #%00111100
d8f6 99 a0 00 sta pal_spr_1_1,y
d8f9 88      dey
d8fa 10 f4    bpl .rng_pal_loop
           ; spawn a sfx every 8th frame
d8fc 20 06 d7 jsr phase_check_spawn_frame
d8ff a5 27    lda rng1
d901 29 03    and #$03
d903 a8      tay
d904 b9 d7 d8 lda phase_boss_dying_sfx_table,y
d907 20 46 f8 jsr sfx_test_delegator
d90a c6 4a    dec state_v0
d90c d0 3e    bne .done

           ; kill everything on screen
d90e a2 00    ldx #$00
d910 86 78    stx enemy_slot_id
d912 86 76    stx enemy_ram_offset

.iteration_loop
d914 a6 78    ldx enemy_slot_id
d916 bc 39 db ldy enemy_slot_offset_to_oam_offset,x
d919 84 77    sty enemy_oam_offset
d91b a6 76    ldx enemy_ram_offset
d91d bd 00 03 lda enemy_ram_type,x
d920 c9 03    cmp #$03
d922 90 03    bcc .skip_crossbones_transmogrification
d924 20 62 dd jsr enemy_death_handler

.skip_crossbones_transmogrification
           ; setup next loop
d927 e6 78    inc enemy_slot_id
d929 a9 08    lda #$08
d92b 18      clc
d92c 65 76    adc enemy_ram_offset
d92e 85 76    sta enemy_ram_offset
d930 c9 e0    cmp #$e0
d932 d0 e0    bne .iteration_loop
           ; setup starfield deceleration
d934 e6 5c    inc phase_kill_counter
d936 a9 40    lda #$40
d938 85 61    sta phase_spawn_counter
           ; reset palettes
d93a 20 39 d7 jsr phase_palette_load
           ; setup next scene
d93d 20 04 f9 jsr sfx_enemy_death
d940 20 33 d4 jsr powerup_pickup_plus_one
           ; startup in game music
d943 a9 02    lda #song_in_game
d945 20 4b fb jsr song_start
           ; let crossbones animate
d948 a9 00    lda #$00
d94a 85 7b    sta boss_death_happening
.done
d94c 60      rts

```

```

phase_boss_fight_cooldown: subroutine
d94d a5 61    lda phase_spawn_counter
d94f 29 0f    and #$0f
d951 d0 02    bne .dont_slow_stars
d953 c6 06    dec scroll_speed_hi

.dont_slow_stars
d955 c6 61    dec phase_spawn_counter
d957 d0 12    bne .dont_advance
d959 a9 00    lda #$00
d95b 85 7b    sta boss_death_happening
d95d 85 7c    sta boss_heart_stars
d95f 85 7a    sta boss_dmg_handle_true
d961 85 5c    sta phase_kill_counter
d963 a9 02    lda #song_in_game
d965 20 4b fb jsr song_start
d968 20 e2 d6 jsr phase_next

.dont_advance
d96b 60      rts

phase_interval_spawn: subroutine
           ; dont spawn if boss is dying
d96c a5 7b    lda boss_death_happening
d96e d0 56    bne .no_enemy
           ; check for ike's mom spawn
d970 a5 25    lda ftw
           ;cmp #84 ; about 6 minutes
d972 c9 2a    cmp #42 ; about 3 minutes
d974 90 04    bcc .no_ikes_mom
d976 a0 15    ldy #ikes_mom_id
d978 d0 4d    bne phase_interval_spawn_special_4_sprite

.no_ikes_mom
           ; interval enemy spawn?
d97a a5 44    lda timer_frames_1s
d97c c9 60    cmp #char_set_offset
d97e d0 46    bne .no_enemy
d980 a5 45    lda timer_frames_10s
d982 c9 60    cmp #char_set_offset
d984 d0 40    bne .no_enemy
           ; look for 10 second increments
d986 a5 46    lda timer_seconds_1s
           ; if seconds == 0 then starglasses possible
d988 c9 60    cmp #char_set_offset
d98a f0 0a    beq .starglasses_possible
           ; if seconds == 1 or 7 then spawn enemy
d98c c9 61    cmp #char_set_offset+1
d98e f0 14    beq .spawn_enemy
d990 c9 67    cmp #char_set_offset+7
d992 f0 10    beq .spawn_enemy
d994 d0 30    bne .no_enemy

.starglasses_possible
d996 a5 47    lda timer_seconds_10s
d998 c9 63    cmp #char_set_offset+3
d99a f0 04    beq .starglasses_happening
d99c c9 65    cmp #char_set_offset+5

```

```

d99e d0 26      bne .no_enemy
               .starglasses_happening
d9a0 a0 0f      ldy #starglasses_id
d9a2 d0 23      bne phase_interval_spawn_special_4_sprite
               .spawn_enemy
d9a4 a4 5b      ldy phase_level
d9a6 b9 e1 d9    lda level_intervals_lo,y
d9a9 85 88      sta temp02
d9ab b9 e5 d9    lda level_intervals_hi,y
d9ae 85 89      sta temp03
d9b0 a4 62      ldy phase_interval_counter
d9b2 b1 88      lda (temp02),y
d9b4 d0 04      bne .dont_reset_counter
               .reset_counter
d9b6 85 62      sta phase_interval_counter
d9b8 b1 88      lda (temp02),y
               .dont_reset_counter
d9ba a8         tay
d9bb 20 72 dc    jsr enemy_slot_from_type
d9be b0 06      bcs .no_enemy
d9c0 e6 62      inc phase_interval_counter
d9c2 98         tya
d9c3 20 77 da    jsr enemy_spawn_delegator
               .no_enemy
d9c6 60         rts

               phase_interval_spawn_special_4_sprite: subroutine
d9c7 20 b0 dc    jsr get_enemy_slot_4_sprite
d9ca b0 04      bcs .nope
d9cc 98         tya
d9cd 20 77 da    jsr enemy_spawn_delegator
               .nope
d9d0 60         rts

```

phase_table.asm

```

; phase type constants

; "good luck" / "congratation" ; x0
phase_zero_id EQM #$00
; typical combat phases ; x1-xf
phase_galger_id EQM #$01
phase_spawns_id EQM #$02
phase_longspawn_id EQM #$03
phase_bossfight_id EQM #$04

phase_type_table:
d9d1 byte #phase_zero_id ; 0
d9d2 byte #phase_galger_id ; 1
d9d3 byte #phase_spawns_id ; 2
d9d4 byte #phase_galger_id ; 3
d9d5 byte #phase_spawns_id ; 4
d9d6 byte #phase_galger_id ; 5
d9d7 byte #phase_longspawn_id ; 6
d9d8 byte #phase_galger_id ; 7
d9d9 byte #phase_spawns_id ; 8
d9da byte #phase_galger_id ; 9
d9db byte #phase_galger_id ; a
d9dc byte #phase_spawns_id ; b
d9dd byte #phase_galger_id ; c
d9de byte #phase_longspawn_id ; d
d9df byte #phase_galger_id ; e
d9e0 byte #phase_bossfight_id ; f

; OLD DEMO PHASES
; 1: 1 birb
; 2: 2 maggs
; 3: 2 starglasses
; 4: 1 skully
; 5: 2 starglasses, 2 maggs, 4 birbs
; 6: 8 birbs, 4 maggs
; 7: 12 birbs, 3 skullys
; 8: 12 birbs, 1 skully, 2 maggs
; 9: 16 birbs, 6 skullys, 4 maggs, 2 starglasses

level_intervals_lo:
d9e1 byte #<level_0_interval_spawn_table
d9e2 byte #<level_1_interval_spawn_table
d9e3 byte #<level_2_interval_spawn_table
d9e4 byte #<level_3_interval_spawn_table
level_intervals_hi:
d9e5 byte #>level_0_interval_spawn_table
d9e6 byte #>level_1_interval_spawn_table
d9e7 byte #>level_2_interval_spawn_table
d9e8 byte #>level_3_interval_spawn_table

```

		level_0_interval_spawn_table:			; === LEVEL 01 =====
		; spawn count = 5			; phase 12
		; starglasses = 2	da31	byte	birb_id, 1
d9e9	byte	starglasses_id, spark_id	da33	byte	0
d9eb	byte	starglasses_id, spark_id			; phase 14
d9ed	byte	dumbface_id	da34	byte	chomps_id, 4
d9ee	byte	0	da36	byte	0
		level_1_interval_spawn_table:			; phase 18
		; spawn count = 7	da37	byte	skully_id, 2
		; starglasses = 2	da39	byte	0
d9ef	byte	birb_id, starglasses_id, ant_id, zigzag_id			; phase 1b
d9f3	byte	starglasses_id, zigzag_id, dumbface_id	da3a	byte	birb_id, 8
d9f6	byte	0	da3c	byte	skully_id, 2
		level_2_interval_spawn_table:	da3e	byte	birb_id, 12
		; spawn count = 14	da40	byte	chomps_id, 6
		; starglasses = 2	da42	byte	throber_id, 2
d9f7	byte	maggs_id, skeet_id, starglasses_id	da44	byte	0
d9fa	byte	ant_id, dumbface_id, ant_id, starglasses_id			
d9fe	byte	skeet_id, lasso_id, starglasses_id			; === LEVEL 02 =====
da01	byte	chomps_id, skeet_id, mua_id, starglasses_id			; phase 22
da05	byte	0	da45	byte	0
		level_3_interval_spawn_table:	da46	byte	maggs_id, 2
		; spawn count = 14	da48	byte	throber_id, 1
		; starglasses = 2			; phase 24
da06	byte	chomps_id, mua_id, mua_id, starglasses_id	da4a	byte	0
da0a	byte	lasso_id, ant_id, uzi_id, lasso_id, ant_id, uzi_id	da4b	byte	chomps_id, 2
da10	byte	mua_id, skully_id, throber_id, starglasses_id	da4d	byte	throber_id, 1
da14	byte	0			; phase 28
		level_boss_table:	da4f	byte	0
		boss_moufs_id	da50	byte	maggs_id, 2
da15	byte	boss_vamp_id	da52	byte	birb_id, 6
da16	byte	boss_scarab_id	da54	byte	chomps_id, 2
da17	byte	boss_swordtner_id	da56	byte	birb_id, 6
da18	byte		da58	byte	maggs_id, 2
			da5a	byte	birb_id, 6
			da5c	byte	chomps_id, 2
			da5e	byte	birb_id, 6
		phase_enemy_table:			; phase 2b
		; === LEVEL 00 =====	da60	byte	birb_id, 8
		; phase 02	da62	byte	throber_id, 2
da19	byte	birb_id, 4	da64	byte	birb_id, 12
da1b	byte	0	da66	byte	0
		; phase 04			
da1c	byte	maggs_id, 2			; === LEVEL 03 =====
da1e	byte	0			; phase 32
		; phase 08	da67	byte	birb_id, 32
da1f	byte	birb_id, 3	da69	byte	0
da21	byte	maggs_id, 3			; phase 34
da23	byte	birb_id, 6	da6a	byte	throber_id, 4
da25	byte	0	da6c	byte	0
		; phase 0b			; phase 38
da26	byte	maggs_id, 3	da6d	byte	birb_id, 4
da28	byte	birb_id, 8	da6f	byte	0
da2a	byte	maggs_id, 4			; phase 3b
da2c	byte	skully_id, 1	da70	byte	uzi_id, 1
da2e	byte	birb_id, 12	da72	byte	birb_id, 24
da30	byte	0	da74	byte	throber_id, 2
			da76	byte	0

enemy_handler.dasm

```

;;; Constants

ENEMY_HIT_PALETTE_FRAMES EQM $05

02 03      oam_ram_x      = $0203
02 00      oam_ram_y      = $0200
02 01      oam_ram_spr    = $0201
02 02      oam_ram_att    = $0202

02 20      oam_enemy_sprites = $0220

03 00      enemy_ram_type = $0300
03 01      enemy_ram_hp   = $0301
03 02      enemy_ram_x    = $0302
03 03      enemy_ram_y    = $0303
03 04      enemy_ram_pc   = $0304
03 05      enemy_ram_ac   = $0305
03 06      enemy_ram_hc   = $0306
03 07      enemy_ram_ex   = $0307

enemy_spawn_delegator: subroutine
; a = enemy type id
; x = enemy ram offset
da77 9d 00 03 sta enemy_ram_type,x
da7a a8      tay
da7b 84 86   sty temp00 ; enemy type
; do hit point maths
; hp * (game difficulty + 1)
da7d b9 c5 dc lda enemy_hitpoints_table,y
da80 85 87   sta temp01 ; enemy hp
da82 a9 01   lda #$01
da84 18      clc
da85 65 23   adc game_difficulty
da87 a8      tay
da88 a9 00   lda #$00
.enemy_hp_loop
da8a 65 87   adc temp01
da8c 88      dey
da8d d0 fb   bne .enemy_hp_loop
; save hp result
da8f 9d 01 03 sta enemy_ram_hp,x
; jump to enemy type spawner
da92 a5 86   lda temp00 ; enemy type
da94 18      clc
da95 69 1b   adc #enemy_spawn_jump_table_offset
da97 4c f1 c4 jmp jump_to_subroutine

enemy_do_nothing: subroutine
da9a 4c 92 db jmp update_enemies_handler_next

```

```

sprite_4_set_palette: subroutine
; a = palette id
; x = enemy ram offset
; y = oam ram offset
da9d 20 39 dc jsr enemy_set_palette
daa0 99 06 02 sta oam_ram_att+$04,y
daa3 99 0a 02 sta oam_ram_att+$08,y
daa6 99 0e 02 sta oam_ram_att+$0c,y
daa9 60      rts

sprite_4_set_palette_no_process: subroutine
; a = palette id
; y = oam ram offset
daaa 99 02 02 sta oam_ram_att+$00,y
daad 99 06 02 sta oam_ram_att+$04,y
dab0 99 0a 02 sta oam_ram_att+$08,y
dab3 99 0e 02 sta oam_ram_att+$0c,y
dab6 60      rts

sprite_4_set_sprite: subroutine
; a = top left tile id
; y = oam ram offset
dab7 99 01 02 sta oam_ram_spr,y
daba 18      clc
dabb 69 01   adc #$01
dabd 99 05 02 sta oam_ram_spr+$04,y
dac0 69 0f   adc #$0f
dac2 99 09 02 sta oam_ram_spr+$08,y
dac5 69 01   adc #$01
dac7 99 0d 02 sta oam_ram_spr+$0c,y
daca 60      rts

sprite_4_set_sprite_flip: subroutine
; a = top left tile id
; y = oam ram offset
dacb 99 09 02 sta oam_ram_spr+$08,y
dace 18      clc
dacf 69 01   adc #$01
dad1 99 0d 02 sta oam_ram_spr+$0c,y
dad4 69 0f   adc #$0f
dad6 99 01 02 sta oam_ram_spr,y
dad9 69 01   adc #$01
dadb 99 05 02 sta oam_ram_spr+$04,y
dade 60      rts

sprite_4_set_x: subroutine
; a = x pos
; y = oam ram offset
dadf 99 03 02 sta oam_ram_x,y
dae2 99 0b 02 sta oam_ram_x+$08,y
dae5 18      clc
dae6 69 08   adc #$08
dae8 99 07 02 sta oam_ram_x+$04,y
daeb 99 0f 02 sta oam_ram_x+$0c,y
daee 60      rts

```



```

        sprite_4_set_y: subroutine
            ; a = y pos
            ; y = oam ram offset
daef 99 00 02    sta oam_ram_y,y
daf2 99 04 02    sta oam_ram_y+$04,y
daf5 18          clc
daf6 69 08       adc #$08
daf8 99 08 02    sta oam_ram_y+$08,y
dafb 99 0c 02    sta oam_ram_y+$0c,y
dafe 60          rts

        enemy_fix_y_visible: subroutine
            ; a = y position to check
            ; returns new y position in a
daff c9 f7       cmp #255-8
db01 90 0f       bcc .not_wrapped_from_top
db03 85 86       sta temp00
db05 a9 ff       lda #255
db07 38          sec
db08 e9 b6       sbc #sprite_0_y
db0a 85 87       sta temp01
db0c a5 86       lda temp00
db0e 38          sec
db0f e5 87       sbc temp01
db11 60          rts
        .not_wrapped_from_top
db12 c9 b6       cmp #sprite_0_y
db14 90 03       bcc .not_wrapped_from_bottom
db16 38          sec
db17 e9 b6       sbc #sprite_0_y
        .not_wrapped_from_bottom
db19 60          rts

        enemy_spawn_set_x_y_rng: subroutine
            ; x = enemy_ram_offset
            jsr get_oam_offset_from_ram_offset
db1a 20 52 dc     jsr get_next_random
db1d 20 52 c1     bmi .rng_y
db20 30 0b       .rng_x
db22 a5 27       lda rng1
db24 99 03 02    sta oam_ram_x,y
db27 a9 00       lda #$00
db29 99 00 02    sta oam_ram_y,y
db2c 60          rts
        .rng_y
db2d a9 00       lda #$00
db2f 99 03 02    sta oam_ram_x,y
db32 20 d1 db     jsr enemy_spawn_random_y_pos
db35 99 00 02    sta oam_ram_y,y
db38 60          rts

```

```

        enemy_slot_offset_to_oam_offset:
            ; slots leftshift 3 times to get enemy_ram_offset
            ; 1 sprite slots : start at $300
            ; 16 slots using 1 sprite
            $20,$24,$28,$2c,$30,$34,$38,$3c
db39 byte
db41 byte
            $40,$44,$48,$4c,$50,$54,$58,$5c
            ; 2 sprite slots : start at $380
            ; 4 slots using 2 sprites
            $60,$68,$70,$78
db49 byte
            ; 4 sprite slots : start at $3a0
            ; 8 slots using 4 sprites
            $80,$90,$a0,$b0,$c0,$d0,$e0,$f0
db4d byte
            ; null terminator
db55 byte
            $00

        enemies_update_all: subroutine
db56 a5 7b       lda boss_death_happening
db58 d0 4a       bne .done
            ; set max darts launched per frame
            lda #$03
db5a a9 03       sta dart_frame_max
db5c 85 70       ; if player is dead despawn
db5e a5 0b       lda player_health
db60 d0 02       bne .player_alive
db62 f0 41       beq player_dead_enemies_despawn
        .player_alive
        update_enemies_reset:
            ; set vars for processing
            ldx #$00
db64 a2 00       stx enemy_slot_id
db66 86 78       stx enemy_ram_offset
db68 86 76       ldy enemy_slot_offset_to_oam_offset,x
db6a bc 39 db     sty enemy_oam_offset
            ; cycle through all the slots
        .handler_loop
db6f a6 76       ldx enemy_ram_offset
            ; get enemy type
            lda enemy_ram_type,x
db71 bd 00 03     ; not defined go next slot
            beq .handler_next
db74 f0 1c       tax
db76 aa          lda enemy_update_jump_table_lo,x
db77 bd 03 c5     sta temp00
db7a 85 86       lda enemy_update_jump_table_hi,x
db7c bd 92 c5     sta temp01
db7f 85 87       ; setup collision detection enemy origin pos
            ldx enemy_ram_offset
db81 a6 76       ldy enemy_oam_offset
db83 a4 77       lda oam_ram_x,y
db85 b9 03 02    sta collision_0_x
db88 85 3b       lda oam_ram_y,y
db8a b9 00 02    sta collision_0_y
db8d 85 3c       jmp (temp00)
db8f 6c 86 00    update_enemies_handler_next:
            .handler_next

```



```

; go to next enemy slot
db92 e6 78      inc enemy_slot_id
db94 a5 78      lda enemy_slot_id
db96 0a         asl
db97 0a         asl
db98 0a         asl
db99 85 76      sta enemy_ram_offset
db9b a6 78      ldx enemy_slot_id
db9d bc 39 db    ldy enemy_slot_offset_to_oam_offset,x
dba0 84 77      sty enemy_oam_offset
dba2 d0 cb      bne .handler_loop

.done
dba4 60         rts

player_dead_enemies_despawn: subroutine
; enemies slide off left side of screen
; after player dies
dba5 a2 df      ldx #$df
.loop
dba7 bd 20 02    lda oam_enemy_sprites,x
dbaa f0 14      beq .next
dbac 38         sec
dbad e5 65      sbc player_death_flag
dbaf 9d 20 02    sta oam_enemy_sprites,x
dbb2 c9 f0      cmp #$f0
dbb4 90 0a      bcc .next
.despawn
dbb6 a9 00      lda #$00
dbb8 9d 20 02    sta oam_enemy_sprites,x
dbbb a9 ff      lda #$ff
dbbd 9d 1d 02    sta oam_enemy_sprites-3,x
.next
dbc0 ca         dex
dbc1 ca         dex
dbc2 ca         dex
dbc3 ca         dex
dbc4 e0 ff      cpx #$ff
dbc6 d0 df      bne .loop
dbc8 a5 24      lda wtf
dbca 29 01      and #$01
dbcc d0 02      bne .dont_increase_velocity
dbce e6 65      inc player_death_flag
.dont_increase_velocity
dbd0 60         rts

enemy_spawn_random_y_pos: subroutine
dbd1 20 52 c1    jsr get_next_random
dbd4 4a         lsr
dbd5 18         clc
dbd6 69 10      adc #$10
dbd8 60         rts

```

```

game_difficulty_damage_table:
;hits to die 17 10 7 6
;#15, #26, #37, #48
dbd9 byte      ;hits to die 13 10 7 5
;byte #20, #26, #37, #58
;hits to die 23 17 11 5
;byte #11, #15, #25, #58
;hits to die 5 4 3 2
;byte #58, #77, #109, #191

enemy_gives_damage: subroutine
dbdd a4 23      ldy game_difficulty
dbdf b9 d9 db    lda game_difficulty_damage_table,y
dbe2 85 21      sta player_damage
dbe4 20 5e f2    jsr player_take_damage
dbe7 20 6f f8    jsr sfx_player_damage
dbea 60         rts

enemy_handle_damage_and_death: subroutine
; x = enemy ram offset
; y = oam ram offset
dbeb a9 00      lda #$00
dbed 85 79      sta enemy_dmg_accumulator
; handle bullet collisions
dbef 20 04 f3    jsr player_bullet_collision_handler
dbf2 20 35 cf    jsr player_collision_detect
dbf5 f0 0a      beq .no_collision
; handle player collision
; player damage
dbf7 20 dd db    jsr enemy_gives_damage
; enemy damage
dbfa a5 79      lda enemy_dmg_accumulator
dbfc 18         clc
dbfd 69 02      adc #$02
dbff 85 79      sta enemy_dmg_accumulator
.no_collision
; handle bomb explosion
dc01 a5 6a      lda bomb_counter
dc03 f0 08      beq .no_bomb_damage
dc05 a5 24      lda wtf
dc07 29 01      and #1
dc09 f0 02      beq .no_bomb_damage
dc0b e6 79      inc enemy_dmg_accumulator
.no_bomb_damage
; fix y register
dc0d a4 77      ldy enemy_oam_offset
; handle damage accumulated
dc0f a5 79      lda enemy_dmg_accumulator
dc11 d0 01      bne .hit
dc13 60         rts
.hit
dc14 bd 01 03    lda enemy_ram_hp,x
dc17 38         sec
dc18 e5 79      sbc enemy_dmg_accumulator

```

```

dc1a 9d 01 03    sta enemy_ram_hp,x
dc1d 90 09      bcc .dead

dc1f a9 05      lda #ENEMY_HIT_PALETTE_FRAMES
dc21 9d 06 03    sta enemy_ram_hc,x
dc24 20 e3 f8    jsr sfx_enemy_damage
dc27 60         rts

        .dead
        ; boss death check
dc28 a5 7a      lda boss_dmg_handle_true
dc2a f0 05      beq .not_boss_death
dc2c e6 7b      inc boss_death_happening
dc2e e6 7c      inc boss_heart_stars
dc30 60         rts

        .not_boss_death
dc31 20 62 dd    jsr enemy_death_handler
        ; return to enemy handler not cycle
dc34 68         pla
dc35 68         pla
dc36 4c 92 db    jmp update_enemies_handler_next

        enemy_set_palette: subroutine
        ; checks/handles hit palette counter
        ; a = enemy normal palette
        ; x = enemy ram offset
        ; y = oam ram offset
        ; leaves correct palette id in a
        ; can be applied to extra sprites of larger enemy
dc39 18         clc ; set carry for branching on return
dc3a 85 86      sta temp00
dc3c bd 06 03    lda enemy_ram_hc,x
dc3f f0 0b      beq .use_enemy_palette

        .use_hit_palette
dc41 de 06 03    dec enemy_ram_hc,x
dc44 a5 86      lda temp00
dc46 29 fc      and #%11111100
dc48 99 02 02    sta oam_ram_att,y
dc4b 60         rts

        .use_enemy_palette
dc4c a5 86      lda temp00
dc4e 99 02 02    sta oam_ram_att,y
dc51 60         rts

        ; 16 1 sprite enemies
        ; 4 2 sprite enemies
        ; 8 4 sprite enemies
        ; 28 total on screen enemies / entities
        ; 1 sprite can occupy all slots
        ; 2 sprite can occupy 12 slots
        ; 4 sprite restricted to their 8 slots

```

```

get_oam_offset_from_ram_offset: subroutine
    ; x = slot offset
    ; returns oam offset in y
dc52 86 86      stx temp00
dc54 8a         txa
dc55 4a         lsr
dc56 4a         lsr
dc57 4a         lsr
dc58 aa         tax
dc59 bc 39 db    ldy enemy_slot_offset_to_oam_offset,x
dc5c a6 86      ldx temp00
dc5e 60         rts

```

```

get_enemy_slot_next: subroutine
    ; returns enemy_ram_offset in x
    ; works from the top down
dc5f a9 d8      lda #$d8
dc61 38         sec

        .slot_loop
dc62 aa         tax
dc63 86 86      stx temp00
dc65 bd 00 03    lda $0300,x
dc68 f0 07      beq .slot_found
dc6a a5 86      lda temp00
dc6c e9 08      sbc #$08
dc6e b0 f2      bcs .slot_loop

        .no_slot
dc70 18         clc

        .slot_found
dc71 60         rts

```

```

enemy_slot_from_type: subroutine
    ; y = enemy type id
    ; returns slot offset in x
dc72 be e0 dc    ldx enemy_size_table,y
dc75 e0 01      cpx #1
dc77 d0 03      bne .not_size_1
dc79 4c 86 dc    jmp get_enemy_slot_1_sprite

        .not_size_1
dc7c e0 02      cpx #2
dc7e d0 03      bne .not_size_2
dc80 4c 9b dc    jmp get_enemy_slot_2_sprite

        .not_size_2
dc83 4c b0 dc    jmp get_enemy_slot_4_sprite

```

enemy_tables.dasm

```

get_enemy_slot_1_sprite: subroutine
    ; returns enemy_ram offset in x
    ; carry cleared if found
    ; carry set if unfound
dc86  a9 00    lda #$00
dc88  18       clc
        .slot_loop
dc89  aa       tax
dc8a  86 86    stx temp00
dc8c  bd 00 03  lda $0300,x
dc8f  f0 09    beq .slot_found
dc91  a9 08    lda #$08
dc93  65 86    adc temp00
dc95  c9 80    cmp #$80
dc97  d0 f0    bne .slot_loop
        .no_available_slot
dc99  38       sec
        .slot_found
dc9a  60       rts

get_enemy_slot_2_sprite: subroutine
    ; returns enemy_ram offset in x
dc9b  a9 80    lda #$80
dc9d  18       clc
        .slot_loop
dc9e  aa       tax
dc9f  86 86    stx temp00
dca1  bd 00 03  lda $0300,x
dca4  f0 09    beq .slot_found
dca6  a9 08    lda #$08
dca8  65 86    adc temp00
dcaa  c9 a0    cmp #$a0
dcac  d0 f0    bne .slot_loop
        .no_available_slot
dcae  38       sec
        .slot_found
dcac  60       rts

get_enemy_slot_4_sprite: subroutine
    ; returns enemy_ram offset in x
dcb0  a9 a0    lda #$a0
dcb2  18       clc
        .slot_loop
dcb3  aa       tax
dcb4  86 86    stx temp00
dcb6  bd 00 03  lda $0300,x
dcb9  f0 09    beq .slot_found
dcbb  a9 08    lda #$08
dcbd  65 86    adc temp00
dcbf  c9 e0    cmp #$e0
dcc1  d0 f0    bne .slot_loop
        .no_available_slot
dcc3  38       sec
        .slot_found
dcc4  60       rts

```

; 27 enemy entities defined

; enemy id list:

null_id	EQM \$00
crossbones_id	EQM \$01
powerups_id	EQM \$02
boss_assist_id	EQM \$03
birb_id	EQM \$04
spark_id	EQM \$05
zigzag_id	EQM \$06
skeet_id	EQM \$07
galger_id	EQM \$08
bullet_id	EQM \$09
dart_id	EQM \$0a
boss_vamp_bat_id	EQM \$0b
chomps_id	EQM \$0c
maggs_id	EQM \$0d
muya_id	EQM \$0e
starglasses_id	EQM \$0f
skully_id	EQM \$10
dumbface_id	EQM \$11
throber_id	EQM \$12
ant_id	EQM \$13
lasso_id	EQM \$14
ikes_mom_id	EQM \$15
uzi_id	EQM \$16
boss_moufs_id	EQM \$17
boss_vamp_id	EQM \$18
boss_scarab_id	EQM \$19
boss_swordtner_id	EQM \$1a

```

enemy_hitpoints_table:
dcc5 byte $00 ; null
dcc6 byte $00 ; crossbones
dcc7 byte $00 ; powerups
dcc8 byte $00 ; boss_assist
dcc9 byte $02 ; birb
dcca byte $03 ; spark
dccb byte $03 ; zigzag
dccc byte $03 ; skeet
dccd byte $02 ; galger
dcce byte $01 ; bullet
dccf byte $01 ; dart
dcd0 byte $0a ; boss_vamp_bat
dcd1 byte $07 ; chomps
dcd2 byte $09 ; maggs
dcd3 byte $0d ; muya
dcd4 byte $05 ; starglasses
dcd5 byte $1f ; skully
dcd6 byte $0b ; dumbface
dcd7 byte $0d ; throber
dcd8 byte $11 ; ant
dcd9 byte $35 ; lasso
dcda byte $25 ; ikes_mom
dcdb byte $1d ; uzi
dcdc byte $3f ; boss_moufs
dcdd byte $3f ; boss_vamp
dcde byte $3f ; boss_scarab
dcdf byte $3f ; boss_swordtner

```

```

enemy_size_table:
dce0 byte $00 ; null
dce1 byte $00 ; crossbones
dce2 byte $00 ; powerups
dce3 byte $00 ; boss_assist
dce4 byte $01 ; birb
dce5 byte $01 ; spark
dce6 byte $01 ; zigzag
dce7 byte $01 ; skeet
dce8 byte $01 ; galger
dce9 byte $01 ; bullet
dcea byte $01 ; dart
dceb byte $01 ; boss_vamp_bat
dcec byte $02 ; chomps
dced byte $02 ; maggs
dcee byte $02 ; muya
dcef byte $04 ; starglasses
dcf0 byte $04 ; skully
dcf1 byte $04 ; dumbface
dcf2 byte $04 ; throber
dcf3 byte $04 ; ant
dcf4 byte $04 ; lasso
dcf5 byte $04 ; ikes_mom
dcf6 byte $04 ; uzi
dcf7 byte $04 ; boss_moufs
dcf8 byte $04 ; boss_vamp
dcf9 byte $04 ; boss_scarab
dcfa byte $04 ; boss_swordtner

```

enemy_death.asm

```

enemy_accelerates_upwards: subroutine
dcfb fe 05 03 inc enemy_ram_ac,x
dcfe bd 05 03 lda enemy_ram_ac,x
dd01 4a      lsr
dd02 4a      lsr
dd03 4a      lsr
dd04 9d 03 03 sta enemy_ram_y,x
dd07 b9 00 02 lda oam_ram_y,y
dd0a 38      sec
dd0b fd 03 03 sbc enemy_ram_y,x
dd0e c9 b6   cmp #sprite_0_y
dd10 60      rts

```

```

crossbones_cycle: subroutine
dd11 20 fb dc jsr enemy_accelerates_upwards
dd14 b0 0f    bcs .crossbones_death
dd16 99 00 02 sta oam_ram_y,y
dd19 a9 c1    lda #$c1
dd1b 99 01 02 sta oam_ram_spr,y
dd1e a9 03    lda #$03
dd20 99 02 02 sta oam_ram_att,y
dd23 d0 0a    bne .crossbones_done
        .crossbones_death
dd25 a5 bd    lda sfx_noi_counter
dd27 d0 03    bne .die_already
dd29 20 2c fb jsr sfx_ghost_snare
        .die_already
dd2c 20 47 dd jsr enemy_clear
        .crossbones_done
dd2f 4c 92 db jmp update_enemies_handler_next

```

```

clear_all_enemies: subroutine
                ; clears all enemy ram
dd32 a2 f0    ldx #$f0
dd34 a9 00    lda #$00
        .enemy_loop
dd36 ca      dex
dd37 9d 00 03 sta enemy_ram_type,x
dd3a d0 fa    bne .enemy_loop

```

```

; clears all enemy sprites
dd3c a9 ff lda #$ff
dd3e a2 20 ldx #$20
; sprite_loop
dd40 9d 00 02 sta $0200,x
dd43 e8 inx
dd44 d0 fa bne .sprite_loop
dd46 60 rts

enemy_clear: subroutine
; clear OAM data
dd47 a4 77 ldy enemy_oam_offset
dd49 a9 ff lda #$ff
dd4b a2 03 ldx #$03
; enemy_oam_loop
dd4d 99 00 02 sta oam_ram_y,y
dd50 c8 iny
dd51 ca dex
dd52 d0 f9 bne .enemy_oam_loop
; clear enemy data
dd54 a6 76 ldx enemy_ram_offset
dd56 a9 00 lda #$00
dd58 a0 06 ldy #$06 ; 7 would clear _ex
; enemy_ram_loop
dd5a 9d 00 03 sta enemy_ram_type,x
dd5d e8 inx
dd5e 88 dey
dd5f d0 f9 bne .enemy_ram_loop
; enemy_ram_ex is allowed to carry over
; entities using _ex should set it on spawn
dd61 60 rts

enemy_death_handler: subroutine
; requires
; enemy_ram_offset
; enemy_oam_offset

; don't handle a non entity
dd62 a6 76 ldx enemy_ram_offset
dd64 bd 00 03 lda enemy_ram_type,x
dd67 f0 59 beq .done

dd69 e6 64 inc player_frag_counter

; track deaths of phase spawns
dd6b a4 78 ldy enemy_slot_id
dd6d b9 50 01 lda phase_spawn_table,y
dd70 f0 0b beq .not_phase_spawn
dd72 a9 00 lda #$00
dd74 99 50 01 sta phase_spawn_table,y

```

```

dd77 c6 5c dec phase_kill_counter

; if less than 0 set to 0
dd79 10 02 bpl .not_phase_spawn
dd7b 85 5c sta phase_kill_counter
; not_phase_spawn

dd7d a6 76 ldx enemy_ram_offset
dd7f a4 77 ldy enemy_oam_offset
; 4 sprite cleanup
dd81 e0 a0 cpx #$a0
dd83 90 1f bcc .dont_cleanup_4_sprite
dd85 a9 04 lda #$04 ; move first sprite over 4 pixels
dd87 18 clc
dd88 79 03 02 adc oam_ram_x,y
dd8b 99 03 02 sta oam_ram_x,y
dd8e a9 04 lda #$04 ; move first sprite down 4 pixels
dd90 18 clc
dd91 79 00 02 adc oam_ram_y,y
dd94 99 00 02 sta oam_ram_y,y
; move other sprites off screen
dd97 a9 ff lda #$ff
dd99 99 04 02 sta oam_ram_y+4,y
dd9c 99 08 02 sta oam_ram_y+8,y
dd9f 99 0c 02 sta oam_ram_y+12,y
dda2 d0 09 bne .check_for_powerup_drop
; dont_cleanup_4_sprite

; 2 sprite cleanup
dda4 e0 80 cpx #$80
dda6 90 05 bcc .dont_cleanup_2_sprite
dda8 a9 ff lda #$ff
ddaa 99 04 02 sta oam_ram_y+4,y
; dont_cleanup_2_sprite

; check_for_powerup_drop
; starglasses
ddad bd 00 03 lda enemy_ram_type,x
ddb0 c9 0f cmp #starglasses_id
ddb2 d0 03 bne .not_starglasses
; spawn powerup
ddb4 4c 18 d3 jmp powerup_from_starglasses
; not_starglasses

ddb7 20 04 f9 jsr sfx_enemy_death
; change it into crossbones!
ddba a9 01 lda #crossbones_id
ddbc 9d 00 03 sta enemy_ram_type,x
ddbf 9d 05 03 sta enemy_ram_ac,x
; done
ddc2 60 rts

```

enemy_boss_moufs.asm

```

; boss_x : x origin of sine pattern
; boss_y : y origin of sine pattern
; state_v0 : bottom lip position
; state_v1 : bottom lip animation counter
; state_v2 :
; state_v3 : y bounce sine direction
; state_v4 : sprite x position
; state_v5 : sprite y position
; state_v6 : upper lip up sprite offset

```

```

boss_moufs_spawn: subroutine
ddc3 a9 03      lda #boss_assist_id
; eyeballs
ddc5 8d 40 03   sta $0340
ddc8 8d 48 03   sta $0348
; lip rows
ddcb 8d c0 03   sta $03c0
ddce 8d c8 03   sta $03c8
ddd1 8d d0 03   sta $03d0
ddd4 a9 40      lda #$40
ddd6 9d 02 03   sta enemy_ram_x,x
ddd9 85 4e      sta state_v4
dddb 9d 03 03   sta enemy_ram_y,x
ddde 85 4f      sta state_v5
dde0 a9 08      lda #$08
dde2 85 7d      sta boss_x
dde4 a9 20      lda #$20
dde6 85 7e      sta boss_y
dde8 60         rts

```

```

moufs_oam_offset_table:
dde9 byte      $b0, $c0, $e0, $d0
moufs_sprite_offset_table:
dded byte      $00, $10, $20, $30
moufs_y_offset_table:
ddf1 byte      $00, $08, $0d, $10

```

```

boss_moufs_cycle: s
ubroutine
ddf5 a5 4e      lda state_v4
ddf7 85 3b      sta collision_0_x
ddf9 a5 4f      lda state_v5
ddfb 85 3c      sta collision_0_y
ddfd a9 14      lda #$14
ddff 85 3d      sta collision_0_w
de01 a9 20      lda #$20
de03 85 3e      sta collision_0_h

```

```

de05 e6 7a      inc boss_dmg_handle_true
de07 20 eb db   jsr enemy_handle_damage_and_death
de0a c6 7a      dec boss_dmg_handle_true

; set upper lip default sprite
de0c a9 00      lda #$00
de0e 85 50      sta state_v6

; sine bounce movement
; ac does x axis
de10 fe 05 03   inc enemy_ram_ac,x
; pc does y axis
de13 a5 4d      lda state_v3
de15 29 01      and #$01
de17 d0 0f      bne .y_increase
.y_decrease
; arc backwards
de19 de 04 03   dec enemy_ram_pc,x
de1c bd 04 03   lda enemy_ram_pc,x
de1f c9 b3      cmp #$b3
de21 b0 11      bcs .y_change_done
de23 e6 50      inc state_v6
de25 4c 34 de   jmp .y_change_done
.y_increase
; arc forwards
de28 fe 04 03   inc enemy_ram_pc,x
de2b bd 04 03   lda enemy_ram_pc,x
de2e c9 ce      cmp #$ce
de30 90 02      bcc .y_change_done
de32 e6 50      inc state_v6
.y_change_done
de34 bc 04 03   ldy enemy_ram_pc,x
de37 b9 00 fe   lda sine_table,y
de3a a4 77      ldy enemy_oam_offset
de3c c9 80      cmp #$80
de3e d0 02      bne .dont_change_dir
de40 e6 4d      inc state_v3
.dont_change_dir
; x
de42 bd 05 03   lda enemy_ram_ac,x
de45 18         clc
de46 69 40      adc #$40
de48 4a         lsr
de49 aa         tax
de4a bd 00 05   lda sine_6bits,x
de4d 18         clc
de4e 65 7d      adc boss_x
de50 85 4e      sta state_v4
de52 a6 76      ldx enemy_ram_offset
; y
de54 bd 04 03   lda enemy_ram_pc,x
de57 aa         tax
de58 bd 00 fe   lda sine_table,x
de5b 18         clc
de5c 65 7e      adc boss_y
de5e 85 4f      sta state_v5
de60 a6 76      ldx enemy_ram_offset

```

```

; palette
de62 a9 01    lda #$01
de64 20 9d da  jsr sprite_4_set_palette
de67 85 87    sta temp01

```

```

; EYEBALLLZ

```

```

de69 a5 15    lda player_y_hi
de6b c5 4f    cmp state_v5
de6d b0 04    bcs .look_down
               .look_up
de6f a9 c6    lda #$c6
de71 d0 02    bne .look_set_sprites
               .look_down
de73 a9 d6    lda #$d6
               .look_set_sprites
de75 8d 41 02  sta oam_ram_spr+$40
de78 8d 45 02  sta oam_ram_spr+$44
de7b ee 45 02  inc oam_ram_spr+$44

de7e a5 4e    lda state_v4
de80 8d 43 02  sta oam_ram_x+$40
de83 18       clc
de84 69 11    adc #$11
de86 8d 47 02  sta oam_ram_x+$44
de89 a5 4f    lda state_v5
de8b e9 04    sbc #$04
de8d 8d 40 02  sta oam_ram_y+$40
de90 8d 44 02  sta oam_ram_y+$44
de93 a5 87    lda temp01
de95 f0 02    beq .hit
de97 a9 02    lda #$02
               .hit
de99 8d 42 02  sta oam_ram_att+$40
de9c 8d 46 02  sta oam_ram_att+$44

```

```

; LIPPPPS

```

```

; y row 3
de9f a5 4b    lda state_v1
dea1 4a       lsr
dea2 aa       tax
dea3 bd 00 07  lda sine_2bits,x
dea6 85 88    sta temp02

dea8 e6 4b    inc state_v1
deaa e6 4b    inc state_v1
deac e6 4b    inc state_v1

```

```

; y row 4
deae a5 4b    lda state_v1
deb0 4a       lsr
deb1 aa       tax
deb2 bd 80 06  lda sine_3bits,x
deb5 85 4a    sta state_v0
deb7 85 89    sta temp03

```

```

deb9 e6 4b    inc state_v1
debb e6 4b    inc state_v1
debd e6 4b    inc state_v1

```

```

; row 1 sprite
debf a0 b0    ldy #$b0
dec1 a5 50    lda state_v6
dec3 f0 06    beq .lip_up
               .lip_down
dec5 a9 65    lda #$65
dec7 85 50    sta state_v6
dec9 d0 04    bne .upper_lip_set_sprite
               .lip_up
decb a9 86    lda #$86
decd 85 50    sta state_v6
               .upper_lip_set_sprite

```

```

decf a2 00    ldx #$00
               .mouf_sprites_loop
ded1 bc e9 dd  ldy moufs_oam_offset_table,x
               ; sprite
               ; a = left tile id
               ; y = oam ram offset
ded4 e0 02    cpx #$02
ded6 b0 04    bcs .sprite_normal
               .sprite_custom
ded8 a5 50    lda state_v6
deda d0 02    bne .sprite_chosen
               .sprite_normal
dedc a9 86    lda #$86
               .sprite_chosen
dede 18       clc
dedf 7d ed dd  adc moufs_sprite_offset_table,x
dee2 99 01 02  sta oam_ram_spr,y
dee5 18       clc
dee6 69 01    adc #$01
dee8 99 05 02  sta oam_ram_spr+$04,y
deeb 69 01    adc #$01
deed 99 09 02  sta oam_ram_spr+$08,y
               ; x
               ; a = x pos
               ; y = oam ram offset
def0 a5 4e    lda state_v4
def2 99 03 02  sta oam_ram_x,y
def5 18       clc
def6 69 08    adc #$08
def8 99 07 02  sta oam_ram_x+$04,y
defb 69 08    adc #$08

```

```

defd 99 0b 02    sta oam_ram_x+$08,y
df00 69 08      adc #$08
df02 99 0f 02    sta oam_ram_x+$0c,y
                ; y
                ; a = y pos
                ; y = oam ram offset
df05 a5 4f      lda state_v5
df07 18         clc
df08 7d f1 dd    adc moufs_y_offset_table,x
df0b e0 02      cpx #$02
df0d 90 0a      bcc .no_y_adjustments
df0f e0 03      cpx #$03
df11 f0 04      beq .bottom_sprite
                .almost_bottom_sprite
df13 65 88      adc temp02
df15 d0 02      bne .no_y_adjustments
                .bottom_sprite
df17 65 89      adc temp03
                .no_y_adjustments
df19 99 00 02    sta oam_ram_y,y
df1c 99 04 02    sta oam_ram_y+$04,y
df1f 99 08 02    sta oam_ram_y+$08,y
                ; pal / attr
df22 a5 87      lda temp01
df24 20 aa da    jsr sprite_4_set_palette_no_process
                ; check next
df27 e8         inx
df28 e0 04      cpx #$04
df2a d0 a5      bne .mouf_sprites_loop

df2c a6 76      ldx enemy_ram_offset

                ; fire time?
df2e a5 4a      lda state_v0
df30 c9 07      cmp #$07
df32 d0 38      bne .done
df34 a5 24      lda wtf
df36 29 07      and #$07
df38 d0 32      bne .done

                .dart_fire
                ; x
df3a a5 4e      lda state_v4
df3c 18         clc
df3d 69 0a      adc #$0a
df3f 85 71      sta dart_x_origin
                ; y
df41 a5 4f      lda state_v5
df43 18         clc
df44 69 0f      adc #$0f
df46 85 72      sta dart_y_origin
                ; velocity
df48 bd 04 03    lda enemy_ram_pc,x
df4b e9 80      sbc #$80

```

```

df4d 4a         lsr
df4e 4a         lsr
df4f 4a         lsr
df50 4a         lsr
df51 85 73      sta dart_velocity
                ; sprite
df53 a9 fe      lda #$fe
                ;lda #$00
df55 85 74      sta dart_sprite
                ; dir adjustor
df57 a9 00      lda #$00
df59 85 75      sta dart_dir_adjust
df5b 20 a8 e7    jsr dart_spawn
df5e a9 ff      lda #$ff
df60 85 75      sta dart_dir_adjust
df62 20 a8 e7    jsr dart_spawn
df65 a9 01      lda #$01
df67 85 75      sta dart_dir_adjust
df69 20 a8 e7    jsr dart_spawn

                .done

df6c 4c 92 db    jmp update_enemies_handler_next

```


enemy_boss_vamp.asm

```

; things the bat entities need to know
; boss_x : center offset
; boss_y : center offset
; state_v0 : state
; state_v1 : spawn x to target
; state_v2 : x sine position
; state_v3 : mouth position
;          ( 0 = closed ; < 8 midframe ; > 8 open )
; state_v4 : bat circle size / other states counter
; state_v5 : bat visibility
; state_v6 : rng target
; state_v7 : mouth frame current
; enemy_ram_pc : mouth frame target
; enemy_ram_ac : idle_update counter
; enemy_ram_ex : arctang direction

boss_vamp_bat_spawn: subroutine
; x = bat slot in enemy ram
df6f a9 0b lda #boss_vamp_bat_id
df71 9d 00 03 sta enemy_ram_type,x
df74 60 rts

boss_vamp_bat_cycle: subroutine
; bats visible?
df75 a5 4f lda state_v5
df77 f0 69 beq .bats_not_visible
; collision detection
df79 a9 08 lda #$08
df7b 85 3d sta collision_0_w
df7d 85 3e sta collision_0_h
df7f 20 eb db jsr enemy_handle_damage_and_death

; update x pos
df82 bd 07 03 lda enemy_ram_ex,x
df85 85 87 sta temp01 ; position on circle
df87 a5 4e lda state_v4
df89 dd 05 03 cmp enemy_ram_ac,x
df8c 90 0c bcc .use_vamp_circle_size
df8e bd 05 03 lda enemy_ram_ac,x
df91 fe 05 03 inc enemy_ram_ac,x
df94 fe 05 03 inc enemy_ram_ac,x
df97 fe 05 03 inc enemy_ram_ac,x
.use_vamp_circle_size
df9a 85 88 sta temp02 ; size of circle
df9c 4a lsr
df9d 85 89 sta temp03 ; half of circle size
df9f a5 88 lda temp02
dfa1 a6 87 ldx temp01
dfa3 20 ad d4 jsr sine_of_scale
dfa6 18 clc
dfa7 65 7d adc boss_x

dfa9 38 sec
dfaa e5 89 sbc temp03
dfac a6 76 ldx enemy_ram_offset
dfae 99 03 02 sta oam_ram_x,y
; update y pos
dfb1 a5 87 lda temp01
dfb3 18 clc
dfb4 69 40 adc #$40
dfb6 aa tax
dfb7 a5 88 lda temp02
dfb9 20 ad d4 jsr sine_of_scale
dfbc 18 clc
dfbd 65 7e adc boss_y
dfbf 38 sec
dfc0 e5 89 sbc temp03
dfc2 a6 76 ldx enemy_ram_offset
dfc4 99 00 02 sta oam_ram_y,y
; interpret animation frame
dfc7 bd 07 03 lda enemy_ram_ex,x
dfca 4a lsr
dfcb 4a lsr
dfcc 29 03 and #%00000011
dfce 18 clc
dfcf 69 38 adc #$38
dfd1 99 01 02 sta oam_ram_spr,y
; palette
dfd4 ad be 03 lda $03be ; use vamp hc
dfd7 9d 06 03 sta enemy_ram_hc,x
dfda a9 02 lda #$02
dfdc 20 39 dc jsr enemy_set_palette
.done
dfdf 4c 92 db jmp update_enemies_handler_next

.bats_not_visible
dfe2 a9 ff lda #$ff
dfe4 99 00 02 sta oam_ram_y,y
dfe7 d0 f6 bne .done

; I WANT TO
; SUCK YOUR
; BLOODS !!
boss_vamp_spawn: subroutine
; claim eyeball slot
dfe9 a9 03 lda #boss_assist_id
dfef 8d d8 03 sta $03d8
; clear eyeball slot sprites
dfef a9 f0 lda #$f0 ; y pos of metasprite
dff0 a8 tay ; happens to match oam offset
dff1 20 ef da jsr sprite_4_set_y
; set eye color
dff4 a9 02 lda #$02
dff6 85 7f sta boss_eyes_pal
; set spawn x
dff8 a9 a0 lda #$a0
dff9 85 4c sta state_v2
; set spawn y
```

```

dffc a9 15      lda #$15
dffe 9d 03 03   sta enemy_ram_y,x
                ; bats are visible
e001 a9 01      lda #$01
e003 85 4f      sta state_v5
                ; minimum bat circle size
e005 a9 40      lda #$40
e007 85 4e      sta state_v4
                ; rando target
e009 a9 80      lda #$80
e00b 85 50      sta state_v6
                ; target count of bat underlings;
                ; #$0d = 13 but we `bpl` loopin`
e00d a0 0c      ldy #$0c
e00f 85 86      sta temp00
                ; set temp counters to 0
e011 a9 00      lda #$00
e013 85 87      sta temp01
e015 85 88      sta temp02
                .bat_spawn_loop
                ; a = animation counter / v1
                ; x = slot in enemy ram
e017 a5 87      lda temp01
e019 a6 88      ldx temp02
e01b 9d 07 03   sta enemy_ram_ex,x
e01e 18         clc
e01f 69 14      adc #$14
e021 85 87      sta temp01
e023 a9 08      lda #$08
e025 65 88      adc temp02
e027 85 88      sta temp02
e029 88         dey
e02a 10 eb      bpl .bat_spawn_loop
                .done
e02c 60         rts

boss_vamp_cycle: subroutine
e02d a9 0d      lda #$0d
e02f 85 3d      sta collision_0_w
e031 a9 10      lda #$10
e033 85 3e      sta collision_0_h

e035 e6 7a      inc boss_dmg_handle_true
e037 20 eb db   jsr enemy_handle_damage_and_death
e03a c6 7a      dec boss_dmg_handle_true
                ; IS DEAD?
e03c a5 7b      lda boss_death_happening
e03e f0 1b      beq .not_dead
e040 c9 02      cmp #$02
e042 f0 17      beq .bats_spawned
e044 e6 4f      inc state_v5 ; bats are visible
e046 a9 56      lda #$56
e048 85 4e      sta state_v4 ; bat circle size
e04a 85 51      sta state_v7 ; mouf open
                ; track bats are gonna spawn
e04c e6 7b      inc boss_death_happening

```

```

e04e b9 03 02   lda oam_ram_x,y
e051 85 7d      sta boss_x
e053 b9 00 02   lda oam_ram_y,y
e056 85 7e      sta boss_y
                ; reset enemy handler so bats show
e058 4c 64 db   jmp update_enemies_reset
                .bats_spawned
                .not_dead

                ; MOUTH HANDLER
                ; state_v7 : mouth frame current
                ; enemy_ram_pc : mouth frame target
e05b a5 51      lda state_v7
e05d dd 04 03   cmp enemy_ram_pc,x
e060 f0 08      beq .mouth_is_fine
e062 b0 04      bcs .mouth_close
                .mouth_open
e064 e6 51      inc state_v7
e066 d0 02      bne .mouth_is_fine
                .mouth_close
e068 c6 51      dec state_v7
                .mouth_is_fine

                ; STATE behavior
e06a 20 aa e0   jsr boss_vamp_update_state_delegator
e06d a6 76      ldx enemy_ram_offset

                ; SPRITE tiles
e06f a5 51      lda state_v7
e071 4a         lsr
e072 4a         lsr
e073 4a         lsr
e074 29 03      and #$03
e076 c9 03      cmp #$03
e078 d0 02      bne .good_frame
e07a a9 02      lda #$02
                .good_frame
e07c 0a         asl
e07d 18         clc
e07e 69 9a      adc #$9a
e080 20 b7 da   jsr sprite_4_set_sprite

                ; palette
e083 a9 01      lda #$01
e085 20 9d da   jsr sprite_4_set_palette
e088 d0 04      bne .not_hit
e08a a9 80      lda #$80
e08c 85 51      sta state_v7
                .not_hit

                ; eyeballs
e08e b9 03 02   lda oam_ram_x,y
e091 18         clc
e092 69 03      adc #$03
e094 85 86      sta temp00
e096 b9 00 02   lda oam_ram_y,y
e099 85 87      sta temp01
e09b b9 01 02   lda oam_ram_spr,y

```

```

e09e c9 9e      cmp #$9e
e0a0 d0 02      bne .dont_adjust_for_open_mouth
e0a2 c6 87      dec temp01
e0a4 20 b6 e5   .dont_adjust_for_open_mouth
                    jsr enemy_boss_eyes
e0a7 4c 92 db   jmp update_enemies_handler_next

```

```

                    boss_vamp_update_state_delegator:
e0aa a5 4a      lda state_v0
e0ac 18         clc
e0ad 69 36      adc #boss_vamp_state_jump_table_offset
e0af 85 86      sta temp00
e0b1 4c f1 c4   jmp jump_to_subroutine

```

```

                    boss_vamp_calc_boss_x_y: subroutine
                    ; calc x
e0b4 a5 4c      lda state_v2 ; x sine
e0b6 4a         lsr
e0b7 a8         tay
e0b8 bd 02 03   lda enemy_ram_x,x
e0bb 18         clc
e0bc 79 00 05   adc sine_6bits,y
e0bf 85 7d      sta boss_x
e0c1 a4 77      ldy enemy_oam_offset
e0c3 20 df da   jsr sprite_4_set_x
e0c6 a5 7d      lda boss_x
e0c8 18         clc
e0c9 69 04      adc #$04 ; add half of vampire size and subtract
half of bat size?
e0cb 85 7d      sta boss_x
                    ; calc y
e0cd bd 05 03   lda enemy_ram_ac,x
e0d0 aa         tax
e0d1 bd 00 fe   lda sine_table,x
e0d4 4a         lsr
e0d5 a6 76      ldx enemy_ram_offset
e0d7 18         clc
e0d8 7d 03 03   adc enemy_ram_y,x
e0db 85 7e      sta boss_y
e0dd 20 ef da   jsr sprite_4_set_y
e0e0 a5 7e      lda boss_y
e0e2 18         clc
e0e3 69 0a      adc #$0a ; add half of vampire size and subtract
half of bat size?
e0e5 85 7e      sta boss_y
e0e7 60         rts

```

```

                    boss_vamp_state_idle_update: subroutine
                    ; update x home pos
e0e8 bd 02 03   lda enemy_ram_x,x
e0eb c9 40      cmp #$40
e0ed b0 05      bcs .dont_inc_x
e0ef fe 02 03   inc enemy_ram_x,x
e0f2 e6 4b      inc state_v1
                    .dont_inc_x
                    .done_with_x
                    ; update x pos sine offset
e0f4 18         clc
e0f5 a9 03      lda #$03
e0f7 65 4c      adc state_v2
e0f9 85 4c      sta state_v2
e0fb 20 b4 e0   jsr boss_vamp_calc_boss_x_y
                    .next_state_check
e0fe fe 05 03   inc enemy_ram_ac,x
e101 bd 05 03   lda enemy_ram_ac,x
e104 c5 50      cmp state_v6
e106 d0 06      bne .dont_inc_state
                    .setup_next_state
e108 a5 26      lda rng0
e10a 85 50      sta state_v6
e10c e6 4a      inc state_v0
                    .dont_inc_state
e10e bd 05 03   lda enemy_ram_ac,x
e111 0a         asl
e112 0a         asl
e113 c9 80      cmp #$80
e115 b0 05      bcs .shrink_bat_circle
                    .grow_bat_circle
e117 e6 4e      inc state_v4
e119 4c 1e e1   jmp .bat_circle_adjust_done
                    .shrink_bat_circle
e11c c6 4e      dec state_v4
                    .bat_circle_adjust_done
                    ; BAT UPDATE LOOP
e11e a9 00      lda #$00
e120 85 86      sta temp00
                    .bat_update_loop
e122 a6 86      ldx temp00
                    ; update circle position
e124 fe 07 03   inc enemy_ram_ex,x
                    ; respawn if slot is empty
e127 bd 00 03   lda enemy_ram_type,x
e12a d0 07      bne .dont_respawn_bat
e12c 20 6f df   jsr boss_vamp_bat_spawn
e12f a9 20      lda #$20
e131 85 51      sta state_v7
                    .dont_respawn_bat
e133 a9 08      lda #$08
e135 18         clc
e136 65 86      adc temp00
e138 85 86      sta temp00
e13a c9 68      cmp #$68
e13c d0 e4      bne .bat_update_loop
e13e 60         rts

```

```

boss_vamp_state_suck_bats: subroutine
    ; open mouth
    lda #$40
    sta enemy_ram_pc,x
    lda #$10
    sta state_v7
    lda state_v4
    sec
    sbc #$05
    sta state_v4
    bcs .done
    .setup_next_state
    inc state_v0 ; next state
    lda #$00
    sta state_v4
    dec state_v5 ; hide bats
    lda oam_ram_x,y
    sta boss_x
    lda oam_ram_y,y
    sta boss_y
    .done
    rts

boss_vamp_state_shake: subroutine
    ; close mouth
    lda #$00
    sta enemy_ram_pc,x
    ; x shake
    jsr shake_8
    adc boss_x
    jsr sprite_4_set_x
    ; y shake
    jsr shake_8
    adc boss_y
    jsr sprite_4_set_y
    ; counter
    inc state_v4
    lda #15
    cmp state_v4
    bcs .done
    .setup_next_state
    ; next state
    inc state_v0
    ; reset v4 counter
    lda #$00
    sta state_v4
    ; load arctang dir into v6
    jsr enemy_get_direction_of_player
    sta enemy_ram_ex,x
    .done
    rts

```

```

boss_vamp_state_lunge: subroutine
    lda #<arctang_velocity_6.66
    sta arctang_velocity_lo
    jsr arctang_enemy_update
    ldy enemy_oam_offset
    lda oam_ram_x,y
    jsr sprite_4_set_x
    lda oam_ram_y,y
    jsr sprite_4_set_y
    inc state_v4
    lda #$14
    cmp state_v4
    bcs .done
    .setup_next_state
    lda enemy_ram_ex,x
    sec
    sbc #12
    bcs .valid_range
    adc #24
    .valid_range
    sta enemy_ram_ex,x
    inc state_v0
    .done
    rts

boss_vamp_state_retreat: subroutine
    lda #<arctang_velocity_6.66
    sta arctang_velocity_lo
    jsr arctang_enemy_update
    ldy enemy_oam_offset
    lda oam_ram_x,y
    jsr sprite_4_set_x
    lda oam_ram_y,y
    jsr sprite_4_set_y
    dec state_v4
    bne .done
    .setup_next_state
    inc state_v0
    lda #$00
    sta state_v4
    inc state_v5 ; show bats
    lda state_v1 ; x home on the way to #$60
    sta enemy_ram_x,x
    lda #$15
    sta enemy_ram_y,x
    jsr boss_vamp_calc_boss_x_y
    ; open mouth then close
    ; state_v7 : mouth frame current
    ; enemy_ram_pc : mouth frame target
    lda #$00
    sta enemy_ram_pc,x
    lda #$20
    sta state_v7
    .done
    rts

```

```

boss_vamp_state_blow_bats: subroutine
e1f3 e6 4e      inc state_v4
e1f5 e6 4e      inc state_v4
e1f7 e6 4e      inc state_v4
e1f9 a9 40      lda #$40
e1fb c5 4e      cmp state_v4
e1fd b0 04      bcs .done
               .setup_next_state
e1ff a9 00      lda #$00
e201 85 4a      sta state_v0
               .done
e203 60        rts

```

enemy_boss_scarab.asm

```

;
; boss_x : topleft offset
; boss_y : topleft offset
; state_v2 : x sine position
; state_v3 : y sine position
; state_v4 : x body position
; state_v5 : y body position
; state_v6 : x wing position
; state_v7 : y wing position

; scarab palette
; $0c, $27, $38

boss_scarab_spawn: subroutine
; claim highest 4 sprite slot for beetle body
; ldx #boss_scarab_id
; stx $03b8
; claim 4 more slots for the wings
e204 a9 03      lda #boss_assist_id
e206 8d c0 03    sta $03c0
e209 8d c8 03    sta $03c8
e20c 8d d0 03    sta $03d0
e20f 8d d8 03    sta $03d8
; setup initial state
e212 a9 10      lda #$10
e214 85 7d      sta boss_x
e216 a9 1c      lda #$1c
e218 85 7e      sta boss_y
e21a a9 00      lda #$00
e21c 85 4d      sta state_v3
e21e 85 50      sta state_v6
e220 85 51      sta state_v7
e222 a9 c0      lda #$c0
e224 85 4c      sta state_v2
e226 60        rts

; KNOW
; THYSELF
; DEATHLESS

scarab_oam_offset_table:
e227 byte      $c0, $d0, $e0, $f0
scarab_wing_y_offset_table:
e22b byte      $f4, $e4, $0c, $1c
scarab_wing_sprite_table:
e22f byte      $80, $60, $80, $60

```

```

boss_scarab_cycle: subroutine
e233 a9 10    lda #$10
e235 85 3d    sta collision_0_w
e237 85 3e    sta collision_0_h

e239 e6 7a    inc boss_dmg_handle_true
e23b 20 eb db jsr enemy_handle_damage_and_death
e23e c6 7a    dec boss_dmg_handle_true

; MAIN BODY

; x pos
e240 e6 4c    inc state_v2
e242 a6 4c    ldx state_v2
e244 bd 00 fe lda sine_table,x
e247 4a       lsr
e248 4a       lsr
e249 18       clc
e24a 65 7d    adc boss_x
e24c a6 76    ldx enemy_ram_offset
e24e 9d 02 03 sta enemy_ram_x,x
e251 85 4e    sta state_v4
e253 20 df da jsr sprite_4_set_x

; y pos
e256 e6 4d    inc state_v3
e258 e6 4d    inc state_v3
e25a a6 4d    ldx state_v3
e25c bd 00 fe lda sine_table,x
e25f 4a       lsr
e260 18       clc
e261 65 7e    adc boss_y
e263 a6 76    ldx enemy_ram_offset
e265 9d 03 03 sta enemy_ram_y,x
e268 85 4f    sta state_v5
e26a 20 ef da jsr sprite_4_set_y

; sprite
e26d a9 a0    lda #$a0
e26f 20 b7 da jsr sprite_4_set_sprite

; WINGS
; twitch offset
e272 e6 51    inc state_v7
e274 a5 51    lda state_v7
e276 c9 03    cmp #3
e278 d0 04    bne .state_v7_dont_reset
e27a a9 00    lda #0
e27c 85 51    sta state_v7
.state_v7_dont_reset
e27e a5 24    lda wtf
e280 29 01    and #1
e282 85 50    sta state_v6

```

```

; palette
e284 a9 02    lda #$02
e286 20 9d da jsr sprite_4_set_palette
e289 f0 03    beq .hit
e28b 38       sec
e28c e9 01    sbc #$01

.hit
e28e 85 87    sta temp01

; calc x pos of wing set
e290 a5 4e    lda state_v4
e292 38       sec
e293 e9 06    sbc #$6
e295 18       clc
e296 65 50    adc state_v6
e298 85 86    sta temp00

e29a a2 03    ldx #$03
.wing_sprites_loop
e29c bc 27 e2 ldy scarab_oam_offset_table,x
; x
e29f a5 86    lda temp00
e2a1 20 df da jsr sprite_4_set_x
; y
e2a4 a5 4f    lda state_v5
e2a6 18       clc
e2a7 7d 2b e2 adc scarab_wing_y_offset_table,x
e2aa 20 ef da jsr sprite_4_set_y
; sprite
e2ad bd 2f e2 lda scarab_wing_sprite_table,x
; flipped wing?
e2b0 e0 02    cpx #$02
e2b2 90 09    bcc .no_flip
e2b4 20 cb da jsr sprite_4_set_sprite_flip
; att flipped
e2b7 a5 87    lda temp01
e2b9 09 80    ora #$80
e2bb d0 05    bne .sprite_done
.no_flip
e2bd 20 b7 da jsr sprite_4_set_sprite
; att
e2c0 a5 87    lda temp01
.sprite_done
e2c2 20 aa da jsr sprite_4_set_palette_no_process
e2c5 ca       dex
e2c6 10 d4    bpl .wing_sprites_loop

```

enemy_boss_swordtner.asm

```

; SHOOT
e2c8 a5 24    lda wtf
e2ca 29 03    and #$03
e2cc d0 23    bne .done
e2ce a5 24    lda wtf
e2d0 29 20    and #$20
e2d2 d0 1d    bne .done

.dart_fire
e2d4 a5 4e    lda state_v4
e2d6 18       clc
e2d7 69 10    adc #$10
e2d9 85 71    sta dart_x_origin
e2db a5 4f    lda state_v5
e2dd 18       clc
e2de 69 04    adc #$04
e2e0 85 72    sta dart_y_origin
e2e2 a9 02    lda #$02
e2e4 85 73    sta dart_velocity
e2e6 a9 fe    lda #$fe
e2e8 85 74    sta dart_sprite
e2ea a9 00    lda #$00
e2ec 85 75    sta dart_dir_adjust
e2ee 20 a8 e7 jsr dart_spawn

.done
e2f1 a5 24    lda wtf
e2f3 29 0f    and #$0f
e2f5 d0 08    bne .done_x_entrance
e2f7 a5 7d    lda boss_x
e2f9 c9 40    cmp #$40
e2fb f0 02    beq .done_x_entrance
e2fd e6 7d    inc boss_x
.done_x_entrance
e2ff 4c 92 db jmp update_enemies_handler_next

```

```

; things the bat entities need to know
; boss_x : x body position
; boss_y : y body position

```

```

; state_v0 : current mode
; 0 - bounce around and fire
; 1 - lunge at player and around to box
; 2 - aim and shake
; state_v1 : cycle counter
; state_v2 : velocity
; state_v4 : emitter type
; state_v5 : scratch for spawn loop
; state_v6 : alt face countdown
; state_v7 : 2nd form true

```

```

; swordtner palette
; $07, $17, $3d
; $0c, $2d, $31

```

```

boss_swordtner_spawn: subroutine
; claim 4 more slots for rest of body
e302 a9 03    lda #boss_assist_id
e304 8d c0 03 sta $03c0
e307 8d c8 03 sta $03c8
e30a 8d d0 03 sta $03d0
e30d 8d d8 03 sta $03d8
; claim 1 more slot for eyeballs
e310 8d 40 03 sta $0340
; setup initial state
jsr get_oam_offset_from_ram_offset
e313 20 52 dc lda #$20
e316 a9 20    sta oam_ram_x,y
e318 99 03 02 sta oam_ram_y,y
e31b 99 00 02 sta oam_ram_y,y
e31e a9 15    lda #$15
e320 9d 07 03 sta enemy_ram_ex,x
; set eye color
e323 a9 01    lda #$01
e325 85 7f    sta boss_eyes_pal
; offset song position
e327 a9 0e    lda #$0e
e329 85 c3    sta audio_pattern_pos
e32b 60       rts

```

```

; SWORDTNER
; PROVERB
; HERE

```

```

arc_bounce_x:
; if ex < $0c then ex = $0c - ex
e32c hex      0b 0b 0a 09 08 07 05 05 04 03 02 01
; if ex == $0c then ex = 0
; if ex > $0c then ex = $17 - (ex - $0d)
e338 hex      01 17 16 15 14 13 13 11 10 0f 0e 0d
arc_bounce_y:
; if ex == $00 then ex = 0
; if ex < $0c then ex = $18 - ex
e344 hex      01 17 16 15 14 13 13 11 10 0f 0e 0d
; if ex > $0c then ex = $24 - ex
e350 hex      0b 0b 0a 09 08 07 05 05 04 03 02 01

```

```

swordtner metasprite_offset:
e35c byte      #$10, #$20, #$30, #$40
swordtner metasprite_id:
e360 byte      #$82, #$a2, #$a2, #$c2

```

```

swordtner_rng_bullet:
e364 20 52 c1  jsr get_next_random
e367 29 07      and #$07
e369 18         clc
e36a 69 24      adc #$24
e36c 85 74      sta dart_sprite
e36e 60         rts

```

```

boss_swordtner_cycle: subroutine
; shrink width of hit box
e36f a9 04      lda #$04
e371 85 3d      sta collision_0_w
e373 65 3b      adc collision_0_x
e375 85 3b      sta collision_0_x
; check for player collision with blade
e377 a9 50      lda #$50
e379 85 3e      sta collision_0_h
e37b 20 35 cf    jsr player_collision_detect
e37e f0 0f      beq .no_collision
e380 20 dd db    jsr enemy_gives_damage
e383 a9 00      lda #$00
e385 85 4b      sta state_v1
e387 a9 01      lda #$01
e389 85 4a      sta state_v0
; change facial expression
e38b a9 20      lda #$20
e38d 85 50      sta state_v6
.no_collision

```

```

; hitbox is face of swordtner
e38f a9 0f      lda #$0f
e391 85 3e      sta collision_0_h
e393 a9 12      lda #$12
e395 65 3c      adc collision_0_y
e397 85 3c      sta collision_0_y
e399 e6 7a      inc boss_dmg_handle_true
e39b 20 eb db    jsr enemy_handle_damage_and_death
e39e c9 00      cmp #$00
e3a0 f0 04      beq .dont_change_face
e3a2 a9 10      lda #$10
e3a4 85 50      sta state_v6
.dont_change_face
e3a6 c6 7a      dec boss_dmg_handle_true

```

```

; check for 2nd Form
e3a8 a5 51      lda state_v7
e3aa d0 11      bne .2nd_form_done
e3ac bd 01 03    lda enemy_ram_hp,x
e3af c9 19      cmp #25
e3b1 b0 0a      bcs .2nd_form_done
e3b3 a9 64      lda #100
e3b5 7d 01 03    adc enemy_ram_hp,x
e3b8 9d 01 03    sta enemy_ram_hp,x
e3bb e6 51      inc state_v7
.2nd_form_done

```

```

; both modes use this
; x
e3bd a5 7d      lda boss_x
e3bf 18         clc
e3c0 69 04      adc #$04
e3c2 85 71      sta dart_x_origin
; y
e3c4 a5 7e      lda boss_y
e3c6 18         clc
e3c7 69 0c      adc #$0c
e3c9 85 72      sta dart_y_origin

```

```

; act on current mode
e3cb a5 4a      lda state_v0
e3cd 18         clc
e3ce 69 3c      adc #boss_swordtner_state_jump_table_offset
e3d0 20 f1 c4    jsr jump_to_subroutine

```

```

; y
e3d3 b9 00 02    lda oam_ram_y,y
; y check for poor range
.check_too_low
e3d6 c9 88      cmp #$88
e3d8 90 05      bcc .check_too_high
e3da a9 8a      lda #$8a
e3dc 99 00 02    sta oam_ram_y,y
.check_too_high

```



```

e3df c9 08      cmp #$08
e3e1 b0 05      bcs .check_y_softer_bounce
e3e3 a9 06      lda #$06
e3e5 99 00 02   sta oam_ram_y,y
               .check_y_softer_bounce
               ; y check for bounce
e3e8 c9 08      cmp #$08
e3ea 90 06      bcc .bounce_y
e3ec c9 88      cmp #$88
e3ee b0 02      bcs .bounce_y
e3f0 90 0b      bcc .dont_bounce_y
               .bounce_y
e3f2 bc 07 03   ldy enemy_ram_ex,x
e3f5 b9 44 e3   lda arc_bounce_y,y
e3f8 9d 07 03   sta enemy_ram_ex,x
e3fb a4 77      ldy enemy_oam_offset
               .dont_bounce_y

               .set_boss_x_y
               ; set boxx x,y
e3fd b9 03 02   lda oam_ram_x,y
e400 85 7d      sta boss_x
e402 20 df da   jsr sprite_4_set_x
e405 b9 00 02   lda oam_ram_y,y
e408 85 7e      sta boss_y
e40a 20 ef da   jsr sprite_4_set_y

               ; SWORDTNER
               ; MAIN BODY

               ; handle sprite
e40d a9 62      lda #$62
e40f 20 b7 da   jsr sprite_4_set_sprite

e412 a2 03      ldx #$03
               .next_meta_sprite
e414 bd 5c e3   lda swordtner metasprite_offset,x
e417 18         clc
e418 65 77      adc enemy_oam_offset
e41a a8         tay
e41b a5 7d      lda boss_x
e41d 20 df da   jsr sprite_4_set_x
e420 a5 7e      lda boss_y
e422 7d 5c e3   adc swordtner metasprite_offset,x
e425 20 ef da   jsr sprite_4_set_y
               ; check for face animation
e428 e0 00      cpx #$00
e42a d0 0a      bne .not_face_sprite
e42c a5 50      lda state_v6
e42e f0 12      beq .normal_sprite
e430 c6 50      dec state_v6
e432 a9 84      lda #$84
e434 d0 0f      bne .plot_sprite

```

```

               .not_face_sprite
               ; check for 2nd form
e436 a5 51      lda state_v7
e438 f0 08      beq .normal_sprite
e43a bd 60 e3   lda swordtner metasprite_id,x
e43d 18         clc
e43e 69 02      adc #$02
e440 d0 03      bne .plot_sprite
               .normal_sprite
e442 bd 60 e3   lda swordtner metasprite_id,x
               .plot_sprite
e445 20 b7 da   jsr sprite_4_set_sprite
e448 ca         dex
e449 10 c9      bpl .next_meta_sprite

e44b a6 76      ldx enemy_ram_offset
e44d a4 77      ldy enemy_oam_offset

               ; palette
e44f a9 01      lda #$01
e451 20 9d da   jsr sprite_4_set_palette
e454 f0 03      beq .hit
e456 18         clc
e457 69 01      adc #$01
               .hit
e459 a0 c0      ldy #$c0
e45b 20 aa da   jsr sprite_4_set_palette_no_process
e45e a0 d0      ldy #$d0
e460 20 aa da   jsr sprite_4_set_palette_no_process
e463 a0 e0      ldy #$e0
e465 20 aa da   jsr sprite_4_set_palette_no_process
e468 a0 f0      ldy #$f0
e46a 20 aa da   jsr sprite_4_set_palette_no_process

               ; move last sprite to higher spot
               ; make room for eyeballs
e46d a2 03      ldx #$03
               .migrate_sprite_loop
e46f bd fc 02   lda $02fc,x
e472 9d 40 02   sta $0240,x
e475 ca         dex
e476 10 f7      bpl .migrate_sprite_loop

               ;ldx enemy_ram_offset
               ;ldy enemy_oam_offset

               ; eyeballs setup
e478 a4 77      ldy enemy_oam_offset
e47a b9 03 02   lda oam_ram_x,y
e47d 18         clc
e47e 69 04      adc #$04
e480 85 86      sta temp00
e482 b9 00 02   lda oam_ram_y,y
e485 18         clc
e486 69 10      adc #$10
e488 85 87      sta temp01
e48a 20 b6 e5   jsr enemy_boss_eyes

```

```

.done
e48d 4c 92 db jmp update_enemies_handler_next

```

```

swordtner_inside_bouncy_box_x: subroutine
; returns 0 in a if false
e490 b9 03 02 lda oam_ram_x,y
e493 c9 10 cmp #$10
e495 90 08 bcc .outside_bouncy_box_x
e497 c9 60 cmp #$60
e499 b0 04 bcs .outside_bouncy_box_x
e49b a9 00 lda #$00
e49d 90 02 bcc .inside_bouncy_box_x
.outside_bouncy_box_x
e49f a9 ff lda #$ff
.inside_bouncy_box_x
e4a1 60 rts

```

```

swordtner_emit_enemies: subroutine
e4a2 a9 02 lda #$02
e4a4 85 4f sta state_v5
.spawn_loop
e4a6 20 86 dc jsr get_enemy_slot_1_sprite
e4a9 b0 19 bcs .done
e4ab 18 clc
e4ac a9 05 lda #spark_id
e4ae 65 4f adc state_v5
e4b0 20 77 da jsr enemy_spawn_delegator
e4b3 20 52 dc jsr get_oam_offset_from_ram_offset
e4b6 a5 71 lda dart_x_origin
e4b8 99 03 02 sta oam_ram_x,y
e4bb a5 72 lda dart_y_origin
e4bd 99 00 02 sta oam_ram_y,y
e4c0 c6 4f dec state_v5
e4c2 10 e2 bpl .spawn_loop
.done
e4c4 20 5e fa jsr sfx_shoot_bullet
e4c7 a6 76 ldx enemy_ram_offset
e4c9 a4 77 ldy enemy_oam_offset
e4cb 60 rts

```

```

swordtner_emit_projectiles: subroutine
; velocity
e4cc a9 02 lda #$02
e4ce 85 73 sta dart_velocity
; sprite
e4d0 a9 fe lda #$fe
;lda #$00
e4d2 85 74 sta dart_sprite

```

```

; dir adjustor
e4d4 a9 00 lda #$00
e4d6 85 75 sta dart_dir_adjust
e4d8 20 64 e3 jsr swordtner_rng_bullet
e4db 20 a8 e7 jsr dart_spawn
e4de a9 ff lda #$ff
e4e0 85 75 sta dart_dir_adjust
e4e2 20 64 e3 jsr swordtner_rng_bullet
e4e5 20 a8 e7 jsr dart_spawn
e4e8 a9 01 lda #$01
e4ea 85 75 sta dart_dir_adjust
e4ec 20 64 e3 jsr swordtner_rng_bullet
e4ef 20 a8 e7 jsr dart_spawn
e4f2 60 rts

```

```

; mode 0 - bounce around and fire
boss_swordtner_mode_0: subroutine

; fix x position after player death
e4f3 b9 03 02 lda oam_ram_x,y
e4f6 d0 08 bne .dont_fix_x
e4f8 a9 20 lda #$20
e4fa 99 03 02 sta oam_ram_x,y
e4fd 99 00 02 sta oam_ram_y,y
.dont_fix_x

.bounce_and_fire
; change face before firing
e500 a5 c3 lda audio_pattern_pos
e502 c9 09 cmp #$09
e504 d0 04 bne .no_face_change
e506 a9 05 lda #$05
e508 85 50 sta state_v6
.no_face_change
e50a a5 c3 lda audio_pattern_pos
e50c c9 08 cmp #8
e50e 90 07 bcc .before_snare
e510 c9 0c cmp #12
e512 b0 0b bcs .after_snare
e514 4c 3a e5 jmp .movement_skip

.before_snare
e517 aa tax
e518 bd f0 ff lda arctang_velocities_lo,x
e51b 85 4c sta state_v2
e51d d0 04 bne .bounce_movement

```

```

.after_snare
e51f a9 80 lda #<arctang_velocity_6.66
e521 85 4c sta state_v2

```

```

; MOVEMENT
.bounce_movement
e523 a6 76    ldx enemy_ram_offset
e525 85 84    sta arctang_velocity_lo
e527 20 f6 ce  jsr arctang_enemy_update
; x
e52a 20 90 e4  jsr swordtner_inside_bouncy_box_x
e52d f0 0b    beq .dont_bounce_x
.bounce_x
e52f bc 07 03  ldy enemy_ram_ex,x
e532 b9 2c e3  lda arc_bounce_x,y
e535 9d 07 03  sta enemy_ram_ex,x
e538 a4 77    ldy enemy_oam_offset
.dont_bounce_x

.movement_skip

; spawn/fire on boss fight music main snare
e53a a5 c1    lda audio_frame_counter
e53c c9 05    cmp #5
e53e d0 24    bne .dont_emit
e540 a5 c3    lda audio_pattern_pos
e542 c9 0a    cmp #10
e544 d0 1e    bne .dont_emit
e546 a5 4b    lda state_v1
e548 c9 02    cmp #$02
e54a d0 07    bne .dont_next_state
e54c e6 4a    inc state_v0
e54e a9 00    lda #$00
e550 85 4b    sta state_v1
e552 60      rts
.dont_next_state
e553 a5 4e    lda state_v4
e555 29 01    and #$01
e557 d0 06    bne .emit_enemies
.emit_projectiles
e559 20 cc e4  jsr swordtner_emit_projectiles
e55c 4c 62 e5  jmp .inc_state
.emit_enemies
e55f 20 a2 e4  jsr swordtner_emit_enemies
.inc_state
e562 e6 4b    inc state_v1
.dont_emit

e564 60      rts

```

```

; mode 1 - aim for player and warn of attack
boss_swordtner_mode_shake: subroutine
e565 a5 4b    lda state_v1
e567 d0 13    bne .init_done
.init
e569 e6 4b    inc state_v1
e56b 20 12 ce  jsr enemy_get_direction_of_player
e56e 18        clc
e56f 69 01    adc #$01
e571 20 ae ce  jsr arctang_bound_dir
e574 9d 07 03  sta enemy_ram_ex,x
e577 ad f1 ff  lda arctang_velocities_lo+1
e57a 85 4c    sta state_v2
.init_done
; x shake
e57c 20 66 c1  jsr shake_8
e57f 65 7d    adc boss_x
e581 20 df da  jsr sprite_4_set_x
; y shake
e584 20 66 c1  jsr shake_8
e587 65 7e    adc boss_y
e589 20 ef da  jsr sprite_4_set_y
; check if done
e58c e6 4b    inc state_v1
e58e a5 4b    lda state_v1
e590 c9 10    cmp #$10
e592 d0 04    bne .done
e594 a9 02    lda #$02
e596 85 4a    sta state_v0
.done
e598 60      rts

; mode 2 - lunges at player
boss_swordtner_mode_2: subroutine
e599 e6 4b    inc state_v1
e59b a5 4b    lda state_v1
e59d c9 40    cmp #$40
e59f 90 0d    bcc .lunge_keep_going
e5a1 20 90 e4  jsr swordtner_inside_bouncy_box_x
e5a4 d0 08    bne .lunge_keep_going
; next state
e5a6 e6 4e    inc state_v4
e5a8 a9 00    lda #$00
e5aa 85 4a    sta state_v0
e5ac 85 4b    sta state_v1
.lunge_keep_going
e5ae a5 4c    lda state_v2
e5b0 85 84    sta arctang_velocity_lo
e5b2 20 f6 ce  jsr arctang_enemy_update
.lunge_done
e5b5 60      rts

```

enemy_boss_eyes.asm

```
        ; temp00 = eye sprite x origin
        ; temp01 = eye sprite y origin

enemy_boss_eyes: subroutine
        ; EYE SPRITE
        .now_lets_add_eyes
e5b6  a2 fc      ldx #$fc
e5b8  a9 a9      lda #$a9
e5ba  9d 01 02   sta oam_ram_spr,x
e5bd  a5 7f      lda boss_eyes_pal
e5bf  9d 02 02   sta oam_ram_att,x
        ; find x
e5c2  a5 86      lda temp00
e5c4  9d 03 02   sta oam_ram_x,x
        ; check if rudy is left of vampire
e5c7  c5 13      cmp player_x_hi
e5c9  90 05      bcc .looking_right
        .looking_left
e5cb  a9 99      lda #$99
e5cd  9d 01 02   sta oam_ram_spr,x
        ;inc oam_ram_x,x
        .looking_right
        ; find y
e5d0  a4 87      ldy temp01
e5d2  c8         iny
e5d3  98         tya
e5d4  a4 77      ldy enemy_oam_offset
e5d6  9d 00 02   sta oam_ram_y,x
e5d9  18         clc
e5da  69 1c      adc #$1c
e5dc  38         sec
e5dd  e5 15      sbc player_y_hi
e5df  90 14      bcc .looking_down
e5e1  18         clc
e5e2  69 cc      adc #$cc
e5e4  b0 0b      bcs .looking_up
        .looking_across
e5e6  bd 01 02   lda oam_ram_spr,x
e5e9  c9 99      cmp #$99
e5eb  f0 03      beq .adjust_for_left_looking
e5ed  4c f8 e5   jmp .done
        .adjust_for_left_looking
e5f0  60         rts
        .looking_up
e5f1  de 00 02   dec oam_ram_y,x
e5f4  60         rts
        .looking_down
e5f5  fe 00 02   inc oam_ram_y,x
        .done
e5f8  60         rts
```

enemy_ant.asm

```
ant_spawn: subroutine
        ; x is set by enemy spawner
e5f9  a9 fc      lda #$fc
e5fb  9d 02 03   sta enemy_ram_x,x ; x pos
e5fe  a9 00      lda #$00
e600  9d 05 03   sta enemy_ram_ac,x ; animation counter
e603  8a         txa
e604  4a         lsr
e605  4a         lsr
e606  4a         lsr
e607  29 01      and #1
e609  f0 0e      beq .dash_ant
        .ceiling_ant
e60b  9d 07 03   sta enemy_ram_ex,x
e60e  a9 03      lda #$03
e610  9d 03 03   sta enemy_ram_y,x ; y ant pos
e613  a9 0c      lda #$0c
e615  9d 04 03   sta enemy_ram_pc,x ; y dart origin
e618  60         rts
        .dash_ant
e619  9d 07 03   sta enemy_ram_ex,x
e61c  a9 ad      lda #$ad
e61e  9d 03 03   sta enemy_ram_y,x ; y ant pos
e621  9d 04 03   sta enemy_ram_pc,x ; y dart origin
e624  60         rts

ant_cycle: subroutine
e625  a9 10      lda #$10
e627  85 3d      sta collision_0_w
e629  a9 10      lda #$10
e62b  85 3e      sta collision_0_h
e62d  20 eb db   jsr enemy_handle_damage_and_death

e630  a5 24      lda wtf
e632  4a         lsr
e633  4a         lsr
e634  29 3f      and #$3f
e636  c9 37      cmp #$37
e638  90 41      bcc .normal_walking
        .stop_and_shoot
        ; frames default to spazzing
        ; frames f0-f8 butt up
        ; frame f8 shoot dart
        ; frames f9-ff butt down
e63a  a5 24      lda wtf
e63c  c9 f9      cmp #$f9
e63e  b0 34      bcs .butt_down
e640  c9 f8      cmp #$f8
e642  b0 17      bcs .dart_fire
e644  c9 f0      cmp #$f0
e646  b0 0c      bcs .butt_up
        .butt_shake
e648  4a         lsr
e649  29 01      and #$01
```

```

e64b 0a      asl
e64c 18      clc
e64d 69 e8   adc #$e8
e64f 9d 05 03 sta enemy_ram_ac,x
e652 d0 39   bne .dont_advance

      .butt_up
e654 a9 ea   lda #$ea
e656 9d 05 03 sta enemy_ram_ac,x
e659 d0 32   bne .dont_advance

      .dart_fire
e65b bd 02 03 lda enemy_ram_x,x
e65e 85 71   sta dart_x_origin
e660 bd 04 03 lda enemy_ram_pc,x
e663 85 72   sta dart_y_origin
e665 a9 03   lda #$03
e667 85 73   sta dart_velocity
e669 a9 00   lda #$00
e66b 85 74   sta dart_sprite
e66d 85 75   sta dart_dir_adjust
e66f 20 a8 e7 jsr dart_spawn
e672 d0 19   bne .dont_advance

      .butt_down
e674 a9 e8   lda #$e8
e676 9d 05 03 sta enemy_ram_ac,x
e679 d0 12   bne .dont_advance

      .normal_walking
e67b 29 03   and #$03
e67d 0a      asl
e67e 18      clc
e67f 69 e0   adc #$e0
e681 9d 05 03 sta enemy_ram_ac,x
e684 a5 24   lda wtf
e686 29 03   and #$03
e688 d0 03   bne .dont_advance
e68a fe 02 03 inc enemy_ram_x,x

      .dont_advance
e68d bd 02 03 lda enemy_ram_x,x
e690 20 df da jsr sprite_4_set_x
e693 bd 03 03 lda enemy_ram_y,x
e696 20 ef da jsr sprite_4_set_y
e699 bd 07 03 lda enemy_ram_ex,x
e69c d0 0a   bne .ceiling_ant

      .dash_ant
e69e bd 05 03 lda enemy_ram_ac,x
e6a1 20 b7 da jsr sprite_4_set_sprite
e6a4 a9 01   lda #$01
e6a6 d0 08   bne .ant_pos_done

      .ceiling_ant
e6a8 bd 05 03 lda enemy_ram_ac,x
e6ab 20 cb da jsr sprite_4_set_sprite_flip
e6ae a9 81   lda #$81

      .ant_pos_done
e6b0 20 9d da jsr sprite_4_set_palette

      .done
e6b3 4c 92 db jmp update_enemies_handler_next

```

enemy_birb.asm

```

      birb_spawn: subroutine
      ; x is set by enemy spawner

e6b6 a5 27   lda rng1
e6b8 9d 04 03 sta enemy_ram_pc,x ; pattern counter
e6bb a5 28   lda rng2
e6bd 9d 05 03 sta enemy_ram_ac,x ; animation counter
e6c0 20 52 c1 jsr get_next_random
e6c3 4a      lsr
e6c4 18      clc
e6c5 69 0c   adc #$0c
e6c7 9d 03 03 sta enemy_ram_y,x ; y pos
e6ca 60      rts

      ;;;; HANDLING BIRB

      birb_cycle: subroutine
e6cb a9 08   lda #$08
e6cd 85 3d   sta collision_0_w
e6cf a9 05   lda #$05
e6d1 85 3e   sta collision_0_h
e6d3 20 eb db jsr enemy_handle_damage_and_death

      ; update pattern
e6d6 fe 04 03 inc enemy_ram_pc,x
e6d9 fe 04 03 inc enemy_ram_pc,x

      ; set x position
      ; get x pattern position
      ; add it to base x position
e6dc bd 04 03 lda enemy_ram_pc,x
e6df aa      tax
e6e0 bd 00 fe lda sine_table,x
e6e3 4a      lsr
e6e4 4a      lsr
e6e5 4a      lsr
e6e6 a6 76   ldx enemy_ram_offset
e6e8 fe 02 03 inc enemy_ram_x,x
e6eb 18      clc
e6ec 7d 02 03 adc enemy_ram_x,x
e6ef 99 03 02 sta oam_ram_x,y

      ; set y position
e6f2 bd 04 03 lda enemy_ram_pc,x
e6f5 18      clc
e6f6 69 40   adc #$40
e6f8 aa      tax
e6f9 bd 00 fe lda sine_table,x
e6fc 4a      lsr

```

```

e6fd 4a      lsr
e6fe 4a      lsr
e6ff a6 76   ldx enemy_ram_offset
e701 18      clc
e702 7d 03 03 adc enemy_ram_y,x
e705 99 00 02 sta oam_ram_y,y

; current sprite
e708 bd 05 03 lda enemy_ram_ac,x
e70b 18      clc
e70c 69 14   adc #$14
e70e 9d 05 03 sta enemy_ram_ac,x
e711 4a      lsr
e712 4a      lsr
e713 4a      lsr
e714 4a      lsr
e715 4a      lsr
e716 4a      lsr
e717 18      clc
e718 69 2c   adc #$2c
e71a 99 01 02 sta oam_ram_spr,y
.frame_done
e71d a9 01   lda #$01
e71f 20 39 dc jsr enemy_set_palette
.done
e722 4c 92 db jmp update_enemies_handler_next

```

enemy_chomps.asm

```

chomps_spawn: subroutine
; x is set by enemy spawner
e725 a9 c1   lda #$c1
e727 9d 05 03 sta enemy_ram_ac,x
e72a 20 52 dc jsr get_oam_offset_from_ram_offset
e72d a9 00   lda #$00
e72f 99 03 02 sta oam_ram_x,y
e732 60      rts

chomps_cycle: subroutine
e733 a9 08   lda #$08
e735 85 3d   sta collision_0_w
e737 85 3e   sta collision_0_h
e739 20 eb db jsr enemy_handle_damage_and_death

e73c b9 03 02 lda oam_ram_x,y
e73f c9 01   cmp #$01
e741 b0 09   bcs .dont_reset_y
; y
e743 20 d1 db jsr enemy_spawn_random_y_pos
e746 99 00 02 sta oam_ram_y,y
e749 99 04 02 sta oam_ram_y+4,y
.dont_reset_y

; x movement
e74c fe 04 03 inc enemy_ram_pc,x ; actual x origin position
e74f fe 05 03 inc enemy_ram_ac,x
e752 fe 05 03 inc enemy_ram_ac,x
e755 a5 24   lda wtf
e757 29 01   and #$01
e759 d0 03   bne .no_extra_inc
e75b fe 05 03 inc enemy_ram_ac,x
.no_extra_inc
e75e bd 05 03 lda enemy_ram_ac,x
e761 aa      tax
e762 bd 00 fe lda sine_table,x
e765 4a      lsr
e766 4a      lsr
e767 a6 76   ldx enemy_ram_offset
e769 18      clc
e76a 7d 04 03 adc enemy_ram_pc,x
; x
e76d 99 03 02 sta oam_ram_x,y
e770 18      clc
e771 69 01   adc #$01
e773 99 07 02 sta oam_ram_x+4,y

; winder sprite
e776 a5 24   lda wtf
e778 4a      lsr
e779 4a      lsr

```

```

e77a 29 03      and #$03
e77c 18         clc
e77d 69 34      adc #$34
e77f 99 01 02   sta oam_ram_spr,y

; mouth sprite
e782 bd 05 03   lda enemy_ram_ac,x
e785 18         clc
e786 69 d0      adc #$d0
e788 c9 80      cmp #$80
e78a b0 04      bcs .not_closed
e78c a9 58      lda #$58
e78e d0 0a      bne .sprite_done
; .not_closed
e790 c9 a0      cmp #$a0
e792 b0 04      bcs .half_open
e794 a9 68      lda #$68
e796 d0 02      bne .sprite_done
; .half_open
e798 a9 78      lda #$78
; .sprite_done
e79a 99 05 02   sta oam_ram_spr+4,y

; .frame_done
e79d a9 01      lda #$01
e79f 20 39 dc   jsr enemy_set_palette
e7a2 99 06 02   sta oam_ram_att+4,y
; .done
e7a5 4c 92 db   jmp update_enemies_handler_next

```

enemy_dart.asm

```

; set velocity after calling spawn with :
; lda #<arctang_velocity_3.33
; sta enemy_ram_pc,x

; spawn settings go in:
;   dart_velocity
;   dart_x_origin
;   dart_y_origin
;   dart_dir_adjust
;   dart_sprite

dart_spawn: subroutine
; put x,y origin in:
; collision_0_x
; collision_0_y
; .checks
;   dart_frame_max is set at top of enemy_handler
e7a8 a5 70      lda dart_frame_max
e7aa f0 5f      beq .done
e7ac c6 70      dec dart_frame_max
e7ae 20 5f dc   jsr get_enemy_slot_next
e7b1 90 58      bcc .done
; .spawn
e7b3 a9 0a      lda #dart_id
e7b5 9d 00 03   sta enemy_ram_type,x
e7b8 a9 00      lda #0
e7ba 9d 02 03   sta enemy_ram_x,x ; sub pixel pos
e7bd 9d 03 03   sta enemy_ram_y,x ; sub pixel pos
; velocity
e7c0 a4 73      ldy dart_velocity
e7c2 b9 f0 ff   lda arctang_velocities_lo,y
e7c5 9d 04 03   sta enemy_ram_pc,x
; get y register and set origin
e7c8 20 52 dc   jsr get_oam_offset_from_ram_offset
e7cb a5 71      lda dart_x_origin
e7cd 99 03 02   sta oam_ram_x,y
e7d0 a5 72      lda dart_y_origin
e7d2 99 00 02   sta oam_ram_y,y
; direction
e7d5 20 12 ce   jsr enemy_get_direction_of_player
e7d8 18         clc
e7d9 65 75      adc dart_dir_adjust
e7db 20 ae ce   jsr arctang_bound_dir
e7de 9d 07 03   sta enemy_ram_ex,x
; sprite
e7e1 a5 74      lda dart_sprite
e7e3 d0 17      bne .sprite_custom
e7e5 bd 07 03   lda enemy_ram_ex,x
e7e8 aa         tax
e7e9 bd 0c e8   lda dart_direction_to_sprite_table,x
e7ec 18         clc
e7ed 69 b9      adc #$b9
e7ef 99 01 02   sta oam_ram_spr,y
; attributes
e7f2 bd 24 e8   lda dart_direction_to_attribute_table,x

```



```

e7f5 09 01      ora #$01
e7f7 99 02 02   sta oam_ram_att,y
e7fa d0 08      bne .sprite_done
               .sprite_custom
e7fc 99 01 02   sta oam_ram_spr,y
e7ff a9 01      lda #$01
e801 99 02 02   sta oam_ram_att,y
               .sprite_done
               ; sfx
e804 20 41 fa   jsr sfx_shoot_dart
               ; register happy town
e807 a6 76      ldx enemy_ram_offset
e809 a4 77      ldy enemy_oam_offset
               .done
e80b 60         rts

               dart_direction_to_sprite_table:
e80c byte       0,1,2,3,4,5,6,5,4,3,2,1, 0,1,2,3,4,5,6,5,4,3,2,1
               dart_direction_to_attribute_table:
e824 byte       $00,$00,$00,$00,$00,$00,$40,$40,$40,$40,$40,$40
e830 byte       $c0,$c0,$c0,$c0,$c0,$c0,$80,$80,$80,$80,$80,$80

               dart_cycle: subroutine
               ; check for x despawn
e83c b9 03 02   lda oam_ram_x,y
e83f c9 03      cmp #$03
e841 90 21      bcc .despawn
e843 c9 fc      cmp #$fc
e845 b0 1d      bcs .despawn
               ; check for player collision
e847 a9 04      lda #$04
e849 85 3d      sta collision_0_w
e84b 85 3e      sta collision_0_h
e84d b9 03 02   lda oam_ram_x,y
e850 18         clc
e851 69 02      adc #$02
e853 85 3b      sta collision_0_x
e855 b9 00 02   lda oam_ram_y,y
e858 69 02      adc #$02
e85a 85 3c      sta collision_0_y
e85c 20 35 cf   jsr player_collision_detect
e85f f0 09      beq .no_collision
e861 20 dd db   jsr enemy_gives_damage
               .despawn
e864 20 47 dd   jsr enemy_clear
e867 4c 79 e8   jmp .done
               .no_collision
               ; handle direction movement
e86a bd 04 03   lda enemy_ram_pc,x
e86d 85 84      sta arctang_velocity_lo
e86f 20 f6 ce   jsr arctang_enemy_update
               ; check for y despawn
e872 b9 00 02   lda oam_ram_y,y
e875 c9 06      cmp #$06
e877 90 eb      bcc .despawn
               .done
e879 4c 92 db   jmp update_enemies_handler_next

```

enemy_dumbface.asm

```

               dumbface_spawn: subroutine
               ; x is set by enemy spawner
e87c a5 26      lda rng0
e87e 9d 05 03   sta enemy_ram_ac,x ; animation counter
e881 20 d1 db   jsr enemy_spawn_random_y_pos
e884 9d 03 03   sta enemy_ram_y,x ; y pos
e887 60         rts

               dumbface_cycle: subroutine
e888 a9 10      lda #$10
e88a 85 3d      sta collision_0_w
e88c 85 3e      sta collision_0_h
e88e 20 eb db   jsr enemy_handle_damage_and_death

e891 fe 02 03   inc enemy_ram_x,x
e894 bd 02 03   lda enemy_ram_x,x
e897 20 df da   jsr sprite_4_set_x

e89a bd 07 03   lda enemy_ram_ex,x
e89d 29 01      and #$01
e89f d0 0b      bne .going_down
               .going_up
e8a1 de 03 03   dec enemy_ram_y,x
e8a4 d0 13      bne .updown_done
e8a6 fe 07 03   inc enemy_ram_ex,x
e8a9 4c b9 e8   jmp .updown_done
               .going_down
e8ac fe 03 03   inc enemy_ram_y,x
e8af bd 03 03   lda enemy_ram_y,x
e8b2 c9 ad      cmp #$ad
e8b4 d0 03      bne .updown_done
e8b6 fe 07 03   inc enemy_ram_ex,x
               .updown_done
e8b9 bd 03 03   lda enemy_ram_y,x
e8bc 20 ef da   jsr sprite_4_set_y

e8bf fe 05 03   inc enemy_ram_ac,x
e8c2 bd 05 03   lda enemy_ram_ac,x
e8c5 4a         lsr
e8c6 4a         lsr
e8c7 4a         lsr
e8c8 29 03      and #$03
e8ca 0a         asl
e8cb 18         clc
e8cc 69 c8      adc #$c8
e8ce 20 b7 da   jsr sprite_4_set_sprite
e8d1 a9 02      lda #$02
e8d3 20 9d da   jsr sprite_4_set_palette
               .done
e8d6 4c 92 db   jmp update_enemies_handler_next

```


enemy_galger.asm

```

; flight path built on sine table circles

; quadrants clockwise : x val , y val
; 0 or 12-3 : $00-$40 , $c0-$00
; 1 or 3-6 : $40-$80 , $00-$40
; 2 or 6-9 : $80-$c0 , $40-$80
; 3 or 9-12 : $c0-$00 , $80-$c0

; state sequence -- comes in from top
; ; x,y origin ; quadrant(s) ; diameter x,y ; x,y origin diff
; state1 ; 88,-48 ; 1 ; 96, 96 ;
; state2 ; 124, 32 ; 2-3 ; 24, 16 ; +38, +80
; state3 ; 120, 32 ; 0-1 ; 32, 32 ; -4, 0
; state4 ; 104, 16 ; 2-3 ; 64, 48 ; -16, -16
; state5 ; 104, 16 ; 0-1 ; 64, 60 ; 0, 0
; state6 ; 88, 76 ; 3 in reverse ; 96, 52 ; -16, +60
; state7 resets the pattern... -8, 56 ; -96, -20

; enemy_ram_pc = current arc position
; enemy_ram_ac = arc position high byte
; enemy_ram_ex = current arc sequence leg

; before 1st leg translation === saves stuff in enemy ram
; x, y origin: 184, 228 <-- saved in x, y
; how many legs : 6 <-- saved in ex
; initial arc position: $c0 <-- saved in pattern counter

04 00 arc_leg_x_offset EQU $400
04 01 arc_leg_y_offset EQU $401
04 02 arc_leg_x_scale EQU $402
04 03 arc_leg_y_scale EQU $403
04 04 arc_leg_arc_start EQU $404
04 05 arc_leg_arc_target EQU $405
04 06 arc_leg_speed_inc EQU $406
04 07 arc_leg_speed_dec EQU $407

; height of playfield = 182px

; 16 sequences with 46 steps total
arc_sequence_begin:
e8d9 byte $00, $05, $0b, $0f, $11, $13, $15, $1a,
      $1e, $21, $22, $23, $25, $27, $29, $2b
arc_sequence_lengths:
e8e9 byte $05, $06, $04, $02, $02, $02, $05, $04,
      $03, $01, $01, $02, $02, $02, $02, $03
arc_sequence_x_origin:
e8f9 byte $00, $a0, $80, $cd, $98, $35, $be, $d8,
      $22, $f8, $00, $cf, $ba, $e2, $c4, $80
arc_sequence_y_origin:
e909 byte $96, $ec, $2a, $10, $ac, $76, $9e, $a0,
      $a8, $5e, $32, $84, $78, $70, $8c, $3a
```

```
arc_sequences:
; sequence 0 : Spiral Young Buck
      $d2, $a0, $60, $60, $80, $40, $00, $02
e919 byte $60, $10, $40, $40, $c0, $78, $04, $00
e921 byte $10, $10, $30, $30, $80, $f8, $05, $00
e929 byte $00, $00, $20, $20, $00, $78, $07, $00
e931 byte $10, $10, $08, $08, $80, $f0, $10, $00
e939 byte
; sequence 1 : Spiral O.G.
e941 byte $a0, $ec, $60, $60, $40, $80, $03, $00
e949 byte $26, $50, $18, $10, $80, $00, $07, $00
e951 byte $fc, $00, $20, $20, $00, $80, $06, $00
e959 byte $f0, $f0, $40, $30, $80, $00, $05, $00
e961 byte $00, $00, $40, $3c, $00, $80, $04, $00
e969 byte $f0, $3c, $60, $34, $00, $c0, $00, $03
; sequence 2 : Big S
e971 byte $80, $60, $70, $14, $bc, $80, $00, $04
e979 byte $1c, $d2, $3c, $40, $7d, $03, $00, $04
e981 byte $00, $c0, $3c, $40, $80, $fa, $04, $00
e989 byte $dc, $00, $80, $c8, $00, $40, $02, $00
; sequence 3 : Arc and Loop from Behind
e991 byte $a8, $00, $ff, $eb, $00, $c0, $00, $01
e999 byte $00, $5a, $40, $40, $c0, $06, $00, $04
; sequence 4 : Butt Planet
e9a1 byte $28, $b6, $de, $a4, $40, $05, $00, $01
e9a9 byte $60, $f0, $20, $12, $80, $c0, $05, $00
; sequence 5 : Loop Up from Below
e9b1 byte $00, $c0, $40, $80, $80, $fa, $02, $00
e9b9 byte $00, $00, $40, $40, $00, $7d, $03, $00
; sequence 6 : Loop de Loop
e9c1 byte $00, $00, $80, $10, $80, $c0, $03, $00
e9c9 byte $80, $fc, $80, $10, $40, $02, $00, $03
e9d1 byte $00, $81, $80, $80, $80, $7e, $02, $00
e9d9 byte $00, $81, $80, $10, $00, $c0, $00, $03
e9e1 byte $80, $00, $80, $10, $40, $80, $03, $00
; sequence 7 : wide Ying Yang
e9e9 byte $c8, $c0, $c0, $40, $80, $40, $00, $02
e9f1 byte $c0, $00, $40, $40, $c3, $80, $04, $00
e9f9 byte $bc, $00, $c0, $40, $80, $c0, $02, $00
ea01 byte $00, $00, $40, $40, $c3, $00, $04, $00
; sequence 8 : Horse Shoe Loop
ea09 byte $00, $00, $40, $f2, $ca, $80, $00, $02
ea11 byte $d4, $a0, $a0, $50, $80, $02, $00, $02
ea19 byte $30, $00, $40, $f2, $00, $c8, $00, $02
; sequence 9 : Circle LeftDownReverse
ea21 byte $08, $f8, $60, $60, $00, $f9, $06, $00
; sequence 10 : Tunnel Going Right
ea29 byte $08, $00, $29, $50, $80, $87, $00, $07
; sequence 11 : wide S Short
ea31 byte $00, $d8, $64, $28, $80, $40, $00, $02
ea39 byte $64, $00, $64, $28, $c0, $00, $02, $00
; sequence 12 : wide S Medium
ea41 byte $00, $b4, $a0, $50, $80, $40, $00, $02
ea49 byte $a4, $00, $a0, $50, $c0, $00, $02, $00
; sequence 13 : Stair Step Small
ea51 byte $00, $c8, $3c, $3c, $80, $40, $00, $04
ea59 byte $3c, $f8, $3c, $3c, $c0, $00, $04, $00
```

```

; sequence 14 : Stair Step Big
ea61 byte $00, $8c, $78, $78, $80, $40, $00, $03
ea69 byte $78, $f2, $78, $78, $c0, $00, $03, $00
; sequence 15 : w
ea71 byte $80, $d8, $40, $78, $c0, $40, $00, $03
ea79 byte $40, $00, $40, $78, $ba, $40, $00, $03
ea81 byte $40, $28, $80, $28, $c3, $3d, $03, $00

```

```

arc_sequence_set: subroutine
; x holds which sequence
ea89 86 80 stx arc_sequence_id
ea8b a9 e9 lda #>arc_sequences
ea8d 85 82 sta arc_sequence_hi
ea8f bd d9 e8 lda arc_sequence_begin,x
ea92 0a asl
ea93 0a asl
ea94 0a asl
ea95 90 02 bcc .hi_byte_perfect
.hi_byte_increase
ea97 e6 82 inc arc_sequence_hi
.hi_byte_perfect
ea99 18 clc
ea9a 69 19 adc #<arc_sequences
ea9c 85 81 sta arc_sequence_lo
ea9e 90 02 bcc .hi_byte_still_perfect
eaa0 e6 82 inc arc_sequence_hi
.hi_byte_still_perfect
eaa2 bd e9 e8 lda arc_sequence_lengths,x
eaa5 85 83 sta arc_sequence_length
eaa7 60 rts

```

```

galger_spawn: subroutine
; x is set by enemy spawner
eaa8 a9 08 lda #galger_id
eaaa 9d 00 03 sta enemy_ram_type,x
eaad a8 tay
eaae b9 c5 dc lda enemy_hitpoints_table,y
eab1 9d 01 03 sta enemy_ram_hp,x
; arc system setup
eab4 a9 00 lda #0
eab6 9d 05 03 sta enemy_ram_ac,x
eab9 9d 07 03 sta enemy_ram_ex,x
eabc a4 80 ldy arc_sequence_id
eabe b9 f9 e8 lda arc_sequence_x_origin,y
eac1 9d 02 03 sta enemy_ram_x,x
eac4 b9 09 e9 lda arc_sequence_y_origin,y
eac7 9d 03 03 sta enemy_ram_y,x
; angle / pc set in leg init
eaca 20 b0 eb jsr arc_leg_init
eacd 60 rts

```

```

galger_cycle: subroutine
eace a9 08 lda #$08
ead0 85 3d sta collision_0_w
ead2 85 3e sta collision_0_h
ead4 20 eb db jsr enemy_handle_damage_and_death

```

```

; time to shoot a dart?
ead7 a5 26 lda rng0
ead9 4a lsr
eada 29 3f and #$3f
eadc d0 19 bne .dont_fire
.dart_fire
eade b9 03 02 lda oam_ram_x,y
eae1 85 71 sta dart_x_origin
eae3 b9 00 02 lda oam_ram_y,y
eae6 85 72 sta dart_y_origin
eae8 a9 04 lda #$04
eaea 85 73 sta dart_velocity
eaec a9 00 lda #$00
eae6 85 74 sta dart_sprite
eaf0 85 75 sta dart_dir_adjust
eaf2 20 a8 e7 jsr dart_spawn
eaf5 a4 77 ldy enemy_oam_offset
.dont_fire
eaf7 a6 76 ldx enemy_ram_offset
eaf9 20 0e eb jsr arc_leg
; current sprite
eafc a9 7f lda #$7f
eafe 99 01 02 sta oam_ram_spr,y
; do palette
eb01 bd 04 03 lda enemy_ram_pc,x
eb04 29 c0 and #$c0
eb06 09 01 ora #$01
eb08 20 39 dc jsr enemy_set_palette
.done
eb0b 4c 92 db jmp update_enemies_handler_next

```

```

arc_leg: subroutine
; set x position
eb0e bd 04 03 lda enemy_ram_pc,x
eb11 85 86 sta temp00
eb13 bd 02 04 lda arc_leg_x_scale,x
eb16 a6 86 ldx temp00
eb18 20 ad d4 jsr sine_of_scale
eb1b a6 76 ldx enemy_ram_offset
eb1d 18 clc
eb1e 7d 02 03 adc enemy_ram_x,x
eb21 99 03 02 sta oam_ram_x,y
; set y position
eb24 bd 04 03 lda enemy_ram_pc,x
eb27 18 clc
eb28 69 c0 adc #$c0
eb2a 85 86 sta temp00
eb2c bd 03 04 lda arc_leg_y_scale,x
eb2f a6 86 ldx temp00
eb31 20 ad d4 jsr sine_of_scale

```

```

eb34 a6 76      ldx enemy_ram_offset
eb36 18         clc
eb37 7d 03 03   adc enemy_ram_y,x
eb3a 99 00 02   sta oam_ram_y,y

eb3d c9 f0      cmp #240
eb3f 90 11      bcc .y_under_240
eb41 38         sec
eb42 e9 4a      sbc #sprite_0_y_diff
eb44 99 00 02   sta oam_ram_y,y
eb47 bd 03 03   lda enemy_ram_y,x
eb4a 38         sec
eb4b e9 4a      sbc #sprite_0_y_diff
eb4d 9d 03 03   sta enemy_ram_y,x
eb50 b0 13      bcs .y_check_done
               .y_under_240
eb52 c9 b6      cmp #sprite_0_y
eb54 90 0f      bcc .y_check_done
eb56 18         clc
eb57 69 4a      adc #sprite_0_y_diff
eb59 99 00 02   sta oam_ram_y,y
eb5c bd 03 03   lda enemy_ram_y,x
eb5f 18         clc
eb60 69 4a      adc #sprite_0_y_diff
eb62 9d 03 03   sta enemy_ram_y,x
               .y_check_done

               ; forward arc position
eb65 bd 05 03   lda enemy_ram_ac,x
eb68 f0 22      beq .speed_handle_normal
               .speed_handle_abnormal
eb6a bd 06 04   lda arc_leg_speed_inc,x
eb6d c9 00      cmp #$00
eb6f f0 0b      beq .speed_ab_dec
               .speed_ab_inc
eb71 18         clc
eb72 7d 04 03   adc enemy_ram_pc,x
eb75 9d 04 03   sta enemy_ram_pc,x
eb78 90 11      bcc .speed_ab_done
eb7a b0 0c      bcs .speed_dec_ac
               .speed_ab_dec
eb7c bd 04 03   lda enemy_ram_pc,x
eb7f 38         sec
eb80 fd 07 04   sbc arc_leg_speed_dec,x
eb83 9d 04 03   sta enemy_ram_pc,x
eb86 b0 03      bcs .speed_ab_done
               .speed_dec_ac
eb88 de 05 03   dec enemy_ram_ac,x
               .speed_ab_done
eb8b 60         rts

               .speed_handle_normal
eb8c bd 06 04   lda arc_leg_speed_inc,x
eb8f c9 00      cmp #$00
eb91 f0 0d      beq .speed_dec
               .speed_inc

```

```

eb93 18         clc
eb94 7d 04 03   adc enemy_ram_pc,x
eb97 9d 04 03   sta enemy_ram_pc,x
eb9a dd 05 04   cmp arc_leg_arc_target,x
eb9d b0 11      bcs .next_leg
eb9f 60         rts
               .speed_dec
eba0 bd 04 03   lda enemy_ram_pc,x
eba3 38         sec
eba4 fd 07 04   sbc arc_leg_speed_dec,x
eba7 9d 04 03   sta enemy_ram_pc,x
ebaa dd 05 04   cmp arc_leg_arc_target,x
ebad 90 01      bcc .next_leg
ebaf 60         rts
               .next_leg
               arc_leg_init:
ebb0 bd 07 03   lda enemy_ram_ex,x
               ; update arc data in enemy slot
ebb3 0a         asl
ebb4 0a         asl
ebb5 0a         asl
ebb6 a8         tay
ebb7 b1 81      lda (arc_sequence_lo),y
ebb9 9d 00 04   sta arc_leg_x_offset,x
ebbc c8         iny
ebbd b1 81      lda (arc_sequence_lo),y
ebbf 9d 01 04   sta arc_leg_y_offset,x
ebc2 c8         iny
ebc3 b1 81      lda (arc_sequence_lo),y
ebc5 9d 02 04   sta arc_leg_x_scale,x
ebc8 c8         iny
ebc9 b1 81      lda (arc_sequence_lo),y
ebcb 9d 03 04   sta arc_leg_y_scale,x
ebce c8         iny
ebcf b1 81      lda (arc_sequence_lo),y
ebd1 9d 04 04   sta arc_leg_arc_start,x
ebd4 9d 04 03   sta enemy_ram_pc,x
ebd7 c8         iny
ebd8 b1 81      lda (arc_sequence_lo),y
ebda 9d 05 04   sta arc_leg_arc_target,x
ebdd c8         iny
ebde b1 81      lda (arc_sequence_lo),y
ebe0 9d 06 04   sta arc_leg_speed_inc,x
ebe3 c8         iny
ebe4 b1 81      lda (arc_sequence_lo),y
ebe6 9d 07 04   sta arc_leg_speed_dec,x
ebe9 a4 77      ldy enemy_oam_offset
               ; update x origin
ebeb bd 02 03   lda enemy_ram_x,x
ebef 18         clc
ebef 7d 00 04   adc arc_leg_x_offset,x
ebf2 9d 02 03   sta enemy_ram_x,x
               ; update y origin
ebf5 bd 03 03   lda enemy_ram_y,x
ebf8 18         clc
ebf9 7d 01 04   adc arc_leg_y_offset,x
ebfc 9d 03 03   sta enemy_ram_y,x

```

```

; update arc origin
ebff bd 04 04 lda arc_leg_arc_start,x
ec02 9d 04 03 sta enemy_ram_pc,x
ec05 bd 06 04 lda arc_leg_speed_inc,x
ec08 c9 00 cmp #$00
ec0a f0 0d beq .check_speed_dec_count
;check_speed_inc_count
ec0c bd 04 04 lda arc_leg_arc_start,x
ec0f dd 05 04 cmp arc_leg_arc_target,x
ec12 90 10 bcc .arc_leg_done
ec14 fe 05 03 inc enemy_ram_ac,x
ec17 d0 0b bne .arc_leg_done
;check_speed_dec_count
ec19 bd 04 04 lda arc_leg_arc_start,x
ec1c dd 05 04 cmp arc_leg_arc_target,x
ec1f b0 03 bcs .arc_leg_done
ec21 fe 05 03 inc enemy_ram_ac,x
;arc_leg_done
ec24 fe 07 03 inc enemy_ram_ex,x
ec27 a5 83 lda arc_sequence_length
ec29 dd 07 03 cmp enemy_ram_ex,x
ec2c d0 05 bne .dont_wrap_sequence
ec2e a9 00 lda #0
ec30 9d 07 03 sta enemy_ram_ex,x
;dont_wrap_sequence
ec33 60 rts

```

enemy_ikes_mom.asm

```

ikes_mom_spawn:
ec34 20 52 dc jsr get_oam_offset_from_ram_offset
ec37 a9 fa lda #$fa
ec39 99 03 02 sta oam_ram_x,y
ec3c a9 80 lda #$80
ec3e 99 00 02 sta oam_ram_y,y
ec41 a9 05 lda #$05
ec43 9d 07 03 sta enemy_ram_ex,x
; reset spawn counter
ec46 a9 00 lda #$00
ec48 85 25 sta ftw
ec4a 60 rts

ikes_mom_cycle:
ec4b a9 10 lda #$10
ec4d 85 3d sta collision_0_w
ec4f 85 3e sta collision_0_h
ec51 20 eb db jsr enemy_handle_damage_and_death

ec54 a5 24 lda wtf
ec56 29 0f and #$0f
ec58 d0 47 bne .dir_update_done
; next direction
ec5a 20 12 ce jsr enemy_get_direction_of_player
ec5d 85 86 sta temp00
; current direction
ec5f bd 07 03 lda enemy_ram_ex,x
ec62 85 87 sta temp01

; diff 1
ec64 38 sec
ec65 e5 86 sbc temp00
ec67 10 03 bpl .diff_1_set
ec69 18 clc
ec6a 69 18 adc #24
;diff_1_set
ec6c 85 88 sta temp02
; diff 2
ec6e a5 86 lda temp00
ec70 38 sec
ec71 e5 87 sbc temp01
ec73 10 03 bpl .diff_2_set
ec75 18 clc
ec76 69 18 adc #24
;diff_2_set
ec78 85 89 sta temp03
; which dir faster
ec7a c5 88 cmp temp02
ec7c f0 0e beq .dir_set
ec7e b0 07 bcs .counterclockwise

```

```

        .clockwise
ec80  a9 01      lda #$01
ec82  9d 04 03   sta enemy_ram_pc,x
ec85  d0 05      bne .dir_set
        .counterclockwise
ec87  a9 ff      lda #$ff
ec89  9d 04 03   sta enemy_ram_pc,x
        .dir_set
ec8c  bd 07 03   lda enemy_ram_ex,x
ec8f  18         clc
ec90  7d 04 03   adc enemy_ram_pc,x
ec93  10 03      bpl .skip_neg_fix
ec95  18         clc
ec96  69 18      adc #24
        .skip_neg_fix
ec98  c9 18      cmp #24
ec9a  d0 02      bne .skip_over_fix
ec9c  a9 00      lda #0
        .skip_over_fix
ec9e  9d 07 03   sta enemy_ram_ex,x
        .dir_update_done

eca1  ad f2 ff   lda arctang_velocities_lo+2
eca4  85 84     sta arctang_velocity_lo
eca6  20 f6 ce  jsr arctang_enemy_update

```

```

        ; shake?
eca9  a5 24     lda wtf
ecab  c9 c0     cmp #$c0
ecad  90 2f     bcc .dont_banshee
        ; x shake
ecaf  20 66 c1  jsr shake_8
ecb2  79 03 02  adc oam_ram_x,y
ecb5  99 03 02  sta oam_ram_x,y
        ; y shake
ecb8  20 66 c1  jsr shake_8
ecbb  79 00 02  adc oam_ram_y,y
ecbe  99 00 02  sta oam_ram_y,y
        ; sfx (pu1)
ecc1  a9 3f     lda #$3f
ecc3  8d 00 40  sta APU_PULSE1_VOL
ecc6  a9 08     lda #$08
ecc8  8d 01 40  sta APU_PULSE1_SWEEP
eccb  a5 24     lda wtf
eccd  29 3f     and #$3f
eccf  49 3f     eor #$3f
ecd1  4a        lsr
ecd2  69 a0     adc #$a0
ecd4  8d 02 40  sta APU_PULSE1_TIMER_LO
ecd7  a9 08     lda #$08
ecd9  8d 03 40  sta APU_PULSE1_TIMER_HI
ecdc  e6 ac     inc apu_pu1_counter
        .dont_banshee

```

```

        ; x
ecde  b9 03 02  lda oam_ram_x,y
ece1  20 df da  jsr sprite_4_set_x
        ; y
ece4  b9 00 02  lda oam_ram_y,y
ece7  20 ef da  jsr sprite_4_set_y
        ; sprite
ecea  a5 13     lda player_x_hi
ecec  e9 07     sbc #$07
ec ee  d9 03 02  cmp oam_ram_x,y
ecf1  90 04     bcc .face_left
        .face_right
ecf3  a9 22     lda #$22
ecf5  d0 02     bne .set_face
        .face_left
ecf7  a9 20     lda #$20
        .set_face
ecf9  20 b7 da  jsr sprite_4_set_sprite
        ; palette
ecfc  a9 02     lda #$02
ecfe  20 9d da  jsr sprite_4_set_palette

ed01  4c 92 db  jmp update_enemies_handler_next

```

enemy_lasso.asm

```
        lasso_spawn: subroutine
        ; x is set by enemy spawner
ed04    20 52 dc    jsr get_oam_offset_from_ram_offset
ed07    a9 00      lda #$00
ed09    99 03 02   sta oam_ram_x,y
ed0c    60         rts

        lariat_sine_table:
ed0d    byte      #16, #35, #55, #73, #93, #113, #129, #148

        lasso_cycle: subroutine

        ; x pos
ed15    a5 24      lda wtf
ed17    29 01      and #$01
ed19    d0 12      bne .dont_advance
ed1b    b9 03 02   lda oam_ram_x,y
ed1e    18         clc
ed1f    69 01      adc #$01
ed21    99 03 02   sta oam_ram_x,y
ed24    99 07 02   sta oam_ram_x+4,y
ed27    99 0b 02   sta oam_ram_x+8,y
ed2a    99 0f 02   sta oam_ram_x+$c,y
        .dont_advance

        ; y pos
ed2d    fe 04 03   inc enemy_ram_pc,x
ed30    bd 04 03   lda enemy_ram_pc,x
ed33    85 87      sta temp01
ed35    aa         tax
ed36    a9 9f      lda #159
ed38    20 ad d4   jsr sine_of_scale
ed3b    99 00 02   sta oam_ram_y,y
ed3e    18         clc
ed3f    69 08      adc #$08
ed41    99 04 02   sta oam_ram_y+4,y
ed44    85 3e      sta collision_0_h
        ; middle sprite
        ;lda #105
ed46    a5 24      lda wtf
ed48    29 07      and #$07
ed4a    aa         tax
ed4b    bd 0d ed   lda lariat_sine_table,x
ed4e    a6 87      ldx temp01
ed50    20 ad d4   jsr sine_of_scale
ed53    29 fe      and #$fe
ed55    85 88      sta temp02
ed57    38         sec
```

```
ed58    e9 08      sbc #$08
ed5a    99 08 02   sta oam_ram_y+8,y
        ; top sprite
ed5d    a5 88      lda temp02
ed5f    99 0c 02   sta oam_ram_y+$c,y
ed62    a6 76      ldx enemy_ram_offset

        ; collision
ed64    a9 00      lda #$00
ed66    85 3c      sta collision_0_y
ed68    a9 07      lda #$07
ed6a    85 3d      sta collision_0_w
ed6c    20 eb db   jsr enemy_handle_damage_and_death

        ; sprites
ed6f    a9 59      lda #$59
ed71    99 01 02   sta oam_ram_spr,y
ed74    a9 69      lda #$69
ed76    99 05 02   sta oam_ram_spr+4,y
ed79    a9 6a      lda #$6a
ed7b    99 09 02   sta oam_ram_spr+8,y
ed7e    99 0d 02   sta oam_ram_spr+$c,y

        ; palette & attributes
ed81    a9 02      lda #$02
ed83    20 9d da   jsr sprite_4_set_palette

        .done
ed86    4c 92 db   jmp update_enemies_handler_next
```

enemy_maggs.asm

```
maggs_spawn: subroutine
ed89 20 52 c1    jsr get_next_random
ed8c 9d 04 03    sta enemy_ram_pc,x ; pattern counter
ed8f 20 d1 db    jsr enemy_spawn_random_y_pos
ed92 9d 03 03    sta enemy_ram_y,x ; y pos
ed95 60          rts
```

```
;;; HANDLING maggs
maggs_cycle: subroutine
ed96 a9 10      lda #$10
ed98 85 3d      sta collision_0_w
ed9a a9 05      lda #$05
ed9c 85 3e      sta collision_0_h
ed9e 20 eb db   jsr enemy_handle_damage_and_death
```

```
; sprite
eda1 bd 05 03   lda enemy_ram_ac,x
eda4 4a         lsr
eda5 4a         lsr
eda6 4a         lsr
eda7 4a         lsr
eda8 0a         asl
eda9 18         clc
edaa 69 3c      adc #$3c
edac a4 77      ldy enemy_oam_offset
edae 99 01 02   sta oam_ram_spr,y
edb1 69 01      adc #$01
edb3 99 05 02   sta oam_ram_spr+4,y
```

```
; x pos
edb6 bd 02 03   lda enemy_ram_x,x
edb9 99 03 02   sta oam_ram_x,y
edbc 18         clc
edbd 69 08      adc #$08
edbf 99 07 02   sta oam_ram_x+4,y
```

```
; y pos
edc2 a6 76      ldx enemy_ram_offset
edc4 bd 04 03   lda enemy_ram_pc,x
edc7 4a         lsr
edc8 a8         tay
edc9 b9 00 06   lda sine_4bits,y
edcc 18         clc
edcd 7d 03 03   adc enemy_ram_y,x
edd0 a4 77      ldy enemy_oam_offset
edd2 99 00 02   sta oam_ram_y,y
edd5 99 04 02   sta oam_ram_y+4,y
```

```
; update pattern
edd8 fe 04 03   inc enemy_ram_pc,x
eddb fe 04 03   inc enemy_ram_pc,x
; move forward
edde fe 02 03   inc enemy_ram_x,x
; update animation
ede1 bd 05 03   lda enemy_ram_ac,x
ede4 c9 00      cmp #$00
ede6 d0 05      bne .maggs_frame
ede8 a9 20      lda #$20
edea 9d 05 03   sta enemy_ram_ac,x
; .maggs_frame
eded de 05 03   dec enemy_ram_ac,x

edf0 a9 02      lda #$02
edf2 20 39 dc   jsr enemy_set_palette
edf5 99 06 02   sta oam_ram_att+4,y

.done

edf8 4c 92 db   jmp update_enemies_handler_next
```

enemy_muya.asm

```

        mua_spawn_y      EQM $b4

muya_spawn: subroutine
edfb 20 52 dc    jsr get_oam_offset_from_ram_offset
edfe a9 c4      lda #muya_spawn_y+$10
ee00 99 00 02   sta oam_ram_y,y
ee03 60         rts

        ;;;; HANDLING MUYA
muya_cycle: subroutine
ee04 a9 08      lda #$08
ee06 85 3d      sta collision_0_w
ee08 a9 10      lda #$10
ee0a 85 3e      sta collision_0_h
ee0c 20 eb db   jsr enemy_handle_damage_and_death

        ; x pos
ee0f b9 03 02   lda oam_ram_x,y
ee12 99 07 02   sta oam_ram_x+4,y

        ; y pos
ee15 20 fb dc   jsr enemy_accelerates_upwards
ee18 b0 0b      bcs .reset_x_and_y_pos
ee1a 99 00 02   sta oam_ram_y,y
ee1d 18         clc
ee1e 69 08      adc #$08
ee20 99 04 02   sta oam_ram_y+4,y
ee23 d0 13      bne .reset_skip
        .reset_x_and_y_pos
        ; x
ee25 20 52 c1   jsr get_next_random
ee28 4a         lsr
ee29 69 5c      adc #$5c
ee2b 99 03 02   sta oam_ram_x,y
        ; y
ee2e a9 b4      lda #muya_spawn_y
ee30 99 00 02   sta oam_ram_y,y
        ; reset animation counter
ee33 a9 00      lda #$00
ee35 9d 05 03   sta enemy_ram_ac,x
        .reset_skip

        ; sprite
ee38 a9 64      lda #$64
ee3a 99 01 02   sta oam_ram_spr,y
ee3d a9 74      lda #$74
ee3f 99 05 02   sta oam_ram_spr+4,y

        ; palette
ee42 a9 02      lda #$02
ee44 20 39 dc   jsr enemy_set_palette
ee47 99 06 02   sta oam_ram_att+4,y

ee4a 4c 92 db   jmp update_enemies_handler_next

```

enemy_skeet.asm

```

        ; skeet is a buggar that moves in curves
        ;
        ; ac used for both tiles and cycle counter
        ; pc 2bits x velocity / 2bits y velocity
        ; ex holds 4-way direction

skeet_att_table:
        ; r/u l/u l/d r/d
        ; $a2, #d2, #62, #22
skeet_vel_table:
        #0, #5, #6, #1, #6, #5

skeet_spawn: subroutine
        ; set direction
ee57 a5 27      lda rng1
ee59 29 02      and #$02
ee5b d0 04      bne .downward
        .upward
ee5d a9 06      lda #$06
ee5f d0 02      bne .set_updown
        .downward
ee61 a9 12      lda #18
        .set_updown
ee63 9d 07 03   sta enemy_ram_ex,x
        ; set position
ee66 4c 1a db   jmp enemy_spawn_set_x_y_rng

skeet_cycle: subroutine
ee69 a9 08      lda #$08
ee6b 85 3d      sta collision_0_w
ee6d 85 3e      sta collision_0_h
ee6f 20 eb db   jsr enemy_handle_damage_and_death

        ; reset direction?
ee72 bd 05 03   lda enemy_ram_ac,x
ee75 d0 16      bne .skip_reset
ee77 a9 40      lda #$40
ee79 9d 05 03   sta enemy_ram_ac,x
ee7c 20 52 c1   jsr get_next_random
ee7f 10 07      bpl .dir_inc
        .dir_dec
ee81 a9 ff      lda #$ff
ee83 9d 04 03   sta enemy_ram_pc,x
ee86 d0 05      bne .skip_reset
        .dir_inc
ee88 a9 01      lda #$01
ee8a 9d 04 03   sta enemy_ram_pc,x
        .skip_reset
ee8d de 05 03   dec enemy_ram_ac,x
ee90 bd 05 03   lda enemy_ram_ac,x
ee93 29 03      and #$03
ee95 d0 0d      bne .skip_dir_change

```



```

ee97 bd 07 03    lda enemy_ram_ex,x
ee9a 18          clc
ee9b 7d 04 03    adc enemy_ram_pc,x
ee9e 20 ae ce     jsr arctang_bound_dir
eea1 9d 07 03     sta enemy_ram_ex,x
               .skip_dir_change
               ; velocity
eea4 bd 07 03    lda enemy_ram_ex,x
eea7 4a          lsr
eea8 4a          lsr
eea9 aa          tax
eeaa bd 51 ee     lda skeet_vel_table,x
eead aa          tax
eeae bd f0 ff     lda arctang_velocities_lo,x
eeb1 85 84        sta arctang_velocity_lo
eeb3 a6 76        ldx enemy_ram_offset
eeb5 20 f6 ce     jsr arctang_enemy_update
               ; sprite
eeb8 bd 05 03    lda enemy_ram_ac,x
eebb 4a          lsr
eebc aa          tax
eebd bd 00 fe     lda sine_table,x
eec0 29 01        and #1
eec2 18          clc
eec3 69 4e        adc #$4e ; base sprite tile
eec5 a6 76        ldx enemy_ram_offset
eec7 99 01 02     sta oam_ram_spr,y
               ; attr
eeca bd 07 03    lda enemy_ram_ex,x
eecd 69 04        adc #$04
eecf 4a          lsr
eed0 4a          lsr
eed1 4a          lsr
eed2 aa          tax
eed3 bd 4d ee     lda skeet_att_table,x
eed6 a6 76        ldx enemy_ram_offset
eed8 20 39 dc     jsr enemy_set_palette

eedb 4c 92 db     jmp update_enemies_handler_next

```

enemy_skully.asm

```

               skully_spawn: subroutine
               ; x is set by enemy spawner
eede a5 26        lda rng0
eee0 9d 05 03     sta enemy_ram_ac,x ; animation counter
eee3 20 d1 db     jsr enemy_spawn_random_y_pos
eee6 9d 03 03     sta enemy_ram_y,x ; y pos
eee9 60           rts

               ;;;; HANDLING SKULLY
skully_cycle: subroutine
eeea a9 10        lda #$10
eeec 85 3d        sta collision_0_w
eeee 85 3e        sta collision_0_h
eef0 20 eb db     jsr enemy_handle_damage_and_death

eef3 a6 76        ldx enemy_ram_offset
eef5 a4 77        ldy enemy_oam_offset
eef7 a9 01        lda #$01 ; set mirror flag
eef9 9d 07 03     sta enemy_ram_ex,x
               ; let's find what frame we're on
eefc bd 05 03    lda enemy_ram_ac,x
eeff 4a          lsr
ef00 4a          lsr
ef01 4a          lsr
ef02 4a          lsr
ef03 4a          lsr
ef04 0a          asl
               ; accumulator is now in 0..7 range except x2
               ; sprite
ef05 20 b7 da     jsr sprite_4_set_sprite
               ; x pos
ef08 bd 02 03    lda enemy_ram_x,x
ef0b 20 df da     jsr sprite_4_set_x
               ; y pos
ef0e bd 03 03    lda enemy_ram_y,x
ef11 20 ef da     jsr sprite_4_set_y
               ; update spinning counter
ef14 a9 07        lda #$07
ef16 18          clc
ef17 7d 05 03    adc enemy_ram_ac,x
ef1a 9d 05 03     sta enemy_ram_ac,x
               ; move skully
ef1d 20 28 ef     jsr skully_handle_movement
               ; palette
ef20 a9 03        lda #$03
ef22 20 9d da     jsr sprite_4_set_palette
               .skully_done
ef25 4c 92 db     jmp update_enemies_handler_next

```

```

skully_handle_movement: subroutine
ef28 a5 0e    lda attract_true
ef2a d0 3e    bne .demoshit
ef2c a9 69    lda #$69
ef2e bd 04 03  lda enemy_ram_pc,x
ef31 c9 40    cmp #$40
ef33 d0 1d    bne .not_chasing
             ; CHASE MODE
ef35 de 02 03  dec enemy_ram_x,x
ef38 a5 24    lda wtf
ef3a 29 03    and #$03
ef3c d0 2f    bne .done
ef3e bd 03 03  lda enemy_ram_y,x
ef41 18       clc
ef42 69 09    adc #$09
ef44 c5 15    cmp player_y_hi
ef46 f0 25    beq .done
ef48 90 04    bcc .down

             .up
ef4a de 03 03  dec enemy_ram_y,x
ef4d 60       rts

             .down
ef4e fe 03 03  inc enemy_ram_y,x
ef51 60       rts

             .not_chasing
             ; ZOOM MODE
ef52 a9 20    lda #$20
ef54 9d 04 03  sta enemy_ram_pc,x
ef57 a9 04    lda #$04
ef59 18       clc
ef5a 7d 02 03  adc enemy_ram_x,x
ef5d 9d 02 03  sta enemy_ram_x,x
ef60 c9 f0    cmp #240
ef62 90 06    bcc .demoshit
ef64 a9 40    lda #$40
ef66 9d 04 03  sta enemy_ram_pc,x
ef69 60       rts

             .demoshit
             ; DEMO MODE
ef6a fe 02 03  inc enemy_ram_x,x
             .done
ef6d 60       rts

```

enemy_spark.asm

```

             ; U    R    D    L
spark_dir_x_table:
ef6e byte    #$00, #$01, #$00, #$ff
spark_dir_y_table:
ef72 byte    #$ff, #$00, #$01, #$00

spark_spawn: subroutine
             ; x = slot in enemy ram
             ; y = boss slot in enemy ram
ef76 20 52 c1  jsr get_next_random
ef79 9d 02 03  sta enemy_ram_x,x
ef7c a9 04    lda #4
ef7e 9d 03 03  sta enemy_ram_y,x
ef81 a9 00    lda #0
ef83 9d 07 03  sta enemy_ram_ex,x
ef86 60       rts

spark_cycle: subroutine
ef87 a9 08    lda #$08
ef89 85 3d    sta collision_0_w
ef8b 85 3e    sta collision_0_h
ef8d 20 eb db  jsr enemy_handle_damage_and_death
             ; change direction?
ef90 a5 24    lda wtf
ef92 29 07    and #$07
ef94 d0 12    bne .dont_reset_dir
             ; 50% chance of downward movement
ef96 20 52 c1  jsr get_next_random
ef99 10 04    bpl .not_hard_down
ef9b a9 02    lda #$02
ef9d d0 06    bne .set_dir
             .not_hard_down
             ; otherwise random 4 directions
ef9f 20 52 c1  jsr get_next_random
efa2 4a       lsr
efa3 29 03    and #%00000011
             .set_dir
efa5 9d 07 03  sta enemy_ram_ex,x
             .reset_done
             .dont_reset_dir

```

enemy_starglasses.asm

```

; apply direction
efa8 bc 07 03 ldy enemy_ram_ex,x
efab b9 6e ef lda spark_dir_x_table,y
efae 85 86 sta temp00
efb0 b9 72 ef lda spark_dir_y_table,y
efb3 85 87 sta temp01
efb5 a4 77 ldy enemy_oam_offset
efb7 bd 02 03 lda enemy_ram_x,x
efba 18 clc
efbb 65 86 adc temp00
efbd 9d 02 03 sta enemy_ram_x,x
efc0 99 03 02 sta oam_ram_x,y
efc3 bd 03 03 lda enemy_ram_y,x
efc6 18 clc
efc7 65 87 adc temp01
efc9 20 ff da jsr enemy_fix_y_visible
efcc 9d 03 03 sta enemy_ram_y,x
efcf 99 00 02 sta oam_ram_y,y
; attributes
efd2 20 52 c1 jsr get_next_random
efd5 99 02 02 sta oam_ram_att,y
; sprite
efd8 20 52 c1 jsr get_next_random
efdb 29 03 and #$03
efdd 18 clc
efde 69 6c adc #$6c
efe0 99 01 02 sta oam_ram_spr,y

.done

efe3 4c 92 db jmp update_enemies_handler_next

```

```

starglasses_spawn: subroutine
; x is set by enemy spawner
efe6 20 52 c1 jsr get_next_random
efe9 9d 05 03 sta enemy_ram_ac,x ; animation counter
efec a9 c0 lda #$c0
efee 9d 04 03 sta enemy_ram_pc,x ; pattern counter
eff1 a9 f8 lda #$f8
eff3 9d 02 03 sta enemy_ram_x,x ; x pos
eff6 20 52 c1 jsr get_next_random
eff9 4a lsr
effa 4a lsr
effb 18 clc
effc 69 18 adc #$18
effe 9d 03 03 sta enemy_ram_y,x ; y pos
f001 60 rts

;;; HANDLING STARGLASSES
starglasses_cycle: subroutine
f002 a9 10 lda #$10
f004 85 3d sta collision_0_w
f006 85 3e sta collision_0_h
f008 20 eb db jsr enemy_handle_damage_and_death

.x_decide_action
; calc x
f00b bd 02 03 lda enemy_ram_x,x
f00e c9 10 cmp #$10
f010 f0 0c beq .x_sine_pattern
f012 fe 02 03 inc enemy_ram_x,x
f015 bd 02 03 lda enemy_ram_x,x
f018 20 df da jsr sprite_4_set_x
f01b 4c 33 f0 jmp .y_calc_pos

.x_sine_pattern
; using pattern_counter for x sin
f01e fe 04 03 inc enemy_ram_pc,x
f021 bd 04 03 lda enemy_ram_pc,x
f024 aa tax
f025 bd 00 fe lda sine_table,x
f028 4a lsr
f029 4a lsr
f02a a6 76 ldx enemy_ram_offset
f02c 18 clc
f02d 7d 02 03 adc enemy_ram_x,x
f030 20 df da jsr sprite_4_set_x

```

```

        .y_calc_pos
        ; using anim_counter for y sin
f033  fe 05 03    inc enemy_ram_ac,x
f036  fe 05 03    inc enemy_ram_ac,x
        ; calc y
f039  a6 76      ldx enemy_ram_offset
f03b  bd 05 03    lda enemy_ram_ac,x
f03e  aa         tax
f03f  bd 00 fe    lda sine_table,x
f042  4a         lsr
f043  4a         lsr
f044  18         clc
f045  a6 76      ldx enemy_ram_offset
f047  7d 03 03    adc enemy_ram_y,x
f04a  20 ef da    jsr sprite_4_set_y
        ; tiles
f04d  a6 76      ldx enemy_ram_offset
f04f  bd 05 03    lda enemy_ram_ac,x
f052  29 40      and #$40
f054  f0 04      beq .frame1
        .frame0
f056  a9 7a      lda #$7a
f058  d0 02      bne .frame_done
        .frame1
f05a  a9 7c      lda #$7c
        .frame_done
f05c  20 b7 da    jsr sprite_4_set_sprite
f05f  a9 03      lda #$03
f061  20 9d da    jsr sprite_4_set_palette
        .done
f064  4c 92 db    jmp update_enemies_handler_next

```

enemy_throber.asm

```

        throber_spawn: subroutine
        ; x is set by enemy spawner
f067  20 52 dc    jsr get_oam_offset_from_ram_offset
f06a  20 d1 db    jsr enemy_spawn_random_y_pos
f06d  99 00 02    sta oam_ram_y,y ; y pos
f070  a9 00      lda #$00
f072  99 03 02    sta oam_ram_x,y ; x pos
f075  60         rts

        throber_cycle: subroutine
f076  a9 10      lda #$10
f078  85 3d      sta collision_0_w
f07a  85 3e      sta collision_0_h
f07c  20 eb db    jsr enemy_handle_damage_and_death

f07f  fe 04 03    inc enemy_ram_pc,x
f082  bd 04 03    lda enemy_ram_pc,x
f085  4a         lsr
f086  4a         lsr
f087  4a         lsr
f088  29 07      and #$07
f08a  aa         tax
f08b  bd f0 ff    lda arctang_velocities_lo,x
f08e  85 84      sta arctang_velocity_lo
f090  8a         txa
f091  a6 76      ldx enemy_ram_offset
f093  c9 05      cmp #$05
f095  90 0b      bcc .do_movement
f097  d0 06      bne .dont_change_dir
        .update_direction
f099  20 12 ce    jsr enemy_get_direction_of_player
f09c  9d 07 03    sta enemy_ram_ex,x
        .dont_change_dir
f09f  fe 05 03    inc enemy_ram_ac,x
        .do_movement
f0a2  20 f6 ce    jsr arctang_enemy_update
f0a5  bd 05 03    lda enemy_ram_ac,x
f0a8  4a         lsr
f0a9  4a         lsr
f0aa  4a         lsr
f0ab  29 03      and #$03
f0ad  0a         asl
f0ae  18         clc
f0af  69 40      adc #$40
f0b1  20 b7 da    jsr sprite_4_set_sprite
f0b4  b9 03 02    lda oam_ram_x,y
f0b7  20 df da    jsr sprite_4_set_x
f0ba  b9 00 02    lda oam_ram_y,y
f0bd  20 ef da    jsr sprite_4_set_y
f0c0  a9 03      lda #$03
f0c2  20 9d da    jsr sprite_4_set_palette

f0c5  4c 92 db    jmp update_enemies_handler_next

```

enemy_uzi.asm

```

      uzi_spawn: subroutine
f0c8 a9 20      lda #$20
f0ca 9d 02 03   sta enemy_ram_x,x
f0cd a9 24      lda #$24
f0cf 9d 05 03   sta enemy_ram_ac,x
f0d2 60         rts

      uzi_cycle: subroutine
f0d3 a9 10      lda #$10
f0d5 85 3d      sta collision_0_w
f0d7 a9 10      lda #$10
f0d9 85 3e      sta collision_0_h
f0db 20 eb db   jsr enemy_handle_damage_and_death
                ; x pos
f0de bd 02 03   lda enemy_ram_x,x
f0e1 38         sec
f0e2 fd 05 03   sbc enemy_ram_ac,x
f0e5 20 df da   jsr sprite_4_set_x
f0e8 bd 05 03   lda enemy_ram_ac,x
f0eb f0 03      beq .kickback_done
f0ed de 05 03   dec enemy_ram_ac,x
      .kickback_done
                ; y pos
f0f0 fe 04 03   inc enemy_ram_pc,x
f0f3 bd 04 03   lda enemy_ram_pc,x
f0f6 aa         tax
f0f7 bd 00 fe   lda sine_table,x
f0fa 4a         lsr
f0fb 18         clc
f0fc 69 18      adc #$18
f0fe a6 76      ldx enemy_ram_offset
f100 9d 03 03   sta enemy_ram_y,x
f103 20 ef da   jsr sprite_4_set_y
                ; sprite
f106 a9 ec      lda #$ec
f108 20 b7 da   jsr sprite_4_set_sprite
                ; palette
f10b a9 01      lda #$01
f10d 20 9d da   jsr sprite_4_set_palette
      .bullet_spawn
f110 a5 26      lda rng0
f112 4a         lsr
f113 29 07      and #$07
f115 d0 09      bne .dont_shoot
f117 20 23 f1   jsr bullet_spawn
                ; makes for less repetitive patterns
f11a 20 52 c1   jsr get_next_random
f11d 20 52 c1   jsr get_next_random
      .dont_shoot
f120 4c 92 db   jmp update_enemies_handler_next

```

```

      bullet_spawn: subroutine
f123 b9 03 02   lda oam_ram_x,y
f126 85 88      sta temp02
f128 b9 00 02   lda oam_ram_y,y
f12b 85 89      sta temp03
f12d 20 5f dc   jsr get_enemy_slot_next
f130 90 2d      bcc .done
                ; sprite
f132 a9 09      lda #bullet_id
f134 9d 00 03   sta enemy_ram_type,x
f137 a8         tay
f138 b9 c5 dc   lda enemy_hitpoints_table,y
f13b 9d 01 03   sta enemy_ram_hp,x
                ; load y register
f13e 20 52 dc   jsr get_oam_offset_from_ram_offset
                ; x pos
f141 a5 88      lda temp02
f143 18         clc
f144 69 0b      adc #$0b
f146 99 03 02   sta oam_ram_x,y
                ; y pos
f149 a5 89      lda temp03
f14b 18         clc
f14c 69 03      adc #$03
f14e 99 00 02   sta oam_ram_y,y
                ; animate uzi kickback
f151 a6 76      ldx enemy_ram_offset
f153 bd 05 03   lda enemy_ram_ac,x
f156 18         clc
f157 69 05      adc #$05 ; kickback in pixels
f159 9d 05 03   sta enemy_ram_ac,x
                ; trigger sfx
f15c 20 5e fa   jsr sfx_shoot_bullet
      .done
f15f 60         rts

```

enemy_zigzag.asm

```

bullet_cycle: subroutine
f160 a9 08      lda #$08
f162 85 3d      sta collision_0_w
f164 a9 04      lda #$04
f166 85 3e      sta collision_0_h
f168 20 eb db   jsr enemy_handle_damage_and_death
                ; animate gleam
f16b a5 24      lda wtf
f16d 4a         lsr
f16e 4a         lsr
f16f 29 01      and #$01
f171 18         clc
f172 69 ee      adc #$ee
f174 99 01 02   sta oam_ram_spr,y
                ; move to the right
f177 b9 03 02   lda oam_ram_x,y
f17a 69 03      adc #$03
                ; despawn check
f17c c9 04      cmp #$04
f17e b0 06      bcs .dont_despawn
.despawn
f180 20 47 dd   jsr enemy_clear
f183 4c 8e f1   jmp .done
.dont_despawn
f186 99 03 02   sta oam_ram_x,y
f189 a9 01      lda #$01
f18b 20 39 dc   jsr enemy_set_palette
.done
f18e 4c 92 db   jmp update_enemies_handler_next

```

```

                ; pattern counter holds direction index 0..3
zigzag_dir_table:
                ; r/u l/u l/d r/d
f191 byte      #$03, #$09, #$0f, #$15
zigzag_att_table:
                ; r/u l/u l/d r/d
f195 byte      #$83, #$c3, #$43, #$03

```

```

zigzag_spawn: subroutine
                ; set direction
f199 20 52 c1   jsr get_next_random
f19c 4a         lsr
f19d 29 03      and #$03
f19f 9d 04 03   sta enemy_ram_pc,x
f1a2 86 86      stx temp00
f1a4 aa         tax
f1a5 bd 91 f1   lda zigzag_dir_table,x
f1a8 a6 86      ldx temp00
f1aa 9d 07 03   sta enemy_ram_ex,x
                ; set position
f1ad 4c 1a db   jmp enemy_spawn_set_x_y_rng

```

```

zigzag_cycle: subroutine
f1b0 a9 08      lda #$08
f1b2 85 3d      sta collision_0_w
f1b4 85 3e      sta collision_0_h
f1b6 20 eb db   jsr enemy_handle_damage_and_death
f1b9 a5 24      lda wtf
f1bb 29 03      and #$03
f1bd d0 37      bne .dont_reset_ac
                ; animation
f1bf de 05 03   dec enemy_ram_ac,x
f1c2 10 32      bpl .dont_reset_ac
                ; reset ac and pick new direction
f1c4 a9 05      lda #$05
f1c6 9d 05 03   sta enemy_ram_ac,x

```

player_main.dasm

```

; get range 0..2 from 8bit value
f1c9 20 52 c1 jsr get_next_random
f1cc 4a      lsr
f1cd 18      clc
f1ce 65 26   adc rng0
f1d0 29 80   and #$80
f1d2 2a      rol
f1d3 2a      rol ; should have a value between 0 and 2 here
f1d4 c9 02   cmp #$02
f1d6 f0 0d   beq .dir_sub
f1d8 c9 01   cmp #$01
f1da f0 03   beq .dir_add
        .dir_same
f1dc 4c e8 f1 jmp .ac_reset_done
        .dir_add
f1df fe 04 03 inc enemy_ram_pc,x
f1e2 4c e8 f1 jmp .ac_reset_done
        .dir_sub
f1e5 de 04 03 dec enemy_ram_pc,x
        .ac_reset_done
f1e8 bd 04 03 lda enemy_ram_pc,x
f1eb 29 03   and #$03
f1ed a8      tay
f1ee b9 91 f1 lda zigzag_dir_table,y
f1f1 9d 07 03 sta enemy_ram_ex,x
f1f4 a4 77   ldy enemy_oam_offset
        .dont_reset_ac
        ; set sprite
f1f6 bd 05 03 lda enemy_ram_ac,x
f1f9 18      clc
f1fa 69 5a   adc #$5a ; base sprite tile
f1fc 99 01 02 sta oam_ram_spr,y

        ; work out the direction
f1ff a9 c6   lda #<arctang_velocity_1.25
f201 85 84   sta arctang_velocity_lo
f203 20 f6 ce jsr arctang_enemy_update

f206 bd 04 03 lda enemy_ram_pc,x
f209 29 03   and #$03
f20b a8      tay
f20c b9 95 f1 lda zigzag_att_table,y
f20f a4 77   ldy enemy_oam_offset
f211 20 39 dc jsr enemy_set_palette

f214 4c 92 db jmp update_enemies_handler_next

```

```

02 04      oam_ram_rudy EQU $0204

; CONST
player_dead_tile EQM $c0
iframe_attract_length EQM $08
iframe_game_length EQM $04

player_game_reset: subroutine
f217 a9 00   lda #$00
f219 85 25   sta ftw
        ; clearing 18 bytes of game data
        ; see zp_vars.asm for details! ;D
f21b a2 20   ldx #$20
        .clear_loop
f21d 95 59   sta phase_state,x
f21f ca      dex
f220 10 fb   bpl .clear_loop
        ; XXX testing phase behaviours
        ;lda #3
        ;sta phase_level
        ;lda #$3f
        ;sta phase_current
f222 20 39 d7 jsr phase_palette_load
f225 20 95 d6 jsr timer_reset
f228 a9 d0   lda #$d0
f22a 85 13   sta player_x_hi
f22c 85 0f   sta player_demo_x
f22e a9 70   lda #$70
f230 85 15   sta player_y_hi
f232 85 10   sta player_demo_y
f234 a9 ff   lda #$ff
f236 85 0b   sta player_health
        ; set player stats
f238 a9 03   lda #$03
f23a 85 0c   sta player_lives
f23c a6 23   ldx game_difficulty
f23e bd f9 f2 lda player_gun_strength_per_difficulty,x
f241 85 19   sta player_gun_str
f243 a9 0f   lda #$0f
f245 85 20   sta player_autofire_s
f247 60      rts

```

```

player_update_colors: subroutine
    ; dashboard is bg_3
    ; player is spr_0
f248 a5 a9    lda player_color0
f24a 85 9f    sta pal_spr_0_3
    ; darken dash bg color
f24c 38      sec
f24d e9 10    sbc #$10
f24f 85 9a    sta pal_bg_3_1
    ; do the rest of player colours
f251 a5 aa    lda player_color1
f253 85 9b    sta pal_bg_3_2
f255 85 9e    sta pal_spr_0_2
f257 a5 ab    lda player_color2
f259 85 9c    sta pal_bg_3_3
f25b 85 9d    sta pal_spr_0_1
f25d 60      rts

player_take_damage: subroutine
    ; make sure iframes are not enable
f25e a5 53    lda state_iframes
f260 f0 01    beq .not_iframes
f262 60      rts
.not_iframes
f263 a5 6c    lda shroom_counter
f265 f0 01    beq .apply_damage
f267 60      rts
.apply_damage
    ; set damage display counter
f268 a9 10    lda #$10
f26a 85 22    sta player_damage_flash
    ; XXX enemy hits won't trigger iframes if turbo
gun?
    ; check if turbo gun is enabled
f26c a5 1c    lda player_controls
f26e 29 80    and #BUTTON_A
f270 d0 04    bne .skip_iframes
    ; set iframes
f272 a5 54    lda state_iframe_length
f274 85 53    sta state_iframes
.skip_iframes
    ; damage amount in player_damage
f276 a5 0b    lda player_health
f278 38      sec
f279 e5 21    sbc player_damage
f27b 90 05    bcc .player_dead
f27d f0 03    beq .player_dead
f27f 85 0b    sta player_health
f281 60      rts

```

```

.player_dead
    ; player can be saved by mask shield!
f282 a5 6e    lda orbit_shield_speed
f284 f0 0d    beq .player_really_dead
    ; disable shield
f286 a9 00    lda #$00
f288 85 6e    sta orbit_shield_speed
    ; drift off screen to the right
f28a a9 01    lda #1
f28c 85 c6    sta bullet_x_vel
f28e a9 ff    lda #$ff
f290 85 0b    sta player_health
f292 60      rts
.player_really_dead
    ; player_health 00 means DEATH
f293 a9 00    lda #$00
f295 85 0b    sta player_health
f297 60      rts

set_player_sprite: subroutine
f298 a5 55    lda state_render_addr
f29a c9 82    cmp #state_render_jump_table_offset+1
f29c d0 0c    bne .not_menu_screens
.is_menu_screens
    ; sprite (menus)
f29e a9 8e    lda #$8e
f2a0 8d 05 02 sta oam_ram_rudy+1
f2a3 a9 8f    lda #$8f
f2a5 8d 09 02 sta oam_ram_rudy+5
f2a8 d0 0a    bne .sprite_set
.not_menu_screens
    ; sprite (game)
f2aa a9 4a    lda #$4a
f2ac 8d 05 02 sta oam_ram_rudy+1
f2af a9 4b    lda #$4b
f2b1 8d 09 02 sta oam_ram_rudy+5
.sprite_set
    ; x pos
f2b4 a5 13    lda player_x_hi
f2b6 8d 07 02 sta $207
f2b9 18      clc
f2ba 69 08    adc #$08
f2bc 8d 0b 02 sta $20b
    ; y pos
f2bf a5 15    lda player_y_hi
f2c1 8d 04 02 sta $204
f2c4 8d 08 02 sta $208
    ; attributes
f2c7 a9 00    lda #$00
f2c9 8d 06 02 sta $206
f2cc 8d 0a 02 sta $20a

f2cf a5 0b    lda player_health
f2d1 d0 09    bne .not_dead

```



```

.is_dead
f2d3 a9 c0    lda #player_dead_tile
f2d5 8d 09 02 sta oam_ram_rudy+5
f2d8 8d 05 02 sta oam_ram_rudy+1
f2db 60      rts

.not_dead
; check and decrement iframe counter
f2dc a5 53    lda state_iframes
f2de f0 02    beq .not_iframes
f2e0 c6 53    dec state_iframes

.not_iframes
; check and decrement iframe counter
f2e2 a5 22    lda player_damage_flash
f2e4 f0 12    beq .not_flashing
f2e6 c6 22    dec player_damage_flash
; flash rudy based on iframe counter
f2e8 4a      lsr
f2e9 29 01    and #$01
f2eb d0 0b    bne .not_flashing
; iframes
f2ed a9 4c    lda #$4c
f2ef 8d 05 02 sta oam_ram_rudy+1
f2f2 a9 4d    lda #$4d
f2f4 8d 09 02 sta oam_ram_rudy+5
f2f7 60      rts

.not_flashing
f2f8 60      rts

```

player_bullets.dasm

```

;;;;;;;; Constants

; oam ram
bullet0      EQM $020c
bullet1      EQM $0210
bullet2      EQM $0214
bullet3      EQM $0218
bullet4_or_opt EQM $021c

player_bullet_sprite EQM $79

player_gun_strength_per_difficulty:
f2f9 byte    #4, #5, #6, #7

player_bullet_next: subroutine
; increase bullet data offset
f2fd e8      inx
f2fe e8      inx
; increase bullet oam offset
f2ff c8      iny
f300 c8      iny
f301 c8      iny
f302 c8      iny
f303 60      rts

player_bullet_collision_handler: subroutine
f304 18      clc
f305 a5 3b    lda collision_0_x
f307 65 3d    adc collision_0_w
f309 90 01    bcc .in_bounds ; make sure x+w is not less than x
f30b 60      rts

.in_bounds
; bullet dimensions don't change
f30c a9 04    lda #$04
f30e 85 41    sta collision_1_w
f310 85 42    sta collision_1_h

f312 84 86    sty temp00
f314 a0 0c    ldy #$0c

.bullet_loop
f316 b9 03 02 lda oam_ram_x,y
f319 f0 3e    beq .skip_bullet
f31b 85 3f    sta collision_1_x
f31d b9 00 02 lda oam_ram_y,y
f320 85 40    sta collision_1_y

; collision detection
f322 18      clc
f323 a5 3b    lda collision_0_x
f325 65 3d    adc collision_0_w

```

```

; make sure x+w is not less than x
f327 b0 30 bcs .no_collision
f329 c5 3f cmp collision_1_x
f32b 90 2c bcc .no_collision
f32d 18 clc
f32e a5 3f lda collision_1_x
f330 65 41 adc collision_1_w
f332 c5 3b cmp collision_0_x
f334 90 23 bcc .no_collision
f336 18 clc
f337 a5 3c lda collision_0_y
f339 65 3e adc collision_0_h
f33b c5 40 cmp collision_1_y
f33d 90 1a bcc .no_collision
f33f 18 clc
f340 a5 40 lda collision_1_y
f342 65 42 adc collision_1_h
f344 c5 3c cmp collision_0_y
f346 90 11 bcc .no_collision

f348 18 clc
f349 a5 79 lda enemy_dmg_accumulator
f34b 65 19 adc player_gun_str
f34d 85 79 sta enemy_dmg_accumulator
; despawn bullet
f34f a9 00 lda #$00
f351 99 03 02 sta oam_ram_x,y
f354 a9 ff lda #$ff
f356 99 00 02 sta oam_ram_y,y
.no_collision
.skip_bullet
f359 c8 iny
f35a c8 iny
f35b c8 iny
f35c c8 iny
f35d c0 20 cpy #$20
f35f d0 b5 bne .bullet_loop
f361 a4 86 ldy temp00
f363 60 rts

; bullets are sprites #$03 - #$07
;;; bullet directions, sprite base address
; 03 9 o'clock -7, 0, $020c
; 04 10 o'clock -6, -3, $0210
; 05 8 o'clock -6, +3, $0214
; 06 11 o'clock -3, -6, $0218
; 07 7 o'clock -3, +6, $021c

player_gun_bullet_dir_table:
f364 byte #248, #0
f366 byte #249, #252
f368 byte #252, #6
f36a byte #249, #3
f36c byte #252, #249

```

```

player_gun_fire: subroutine
; temp00 holds five_directions == true
f36e a2 00 ldx #$00
f370 a0 0c ldy #$0c
.bullet_loop
f372 b9 00 02 lda oam_ram_y,y
; check if bullet is spawned
f375 c9 ff cmp #$ff
f377 d0 37 bne .next
; trigger pewpew
f379 20 4c f8 jsr sfx_pewpew
f37c a5 13 lda player_x_hi
f37e 99 03 02 sta oam_ram_x,y
f381 a5 15 lda player_y_hi
f383 18 clc
f384 69 04 adc #$04
f386 99 00 02 sta oam_ram_y,y
; direction
f389 a5 86 lda temp00
f38b f0 0c beq .one_direction
.five_directions
f38d bd 64 f3 lda player_gun_bullet_dir_table,x
f390 95 c6 sta bullet_x_vel,x
f392 bd 65 f3 lda player_gun_bullet_dir_table+1,x
f395 95 c7 sta bullet_y_vel,x
f397 d0 0a bne .dir_done
.one_direction
f399 ad 64 f3 lda player_gun_bullet_dir_table
f39c 95 c6 sta bullet_x_vel,x
f39e ad 65 f3 lda player_gun_bullet_dir_table+1
f3a1 95 c7 sta bullet_y_vel,x
.dir_done
; sprite
f3a3 a9 79 lda #$79
f3a5 99 01 02 sta oam_ram_spr,y
; palette
f3a8 a9 00 lda #$00
f3aa 99 02 02 sta oam_ram_att,y
f3ad 4c b7 f3 jmp .done
.next
f3b0 20 fd f2 jsr player_bullet_next
f3b3 c0 20 cpy #$20
f3b5 d0 bb bne .bullet_loop
.done
f3b7 60 rts

```

```

player_bullets_check_controls: subroutine
    ; check for r bag powerup running
f3b8 a5 6b    lda r_bag_counter
f3ba f0 04    beq .no_r_bag
f3bc c6 6b    dec r_bag_counter
f3be d0 19    bne .turbo_fire
.no_r_bag
    ; a button = turbo
    ; no turbo on phase 0
f3c0 a5 5a    lda phase_current
f3c2 f0 1b    beq .no_turbo
    ; check turbo button
f3c4 a5 1c    lda player_controls
f3c6 29 80    and #BUTTON_A
f3c8 f0 15    beq .no_turbo
    ; damage from turbo gun
f3ca a5 24    lda wtf
f3cc 29 02    and #$02
f3ce c9 02    cmp #$02
f3d0 d0 07    bne .turbo_fire
f3d2 a9 03    lda #$03
f3d4 85 21    sta player_damage
f3d6 20 5e f2 jsr player_take_damage
.turbo_fire
f3d9 a9 01    lda #$01
f3db 85 86    sta temp00
f3dd d0 19    bne .pull_that_mfing_trigger
.no_turbo
f3df a5 1d    lda player_controls_debounced
f3e1 29 40    and #BUTTON_B
f3e3 d0 0f    bne .normal_fire
    ; hold b button = retrigger
f3e5 a5 1c    lda player_controls
f3e7 29 40    and #BUTTON_B
f3e9 f0 10    beq .no_fire
f3eb a5 1f    lda player_autofire_c
f3ed 18      clc
f3ee 65 20    adc player_autofire_s
f3f0 85 1f    sta player_autofire_c
f3f2 90 07    bcc .no_fire
.normal_fire
f3f4 a9 00    lda #$00
f3f6 85 86    sta temp00
.pull_that_mfing_trigger
f3f8 20 6e f3 jsr player_gun_fire
.no_fire
f3fb 60      rts

```

```

player_bullets_update: subroutine
f3fc a2 00    ldx #$00
f3fe a0 0c    ldy #$0c
.bullet_loop
    ; handle mask shield
    cpx #$00
f402 d0 07    bne .mask_check_done
f404 a5 6e    lda orbit_shield_speed
f406 f0 03    beq .mask_check_done
f408 4c 45 f4 jmp player_bullets_update_mask_shield
.mask_check_done
    ; check if spawned
f40b b9 00 02 lda oam_ram_y,y
f40e c9 ff    cmp #$ff
f410 f0 2b    beq .next
    ; check for despawn
    ; is x < 8
f412 b9 03 02 lda oam_ram_x,y
f415 c9 08    cmp #$08
f417 90 1c    bcc .despawn
    ; is y > 180
f419 b9 00 02 lda oam_ram_y,y
f41c c9 b4    cmp #180
f41e b0 15    bcs .despawn
    ; move x
f420 b9 03 02 lda oam_ram_x,y
f423 18      clc
f424 75 c6    adc bullet_x_vel,x
f426 99 03 02 sta oam_ram_x,y
    ; move y
f429 b9 00 02 lda oam_ram_y,y
f42c 18      clc
f42d 75 c7    adc bullet_y_vel,x
f42f 99 00 02 sta oam_ram_y,y
f432 4c 3d f4 jmp .next
.despawn
f435 a9 ff    lda #$ff
f437 99 03 02 sta oam_ram_x,y
f43a 99 00 02 sta oam_ram_y,y
player_bullets_update_next:
.next
f43d 20 fd f2 jsr player_bullet_next
f440 c0 20    cpy #$20
f442 d0 bc    bne .bullet_loop
.done
f444 60      rts

```

```

player_bullets_update_mask_shield: subroutine
    ; x = bullet pos offset
    ; y = bullet oam offset
f445 a5 6f    lda orbit_shield_pos
f447 18      clc
f448 65 6e    adc orbit_shield_speed
f44a 85 6f    sta orbit_shield_pos
    ; calc x
f44c a5 13    lda player_x_hi
f44e 38      sec
f44f e9 1c    sbc #$1c
f451 85 86    sta temp00
f453 a5 6f    lda orbit_shield_pos
f455 4a      lsr
f456 aa      tax
f457 bd 00 05 lda sine_6bits,x
f45a 18      clc
f45b 65 86    adc temp00
f45d 99 03 02 sta oam_ram_x,y
    ; calc y
f460 a2 00    ldx #$00
f462 a5 15    lda player_y_hi
f464 38      sec
f465 e9 10    sbc #16
f467 85 86    sta temp00
f469 a5 6f    lda orbit_shield_pos
f46b 18      clc
f46c 69 40    adc #$40
f46e 4a      lsr
f46f aa      tax
f470 bd 80 05 lda sine_5bits,x
f473 18      clc
f474 65 86    adc temp00
f476 99 00 02 sta oam_ram_y,y
    ; set attributes
f479 a9 03    lda #$03
f47b 99 02 02 sta oam_ram_att,y
    ; set sprites
f47e a9 24    lda #$24
f480 99 01 02 sta oam_ram_spr,y
f483 a2 00    ldx #$00
f485 4c 3d f4 jmp player_bullets_update_next

```

player_controls.dasm

```

;;;;; CONTROLLER READING

    ; reading joypad gives us a byte
    ; bit 7 a button
    ; bit 6 b button
    ; bit 5 select
    ; bit 4 start
    ; bit 3 up
    ; bit 2 down
    ; bit 1 left
    ; bit 0 right

controller_poller: subroutine
f488 a2 01    ldx #$01
f48a 8e 16 40 stx JOYPAD1
f48d ca      dex
f48e 8e 16 40 stx JOYPAD1
f491 a2 08    ldx #$08
    .read_loop
f493 ad 16 40 lda JOYPAD1
f496 4a      lsr
f497 26 86    rol temp00
f499 4a      lsr
f49a 26 87    rol temp01
f49c ca      dex
f49d d0 f4    bne .read_loop
f49f a5 86    lda temp00
f4a1 05 87    ora temp01
f4a3 85 86    sta temp00
f4a5 60      rts

player_controls_read: subroutine
f4a6 20 88 f4 jsr controller_poller
    .checksum_loop
f4a9 a4 86    ldy temp00
f4ab 20 88 f4 jsr controller_poller
f4ae c4 86    cpy temp00
f4b0 d0 f7    bne .checksum_loop
f4b2 a5 86    lda temp00
f4b4 a8      tay
f4b5 45 1c    eor player_controls
f4b7 25 86    and temp00
f4b9 85 1d    sta player_controls_debounced
f4bb 84 1c    sty player_controls
f4bd 60      rts

```

player_movement.dasm

```
; playfield = 256 x 192
; player coordinates should be bound
;           from 16 x 16 to 240 x 176
```

player_move_position: subroutine

```
f4be 20 f5 f4    jsr player_direction_handler
                ; check for boundless
f4c1 a5 1e        lda player_boundless
f4c3 d0 28        bne .done

f4c5 a5 13        lda player_x_hi
f4c7 c9 12        cmp #18
f4c9 b0 04        bcs .right_enuff
f4cb a9 12        lda #18
f4cd 85 13        sta player_x_hi
                .right_enuff

f4cf a5 13        lda player_x_hi
f4d1 c9 dd        cmp #221
f4d3 90 04        bcc .left_enuff
f4d5 a9 dd        lda #221
f4d7 85 13        sta player_x_hi
                .left_enuff

f4d9 a5 15        lda player_y_hi
f4db c9 08        cmp #8
f4dd b0 04        bcs .down_enuff
f4df a9 08        lda #8
f4e1 85 15        sta player_y_hi
                .down_enuff

f4e3 a5 15        lda player_y_hi
f4e5 c9 ae        cmp #174
f4e7 90 04        bcc .up_enuff
f4e9 a9 ae        lda #174
f4eb 85 15        sta player_y_hi
                .up_enuff
                .done
f4ed a5 15        lda player_y_hi
f4ef 20 ff da     jsr enemy_fix_y_visible
f4f2 85 15        sta player_y_hi
f4f4 60          rts
```

player_direction_handler: subroutine

```
; shroom handling
f4f5 a5 6c        lda shroom_counter
f4f7 f0 20        beq .not_shrooming
                ; shroom right
f4f9 a9 10        lda #%0010000
f4fb 24 6d        bit shroom_mod
f4fd d0 02        bne .shroom_not_right
f4ff c6 13        dec player_x_hi
                .shroom_not_right
                ; shroom left
f501 a9 08        lda #%0001000
f503 24 6d        bit shroom_mod
f505 d0 02        bne .shroom_not_left
f507 e6 13        inc player_x_hi
                .shroom_not_left
                ; shroom up
f509 a9 02        lda #%0000010
f50b 24 6d        bit shroom_mod
f50d d0 02        bne .shroom_not_up
f50f c6 15        dec player_y_hi
                .shroom_not_up
                ; shroom down
f511 a9 04        lda #%0000100
f513 24 6d        bit shroom_mod
f515 d0 02        bne .shroom_not_down
f517 e6 15        inc player_y_hi
                .shroom_not_down
                .not_shrooming
                ; get diagonal or not
f519 a9 00        lda #$00
f51b 85 86        sta temp00
f51d a5 1c        lda player_controls
f51f 29 03        and #BUTTON_LEFT|BUTTON_RIGHT
f521 f0 02        beq .not_left_or_right
f523 e6 86        inc temp00
                .not_left_or_right
f525 a5 1c        lda player_controls
f527 29 0c        and #BUTTON_UP|BUTTON_DOWN
f529 f0 02        beq .not_up_or_down
f52b e6 86        inc temp00
                .not_up_or_down
f52d a5 86        lda temp00
f52f d0 01        bne .direction_found
f531 60          rts
                .direction_found
f532 38          sec
f533 e9 01        sbc #1 ; 1 for diagonal detection
f535 0a          asl
f536 0a          asl
                ; get speed
f537 18          clc
f538 65 1a        adc player_speed
f53a aa          tax
                ; get add/sub for each axis
                ; LEFT
f53b a5 1c        lda player_controls
```

```

f53d 29 02      and #BUTTON_LEFT
f53f f0 0f      beq .not_left
f541 a5 14      lda player_x_lo
f543 38         sec
f544 fd 98 f5   sbc player_direction_speed_table_lo,x
f547 85 14      sta player_x_lo
f549 a5 13      lda player_x_hi
f54b fd 90 f5   sbc player_direction_speed_table_hi,x
f54e 85 13      sta player_x_hi
               .not_left
               ; RIGHT
f550 a5 1c      lda player_controls
f552 29 01      and #BUTTON_RIGHT
f554 f0 0f      beq .not_right
f556 a5 14      lda player_x_lo
f558 18         clc
f559 7d 98 f5   adc player_direction_speed_table_lo,x
f55c 85 14      sta player_x_lo
f55e a5 13      lda player_x_hi
f560 7d 90 f5   adc player_direction_speed_table_hi,x
f563 85 13      sta player_x_hi
               .not_right
               ; UP
f565 a5 1c      lda player_controls
f567 29 08      and #BUTTON_UP
f569 f0 0f      beq .not_up
f56b a5 16      lda player_y_lo
f56d 38         sec
f56e fd 98 f5   sbc player_direction_speed_table_lo,x
f571 85 16      sta player_y_lo
f573 a5 15      lda player_y_hi
f575 fd 90 f5   sbc player_direction_speed_table_hi,x
f578 85 15      sta player_y_hi
               .not_up
               ; DOWN
f57a a5 1c      lda player_controls
f57c 29 04      and #BUTTON_DOWN
f57e f0 0f      beq .not_down
f580 a5 16      lda player_y_lo
f582 18         clc
f583 7d 98 f5   adc player_direction_speed_table_lo,x
f586 85 16      sta player_y_lo
f588 a5 15      lda player_y_hi
f58a 7d 90 f5   adc player_direction_speed_table_hi,x
f58d 85 15      sta player_y_hi
               .not_down
f58f 60         rts

```

```

; speed - straight / diag
; speed 1  0.75 / 0.53
; speed 2  1.00 / 0.7071
; speed 3  1.75 / 1.2374
; speed 4  2.50 / 1.7677
; 1 192 - 1.75
; 2 223 - 2.87
; 3 128 - 3.5 / 2 121 - 2.475
; 4  51 - 4.2 / 2 248 - 2.97

```

```
;; calc diagonal by dividing straight by 1.414
```

```

; speeds 1 .. 4
player_direction_speed_table_hi:
; straight values
f590 byte      #1, #2, #3, #4
; diagonal values
f594 byte      #1, #2, #2, #2
player_direction_speed_table_lo:
; straight values
f598 byte      #192, #223, #128, #51
; diagonal values
f59c byte      #61, #8, #121, #248

```

```

; old version
; straight/ diag
;speed 1  0.75 / 0.53
;speed 2  1.00 / 0.7071
;speed 3  1.75 / 1.2374
;speed 4  2.50 / 1.7677

```

```

; old old version
;speed 3  2.25 / 1.5909
;speed 4  3.50 / 2.4748

```

```
; OG speeds
```

```

; speed 1
;byte #191,#0,#135,#0
; speed 2
;byte #0,#1,#181,#0
; speed 3
;byte #192,#1,#61,#1
; speed 4
;byte #128,#2,#197,#1

```

player_demo.asm

```

; controls player in demo mode

player_demo_controls: subroutine
    ; PLAYER MOVEMENT
    ; clear player directions
    lda #$00
f5a0 a9 00 sta player_controls
f5a2 85 1c sta player_controls_debounced
f5a4 85 1d ; set flags in y register
f5a6 a8 tay
    ; check x coordinate
    lda player_x_hi
f5a7 a5 13 cmp player_demo_x
f5a9 c5 0f cmp player_demo_x
f5ab f0 20 beq .player_x_equal
f5ad b0 0f bcs .player_x_greater
    .player_x_lesser
    lda player_demo_lr
f5af a5 11 cmp #$ff
f5b1 c9 ff beq .player_x_equal
f5b3 f0 18 ; go right
f5b5 a9 01 lda #BUTTON_RIGHT
f5b7 05 1c ora player_controls
f5b9 85 1c sta player_controls
    ;inc player_x_hi
f5bb 4c ce f5 jmp .player_x_done
    .player_x_greater
    lda player_demo_lr
f5be a5 11 cmp #$00
f5c0 c9 00 beq .player_x_equal
f5c2 f0 09 ; go left
f5c4 a9 02 lda #BUTTON_LEFT
f5c6 05 1c ora player_controls
f5c8 85 1c sta player_controls
    ;dec player_x_hi
f5ca 4c ce f5 jmp .player_x_done
    .player_x_equal
f5cd c8 iny
    .player_x_done
    ; check y coordinate
    lda player_y_hi
f5ce a5 15 cmp player_demo_y
f5d0 c5 10 beq .player_y_equal
f5d2 f0 20 bcs .player_y_greater
    .player_y_lesser
    lda player_demo_ud
f5d6 a5 12 cmp #$ff
f5d8 c9 ff beq .player_y_equal
f5da f0 18 ; go down
f5dc a9 04 lda #BUTTON_DOWN
f5de 05 1c ora player_controls
f5e0 85 1c sta player_controls

```

```

;inc player_y_hi
f5e2 4c f5 f5 jmp .player_y_done
    .player_y_greater
f5e5 a5 12 lda player_demo_ud
f5e7 c9 00 cmp #$00
f5e9 f0 09 beq .player_y_equal
    ; go up
f5eb a9 08 lda #BUTTON_UP
f5ed 05 1c ora player_controls
f5ef 85 1c sta player_controls
    ;dec player_y_hi
f5f1 4c f5 f5 jmp .player_y_done
    .player_y_equal
f5f4 c8 iny
    .player_y_done
    ; check if both coordinates are met
f5f5 c0 02 cpy #$02
f5f7 f0 03 beq .set_demo_new_target
f5f9 4c 34 f6 jmp .done
    .set_demo_new_target

; set x target
f5fc a5 26 lda rng0
f5fe 20 5a c1 jsr NextRandom
f601 85 26 sta rng0
f603 4a lsr
f604 18 clc
f605 69 5e adc #$5e
f607 85 0f sta player_demo_x
f609 c5 13 cmp player_x_hi
f60b b0 07 bcs .player_going_left
    .player_going_right
f60d a9 ff lda #$ff
f60f 85 11 sta player_demo_lr
f611 4c 18 f6 jmp .player_x_dir_done
    .player_going_left
f614 a9 00 lda #$00
f616 85 11 sta player_demo_lr
    .player_x_dir_done

; set y target
f618 a5 26 lda rng0
f61a 20 5a c1 jsr NextRandom
f61d 85 26 sta rng0
f61f 4a lsr
f620 18 clc
f621 69 10 adc #$10
f623 85 10 sta player_demo_y
f625 c5 15 cmp player_y_hi
f627 b0 07 bcs .player_going_up
    .player_going_down
f629 a9 ff lda #$ff
f62b 85 12 sta player_demo_ud
f62d 4c 34 f6 jmp .player_y_dir_done

```

```

        .player_going_up
f630  a9 00      lda #$00
f632  85 12      sta player_demo_ud
        .player_y_dir_done

        .done
f634  20 be f4    jsr player_move_position

        ; PLAYER GUN

f637  a5 24      lda wtf
f639  29 47      and #$47
f63b  c9 40      cmp #$40
f63d  d0 06      bne .fire_not_turbo
f63f  a9 80      lda #BUTTON_A
f641  05 1c      ora player_controls
f643  85 1c      sta player_controls
        .fire_not_turbo
f645  a9 40      lda #BUTTON_B
f647  05 1c      ora player_controls
f649  85 1c      sta player_controls
f64b  a5 24      lda wtf
f64d  29 0f      and #$0f ; should be 7
f64f  c9 00      cmp #$00
f651  d0 04      bne .fire_done
f653  a9 40      lda #BUTTON_B
f655  85 1d      sta player_controls_debounced
        .fire_done
f657  20 b8 f3    jsr player_bullets_check_controls
f65a  60         rts

```

apu_engine.asm

```

        ; PULSE CHANNELS
        ;$4000/$4004 DDLC VVVV
        ;      Duty (D),
        ;      envelope loop / length counter halt (L),
        ;      constant volume (C), volume/envelope (V)
        ;$4001/$4005 EPPP NSSS
        ;      Sweep unit: enabled (E),
        ;      period (P), negate (N), shift (S)
        ;$4002/$4006 TTTT TTTT
        ;      Timer low (T)
        ;$4003/$4007 LLLL LTTT
        ;      Length counter load (L), timer high (T)

40 00      APU_PULSE1_VOL      = $4000
40 01      APU_PULSE1_SWEEP    = $4001
40 02      APU_PULSE1_TIMER_LO  = $4002
40 03      APU_PULSE1_TIMER_HI  = $4003
40 04      APU_PULSE2_VOL      = $4004
40 05      APU_PULSE2_SWEEP    = $4005
40 06      APU_PULSE2_TIMER_LO  = $4006
40 07      APU_PULSE2_TIMER_HI  = $4007

        ; TRIANGLE CHANNEL
        ;$4008 CRRR RRRR
        ;      Length counter halt (C),
        ;      linear counter load (R)
        ;$4009 ---- ---- Unused
        ;$400A TTTT TTTT
        ;      Timer low (T)
        ;$400B LLLL LTTT
        ;      Length counter load (L), timer high (T)

40 08      APU_TRI_CONTROL     = $4008
40 09      APU_TRI_UNUSED      = $4009
40 0a      APU_TRI_TIMER_LO     = $400a
40 0b      APU_TRI_TIMER_HI     = $400b

        ; NOISE CHANNEL
        ;$400C --LC VVVV
        ;      envelope loop / length counter halt (L),
        ;      constant volume (C), volume/envelope (V)
        ;$400D ---- ---- Unused
        ;$400E L--- PPPP
        ;      Loop noise (L), noise period (P)
        ;$400F LLLL L---
        ;      Length counter load (L)

40 0c      APU_NOISE_VOL       = $400c
40 0e      APU_NOISE_FREQ      = $400e
40 0f      APU_NOISE_TIMER     = $400f

```



```

40 10      DMC_FREQ          = $4010
40 15      APU_STATUS        = $4015
40 10      APU_DMC_CTRL      = $4010
40 15      APU_CHAN_CTRL     = $4015
40 17      APU_FRAME         = $4017

01 40      apu_cache         = $0140

apu_init: subroutine
; Init $4000-4013
ldy #$13
.loop
f65d b9 7c f6    lda apu_init_register_values,y
f660 99 40 01    sta apu_cache,y
f663 99 00 40    sta $4000,y
f666 88          dey
f667 10 f4       bpl .loop
; We have to skip over $4014 (OAMDMA)
f669 a9 0f       lda #$0f
f66b 8d 15 40    sta $4015
f66e a9 40       lda #$40
f670 8d 17 40    sta $4017
apu_rng_reset:
; seed rng
f673 a9 11       lda #$11
f675 85 b5       sta apu_rng0
f677 a9 7f       lda #$7f
f679 85 b6       sta apu_rng1
f67b 60          rts

apu_init_register_values:
f67c byte        $30,$08,$00,$00
f680 byte        $30,$08,$00,$00
f684 byte        $80,$00,$00,$00
f688 byte        $30,$00,$00,$00
f68c byte        $00,$00,$00,$00

octoscale:
f690 byte        $00,$02,$03,$05,$06,$08,$09,$0b
f698 byte        $0c,$0e,$0f,$11,$12,$14,$15,$17
f6a0 byte        $18,$1a ; , $1b,$1d,$1e,$20,$21,$23

majpentscale:
f6a2 byte        #0, #2, #4, #7, #9
f6a7 byte        #12,#14,#16,#19,#21
f6ac byte        #24,#26,#28,#31,#33
f6b1 byte        #36,#38,#40,#43,#45
f6b6 byte        #38,#40,#19

```

```

periodTableLo:
;A  A#  B   C   C#  D   D#  E   F   F#  G   G#
f6b9 byte        $f1,$7f,$13,$ad,$4d,$f3,$9d,$4c,$00,$b8,$74,$34
f6c5 byte        $f8,$bf,$89,$56,$26,$f9,$ce,$a6,$80,$5c,$3a,$1a
f6d1 byte        $fb,$df,$c4,$ab,$93,$7c,$67,$52,$3f,$2d,$1c,$0c
f6dd byte        $fd,$ef,$e1,$d5,$c9,$bd,$b3,$a9,$9f,$96,$8e,$86
f6e9 byte        $7e,$77,$70,$6a,$64,$5e,$59,$54,$4f,$4b,$46,$42
f6f5 byte        $3f,$3b,$38 ; , $34,$31,$2f,$2c,$29,$27,$25,$23,$21
; byte $1f,$1d,$1b,$1a,$18,$17,$15,$14

periodTableHi:
f6f8 byte        $07,$07,$07,$06,$06,$05,$05,$05,$05,$04,$04,$04
f704 byte        $03,$03,$03,$03,$03,$02,$02,$02,$02,$02,$02,$02
f710 byte        $01,$01,$01,$01,$01,$01,$01,$01,$01,$01,$01,$01
f71c byte        $00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00
f728 byte        $00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00,$00
f734 byte        $00,$00,$00 ; , $00,$00,$00,$00,$00,$00,$00,$00,$00
; byte $00,$00,$00,$00,$00,$00,$00,$00

apu_set_pitch: subroutine
; x = pitch table offset
; y = channel low byte offset
f737 bd b9 f6    lda periodTableLo,x
f73a 99 40 01    sta apu_cache+0,y
f73d bd f8 f6    lda periodTableHi,x
f740 09 f8       ora #%11111000
f742 99 41 01    sta apu_cache+1,y
; make sure counter resets in engine
f745 a9 ff       lda #$ff
;sta apu_pu1_last_hi-2,y
; this makes "sick dingle" work
f747 85 ae       sta apu_pu1_last_hi
f749 85 b2       sta apu_pu2_last_hi
f74b 60          rts

apu_bend_down: subroutine
f74c ee 42 01    inc $142
f74f ee 46 01    inc $146
f752 ee 4a 01    inc $14a
f755 60          rts

apu_env_run: subroutine
; x = channel counter offset
; envelope type is byte after
; #$00 = pu1
; #$04 = pu2
; #$07 = noise
; returns 4-bit volume in a
f756 b5 ad       lda apu_pu1_envelope,x
f758 18          clc
f759 69 66       adc #apu_env_jump_table_offset
f75b 4c f1 c4    jmp jump_to_subroutine

```

```

        apu_env_lin_long: subroutine
            ; $40 counter = 63 frames / 1 second
f75e b5 ac    lda apu_pu1_counter,x
f760 4a      lsr
f761 4a      lsr
f762 29 0f   and #%00001111
f764 60      rts
            ;apu_env_lin_short: subroutine
            ; $20 counter = 31 frames / 0.5 second
            ;lda apu_pu1_counter,x
            ;lsr
            ;and #%00001111
            ;rts
        apu_env_lin_tiny: subroutine
            ; $10 counter = 15 frames / 0.25 second
f765 b5 ac    lda apu_pu1_counter,x
f767 29 0f   and #%00001111
f769 60      rts
        apu_env_exp_long: subroutine
            ; $40 counter =~ 54 frames / 1 second
f76a b4 ac    ldy apu_pu1_counter,x
f76c b9 c0 fe lda sine_table+$c0,y
f76f 4a      lsr
f770 4a      lsr
f771 4a      lsr
f772 60      rts
        apu_env_exp_short: subroutine
            ; $20 counter =~ 28 frames / 0.5 second
f773 b4 ac    ldy apu_pu1_counter,x
f775 b9 e0 04 lda sine_7bits+$60,y
f778 4a      lsr
f779 4a      lsr
f77a 60      rts
        apu_env_exp_tiny: subroutine
            ; $10 counter =~ 15 frames / 0.25 second
f77b b4 ac    ldy apu_pu1_counter,x
f77d b9 81 f7 lda apu_env_exp_tiny_table,y
f780 60      rts
        apu_env_exp_tiny_table:
f781 byte    $00,$01,$01,$01,$01,$01,$02,$02,
f78a byte    $03,$04,$05,$07,$0a,$0c,$0f
        apu_env_exp_pico: subroutine
            ; $06 counter =~ 15 frames / 0.25 second
f791 b4 ac    ldy apu_pu1_counter,x
f793 b9 97 f7 lda apu_env_exp_pico_table,y
f796 60      rts
        apu_env_exp_pico_table:
f797 byte    $00,$01,$04,$07,$0a,$0f

```

```

apu_update: subroutine
    ; MUSIC
    ; SFX Pulse 2
    ; SFX Noise
    ; MIX and Write to APU

    ; SFX Update Delegator
f79d a5 be    lda sfx_pu2_update_type
f79f 18      clc
f7a0 69 6c    adc #sfx_update_jump_table_offset
f7a2 20 f1 c4 jsr jump_to_subroutine
f7a5 a5 bf    lda sfx_noi_update_type
f7a7 18      clc
f7a8 69 6c    adc #sfx_update_jump_table_offset
f7aa 20 f1 c4 jsr jump_to_subroutine

    ; Pulse Channels Counter / Envelope
f7ad a2 00    ldx #$00
        .pulse_channels_loop
f7af b5 ac    lda apu_pu1_counter,x
f7b1 f0 2e    beq .pulse_skip
f7b3 d6 ac    dec apu_pu1_counter,x
f7b5 d0 08    bne .pulse_enabled
        .pulse_disabled
f7b7 a9 30    lda #$30
f7b9 9d 00 40 sta $4000,x
f7bc 4c e1 f7 jmp .pulse_skip
        .pulse_enabled
f7bf 20 56 f7 jsr apu_env_run
f7c2 09 b0    ora #%10110000
f7c4 9d 00 40 sta $4000,x
f7c7 a9 08    lda #$08
f7c9 9d 01 40 sta $4001,x
f7cc bd 42 01 lda apu_cache+2,x
f7cf 18      clc
f7d0 65 6d    adc shroom_mod
f7d2 9d 02 40 sta $4002,x
f7d5 bd 43 01 lda apu_cache+3,x
f7d8 d5 ae    cmp apu_pu1_last_hi,x
f7da f0 05    beq .pulse_skip
f7dc 9d 03 40 sta $4003,x
f7df 95 ae    sta apu_pu1_last_hi,x
        .pulse_skip
f7e1 e0 00    cpx #$00
f7e3 d0 04    bne .pulse_channels_done
f7e5 a2 04    ldx #$04
f7e7 d0 c6    bne .pulse_channels_loop
        .pulse_channels_done
    ; Triangle Counter
f7e9 a5 af    lda apu_tri_counter
f7eb f0 1a    beq .triangle_skip
f7ed c6 af    dec apu_tri_counter
f7ef d0 08    bne .triangle_enabled
        .triangle_disabled
f7f1 a9 00    lda #$00
f7f3 8d 08 40 sta $4008

```

```

f7f6 4c 07 f8      jmp .triangle_skip
                .triangle_enabled
f7f9 a9 7f          lda #$7f
f7fb 8d 08 40      sta $4008
f7fe ad 4a 01      lda apu_cache+$a
f801 18            clc
f802 65 6d          adc shroom_mod
f804 8d 0a 40      sta $400a
                .triangle_skip
                ; Noise Counter
f807 a5 b3          lda apu_noi_counter
f809 f0 16          beq .noise_skip
f80b c6 b3          dec apu_noi_counter
f80d d0 08          bne .noise_enabled
                .noise_disabled
f80f a9 10          lda #%00010000
f811 8d 4c 01      sta apu_cache+12
f814 4c 21 f8      jmp .noise_skip
                .noise_enabled
f817 a2 07          ldx #$07
f819 20 56 f7      jsr apu_env_run
f81c 09 10          ora #%00010000
f81e 8d 4c 01      sta apu_cache+12
                .noise_skip
                ; copy cache to apu
f821 a0 05          ldy #$05
                .cache_to_apu_loop
f823 b9 4b 01      lda apu_cache+$b,y
f826 99 0b 40      sta $4000+$b,y
f829 88            dey
f82a 10 f7          bpl .cache_to_apu_loop
                ; RNG updates
f82c a5 b5          lda apu_rng0
f82e 20 5a c1      jsr NextRandom
f831 85 b5          sta apu_rng0
f833 a5 b6          lda apu_rng1
f835 20 60 c1      jsr PrevRandom
f838 85 b6          sta apu_rng1
                ; SFX counter updates
f83a a2 02          ldx #2
                .sfx_counter_update
f83c b5 bb          lda sfx_pu1_counter,x
f83e f0 02          beq .skip_sfx_counter
f840 d6 bb          dec sfx_pu1_counter,x
                .skip_sfx_counter
f842 ca            dex
f843 10 f7          bpl .sfx_counter_update
f845 60            rts

```

```

; envelope lengths
;
; | 0 1 2 3 4 5 6 7 8 9 A B C D E F
;-----+-----
;00-0F 10,254, 20, 2, 40, 4, 80, 6, 160, 8, 60, 10, 14, 12, 26, 14
;10-1F 12, 16, 24, 18, 48, 20, 96, 22, 192, 24, 72, 26, 16, 28, 32, 30

```

```

; envelope length table again

```

	id from table above	duration in ticks	register value
	0x00	10	0x00
	0x01	254	0x08
	0x02	20	0x10
	0x03	2	0x18
	0x04	40	0x20
	0x05	4	0x28
	0x06	80	0x30
	0x07	6	0x38
	0x08	160	0x40
	0x09	8	0x48
	0x0a	60	0x50
	0x0b	10	0x58
	0x0c	14	0x60
	0x0d	12	0x68
	0x0e	26	0x70
	0x0f	14	0x78
	0x10	12	0x80
	0x11	16	0x88
	0x12	24	0x90
	0x13	18	0x98
	0x14	48	0xa0
	0x15	20	0xa8
	0x16	96	0xb0
	0x17	22	0xb8
	0x18	192	0xc0
	0x19	24	0xc8
	0x1a	72	0xd0
	0x1b	26	0xd8
	0x1c	16	0xe0
	0x1d	28	0xe8
	0x1e	32	0xf0
	0x1f	30	0xf8

apu_sfx.asm

```
        ; zp vars
        ; sfx_phase_next_counter
        ; counters to mask other channel audio
        ; sfx_pu1_counter
        ; sfx_pu2_counter
        ; sfx_noi_counter
        ; table offsets for update subroutines
        ; sfx_pu2_update_type
        ; sfx_noi_update_type
        ; these should only use Pulse 2 and Noise channels
        ; unless its a non-music moment (like player death)
```

```
        sfx_test_delegator: subroutine
        ; a = sound effect id
f846 18      clc
f847 69 52    adc #sfx_test_jump_table_offset
f849 4c f1 c4 jmp jump_to_subroutine
```

```
        ; sound test 00
        sfx_pewpew: subroutine
f84c a5 bc    lda sfx_pu2_counter
f84e d0 1e    bne .no
        ; pulse 2
f850 a9 8f    lda #%10001111
f852 8d 04 40 sta $4004
f855 a9 82    lda #%10000010
f857 8d 05 40 sta $4005
f85a a5 26    lda rng0
f85c 29 3f    and #$3f
f85e 09 08    ora #$08
f860 8d 06 40 sta $4006
f863 a9 10    lda #%00010000
f865 8d 07 40 sta $4007
f868 a9 00    lda #0
f86a 85 b0    sta apu_pu2_counter
f86c 85 be    sta sfx_pu2_update_type
        .no
f86e 60      rts
```

```
        ; sound test 01
        sfx_player_damage: subroutine
        ; noise
f86f a5 26    lda rng0
f871 29 8f    and #$8f
f873 09 0c    ora #$0c
f875 8d 4e 01 sta apu_cache+$e
f878 a9 10    lda #$10
f87a 85 b3    sta apu_noi_counter
f87c a9 01    lda #$01
f87e 85 b4    sta apu_noi_envelope
f880 a9 10    lda #$10
f882 85 bd    sta sfx_noi_counter
f884 60      rts
```

```
        ; sound test 02
        sfx_player_death: subroutine
        ; setup pulse 1
f885 a9 8f    lda #%10001111
f887 8d 00 40 sta $4000
f88a a9 87    lda #%10000111
f88c 8d 01 40 sta $4001
f88f a9 fc    lda #$fc
f891 8d 02 40 sta $4002
f894 a9 0a    lda #%00001010
f896 8d 03 40 sta $4003
        ; setup pulse 2
f899 a9 8f    lda #%10001111
f89b 8d 04 40 sta $4004
f89e a9 87    lda #%10000111
f8a0 8d 05 40 sta $4005
f8a3 a9 fb    lda #$fb
f8a5 8d 06 40 sta $4006
f8a8 a9 09    lda #%00001001
f8aa 8d 07 40 sta $4007
        ; setup noise handler
f8ad a9 01    lda #$01
f8af 85 bf    sta sfx_noi_update_type
f8b1 a9 00    lda #$00
f8b3 85 be    sta sfx_pu2_update_type
f8b5 a9 00    lda #$00
f8b7 85 b8    sta sfx_temp00 ; volume
f8b9 a9 80    lda #$80
f8bb 85 b9    sta sfx_temp01 ; pitch
        ; sta sfx_noi_counter
f8bd 60      rts
```

```
        sfx_player_death_update: subroutine
f8be a5 b8    lda sfx_temp00 ; vol
f8c0 4a      lsr
f8c1 4a      lsr
f8c2 4a      lsr
f8c3 4a      lsr
f8c4 29 10    and #%00010000
f8c6 8d 4c 01 sta apu_cache+$c
f8c9 a5 b9    lda sfx_temp01 ; pitch
f8cb 4a      lsr
f8cc 4a      lsr
f8cd 4a      lsr
f8ce 8d 4e 01 sta apu_cache+$e
f8d1 e6 b9    inc sfx_temp01 ; pitch
f8d3 e6 b8    inc sfx_temp00 ; vol
f8d5 e6 b8    inc sfx_temp00 ; vol
f8d7 d0 09    bne .dont_kill_player_death_sound
f8d9 a9 10    lda #$10
f8db 8d 4c 01 sta apu_cache+$c
f8de a9 00    lda #$00
f8e0 85 bf    sta sfx_noi_update_type
        .dont_kill_player_death_sound
f8e2 60      rts
```

```

; sound test 03
sfx_kick:
sfx_enemy_damage: subroutine
f8e3 a5 bc    lda sfx_pu2_counter
f8e5 d0 1c    bne .no
; pulse 2
f8e7 a9 8f    lda #%10001111
f8e9 8d 04 40 sta $4004
f8ec a9 82    lda #%10000010
f8ee 8d 05 40 sta $4005
f8f1 a5 26    lda rng0
f8f3 8d 06 40 sta $4006
f8f6 a9 11    lda #%00010001
f8f8 8d 07 40 sta $4007
f8fb a9 08    lda #$08
f8fd 85 bc    sta sfx_pu2_counter
f8ff a9 00    lda #0
f901 85 be    sta sfx_pu2_update_type
; no
f903 60      rts

```

```

; sound test 04
sfx_enemy_death: subroutine
f904 a9 1f    lda #%00011111
f906 8d 4c 01 sta apu_cache+$c
f909 a9 0f    lda #$0f
f90b 8d 4e 01 sta apu_cache+$e
f90e a9 01    lda #$01
f910 85 b4    sta apu_noi_envelope
f912 a9 02    lda #$02
f914 85 bf    sta sfx_noi_update_type
f916 a9 10    lda #$10
f918 85 b3    sta apu_noi_counter
f91a 85 bd    sta sfx_noi_counter
f91c 60      rts

```

```

sfx_enemy_death_update: subroutine
f91d ad 4c 01 lda apu_cache+$c
f920 8d 4e 01 sta apu_cache+$e
f923 29 0f    and #%00001111
f925 f0 01    beq sfx_noi_update_clear
f927 60      rts

```

```

sfx_noi_update_clear: subroutine
f928 a9 10    lda #%00010000
f92a 8d 4c 01 sta apu_cache+$c
f92d a9 00    lda #$00
f92f 85 bf    sta sfx_noi_update_type
f931 60      rts

```

```

; sound test 05
sfx_powerup_hit: subroutine
f932 a9 82    lda #$82
f934 8d 4e 01 sta apu_cache+$e
f937 a9 20    lda #$20
f939 85 b3    sta apu_noi_counter
f93b a9 03    lda #$03
f93d 85 b4    sta apu_noi_envelope
f93f a9 16    lda #$16
f941 85 bd    sta sfx_noi_counter
f943 60      rts

```

```

; sound test 06
sfx_powerup_bomb: subroutine
f944 a9 2c    lda #44
f946 85 6a    sta bomb_counter
f948 85 bd    sta sfx_noi_counter
f94a a9 04    lda #$04
f94c 85 bf    sta sfx_noi_update_type
f94e 60      rts

```

```

sfx_powerup_bomb_update: subroutine
f94f a9 1f    lda #%00011111
f951 8d 4c 01 sta apu_cache+$c
f954 a5 6a    lda bomb_counter
f956 29 0f    and #%00001111
f958 09 03    ora #%00000011
f95a 8d 4e 01 sta apu_cache+$e
; set powerup counter
f95d a5 6a    lda bomb_counter
f95f f0 c7    beq sfx_noi_update_clear
f961 60      rts

```

```

; sound test 07
sfx_powerup_mushroom: subroutine
; set powerup counter
f962 a9 7f    lda #$7f
f964 85 6c    sta shroom_counter
; slow pu2 sweep up
f966 a9 8f    lda #%10001111
f968 8d 04 40 sta $4004
f96b a9 aa    lda #%10101010
f96d 8d 05 40 sta $4005
f970 a5 c2    lda audio_root_tone
f972 aa      tax
f973 bd b9 f6 lda periodTableLo,x
f976 8d 06 40 sta $4006
f979 bd f8 f6 lda periodTableHi,x
f97c 09 40    ora #%01000000
f97e 8d 07 40 sta $4007
f981 a9 10    lda #$10
f983 85 bc    sta sfx_pu2_counter
f985 60      rts

```

```

; sound test 08
sfx_powerup_mask: subroutine
; fast pu2 sweep up
f986 a9 8f    lda #%10001111
f988 8d 04 40 sta $4004
f98b a9 f9    lda #%11111001
f98d 8d 05 40 sta $4005
f990 a5 26    lda rng0
f992 29 3f    and #$3f
f994 09 08    ora #$08
f996 8d 06 40 sta $4006
f999 a9 14    lda #%00010100
f99b 8d 07 40 sta $4007
f99e a9 10    lda #$10
f9a0 85 bc    sta sfx_pu2_counter
f9a2 60      rts

```

```

; sound test 09
sfx_powerup_1up: subroutine
; imperial jingle
; root note -- x - x x x
f9a3 a9 00    lda #$00
f9a5 85 b8    sta sfx_temp00
f9a7 a9 05    lda #$05
f9a9 85 be    sta sfx_pu2_update_type
f9ab 60      rts

sfx_powerup_1up_update: subroutine
f9ac a5 b8    lda sfx_temp00
f9ae f0 0e    beq .trigger_lower_note
f9b0 c9 10    cmp #$10
f9b2 f0 0a    beq .trigger_lower_note
f9b4 c9 18    cmp #$18
f9b6 f0 06    beq .trigger_lower_note
f9b8 c9 20    cmp #$20
f9ba f0 09    beq .trigger_higher_note
f9bc d0 2d    bne .done

.trigger_lower_note
f9be a5 c2    lda audio_root_tone
f9c0 18      clc
f9c1 69 18    adc #$18
f9c3 d0 09    bne .trigger_note

.trigger_higher_note
f9c5 a9 00    lda #$00
f9c7 85 be    sta sfx_pu2_update_type
f9c9 a5 c2    lda audio_root_tone
f9cb 18      clc
f9cc 69 24    adc #$24

.trigger_note
f9ce aa      tax
f9cf a9 83    lda #%10000011
f9d1 8d 04 40 sta $4004
f9d4 a9 00    lda #$00
f9d6 8d 05 40 sta $4005
f9d9 bd b9 f6 lda periodTableLo,x
f9dc 8d 06 40 sta $4006
f9df bd f8 f6 lda periodTableHi,x
f9e2 09 40    ora #%01000000

```

```

f9e4 8d 07 40 sta $4007
f9e7 a9 10    lda #$10
f9e9 85 bc    sta sfx_pu2_counter

.done
f9eb e6 b8    inc sfx_temp00
f9ed 60      rts

```

```

; sound test 0a, 0b, 0c
sfx_powerup_battery_25: subroutine
f9ee a9 08    lda #$08
f9f0 85 b8    sta sfx_temp00 ; counter
f9f2 d0 0a    bne sfx_powerup_battery_set_update_type
; sound test 0b
sfx_powerup_battery_50: subroutine
f9f4 a9 04    lda #$04
f9f6 85 b8    sta sfx_temp00 ; counter
f9f8 d0 04    bne sfx_powerup_battery_set_update_type
; sound test 0c
sfx_powerup_battery_100: subroutine
f9fa a9 00    lda #$00
f9fc 85 b8    sta sfx_temp00 ; counter
sfx_powerup_battery_set_update_type:
f9fe a9 03    lda #$03
fa00 85 be    sta sfx_pu2_update_type
fa02 60      rts

```

```

sfx_powerup_battery_arp:
fa03 byte    #$18, #$1c, #$1f, #$24

sfx_powerup_battery_update: subroutine
fa07 a5 b8    lda sfx_temp00
fa09 29 03    and #%00000011
fa0b d0 2c    bne .dont_trigger
fa0d a5 b8    lda sfx_temp00
fa0f 4a      lsr
fa10 4a      lsr
fa11 c9 04    cmp #$04
fa13 f0 27    beq .end_sound
fa15 aa      tax
fa16 a5 c2    lda audio_root_tone
fa18 18      clc
fa19 7d 03 fa adc sfx_powerup_battery_arp,x
fa1c aa      tax
fa1d a9 83    lda #%10000011
fa1f 8d 04 40 sta $4004
fa22 a9 00    lda #$00
fa24 8d 05 40 sta $4005
fa27 bd b9 f6 lda periodTableLo,x
fa2a 8d 06 40 sta $4006
fa2d bd f8 f6 lda periodTableHi,x
fa30 09 40    ora #%01000000
fa32 8d 07 40 sta $4007
fa35 a9 10    lda #$10
fa37 85 bc    sta sfx_pu2_counter

.dont_trigger
fa39 e6 b8    inc sfx_temp00 ; counter
fa3b 60      rts

```

```

        .end_sound
fa3c  a9 00      lda #$00
fa3e  85 be      sta sfx_pu2_update_type
fa40  60         rts

```

```

        ; sound test 0d
sfx_shoot_dart: subroutine
        ; pulse 1
fa41  a9 0f      lda #%00001111
fa43  8d 00 40   sta $4000
fa46  a9 83      lda #%10000011
fa48  8d 01 40   sta $4001
fa4b  a5 26      lda rng0
fa4d  29 3f      and #$3f
fa4f  09 08      ora #$08
fa51  8d 02 40   sta $4002
fa54  a9 08      lda #%00001000
fa56  8d 03 40   sta $4003
fa59  a9 10      lda #$10
fa5b  85 bb      sta sfx_pu1_counter
fa5d  60         rts

```

```

        ; sound test 0e
sfx_shoot_bullet: subroutine
fa5e  a9 1f      lda #%00011111
fa60  8d 4c 01   sta apu_cache+$c
fa63  a5 24      lda wtf
fa65  29 03      and #$03
fa67  18         clc
fa68  69 09      adc #$09
fa6a  8d 4e 01   sta apu_cache+$e
fa6d  a9 03      lda #$03
fa6f  85 b4      sta apu_noi_envelope
fa71  a9 20      lda #$20
fa73  85 b3      sta apu_noi_counter
fa75  60         rts

```

```

        ; sound test 0f
sfx_rng_chord: subroutine
        ; used hardware envelope was 1 second
        ; ~ 64 frame fade
        ; triangle cuts off at 32 frames
        ; setup pulse 1 + 2
fa76  a9 40      lda #$40
fa78  85 ac      sta apu_pu1_counter
fa7a  85 b0      sta apu_pu2_counter
fa7c  a9 00      lda #$00
fa7e  85 ad      sta apu_pu1_envelope
fa80  85 b1      sta apu_pu2_envelope
        ; pulse 1 pitch
fa82  a5 26      lda rng0
fa84  29 0f      and #%00001111
fa86  18         clc
fa87  69 10      adc #$10
fa89  aa         tax

```

```

fa8a  a0 02      ldy #$02
fa8c  20 37 f7   jsr apu_set_pitch
        ; pulse 2 pitch
fa8f  a5 27      lda rng1
fa91  29 0f      and #%00001111
fa93  18         clc
fa94  69 08      adc #$08
fa96  aa         tax
fa97  a0 06      ldy #$06
fa99  20 37 f7   jsr apu_set_pitch
        ; setup triangle
fa9c  a9 20      lda #$20
fa9e  85 af      sta apu_tri_counter
faa0  a5 28      lda rng2
faa2  29 0f      and #%00001111
faa4  18         clc
faa5  69 08      adc #$08
faa7  aa         tax
faa8  a0 0a      ldy #$0a
faaa  20 37 f7   jsr apu_set_pitch
faad  60         rts

```

```

        ; sound test 10
sfx_phase_next: subroutine
faae  a9 00      lda #$00
fab0  85 ba      sta sfx_phase_next_counter
fab2  a9 06      lda #$06
fab4  85 be      sta sfx_pu2_update_type
fab6  60         rts

```

```

sfx_phase_next_update: subroutine
fab7  a5 ba      lda sfx_phase_next_counter
fab9  f0 06      beq .trigger_first
fabb  c9 04      cmp #$04
fabd  f0 1b      beq .trigger_last
fabf  d0 46      bne .done
.trigger_first
fac1  a9 40      lda #$40
fac3  85 b0      sta apu_pu2_counter
fac5  a9 20      lda #$20
fac7  85 bc      sta sfx_pu2_counter
fac9  a9 02      lda #2
facb  85 b1      sta apu_pu2_envelope
facd  a5 c2      lda audio_root_tone
facf  18         clc
fad0  69 18      adc #24 ; two octaves
fad2  aa         tax
fad3  a0 06      ldy #6
fad5  20 37 f7   jsr apu_set_pitch
fad8  d0 2d      bne .done
.trigger_last
fada  a9 40      lda #$40
fadc  85 ac      sta apu_pu1_counter
fade  85 b0      sta apu_pu2_counter
fae0  a9 20      lda #$20
fae2  85 bc      sta sfx_pu2_counter
fae4  a9 02      lda #2

```



```

fae6 85 ad      sta apu_pu1_envelope
fae8 85 b1      sta apu_pu2_envelope
faea a5 c2      lda audio_root_tone
faec 18         clc
faed 69 1f      adc #31 ; two octaves + 5th
faef aa        tax
faf0 a0 02      ldy #2
faf2 20 37 f7   jsr apu_set_pitch
faf5 a0 06      ldy #6
faf7 20 37 f7   jsr apu_set_pitch
fafa ee 46 01   inc apu_cache+6
fafd ee 46 01   inc apu_cache+6
fb00 ee 46 01   inc apu_cache+6
                ; trigger next phase and kill sfx
fb03 a9 00      lda #$00
fb05 85 be      sta sfx_pu2_update_type
                .done
fb07 e6 ba      inc sfx_phase_next_counter
fb09 60         rts

                ; sound test 11
sfx_snare: subroutine
fb0a a9 0a      lda #$a
fb0c 8d 4e 01   sta apu_cache+$e
fb0f a9 06      lda #$06
fb11 85 b3      sta apu_noi_counter
fb13 a9 05      lda #$05
fb15 85 b4      sta apu_noi_envelope
fb17 60         rts

                ; sound test 12
sfx_hat: subroutine
fb18 a5 bd      lda sfx_noi_counter
fb1a d0 0f      bne .no
fb1c a5 b6      lda apu_rng1
fb1e 29 03      and #3
fb20 8d 4e 01   sta apu_cache+$e
fb23 a9 0e      lda #$e
fb25 85 b3      sta apu_noi_counter
fb27 a9 04      lda #$04
fb29 85 b4      sta apu_noi_envelope
                .no
fb2b 60         rts

                ; sound test 13
sfx_ghost_snare: subroutine
fb2c a5 28      lda rng2
fb2e 29 01      and #$01
fb30 69 0b      adc #$0b
fb32 8d 4e 01   sta apu_cache+$e
fb35 a9 04      lda #$04
fb37 85 b3      sta apu_noi_counter
fb39 a9 05      lda #$05
fb3b 85 b4      sta apu_noi_envelope
fb3d 60         rts

```

apu_song.asm

```

                song_rng_chord      EQM $00
                song_sick_dingle   EQM $01
                song_in_game       EQM $02
                song_boss_intro    EQM $03
                song_boss_fight    EQM $04
                song_game_over     EQM $05
                song_end_bad       EQM $06
                song_end_ok        EQM $07
                song_end_good      EQM $08

song_update: subroutine
fb3e a5 2b      lda options_music_on
fb40 d0 01      bne .music_on
fb42 60         rts
                .music_on
fb43 a5 c0      lda audio_song_id
fb45 18         clc
fb46 69 73      adc #song_update_jump_table_offset
fb48 4c f1 c4   jmp jump_to_subroutine

song_start: subroutine
                ; a = song id
fb4b 85 c0      sta audio_song_id
fb4d d0 04      bne .not_rng_chords
fb4f 20 76 fa   jsr sfx_rng_chord
fb52 60         rts
                .not_rng_chords
fb53 a9 01      lda #$01
fb55 85 2b      sta options_music_on
fb57 c5 c0      cmp audio_song_id
fb59 d0 15      bne .normal
                ; sick dingle music init
fb5b a9 70      lda #$70
fb5d 85 c1      sta audio_frame_counter
fb5f a9 02      lda #$02
fb61 85 c2      sta audio_root_tone
fb63 a9 04      lda #$04
fb65 85 c3      sta audio_pattern_pos
fb67 a9 30      lda #$30
fb69 85 b6      sta apu_rng1
fb6b a9 44      lda #$44
fb6d 85 b5      sta apu_rng0
fb6f 60         rts
                .normal
fb70 a9 00      lda #0
fb72 85 c1      sta audio_frame_counter
fb74 85 c2      sta audio_root_tone
fb76 85 c3      sta audio_pattern_pos
fb78 20 73 f6   jsr apu_rng_reset
fb7b 60         rts

```



```

song_stop: subroutine
fb7c a9 00    lda #$00
fb7e 85 2b    sta options_music_on
fb80 60        rts

song_unstop: subroutine
fb81 a9 01    lda #$01
fb83 85 2b    sta options_music_on
fb85 60        rts

song_01_lda_note: subroutine
; x = note offset
; returns note id in a
fb86 a5 b7    lda apu_temp
fb88 f0 04    beq .octoscale
                .majorpentscale
fb8a bd a2 f6    lda majpentscale,x
fb8d 60        rts
                .octoscale
fb8e bd 90 f6    lda octoscale,x
fb91 60        rts

; sick dingle
song_01: subroutine
fb92 a9 11    lda #$11
fb94 18        clc
fb95 65 c1    adc audio_frame_counter
fb97 85 c1    sta audio_frame_counter
fb99 90 6b    bcc .do_nothing
fb9b a9 0a    lda #$0a
fb9d 85 c1    sta audio_frame_counter
fb9f a5 c3    lda audio_pattern_pos
fba1 d0 0e    bne .trigger_note
                ; change the note
fba3 a9 08    lda #$08
fba5 85 c3    sta audio_pattern_pos
                .update_root_note
fba7 a5 b6    lda apu_rng1
fba9 29 07    and #%00000111
fbab aa        tax
fbac 20 86 fb    jsr song_01_lda_note
fbaf 85 c2    sta audio_root_tone
                .trigger_note
                ; pulse 1
fbb1 a5 b6    lda apu_rng1
fbb3 29 01    and #%0000001
fbb5 d0 1c    bne .no_pulse_lead
fbb7 a9 20    lda #$20
fbb9 85 ac    sta apu_pu1_counter
fbbb a9 00    lda #$00
fbbd 85 ad    sta apu_pu1_envelope
fbbf a5 b5    lda apu_rng0
fbc1 29 07    and #%00000111
fbc3 18        clc

```

```

fbc4 65 c2    adc audio_root_tone
fbc6 aa        tax
fbc7 20 86 fb    jsr song_01_lda_note
fbca 18        clc
fbcb 69 0c    adc #12
fbcd aa        tax
fbce a0 02    ldy #$02
fbd0 20 37 f7    jsr apu_set_pitch
                .no_pulse_lead
                ; pulse 2
fbd3 a5 bc    lda sfx_pu2_counter
fbd5 d0 1e    bne .no_pulse_rhythm
fbd7 a9 04    lda #$04
fbd9 25 b6    and apu_rng1
fbdb 24 c3    bit audio_pattern_pos
fbdd f0 16    beq .no_pulse_rhythm
fbd f a9 10    lda #$10
fbd1 85 b0    sta apu_pu2_counter
fbd3 a9 00    lda #$00
fbd5 85 b1    sta apu_pu2_envelope
fbd7 a6 c2    ldx audio_root_tone
fbd9 20 86 fb    jsr song_01_lda_note
fbec 18        clc
fbef 69 18    adc #24
fbef aa        tax
fbf0 a0 06    ldy #$06
fbf2 20 37 f7    jsr apu_set_pitch
                .no_pulse_rhythm
                ; triangle
fbf5 a9 03    lda #$03
fbf7 85 af    sta apu_tri_counter
fbf9 a5 c2    lda audio_root_tone
fbfb 18        clc
fbfc 69 18    adc #$18
fbfe aa        tax
fbff a0 0a    ldy #$0a
fc01 20 37 f7    jsr apu_set_pitch
fc04 c6 c3    dec audio_pattern_pos
                .do_nothing
fc06 a9 00    lda #$00
fc08 85 b7    sta apu_temp
fc0a 60        rts

; in game
song_02: subroutine
fc0b a5 24    lda wtf
fc0d 29 07    and #$07
fc0f f0 01    beq .do_this_shit
fc11 60        rts
                .do_this_shit
                ; options screen decrease player health
fc12 a5 55    lda state_render_addr
fc14 c9 82    cmp #state_render_jump_table_offset+1
fc16 d0 02    bne .not_menu_screens
fc18 c6 0b    dec player_health
                .not_menu_screens

```

```

; triangle
fc1a a5 c3 lda audio_pattern_pos
fc1c 45 27 eor rng1
fc1e 29 07 and #7
fc20 f0 2a beq .no_triangle
; change note length based on health?
fc22 a5 0b lda player_health
fc24 49 ff eor #$ff
fc26 4a lsr
fc27 4a lsr
fc28 4a lsr
fc29 4a lsr
fc2a 4a lsr
fc2b 69 02 adc #$02
; lda #$05 ; original length
fc2d 85 af sta apu_tri_counter
fc2f a5 c2 lda audio_root_tone
fc31 18 clc
fc32 69 18 adc #24
fc34 85 b7 sta apu_temp
fc36 a5 c3 lda audio_pattern_pos
fc38 29 01 and #1
fc3a d0 06 bne .no_octave
fc3c a5 b7 lda apu_temp
fc3e 69 0c adc #12
fc40 85 b7 sta apu_temp
.no_octave
fc42 a6 b7 ldx apu_temp
fc44 bd 90 f6 lda octoscale,x
fc47 a0 0a ldy #$0a
fc49 20 37 f7 jsr apu_set_pitch
.no_triangle
; pulse lead
; lda sfx_pu1_counter
; bne .no_pulse_lead
; lda sfx_pu2_counter
; bne .no_pulse_lead
; lda apu_rng0
; and #$01
; bne .no_pulse_lead
; lda apu_rng1
; and #%00000001
; beq .no_pulse_lead
; lda #$0e
; sta apu_pu1_counter
; lda #$0c
; sta apu_pu2_counter
; lda #$05
; sta apu_pu1_envelope
; sta apu_pu2_envelope
; lda apu_rng0
; and #%00000111
; clc
; adc audio_root_tone
; tax
; lda octoscale,x
; clc
; adc #24

```

```

; tax
; ldy #$02
; jsr apu_set_pitch
; txa
; adc #19
; tax
; ldy #$06
; jsr apu_set_pitch
; .no_pulse_lead
; kick
fc4c a5 c3 lda audio_pattern_pos
fc4e d0 03 bne .no_kick
fc50 20 e3 f8 jsr sfx_kick
.no_kick
; check for sfx noise
fc53 a5 bd lda sfx_noi_counter
fc55 d0 1b bne .no_noise
; hats
fc57 a5 b5 lda apu_rng0
fc59 29 06 and #6
fc5b f0 03 beq .no_hat
fc5d 20 18 fb jsr sfx_hat
.no_hat
; ghost snare
fc60 a5 26 lda rng0
fc62 29 0c and #$c
fc64 d0 03 bne .no_g_snare
fc66 20 2c fb jsr sfx_ghost_snare
.no_g_snare
; snare
fc69 a5 c3 lda audio_pattern_pos
fc6b c9 06 cmp #6
fc6d d0 03 bne .no_snare
.do_snare
fc6f 20 0a fb jsr sfx_snare
.no_snare
.no_noise
; next tic
fc72 e6 c3 inc audio_pattern_pos
fc74 a5 c3 lda audio_pattern_pos
fc76 c9 0c cmp #12
fc78 d0 19 bne .done
fc7a a5 b5 lda apu_rng0
fc7c 29 07 and #$07
fc7e 85 c2 sta audio_root_tone
fc80 a9 00 lda #0
fc82 85 c3 sta audio_pattern_pos
fc84 e6 c1 inc audio_frame_counter
fc86 a9 18 lda #24
fc88 c5 c1 cmp audio_frame_counter
fc8a d0 07 bne .done
fc8c 20 73 f6 jsr apu_rng_reset
fc8f a9 00 lda #0
fc91 85 c1 sta audio_frame_counter
.done
fc93 60 rts

```

```

; boss intro
song_03: subroutine
fca4 e6 c3      lda wtf
fca6 a9 00      and #$07
fca8 d0 04      bne .done
fca9 a5 c3      lda audio_pattern_pos
fcae c9 08      cmp #$08
fcae d0 04      bne .next_step
fca0 20 7c fb   jsr song_stop
fca3 60         rts

.next_step
fca4 e6 c3      inc audio_pattern_pos
fca6 a9 00      lda #0
fca8 85 ad      sta apu_pu1_envelope
fcaa 85 b1      sta apu_pu2_envelope
fcac a9 40      lda #$40
fcae 85 ac      sta apu_pu1_counter
fcb0 85 b0      sta apu_pu2_counter
fcb2 85 af      sta apu_tri_counter

.set_pitch
fcb4 e6 c2      inc audio_root_tone
fcb6 a5 c2      lda audio_root_tone
fcb8 18         clc
fcb9 69 04      adc #4
fcbb 85 b7      sta apu_temp
fcbd aa        tax
fcbe bd 90 f6   lda octoscale,x
fcc1 69 0c      adc #12
fcc3 aa        tax
fcc4 a0 02      ldy #2
fcc6 20 37 f7   jsr apu_set_pitch
fcc9 a5 b7      lda apu_temp
fccb 69 0f      adc #15
fccd aa        tax
fcce a0 06      ldy #6
fcd0 20 37 f7   jsr apu_set_pitch
fcd3 a0 0a      ldy #10
fcd5 20 37 f7   jsr apu_set_pitch
; set noise
fcd8 a9 0f      lda #$f
fcda e5 c3      sbc audio_pattern_pos
fcdc 8d 4e 01   sta apu_cache+$e
fcdf a9 00      lda #$00
fce1 85 b4      sta apu_noi_envelope
fce3 a9 40      lda #$40
fce5 85 b3      sta apu_noi_counter

.done
fce7 60         rts

; boss fight
song_04: subroutine
fce8 a5 c1      lda audio_frame_counter
fcea c9 05      cmp #5
fcec f0 03      beq .do_all_the_stuff
fcee 4c 7a fd   jmp .done

```

```

.do_all_the_stuff
fcf1 a9 00      lda #0
fcf3 85 c1      sta audio_frame_counter
fcf5 e6 c3      inc audio_pattern_pos
fcf7 a5 c3      lda audio_pattern_pos
fcf9 c9 12      cmp #18
fcfb d0 12      bne .dont_loop

.dont_loop
; TENSION ANTI_MELODY
fcfd a5 c4      lda audio_pattern_num
fcff 29 04      and #$04
fd01 d0 04      bne .root_raise

.root_lower
fd03 e6 c2      inc audio_root_tone
fd05 d0 02      bne .root_done

.root_raise
fd07 c6 c2      dec audio_root_tone

.root_done
fd09 a9 00      lda #0
fd0b 85 c3      sta audio_pattern_pos
fd0d e6 c4      inc audio_pattern_num

.dont_loop
; KICK (pulse 2)
fd0f a5 c3      lda audio_pattern_pos
fd11 d0 03      bne .no_kick
fd13 20 e3 f8   jsr sfx_kick

.no_kick
; SNARE
fd16 a5 c3      lda audio_pattern_pos
fd18 c9 0a      cmp #10
fd1a f0 08      beq .snare
fd1c a5 26      lda rng0
fd1e 29 07      and #7
fd20 f0 02      beq .snare
fd22 d0 03      bne .no_snare

.snare
fd24 20 0a fb   jsr sfx_snare

.no_snare
; TENSION ANTI_MELODY (pulse 1)
fd27 a5 bb      lda sfx_pu1_counter
fd29 d0 26      bne .no_melody
fd2b a9 05      lda #5
fd2d 85 ad      sta apu_pu1_envelope
fd2f a9 05      lda #$05
fd31 85 ac      sta apu_pu1_counter
fd33 a6 c2      ldx audio_root_tone
fd35 a5 b6      lda apu_rng1
fd37 4a         lsr
fd38 29 08      and #$08
fd3a f0 03      beq .change_root
fd3c 8a         txa
fd3d d0 09      bne .stay_root

.change_root
fd3f a5 b5      lda apu_rng0
fd41 29 07      and #$07
fd43 85 86      sta temp00
fd45 8a         txa
fd46 65 86      adc temp00

```

```

    .stay_root
fd48 18      clc
fd49 69 24   adc #36
fd4b aa      tax
fd4c a0 02   ldy #$02
fd4e 20 37 f7 jsr apu_set_pitch
    .no_melody
    ; BASSLINE
fd51 a6 c3   ldx audio_pattern_pos
fd53 bc 8f fd ldy song_04_length,x
fd56 f0 22   beq .done
    ; chance of staccato
fd58 a5 b5   lda apu_rng0
fd5a 29 0c   and #$0c
fd5c d0 02   bne .full_length
fd5e a0 02   ldy #$02
    .full_length
fd60 84 af   sty apu_tri_counter
fd62 bd 7d fd lda song_04_pitch,x
fd65 18      clc
fd66 69 18   adc #24
fd68 85 b7   sta apu_temp
fd6a a5 c4   lda audio_pattern_num
fd6c 29 04   and #4
fd6e 4a      lsr
fd6f 65 b7   adc apu_temp
fd71 aa      tax
fd72 a0 0a   ldy #10
fd74 20 37 f7 jsr apu_set_pitch
    ; hats in unison with bassline
fd77 20 18 fb jsr sfx_hat
    ; we're done
    .done
fd7a e6 c1   inc audio_frame_counter
fd7c 60      rts
song_04_pitch:
fd7d byte    12,0,17,0,0,12,12,0,17,0,0,15,0,0,13,0,0,12
song_04_length:
fd8f byte    18,0,10,0,0, 3,18,0,10,0,0, 8,0,0, 8,0,0, 3

    ; game over
song_05: subroutine
fda1 a5 c1   lda audio_frame_counter
fda3 c9 ff   cmp #$ff
fda5 d0 05   bne .do_normal
fda7 a9 00   lda #$00
fda9 85 09   sta ppu_mask_emph
fdab 60      rts
    .do_normal
fdac 20 4c f7 jsr apu_bend_down
fdaf 20 4c f7 jsr apu_bend_down
fdb2 a5 c1   lda audio_frame_counter
fdb4 c9 70   cmp #$70
fdb6 f0 13   beq .do_chord
fdb8 c9 40   cmp #$40
fdba f0 0f   beq .do_chord

```

```

fdbc c9 30   cmp #$30
fdbe f0 0b   beq .do_chord
fdc0 c9 10   cmp #$10
fdc2 f0 07   beq .do_chord
fdc4 a5 c1   lda audio_frame_counter
fdc6 d0 0b   bne .done
fdc8 20 0a fb jsr sfx_snare
    .do_chord
fdcb 20 76 fa jsr sfx_rng_chord
fdce 20 bc c1 jsr ppu_mess_emph
fdd1 e6 09   inc ppu_mask_emph
    .done
fdd3 e6 c1   inc audio_frame_counter
fdd5 60      rts

    ; ending bad
song_06: subroutine
    ; rng chord about every 2 seconds (bends down)
fdd6 20 4c f7 jsr apu_bend_down
fdd9 20 4c f7 jsr apu_bend_down
fddc a5 c1   lda audio_frame_counter
fdde 29 7d   and #$7d
fde0 d0 03   bne .no_chord
fde2 20 76 fa jsr sfx_rng_chord
    .no_chord
fde5 e6 c1   inc audio_frame_counter
fde7 60      rts

    ; ending ok
song_07: subroutine
    ; sick dingle but different seed and bends down
fde8 20 4c f7 jsr apu_bend_down
fdeb 20 92 fb jsr song_01
fdee 60      rts

    ; ending good
song_08: subroutine
fdef e6 b7   inc apu_temp
fdf1 20 92 fb jsr song_01
fdf4 60      rts

    WHOOPS:
fdf5 20 4c c1 jsr waitSync
fdf8 20 4c c1 jsr waitSync
fdfb 4c 5b f6 jmp apu_init

```

tables_at_top_of_rom.asm

```

sine_table:
fe00 hex      808386898c8f9295
fe08 hex      989b9ea2a5a7aaad
fe10 hex      b0b3b6b9bcbec1c4
fe18 hex      c6c9cbced0d3d5d7
fe20 hex      dadcdee0e2e4e6e8
fe28 hex      eaebedee0f1f3f4
fe30 hex      f5f6f8f9fafafbfcc
fe38 hex      fdfdfe0efefefefff
fe40 hex      ffffffffefefefd
fe48 hex      fdcfbfafaf9f8f6
fe50 hex      f5f4f3f1f0eeedeb
fe58 hex      eae8e6e4e2e0dedc
fe60 hex      dad7d5d3d0cecbcb9
fe68 hex      c6c4c1bebc9b6b3
fe70 hex      b0adaaa7a5a29e9b
fe78 hex      9895928f8c898683
fe80 hex      807c797673706d6a
fe88 hex      6764615d5a585552
fe90 hex      4f4c494643413e3b
fe98 hex      393634312f2c2a28
fea0 hex      2523211f1d1b1917
fea8 hex      151412110f0e0c0b
feb0 hex      0a09070605050403
feb8 hex      0202010101000000
fec0 hex      0000000001010102
fec8 hex      0203040505060709
fed0 hex      0a0b0c0e0f111214
fed8 hex      1517191b1d1f2123
fee0 hex      25282a2c2f313436
fee8 hex      393b3e414346494c
fef0 hex      4f5255585a5d6164
fef8 hex      676a6d707376797c

```

```

; 0 = right
; 6 = up
; 12 = left
; 18 = down

```

ARCTANG_TRANSLATION_LOOKUP_TABLE:

```

ff00 byte      9, 3,15,21
ff04 byte      10, 2,14,22
ff08 byte      11, 1,13,23
ff0c byte      12, 0,12, 0
ff10 byte      9, 3,15,21
ff14 byte      8, 4,16,20
ff18 byte      7, 5,17,19
ff1c byte      6, 6,18,18

```

```

ff20 byte
ff26 byte
ff2c byte
ff32 byte

```

```

ff38 byte
ff3e byte
ff44 byte
ff4a byte

```

```

ff50 byte
ff56 byte
ff5c byte
ff62 byte

```

```

ff68 byte
ff6e byte
ff74 byte
ff7a byte

```

```

; reduce cycle counts by keeping these
; tables on the same page
ARCTANG_REGION_TO_X_VELOCITY_TABLE:

```

```

0, 1, 2, 3, 4, 5
6, 5, 4, 3, 2, 1
0, 1, 2, 3, 4, 5
6, 5, 4, 3, 2, 1

```

ARCTANG_REGION_TO_Y_VELOCITY_TABLE:

```

6, 5, 4, 3, 2, 1
0, 1, 2, 3, 4, 5
6, 5, 4, 3, 2, 1
0, 1, 2, 3, 4, 5

```

ARCTANG_REGION_X_PLUS_OR_MINUS_TABLE:

```

; 1 = plus
; 0 = minus
1, 1, 1, 1, 1, 1
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
1, 1, 1, 1, 1, 1

```

ARCTANG_REGION_Y_PLUS_OR_MINUS_TABLE:

```

; 1 = plus
; 0 = minus
0, 0, 0, 0, 0, 0
0, 0, 0, 0, 0, 0
1, 1, 1, 1, 1, 1
1, 1, 1, 1, 1, 1

```

arctang_velocity_tables:

```

; region id angle degrees
; 0 0
; 1 15
; 2 30
; 3 45
; 4 60
; 5 75
; 6 90

```

arctang_velocity_6.66:

```

ff80 byte      168, 6
ff82 byte      109, 6
ff84 byte      193, 5
ff86 byte      119, 4
ff88 byte      84, 3
ff8a byte      183, 1
ff8c byte      0, 0

```

arctang_velocity_4.5:

```

ff8e byte      127, 4
ff90 byte      88, 4
ff92 byte      229, 3
ff94 byte      46, 3
ff96 byte      64, 2
ff98 byte      42, 1
ff9a byte      0, 0

```

```

arctang_velocity_3.33:
ff9c byte      85, 3
ff9e byte      56, 3
ffa0 byte     227, 2
ffa2 byte      91, 2
ffa4 byte     171, 1
ffa6 byte     221, 0
ffa8 byte       0, 0
arctang_velocity_2.5:
ffaa byte     127, 2
ffac byte     104, 2
ffae byte      43, 2
ffb0 byte     197, 1
ffb2 byte      64, 1
ffb4 byte     166, 0
ffb6 byte       0, 0
arctang_velocity_1.75:
ffb8 byte     191, 1
ffba byte     176, 1
ffbc byte     131, 1
ffbe byte      61, 1
ffc0 byte     223, 0
ffc2 byte     115, 0
ffc4 byte       0, 0
arctang_velocity_1.25:
ffc6 byte      64, 1
ffc8 byte      53, 1
ffca byte      20, 1
ffcc byte     225, 0
ffce byte     161, 0
ffd0 byte      81, 0
ffd2 byte       0, 0
arctang_velocity_0.75:
ffd4 byte     191, 0
ffd6 byte     184, 0
ffd8 byte     166, 0
ffda byte     135, 0
ffdc byte      96, 0
ffde byte      49, 0
ffe0 byte       0, 0
arctang_velocity_0.33:
ffe2 byte      84, 0
ffe4 byte      81, 0
ffe6 byte      73, 0
ffe8 byte      59, 0
ffea byte      42, 0
ffec byte      22, 0
ffee byte       0, 0

arctang_velocities_lo:
fff0 byte     #<arctang_velocity_6.66
fff1 byte     #<arctang_velocity_4.5
fff2 byte     #<arctang_velocity_3.33
fff3 byte     #<arctang_velocity_2.5
fff4 byte     #<arctang_velocity_1.75
fff5 byte     #<arctang_velocity_1.25
fff6 byte     #<arctang_velocity_0.75
fff7 byte     #<arctang_velocity_0.3

```


